

Tangy World Complete Architecture

System architecture, workflows, database, security, deployment and failure handling, reverse-engineered from the repository

Repository	komalrubyjane/tangy-world
Commit documented	cbd1151 · 4 Oct 2026
Generated	7 Oct 2026
Scope	React SPA · Supabase (Postgres, Auth, Storage, Realtime, Edge Functions) · Razorpay · Resend · Vercel
Contents	34 documents · 90 diagrams
Status	Code verified locally; production deployment not yet performed

Table of Contents

Executive summary and key findings	4
Part I — Overview & Architecture	
Tangy World — Architecture Documentation Package	7
01 — System Overview	9
02 — User / Role Architecture	12
03 — Complete Website Map	22
04 — Frontend Architecture	32
05 — Authentication, Role and Permission Resolution	38
Part II — Major Workflows	
06 — Admin Architecture (the "Control Room")	45
07 — Artist Architecture and Lifecycle	55
08 — Artist Application Workflow (`/artist/apply`)	60
09 — Event Lifecycle	65
10 — Event ↔ Artist Workflow (Availability, Line-up, Booking Requests)	70
11 — Customer Booking Workflow	79
12 — Payment Architecture (Razorpay)	82
13 — Ticket Architecture	88
14 — Check-in Workflow	91
15 — Waitlist Workflow	96
16 — Notification Architecture	100
17 — Email Architecture	105
18 — Messaging Architecture	110
19 — Content / CMS Architecture	117
20 — Storage Architecture	121
Part III — Database & Backend	
21 — Database Architecture	127
22 — Database Function / RPC Map	137
23 — Edge Function Architecture	143
24 — Scheduled Job Architecture	147
Part IV — Security	
25 — Security Architecture	153
Part V — Environments & Deployment	
26 — Local / Demo Architecture	157
27 — Deployment Architecture	162
28 — Environment Architecture	164
Part VI — Failure Flows & Data Flow	
29 — Failure and Recovery Flows	167
30 — Data Flow Diagrams	174
Part VII — Who Calls What	
31 — "Who Calls What" Map	181
Part VIII — Complete Business Flow	
32 — Complete Business Flow	195

Part IX — Explaining & Presenting

33 — How to Explain Tangy World to Someone	198
34 — How I Should Present This System	200

Executive summary and key findings

The system in one paragraph

Tangy World is a React 19 single-page app, built with Vite and configured for Vercel, on top of Supabase. All rules live in Postgres:

- 61 tables, about 228 functions, about 240 RLS policies
- prices, seat holds, waitlist offers and settlement decided under row locks
- roles and permissions in a table, with self-promotion blocked by a trigger
- an append-only audit log

Seven Deno Edge Functions handle Razorpay (order, signature verification, webhook) and email through Resend (ticket, notification drain, approval, invitation). Customers book on the session page and get one QR per booking. Staff, or volunteers with time-boxed access, admit attendees by name. Artists apply through an 8-step application, set their availability, and accept requests that the database protects against double-booking.

Key findings

Major workflows documented

Every workflow below has a diagram:

- authentication and invitations
- role provisioning
- the artist application and review loop
- artist availability
- line-up and booking requests
- the event lifecycle
- customer booking
- Razorpay payment (browser and webhook paths)
- settlement decisions
- ticket issuance and QR
- group check-in and volunteer temporary access
- waitlist offers and expiry
- notifications and preferences
- the ticket and notification email queue
- approval and invitation email
- messaging (support, partner, private)
- content publishing
- storage uploads and signed reads
- scheduled jobs
- local demo and review environments
- deployment
- failure and recovery
- data flows
- the end-to-end business lifecycle

Production vs repository

Item	Repository	Production
Database	36 migrations + a validated production bootstrap package	Not applied (<code>production-bootstrap/README.md</code>)
Edge Functions	7 functions, deploy flags documented	Not deployed ; no <code>supabase/config.toml</code> , no CI (<code>docs/OPERATIONS.md §3a</code>)
Razorpay	full integration	keys, webhook URL and secret not configured ; payload fields to verify against a live webhook

Item	Repository	Production
Email	Resend provider module, outbox, drain	RESEND_API_KEY , verified domain and the drain scheduler not set up
Auth OTP	signInWith0tp / verify0tp	email template must contain the token: unverified
Frontend	Vercel-ready build with guards	canonical domain / SITE_URL not determinable from the repo

Architecture inconsistencies found

1. **Artist approval email gap.** Approving through *Artist Applications* (`review_artist_application`) sends only an in-app notice. The branded `send-approval-email` runs only from the generic *Applications* drawer. See 07.
2. **Ticket holders get no reminders and no cancellation notices.** `send_event_reminders` and `notify_on_event_change` target event *members* only. Cancelling an event does not touch bookings. See 09.
3. **Event sold-out / past statuses are manual.** Capacity is enforced automatically, but the status flags are not updated by any trigger.
4. **Role-string checks in two Edge Functions.** `send-ticket-email` (staff/admin/super_admin) and `send-approval-email` (admin/super_admin) authorize by `profiles.role`, not `has_permission()`, unlike the rest of the system.
5. **Stale documentation.** `.env.example` still calls `content-media` public (private since 0029). The 0017 comment says `role_permissions` is "never changed from the app", but `set_role_permission` (0018) now allows it.
6. **Redundant cron job.** `tangy-log-expired-access` (0018) duplicates work already in `run_platform_jobs`. It is only created if `pg_cron` pre-existed.
7. **Unused legacy code.** - The mock services behind `isMockAuth` (always false). - `assignmentRequestService` / `create_assignment_request` (0008). - `adminApi.createBookingRequest` / `create_booking_request` (0020). - `VITE_RAZORPAY_KEY_ID` (listed in `.env.example`, never read).
8. **No security headers / CSP** in `vercel.json` or `index.html`.
9. **The repository README** is still the default Vite template.

Undocumented / uncertain areas

- **Live configuration:**
- Razorpay payload field names (the webhook source carries its own "verify before going live" note)
- payment auto-capture
- Supabase Auth rate limits and the OTP template
- whether `pg_cron` is available in the production project
- **No ownership information:** the repository has no deployment or monitoring setup for Vercel environments.
- **Tangy AI:** the public assistant is a knowledge-base lookup; the admin AI page is a placeholder.

Diagram inventory

90 Mermaid diagrams (flowcharts, sequence diagrams, state diagrams and ER diagrams), each with a title. All of them validated by rendering with Mermaid 11 in headless Chromium.

Part I — Overview & Architecture

Tangy World — Architecture Documentation Package · 01 — System Overview · 02 — User / Role Architecture · 03 — Complete Website Map · 04 — Frontend Architecture · 05 — Authentication, Role and Permission Resolution

Tangy World — Architecture Documentation Package

This folder is a reverse-engineered, code-verified description of the Tangy World / Tangy Sessions platform as it exists in this repository at commit `cbd1151` (4 Oct 2026). Every table, RPC, Edge Function, route and workflow named here was found in the source. Nothing was inferred from naming alone.

Start with `TANGY_WORLD_COMPLETE_ARCHITECTURE.md` (index) or the PDF `TANGY_WORLD_COMPLETE_ARCHITECTURE.pdf`.

Status labels used throughout

Label	Meaning
IMPLEMENTED	Present in code and wired end-to-end (frontend → service → database / Edge Function).
PARTIALLY IMPLEMENTED	Some of the path exists; a step is missing or manual.
NOT IMPLEMENTED	No code for it in the repository.
DEMO / LOCAL ONLY	Exists only for local development, the demo dataset or review builds; compiled out of / refused for production.
DOCUMENTED BUT NOT VERIFIED IN CODE	Described in a doc/runbook but the repository cannot prove it (e.g. a dashboard setting).
PRODUCTION CONFIGURATION	Needs an operator action (secret, dashboard setting, scheduler) rather than code.
LEGACY	Old route / service kept for compatibility, not the current path.

How the facts were gathered

- **Frontend:** `src/App.jsx` routes, `src/admin/AdminApp.jsx` routes, every `supabase.from()/rpc()/functions.invoke()/storage.from()/channel()` call grouped by file (see `31-who-calls-what.md`).
- **Database:** all 36 migrations in `supabase/migrations/`. The *final* RLS policy set was computed by replaying every `create policy` / `drop policy`. Key functions were read from their *latest* definition (the file each is quoted from is named).
- **Edge Functions:** the 7 functions and 4 shared modules in `supabase/functions/` were read in full.
- **Operations:** `docs/OPERATIONS.md`, `supabase/README.md`, `supabase/production-bootstrap/README.md`, `scripts/`, `vite.config.js`, `vercel.json`, `.env.example`.

Files

#	File	Topic
01	<code>01-system-overview.md</code>	What Tangy World is; system context diagram
02	<code>02-user-roles.md</code>	Every role, its login, routes, permissions, RLS
03	<code>03-website-map.md</code>	Visual sitemap of every route group
04	<code>04-frontend-architecture.md</code>	Providers, router, shells, services
05	<code>05-authentication.md</code>	OTP / password sign-in, profile, role & permission resolution, invitations
06	<code>06-admin-architecture.md</code>	The admin "control room" section by section
07	<code>07-artist-architecture.md</code>	Artist lifecycle and portal
08	<code>08-artist-application.md</code>	<code>/artist/apply</code> 8-step wizard + admin review
09	<code>09-event-lifecycle.md</code>	Event from draft to archive
10	<code>10-event-artist-workflow.md</code>	Availability, line-up, booking requests, locks
11	<code>11-booking-workflow.md</code>	Customer journey from session page to check-in
12	<code>12-payment-architecture.md</code>	Razorpay order, verify, webhook, settlement
13	<code>13-ticket-workflow.md</code>	Ticket issuance and QR
14	<code>14-checkin-workflow.md</code>	Scanner, group check-in, temporary access
15	<code>15-waitlist-workflow.md</code>	Waitlist, offers, expiry

#	File	Topic
16	16-notification-architecture.md	<code>notify()</code> , preferences, in-app + email queue
17	17-email-architecture.md	Every email path, outbox drain, Resend
18	18-messaging-architecture.md	Conversations, participants, RLS
19	19-content-architecture.md	CMS: TV, diary, gallery, programmes, announcements
20	20-storage-architecture.md	Every Storage bucket and its policies
21	21-database-architecture.md	Domain map, ER diagrams, table catalogue
22	22-rpc-function-map.md	RPCs used by the frontend / Edge Functions
23	23-edge-functions.md	All 7 Edge Functions
24	24-scheduled-jobs.md	<code>pg_cron</code> + email drain
25	25-security-architecture.md	Layered security model, keys, secrets
26	26-demo-local-architecture.md	Local stack, demo data, dev role switcher, review mode
27	27-deployment-architecture.md	Vercel + Supabase + Razorpay + Resend
28	28-environment-architecture.md	Local / dev / review / production
29	29-failure-recovery.md	Failure branches for every critical flow
30	30-data-flow.md	12 data-flow diagrams
31	31-who-calls-what.md	Page → service → RPC/table → function → external
32	32-complete-business-flow.md	End-to-end business lifecycle
33	33-explaining-tangy.md	Plain-language explanation
34	34-presentation-guide.md	Recommended 15–30 minute presentation order

`diagrams/` holds every Mermaid diagram from these files as a standalone `.mmd` source plus a rendered `.svg`.

Scope rules this package follows

- No application code, migration, Edge Function, configuration or secret was changed — only `docs/architecture/` was added.
- No secret values appear anywhere. Secret **names** are listed because the code reads them.
- "Deployed" is never claimed: the repository's own runbook (`docs/OPERATIONS.md` §3a) states the backend has not been deployed and production secrets are not configured.

01 — System Overview

What Tangy World is

Tangy World is the web platform for **Tangy Sessions**, an intimate live-music event series in Hyderabad. One React single-page application serves three audiences:

- 1. **The public website:** a scroll-driven, poster-style "museum" of the brand (Hero, Manifesto, Sessions, Calendar, Archive, Crew, Private Sessions, Diary, Inner Circle, Footer), plus content pages (Tangy TV, Diary, Gallery, Artists, Programmes, FAQ, legal).
- 2. **Customer and partner accounts:** customers book tickets for sessions, pay with Razorpay, receive a QR booking pass and can join a waitlist. Artists, sponsors, vendors, venue hosts, crew and volunteers apply, get approved, and work in their own portals.
- 3. **The operations console** (`/admin-portal`) and the **check-in terminal** (`/check-in`): the Tangy team runs events, line-ups, bookings, payments, content, people, messaging, reports and audit.

All business rules live in **Supabase Postgres** (RLS policies, `SECURITY DEFINER` functions, triggers). **Supabase Edge Functions** (Deno) talk to **Razorpay** (payments) and **Resend** (email). The frontend is built with Vite and configured for **Vercel** (`vercel.json`).

Status: IMPLEMENTED in code and verified on a local Supabase stack by the repository's tests. Production is **PRODUCTION CONFIGURATION, NOT DONE:** `docs/OPERATIONS.md` §3a says "Nothing here has been deployed yet: the repository has no `supabase/config.toml`, no linked project ref, no CI and no deploy script". The production Supabase project ref named in the runbooks is `ohyjqxbsgdkzytfnlitf`.

Size of the system (counted from the repository)

Area	Count	Source
SQL migrations	36 (<code>0001</code> – <code>0036</code>)	<code>supabase/migrations/</code>
Tables / views	61 tables (incl. 1 security-audit helper table) / 3 views	migrations
Distinct SQL functions	~228	migrations
RLS policies (final state)	~240 across 58 tables + <code>storage.objects</code>	replayed create/drop policy
Edge Functions	7 (+ 4 shared modules)	<code>supabase/functions/</code>
Storage buckets	6	migrations
SPA routes	134 <code><Route></code> entries in <code>App.jsx</code> ; 54 in the admin console	<code>src/App.jsx</code> , <code>src/admin/AdminApp.jsx</code>
SQL test suites	20 (<code>supabase/tests/*.test.sql</code>)	
Browser E2E suites	21 Playwright suites (<code>e2e/*.mjs</code> minus <code>lib.mjs</code> , <code>make-cam.mjs</code> ; local stack only)	

System context diagram

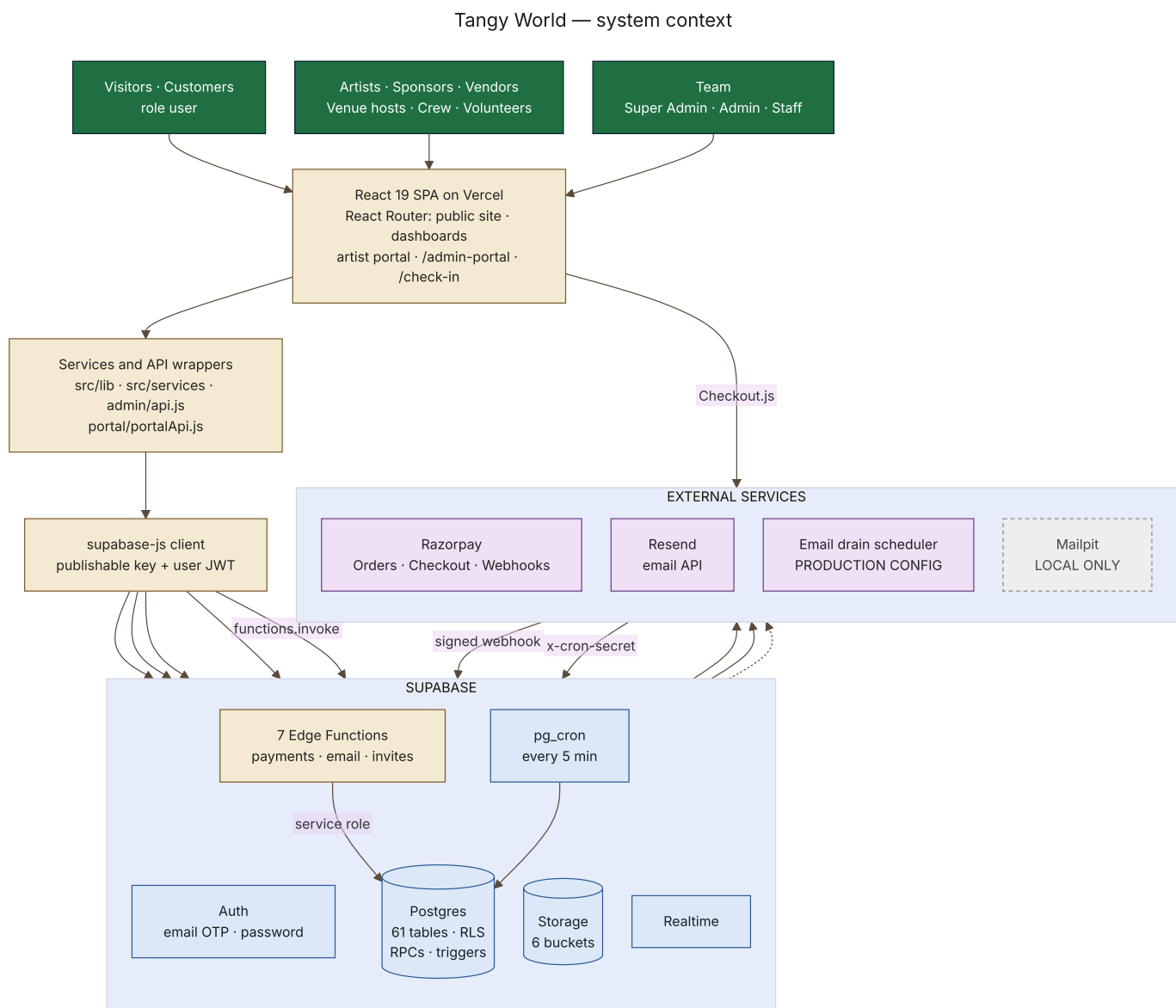


Diagram — Tangy World — system context

Not present in the repository (and therefore not in the diagram): GitHub Actions / any CI, Azure, Netlify, Cloudflare, analytics, SMS/WhatsApp providers, a real AI model provider (see the Tangy AI note below), a CSP / security-headers configuration.

The three layers in one picture

Three layers — experience, operations, partners

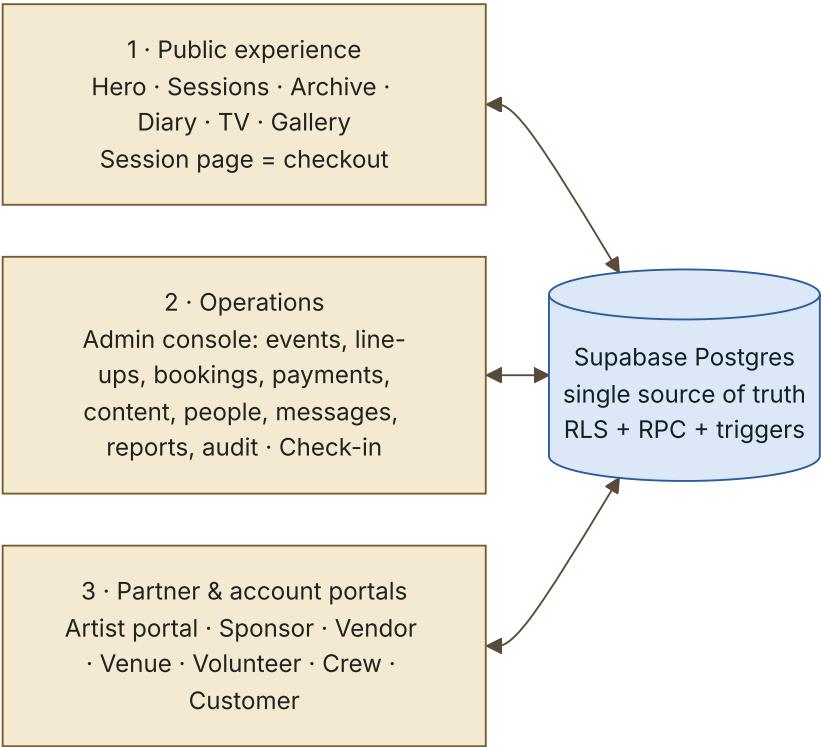


Diagram — Three layers — experience, operations, partners

Key design rules found in the code

Rule	Where it is enforced
The browser holds only the public Supabase key; a secret in any <code>VITE_</code> variable stops <code>vite</code> and <code>vite build</code> .	<code>vite.config.js</code> <code>assertNoSecretInBrowserEnv</code>
Authorization is in the database , never only in the UI. Admin nav and route guards are cosmetic; the DB re-checks every action.	<code>src/admin/rbac.js</code> header, <code>0002_rls.sql</code> header
Prices and capacity are decided by the database (<code>booking_quote</code> , <code>create_pending_booking</code> under an event row lock).	<code>0026</code> , <code>0027</code>
A booking is confirmed only by <code>settle_payment()</code> , called by the Edge Functions after an HMAC check.	<code>0026/0027</code> , <code>razorpay-*</code>
Roles change only through audited RPCs / approvals ; nobody changes their own role.	<code>prevent_role_self_escalation</code> (0030), <code>admin_set_user_role</code> (0034)
Email is queued (<code>email_outbox</code>) and drained server-side; without an email provider nothing is sent and nothing crashes.	<code>notify()</code> , <code>send-notification-emails</code> , <code>_shared/email.ts</code>
No mock data or fake success in production builds . The build fails without a backend; demo and review logins are compiled out.	<code>vite.config.js</code> , <code>commit fix: never ship a build without a backend...</code>

Tangy AI — what it really is

- Public assistant** (`/ai`, floating launcher): **PARTIALLY IMPLEMENTED**. `src/services/aiSupportService.js` is labelled "MOCK AI support engine — a structured knowledge-base lookup, NOT a trained model and NOT connected to any external AI API". Answers come from `src/data/mock/aiKnowledge.js`. The chat transcript uses the mock `messageService` (browser-local). "Request an agent" is **real**: it creates a support conversation through `get_or_create_support_conversation` after sign-in.
- Admin "Tangy AI" page** (`/admin-portal/ai`, `ai.use`): **NOT IMPLEMENTED** as AI. `src/admin/aiService.js` states "NOT connected to any model yet".

02 — User / Role Architecture

Where roles come from

- **Database enum `user_role`** (0001 + 0005 + 0010): `user`, `artist`, `staff`, `admin`, `super_admin`, `vendor`, `sponsor`, `volunteer`, `crew`, `venue`.
- **`profiles.role`** holds one value per account. It is created as `user` by the `handle_new_user()` trigger on `auth.users` insert (0001).
- **Permissions** are rows in `role_permissions` (`role`, `permission`) (0017, 0018, 0028, 0030). `has_permission(p)` checks the caller's role row. `my_permissions()` returns the set to the UI (plus `checkin.perform` while a volunteer has live temporary access).
- **A deactivated account resolves to no role:** `current_role_name()` only returns a role when `profiles.is_active`.
- The UI labels (`src/admin/rbac.js` `ROLE_LABELS`): Super Admin, Admin / Manager, Staff, Patron (`user`), Artist, Vendor, Sponsor, Volunteer, Crew, Venue Partner.
- Review-mode label "Customer" (`src/config/teamReview.js`) is the `user` role. There is **no** separate `customer` enum value.

How a role is obtained

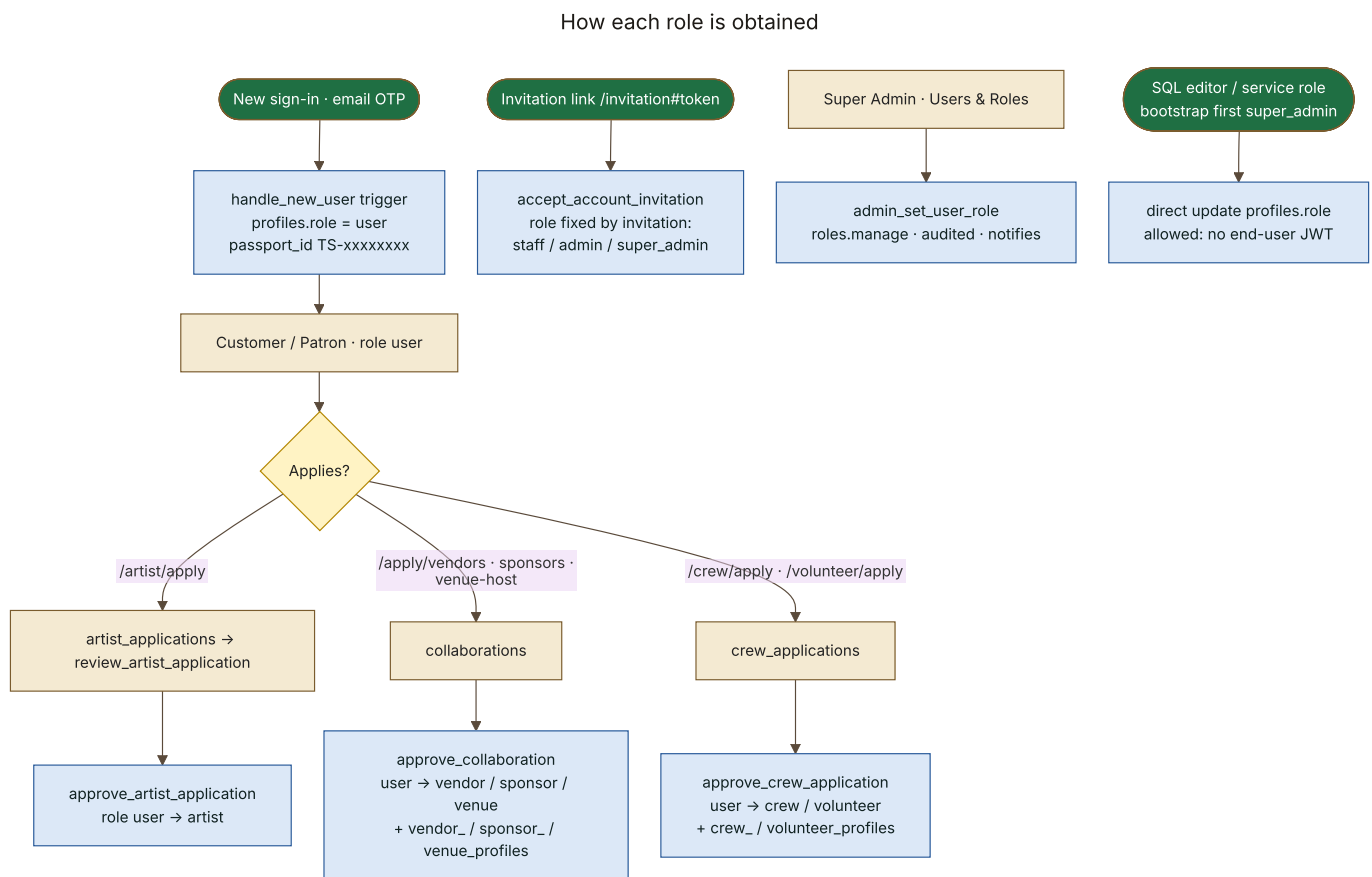


Diagram — How each role is obtained

The guard behind every path is the `prevent_role_self_escalation` trigger (latest in 0030). Its rules:

- Nobody changes their **own** role or active flag, except by accepting an invitation addressed to them.
- An approval may move `user` to a partner or artist role, but only for a caller with `applications.review`.
- Every other role change needs `roles.manage`.
- Changes with no end-user JWT (SQL editor, service role) are allowed. This is how the first super admin is created.

Permission matrix (seeded by migrations)

Permission	super_admin	admin	staff	Notes
dashboard.view	✓	✓	✓	console entry (<code>AdminGate</code> default)
applications.view / applications.review	✓	✓		
events.view_all / events.manage	✓	✓		
events.view_assigned	✓		✓	
bookings.view_all / bookings.manage / payments.view	✓	✓		
attendees.view_all	✓	✓		
attendees.view_assigned	✓		✓	
checkin.perform / checkin.history	✓	✓	✓	volunteers get <code>checkin.perform</code> only while temporary access is live
content.manage · announcements.view	✓	✓	announcements.view	
content.view/create/edit/publish/delete + manage_tv/diary/media/sessions	✓	✓		0028
team.manage · entities.manage	✓	✓		
tasks.view_own	✓		✓	
users.view	✓	✓		
users.manage · roles.manage · audit.view · settings.manage · ai.use	✓			Super Admin only
reports.view · operations.manage	✓	✓		
messages.manage · volunteers.manage · access.grant	✓	✓		0018
staff.invite	✓	✓		0030

A Super Admin can change the **admin** and **staff** rows at runtime with `set_role_permission()` (Roles & Permissions page). `roles.manage` itself and the Super Admin row cannot be edited (0018). The 0017 comment "*changed only by migration/SQL (never from the app)*" predates this. The runtime editor is the current behaviour.

Role → dashboard resolution

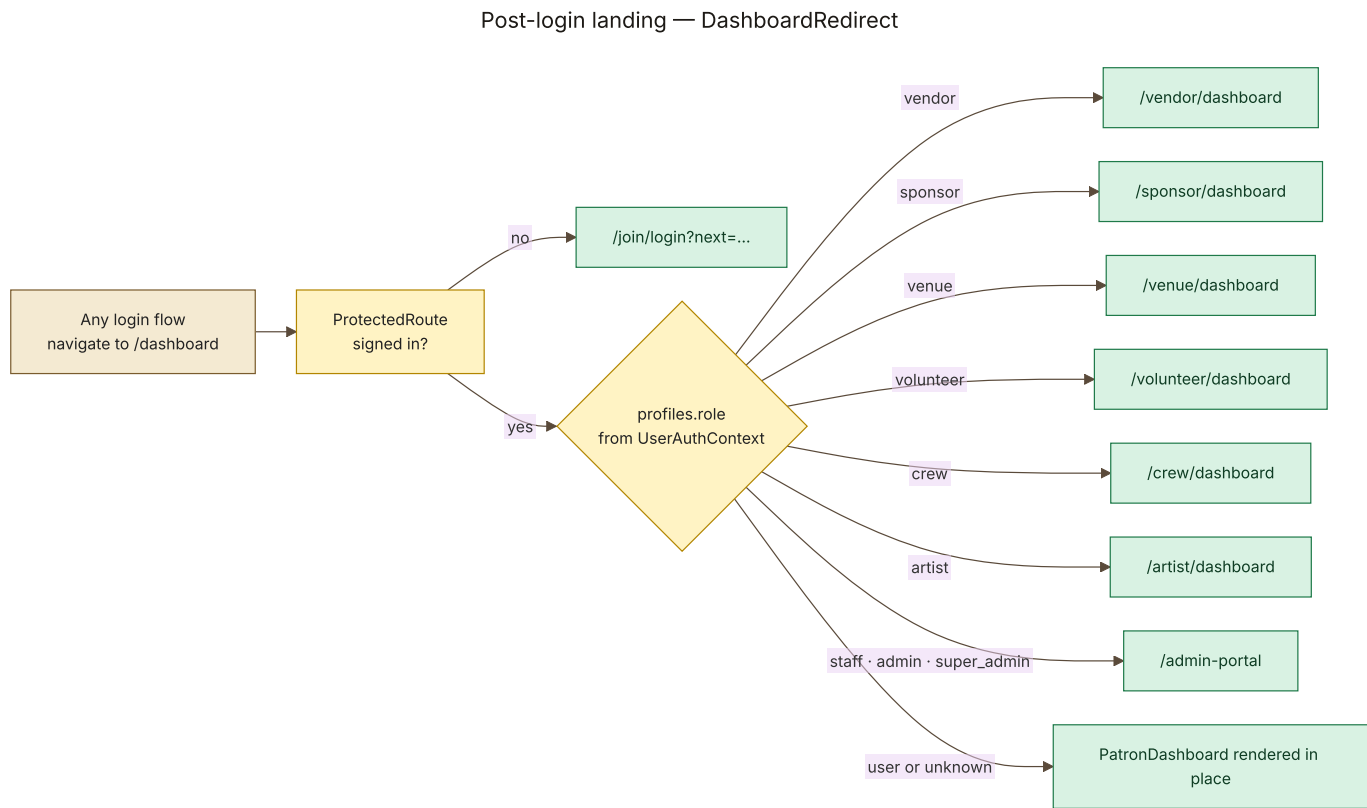


Diagram — Post-login landing — DashboardRedirect

Per-role architecture

Each role follows the same chain: **role** → **login** → **authentication** → **profile** → **role resolution** → **permissions** → **allowed routes** → **allowed actions** → **database / RLS enforcement**. The diagrams below show the concrete values for each role.

Visitor (anonymous)

Visitor — no account

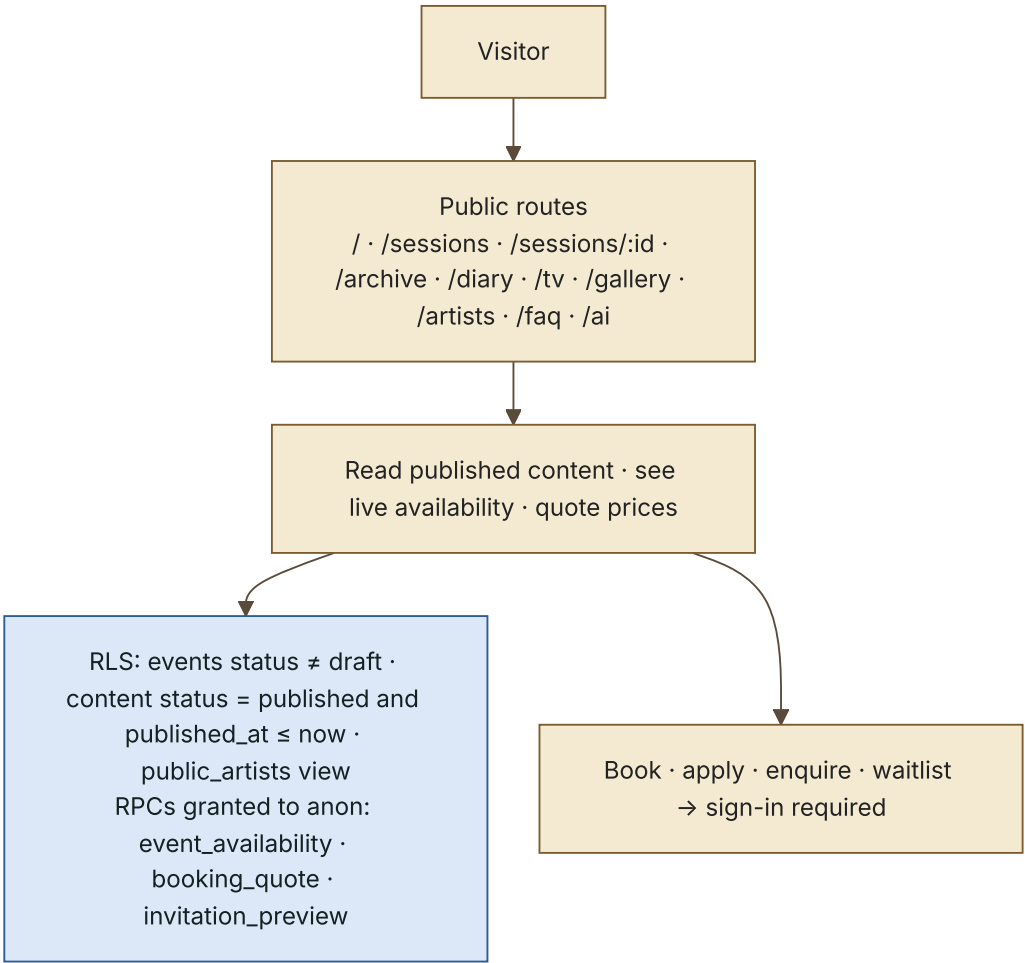


Diagram — Visitor — no account

Customer / Patron (user)

Item	Value
Login	Login modal (<code>UserLoginModal</code> → <code>EmailOtpAuth</code>), <code>/join/login</code> , <code>/join</code>
Dashboard	<code>/dashboard</code> → <code>PatronDashboard</code> (tabs: overview, passport, bookings, waitlist, settings, help); <code>/profile</code>
Main actions	Book and pay (<code>/sessions/:id</code>), see tickets and the booking QR, join or leave the waitlist, apply for partner roles, enquire, request an agent, set a password
DB restrictions	<code>bookings / tickets / checkins</code> : self read only. No insert policy on bookings : bookings are created only by <code>create_pending_booking</code> (service role, via Edge Function). <code>waitlist</code> : self read; joins through <code>join_waitlist()</code>
Security	Cannot change own <code>role / is_active</code> (trigger) or own <code>email / passport_id / member_since</code> (<code>guard_profile_identity</code> , 0035)

Customer — role user

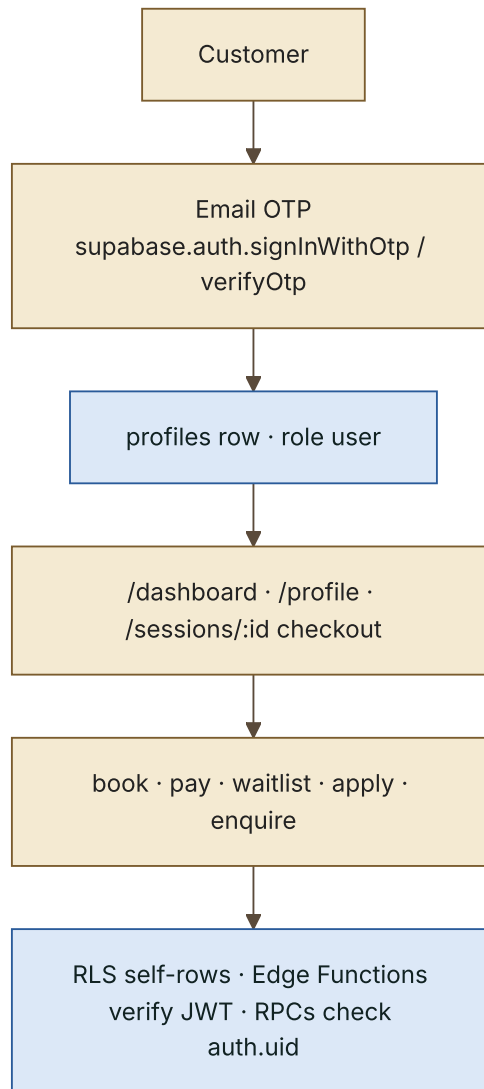


Diagram — Customer — role user

Artist (artist)

Item	Value
Login	/artist/login (EmailOtpAuth)
Before approval	/artist/apply (8-step application, any signed-in person), /artist/application (status)
Dashboard	/artist/dashboard inside ArtistPortalShell: sessions, calendar, requests, availability, media, messages, notifications, profile, documents, settings
Main actions	Edit profile and avatar, set availability (set_artist_availability), accept or decline booking requests (respond_to_booking_request), upload media (artist-media) and documents (artist-documents), message the team, set notification preferences
DB restrictions	artists: self read/update own row (status changes guarded by prevent_artist_status_self_escalation); assignment_requests: read own, non-draft only; artist_availability: self + team only (0033 removed public read); storage folder must be the caller's own artists.id

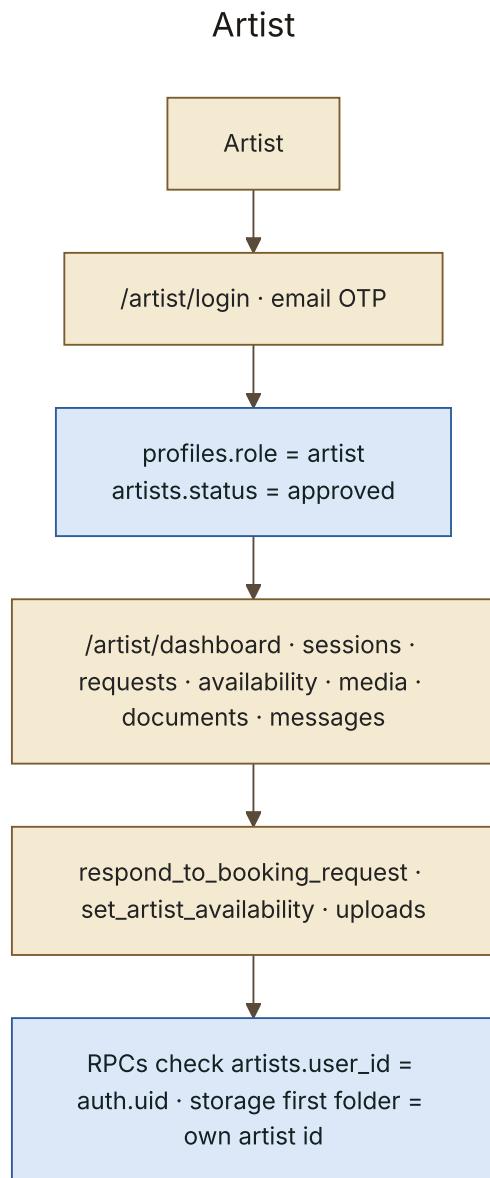


Diagram — Artist

Sponsor / Vendor / Venue host (**sponsor** , **vendor** , **venue**)

Item	Value
Apply	/apply/sponsors , /apply/vendors , /apply/venue-host (sign-in required) → collaborations row (type sponsor / vendor / venue_host)
Login	/join/login (approval email links here)
Dashboard	/sponsor/dashboard , /vendor/dashboard , /venue/dashboard . Pending applicants see their application status; approved accounts get PartnerPortal tabs: overview, events, requirements, messages, documents, sponsor brand assets, payments (invoices), announcements, notifications
Main actions	Respond to event assignments, submit requirements (submit_requirement), upload documents (event-documents) and sponsor assets (sponsor-assets), message the team (start_partner_conversation , send_message), view invoices
DB restrictions	Event access through event_member_kind() (only events they are assigned to); sponsor-assets uploads only by an active sponsor into their own auth.uid() folder

Partners — sponsor · vendor · venue

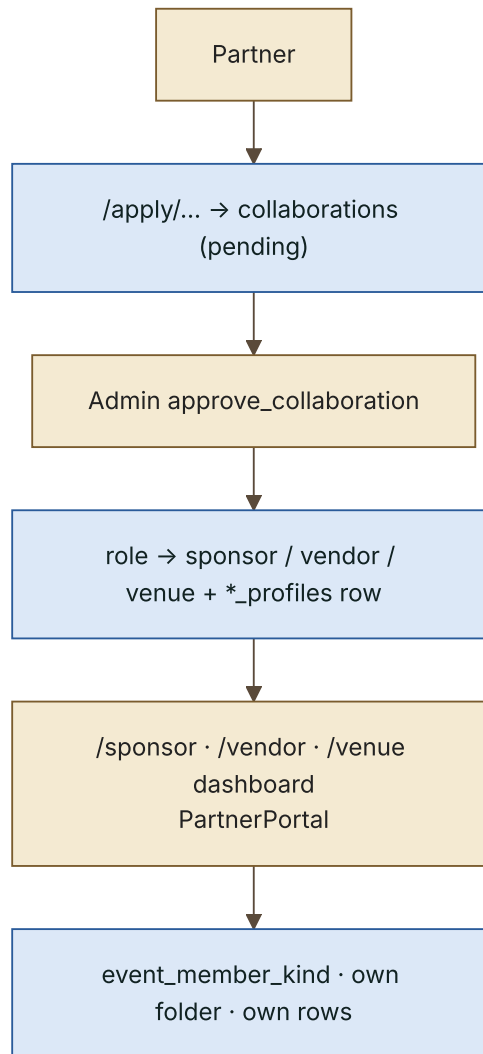


Diagram — Partners — sponsor · vendor · venue

Crew and Volunteer (crew, volunteer)

Item	Value
Apply	/crew/apply (category crew), /volunteer/apply (category volunteer; may name a session) → crew_applications
Dashboard	/crew/dashboard (custom CrewDashboard : assignments, tasks, schedule, applications, profile, messages); /volunteer/dashboard (PartnerPortal kind volunteer: tasks, check-in access, announcements, notifications)
Main actions	Crew: see assignments and tasks. Volunteer: request check-in access (request_checkin_access), use /check-in while a grant is live
DB restrictions	approve_crew_application makes a member a Volunteer and, if a session was named, adds a confirmed event_assignments row. Console accounts keep their console role. A volunteer's check-in right is temporary_access (30 min – 12 h windows)

Volunteer — time-boxed check-in access

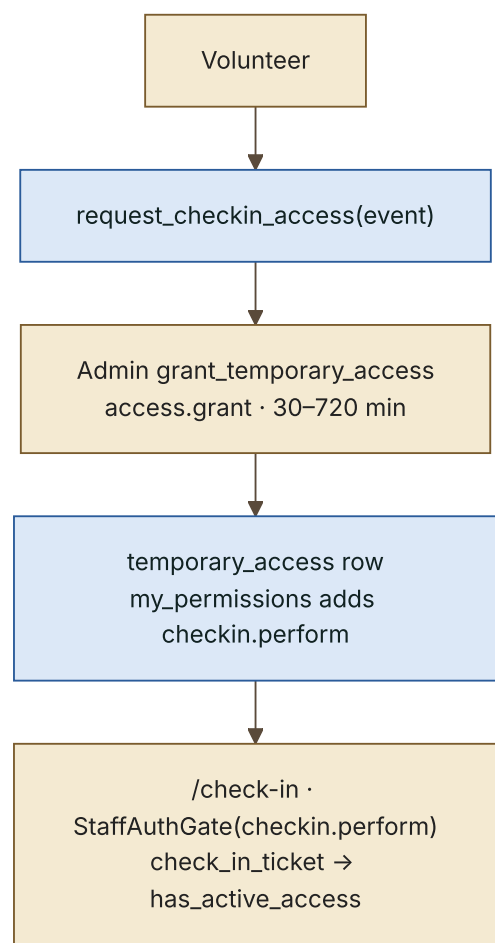


Diagram — Volunteer — time-boxed check-in access

Staff (`staff`)

Item	Value
Login	<code>/admin-portal</code> (<code>AdminLoginPanel</code> : email OTP, <code>allowSignup=false</code> ; password fallback). Accounts are created by invitation
Dashboard	<code>/admin-portal</code> (<code>staff_dashboard</code>), My Events, Event Info, Attendees (assigned), Check-in History, Event Tasks, Announcements, Notifications; <code>/check-in</code>
DB restrictions	Event-scoped: <code>is_assigned_to_event()</code> , <code>can_view_event_attendees()</code> , <code>can_checkin_event()</code> . 0017 removed staff's global access (<code>is_staff_or_admin()</code> now means admin-level only)

Admin / Manager (`admin`)

All of Staff's operational areas for **every** event, plus: applications review, events and line-ups, bookings, payments, waitlist, invoices, people (artists and partners), reviews, volunteers and temporary access, team, staff invitations, content CMS, messages, reports, operations (support inbox, enquiries, email delivery, portal preview). **Not:** `users.manage`, `roles.manage`, `audit.view`, `settings.manage`, `ai.use`.

Super Admin (`super_admin`)

Everything above plus Users & Roles (`admin_set_user_role`, `admin_set_user_active`), Roles & Permissions (`set_role_permission`), Audit Logs, System Settings (`update_system_setting`), Tangy AI page, custom notifications (`send_custom_notification`), private artist conversations (`start_private_artist_conversation`). Only a Super Admin grants or removes Super Admin, and the last active Super Admin cannot be demoted (`admin_set_user_role`, 0034).

Console roles — Staff · Admin · Super Admin

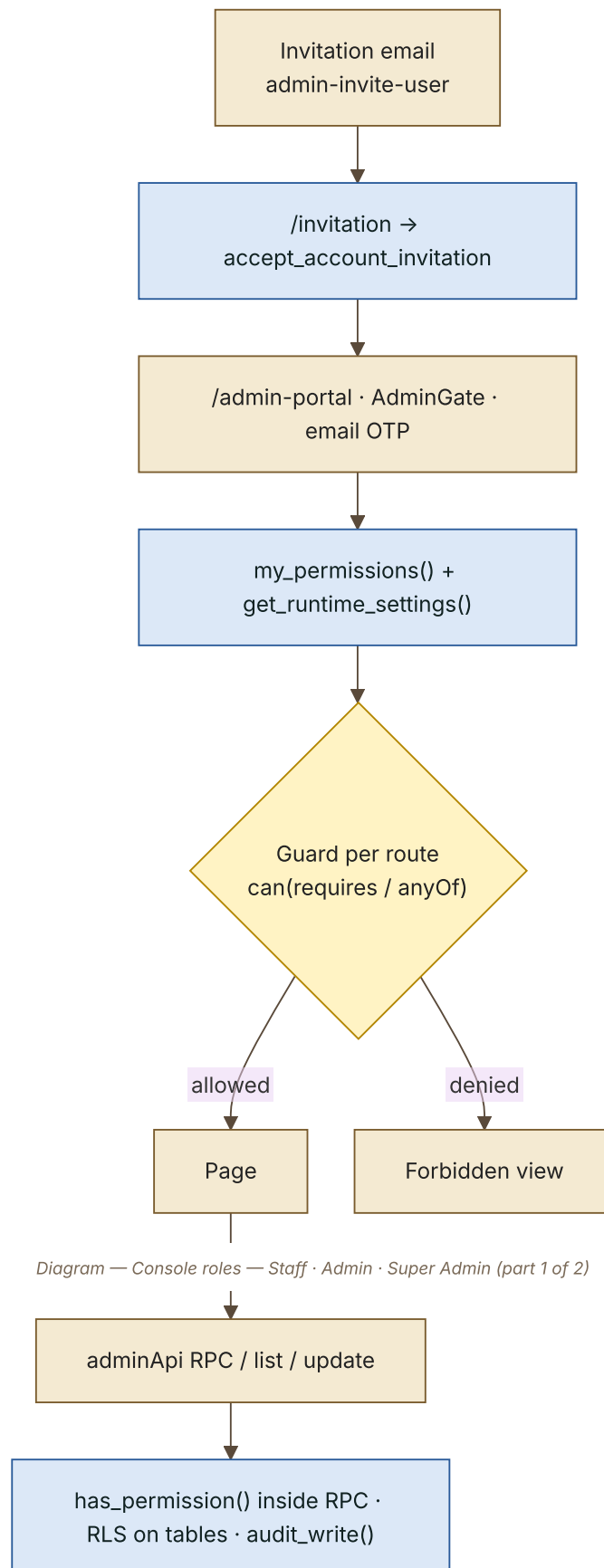


Diagram — Console roles — Staff · Admin · Super Admin (part 1 of 2)

Diagram — Console roles — Staff · Admin · Super Admin (part 2 of 2)

Private-session enquirer

Not a role. A signed-in `user` who submitted `private_enquiries` sees them at `/private/dashboard` (`RoleApplicationDashboard`).

03 — Complete Website Map

Routes come from `src/App.jsx` (134 `<Route>` elements) and `src/admin/AdminApp.jsx` (54). `docs/ROUTES.md` is the repository's own generated inventory (`scripts/route-inventory.mjs`). Every page except the homepage is lazy-loaded (`React.lazy` + `Suspense`).

Top-level hierarchy

Tangy World sitemap — route groups

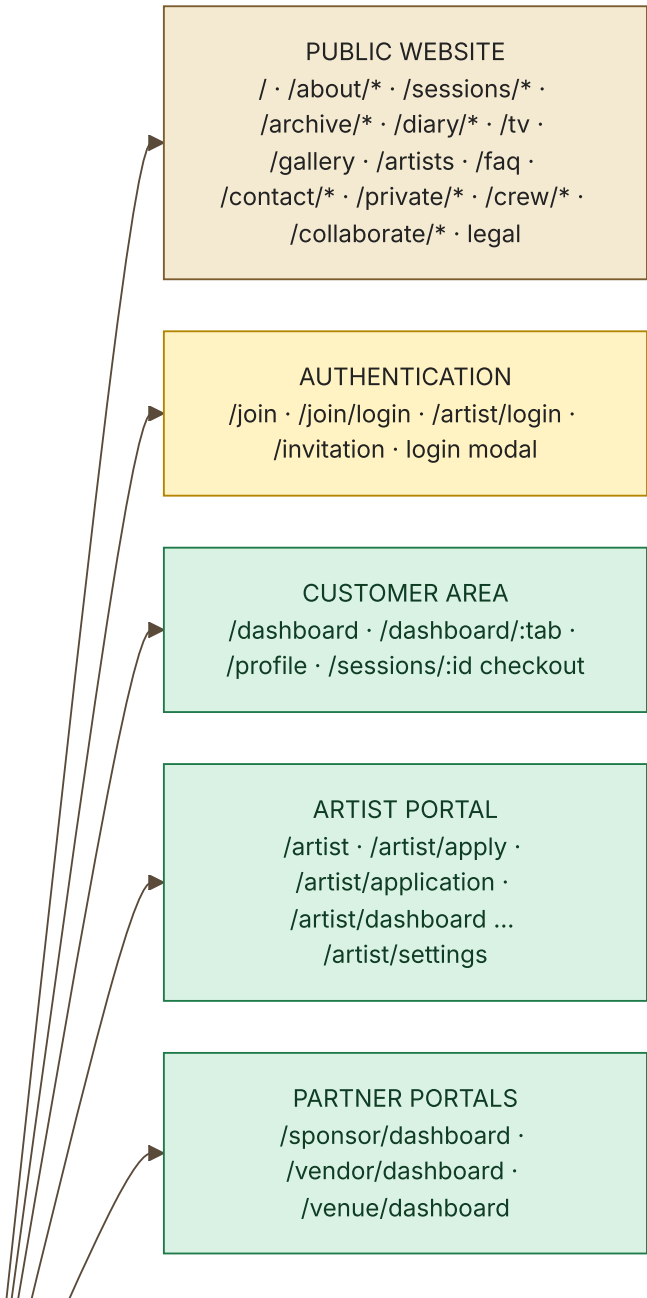


Diagram — Tangy World sitemap — route groups (part 1 of 2)

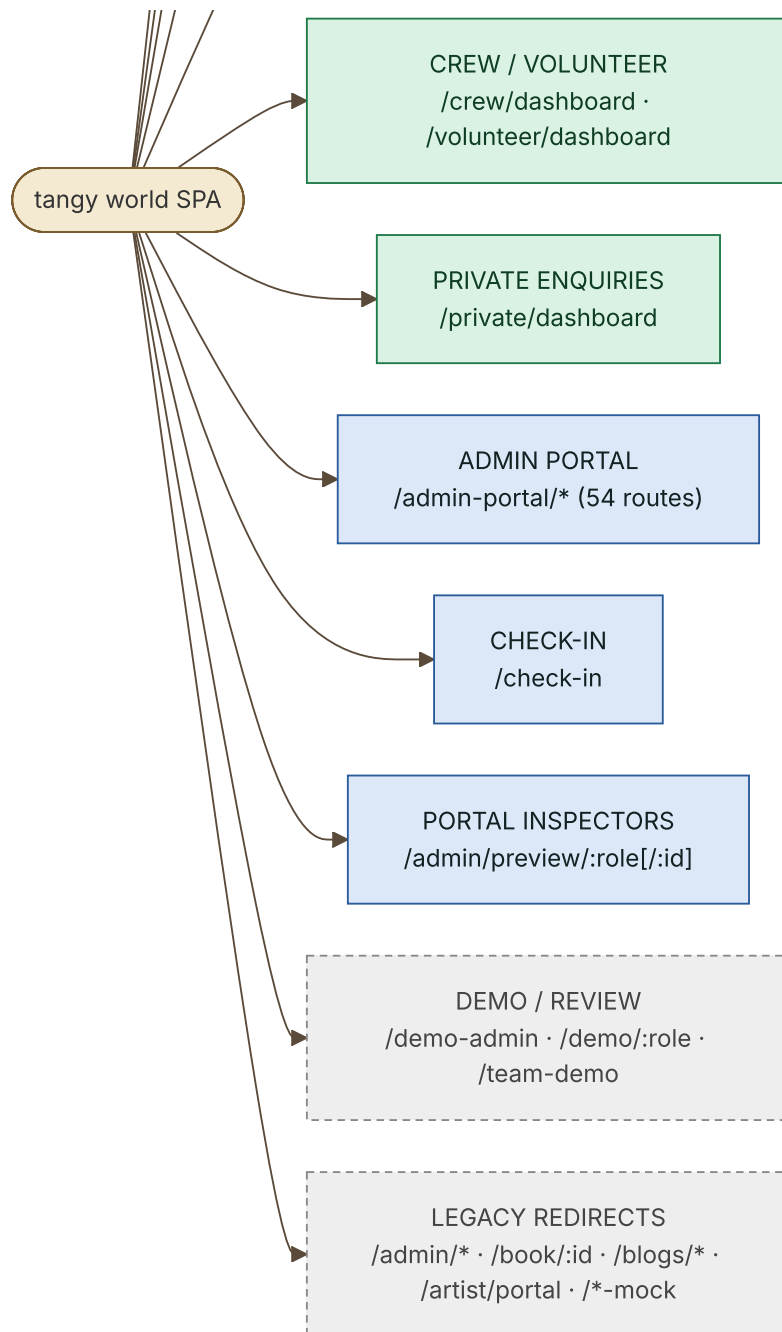


Diagram — Tangy World sitemap — route groups (part 2 of 2)

PUBLIC WEBSITE

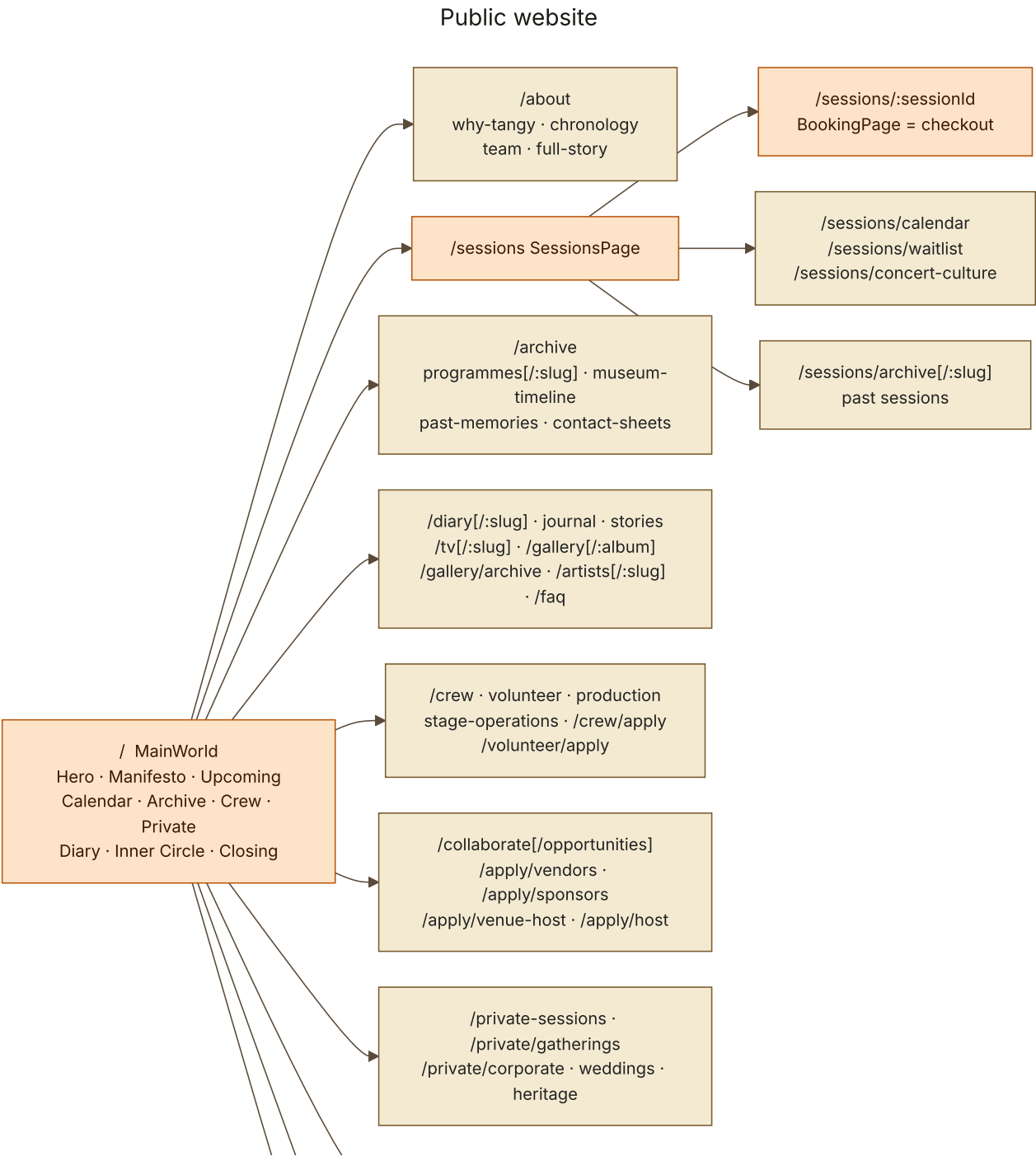


Diagram — Public website (part 1 of 2)

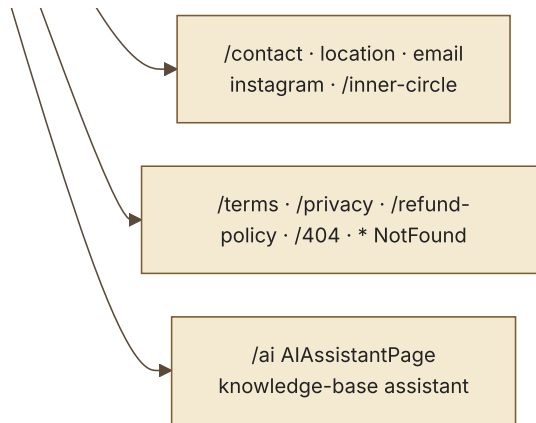


Diagram — Public website (part 2 of 2)

URL	Page	Purpose	Access	Data read	Actions
/	MainWorld	Scroll experience, entry to everything	Public	events (useEvents), published content, announcements	Navigate; open login modal
/sessions	SessionsPage	Upcoming sessions	Public	events (status ≠ draft)	Open a session
/sessions/:sessionId	BookingPage	Session details and checkout	Public read; booking needs sign-in	events, event_artists, public_artists, booking_quote(), event_availability(), my_waitlist(), Realtime event_availability_signal	Pay (Edge Functions), join/leave waitlist
/sessions/calendar	SessionCalendarPage	Calendar of sessions	Public	events (useEvents)	—
/sessions/waitlist	WaitlistPage	Sold-out sessions with waitlists	Public; joining needs sign-in	event_availability(), my_waitlist()	join_waitlist / leave_waitlist
/sessions/archive[:slug]	PreviousSessionsPage, PastSessionPage	Past sessions with line-up, gallery, diary, TV	Public	archiveService reads events, event_artists, gallery_albums, diary_posts, tv_videos, programmes	—
/diary, /tv, /gallery, /artists (+ detail pages)	content pages	CMS content	Public	contentService (published only) + signed URLs for content-media	—
/apply/*, /crew/apply, /volunteer/apply, /contact, /private-sessions	forms	Applications and enquiries	Sign-in required (RequireAuthToApply, 0025)	own previous submissions	insert collaborations / crew_applications / contact_enquiries / private_enquiries

AUTHENTICATION

URL	Page	Purpose	Access	Actions
(modal)	UserLoginModal	Sign in without leaving the page (e.g. "TO BOOK THIS SESSION")	Public	Email OTP
/join	JoinPage	"How are you joining Tangy?" role cards (src/config/joinRoles.js)	Public	Guest signs up by OTP; other cards link to their application
/join/login	JoinLoginPage	Universal sign-in, returns to ?next=	Public	Email OTP
/artist/login	artist LoginPage	Artist sign-in	Public	Email OTP

URL	Page	Purpose	Access	Actions
/invitation	InvitationPage	Accept a console invitation	Signed-in owner of the invited email	invitation_preview, accept_account_invitation
/team-demo	TeamDemoLoginPage	One-click demo roles	DEMO / LOCAL ONLY: review builds only (__TANGY_REVIEW_MODE__)	password sign-in to demo accounts

CUSTOMER / USER AREA

URL	Page	Purpose	Access	Data read	Actions
/dashboard[:tab]	DashboardRedirect → PatronDashboard	Bookings, tickets, booking QR, passport, waitlist, settings	ProtectedRoute	bookings + tickets + events (own), waitlist, my_waitlist(), profiles	Set password (auth.updateUser), leave waitlist
/profile	ProfilePage (passport)	Profile + booking history	ProtectedRoute	profiles, bookingService.getMyBookings	Edit profile fields (identity fields guarded)

Artist routes

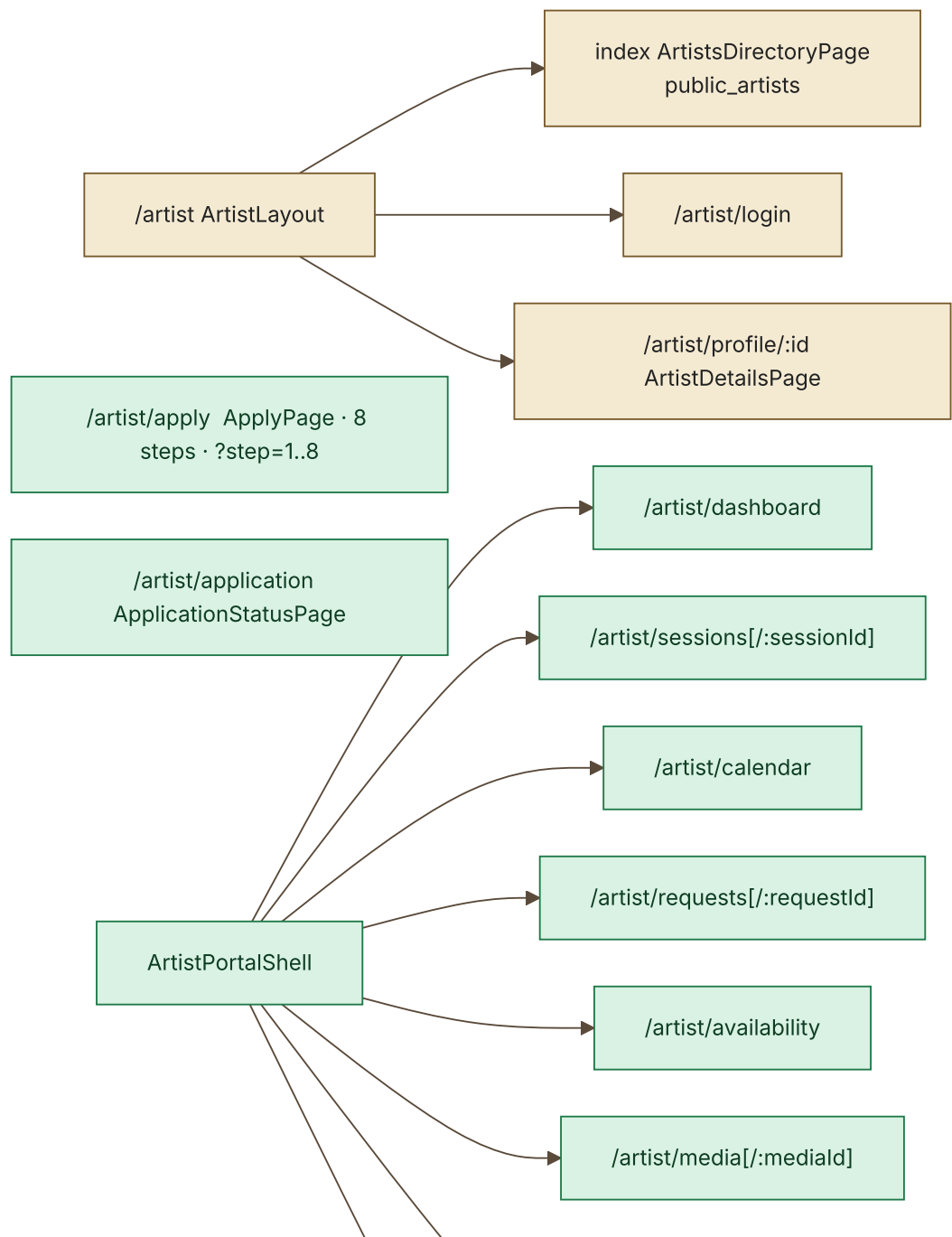


Diagram — Artist routes (part 1 of 2)

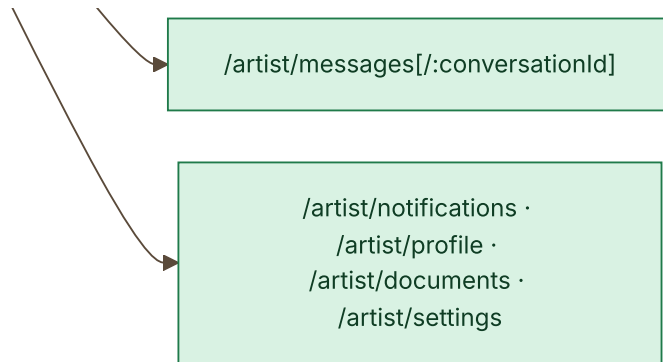


Diagram — Artist routes (part 2 of 2)

URL	Purpose	Data read	Actions
/artist/apply	Multi-step application (any signed-in person)	artist_applications (own)	autosave, update, upload video to artist-media/applications/<uid>/, submit_artist_application, withdraw_artist_application
/artist/calendar	Own calendar (sessions, requests, availability)	artist_calendar() (shared useArtistCalendar)	—
/artist/dashboard	Overview	my_booking_requests, artist_profile_completion, artist_availability_summary	—
/artist/sessions[:id]	Confirmed sessions	event_artists, events, event_artist_details	—
/artist/requests[:id]	Booking requests	my_booking_requests	mark_artist_request_viewed, respond_to_booking_request
/artist/availability	Day-by-day availability	artist_availability, artist_availability_summary	set_artist_availability
/artist/media	Media library	artist_media + signed URLs	upload to artist-media/<artist id>/
/artist/documents	Rider, EPK, tax, identity	artist_documents	upload to artist-documents/<artist id>/
/artist/messages	Threads with the team	my_conversations, conversation_messages	send_message, start_partner_conversation
/artist/settings	Notification preferences	my_notification_preferences	set_notification_preference

SPONSOR · VENDOR · VENUE · VOLUNTEER · CREW · PRIVATE

URL	Component	Access	Data read	Actions
/sponsor/dashboard[:tab[:sub]]	SponsorDashboard → PartnerPortal (kind sponsor)	ProtectedRoute; full portal only when an application is approved	collaborations, sponsor_profiles, sponsor_deliverables, sponsor_assets, my_portal_events, partner_invoices	upload sponsor-assets, submit_requirement, messages
/vendor/dashboard...	VendorDashboard → PartnerPortal (vendor)	same	collaborations, vendor_profiles, event_assignments, invoices	respond to assignments, requirements, documents
/venue/dashboard...	VenueDashboard → PartnerPortal (venue)	same	collaborations, venue_profiles, events	requirements, documents, messages

URL	Component	Access	Data read	Actions
/volunteer/dashboard[:tab]	VolunteerDashboard → PartnerPortal (volunteer)	same	crew_applications, volunteer_profiles, event_assignments, my_checkin_access	request_checkin_access
/crew/dashboard[:tab]	CrewDashboard	ProtectedRoute	crew_applications, crew_profiles, event_assignments, event_tasks	view assignments and tasks
/private/dashboard[:tab]	PrivateDashboard (RoleApplicationDashboard)	ProtectedRoute	own private_enquiries	messages

Route-level role checks are intentionally absent here. A *pending* applicant (still `user`) must reach the page to see their status. RLS on the applicant's own rows is the boundary (`App.jsx` comment above these routes).

ADMIN PORTAL (/admin-portal/*)

Admin console navigation (generated from permissions)

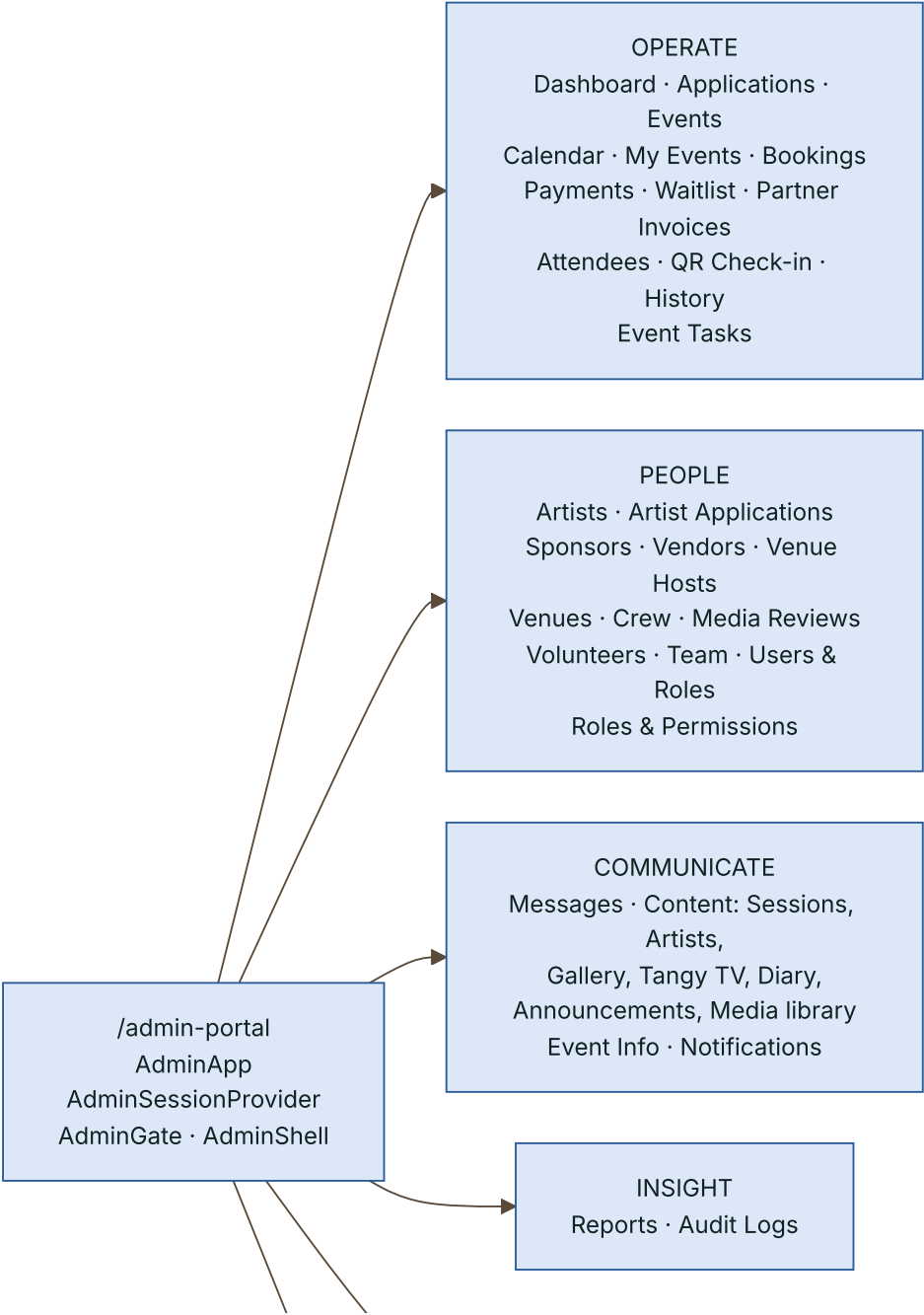


Diagram — Admin console navigation (generated from permissions) (part 1 of 2)

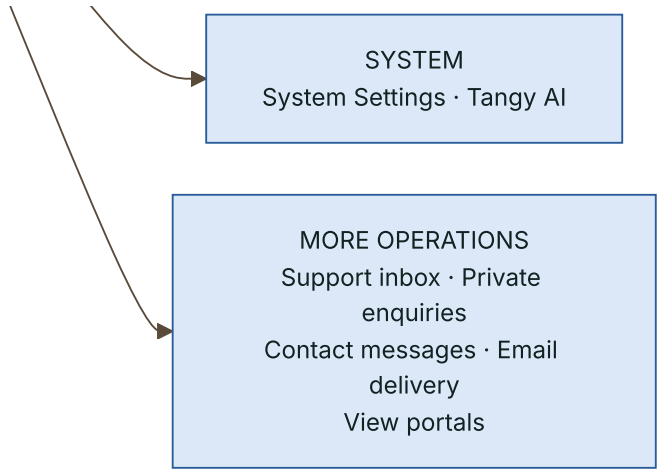


Diagram — Admin console navigation (generated from permissions) (part 2 of 2)

The route-by-route permission table for the console is in [06-admin-architecture.md](#).

CHECK-IN

URL	Component	Access	Data	Actions
/check-in	TangyWorldCheckInPage inside StaffAuthGate (permission checkin.perform)	Staff/Admin/Super Admin, or a volunteer with live temporary_access	my_checkin_events , event_checkin_stats , get_checkin_history , attendee_tickets	camera scan (html5- qrcode) or manual code → check_in_ticket (preview then commit)
/admin/preview/:role[:id] , /admin/preview/artist/:id	AdminEntitySelector , AdminPortalPreview , AdminArtistPreview	Uses admin data access (RLS)	profiles and partner tables	read-only portal inspection

DEMO / REVIEW / LEGACY

URL	Status
/demo-admin , /demo-admin/control-room , /demo/:role , /demo/artist	DEMO / LOCAL ONLY: VITE_DEMO_ADMIN_ENABLED is compiled to false in builds unless TANGY_ALLOW_DEMO_BUILD=1 . In vite dev with dev tools the first two redirect to /admin-portal
/team-demo	DEMO / LOCAL ONLY: review builds only
/admin/* → /admin-portal/* , /book/:id → /sessions/:id , /blogs[:slug] → /diary[:slug] , /artist/portal , /artist-mock/portal , /admin-mock , /crew-mock/dashboard , /artist/dashboard/:tab	LEGACY redirects

04 — Frontend Architecture

Stack (`package.json`): React 19, Vite 8, React Router 7, Tailwind 4 (`@tailwindcss/vite`), `@supabase/supabase-js` 2, GSAP 3 + Lenis + Framer Motion (animation), FullCalendar 6 (calendars), `qrcode` (QR images), `html5-qrcode` (camera scanner), `lucide-react` (icons). Lint: Oxlint. There is no TypeScript and no state library (React context + hooks only).

Boot sequence and provider tree

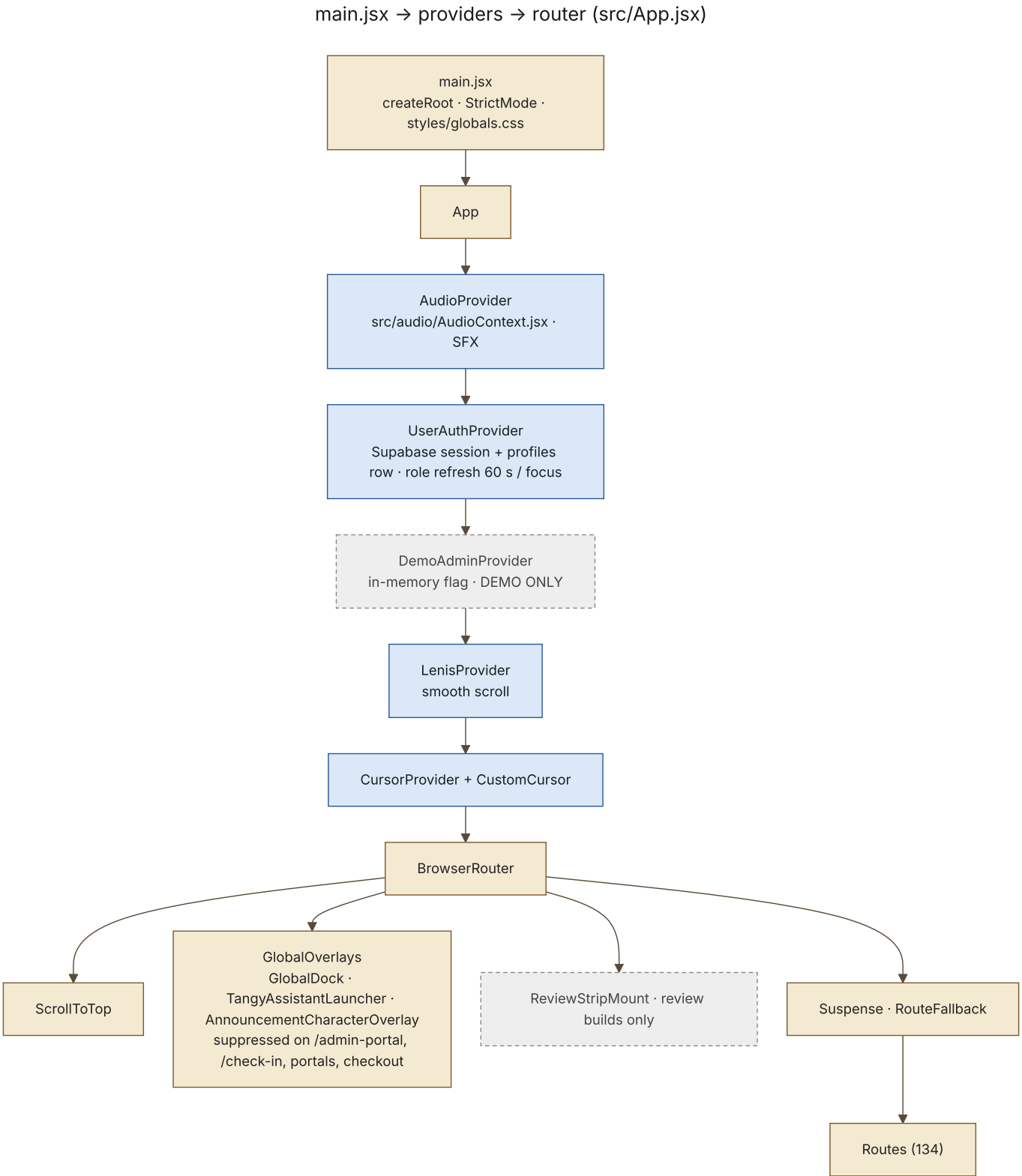


Diagram — main.jsx → providers → router (src/App.jsx)

Route groups and their shells

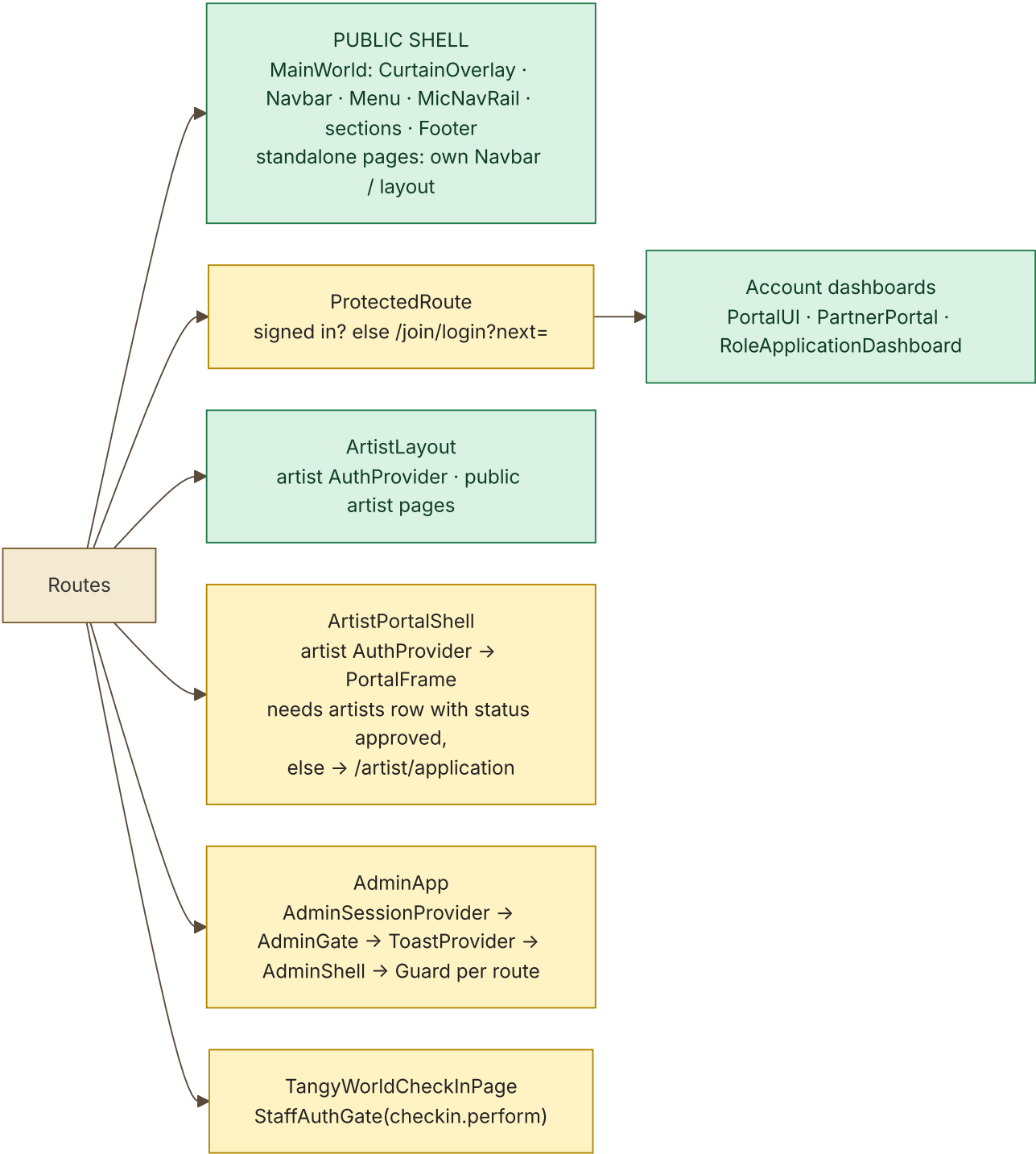


Diagram — Route groups and their shells

Shell	File	What it adds
Public	App.jsx MainWorld, components/layout/*, components/sections/*	Museum modals, curtain intro, navbar, scroll sections
Account	pages/dashboards/*, portal/PartnerPortal.jsx, pages/dashboards/portal/PortalUI.jsx	Tabs, application status, partner sections

Shell	File	What it adds
Artist public	artist/layouts/ArtistLayout.jsx	Artist navbar, artist <code>AuthProvider</code>
Artist portal	artist/portal/ArtistPortalShell.jsx, portalNav.js, kit.jsx	Sidebar nav, request badge, approval gate
Admin	admin/AdminShell.jsx, AdminGate.jsx, AdminSession.jsx, ui.jsx, rbac.js	Permission-generated nav, global search (<code>admin_search</code>), notification bell, idle sign-out (<code>auth.admin_idle_timeout_minutes</code> , default 60)
Check-in	admin/TangyWorldCheckInPage.jsx, StaffAuthGate.jsx	Same sign-in + permission model as the console

Contexts and hooks

Context / hook	File	Purpose
UserAuthContext	src/context/UserAuthContext.jsx	<code>getSession</code> + <code>onAuthStateChange</code> → loads <code>profiles</code> row; exposes <code>user</code> , <code>isLoggedIn</code> , <code>signIn</code> (password), <code>signUp</code> , <code>logout</code> , login modal state; re-reads <code>role</code> , <code>is_active</code> every 60 s and on focus
artist AuthContext	src/artist/context/AuthContext.jsx	Maps the caller's <code>artists</code> row to the portal user; <code>null</code> when no artist row
AdminSessionContext	src/admin/AdminSession.jsx	<code>my_permissions()</code> + <code>get_runtime_settings()</code> , <code>can()</code> , dev mock identity (dev only)
DemoAdminContext	src/context/DemoAdminContext.jsx	DEMO ONLY in-memory flag, grants nothing (no Supabase session)
AudioContext	src/audio/AudioContext.jsx	Sound effects (<code>playSFX</code>)
useEvents	src/hooks/useEvents.js	Public <code>events</code> list
useSessionDetail	src/hooks/useSessionDetail.js	Session page: event, line-up, <code>booking_quote</code> , <code>event_availability</code> , <code>my_waitlist</code> , Realtime subscription <code>availability-<eventId></code> on <code>event_availability_signal</code>
useContent	src/hooks/useContent.js	CMS reads
useAnnouncementTrigger	src/hooks/useAnnouncementTrigger.js	Picks a public announcement for the character overlay
usePageMeta, useReducedMotion, useFocusTrap, useGSAPContext, useReveal, useCursor	src/hooks/	Page titles, a11y, animation
useAdminList, useAsync	src/admin/useAdminList.js, src/admin/hooks.js	Paged admin lists

Service / API layer

Frontend dependency diagram — who talks to Supabase

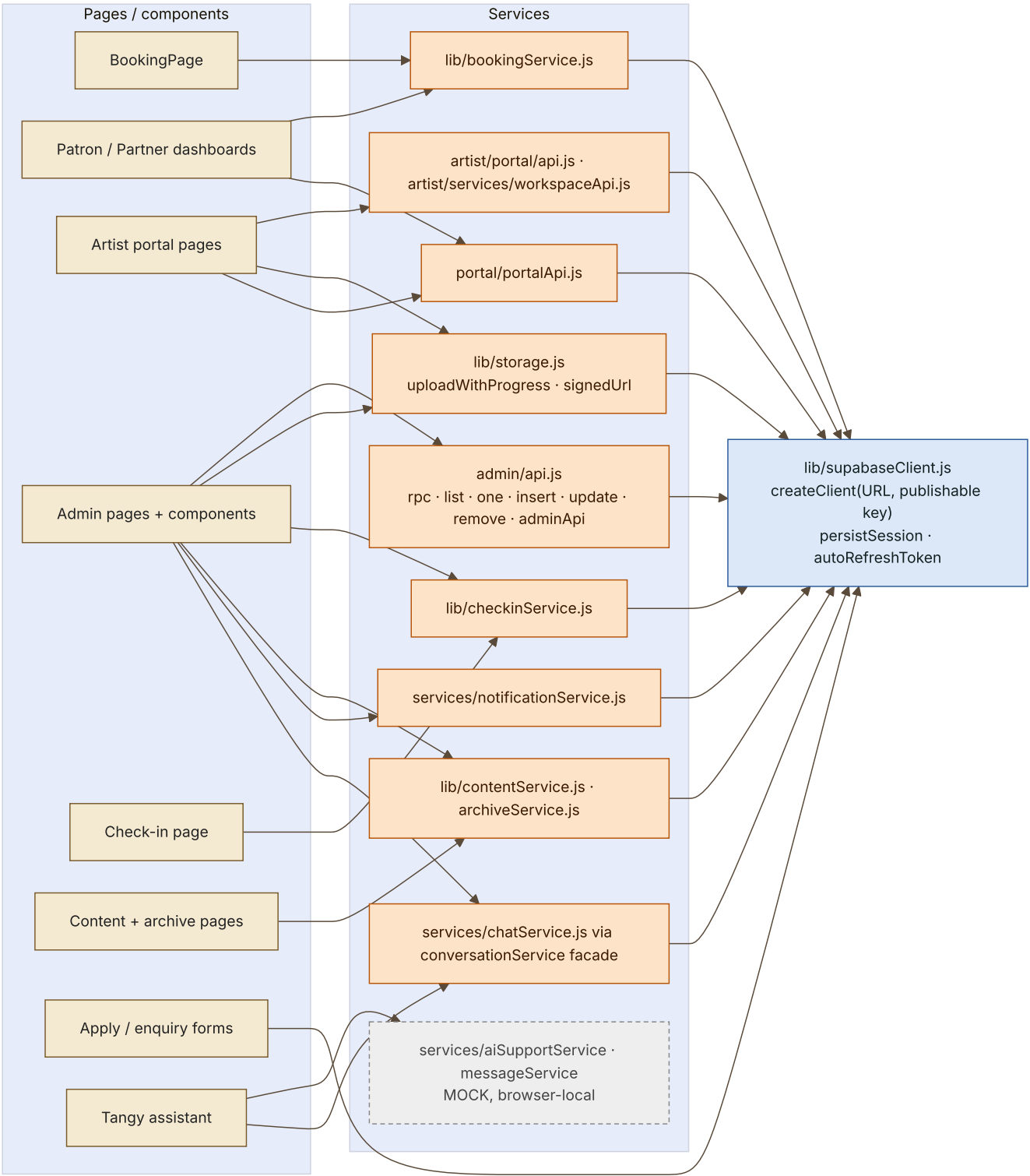


Diagram — Frontend dependency diagram — who talks to Supabase

Module	Talks to
src/lib/supabaseClient.js	Creates the only client. Returns <code>null</code> when <code>VITE_SUPABASE_URL</code> / public key are missing, so the app shows empty or error states instead of crashing
src/lib/bookingService.js	Edge Functions <code>razorpay-create-order</code> , <code>razorpay-verify-payment</code> , <code>send-ticket-email</code> ; table <code>bookings</code> (own)

Module	Talks to
src/lib/checkinService.js	RPCs <code>check_in_ticket</code> , <code>my_checkin_events</code> , <code>event_checkin_stats</code> , <code>get_checkin_history</code> , <code>booking_checkin_history</code> ; view <code>attendee_tickets</code>
src/lib/contentService.js	<code>tv_videos</code> , <code>diary_posts</code> , <code>gallery_albums</code> , <code>gallery_photos</code> , <code>public_artists</code> , <code>event_artists</code> ; storage <code>content-media</code> (signed URLs)
src/lib/archiveService.js	Past <code>events</code> , <code>event_artists</code> , <code>programmes</code> , <code>programme_events</code> , content tables
src/lib/storage.js	Storage REST with the user's JWT (progress events); signed URLs for private buckets
src/admin/api.js	Generic table helpers + typed <code>adminApi</code> RPC wrappers; maps errors to safe messages (<code>friendlyError</code>); invokes <code>admin-invite-user</code>
src/portal/portalApi.js	Partner/artist portal RPCs (events, conversations, notifications, requirements, check-in access) + Realtime inserts
src/artist/portal/api.js, src/artist/services/workspaceApi.js	Artist application, availability, booking requests, profile completion, preferences
src/services/chatService.js (+ conversationService facade)	Support conversations, messages, Realtime channels <code>conversations:inbox</code> , <code>messages:<id></code>
src/services/notificationService.js	<code>application_notifications</code> ; invokes <code>send-approval-email</code>

Mock services: dead code paths

`src/services/{bookingService,collaborationService,enquiryService,eventService,userService,waitlistService}.js` say "MOCK service — not connected to Supabase". They are only reachable behind `if (isMockAuth)`, and `src/config/auth.js` hard-codes `isMockAuth = false`. In the current build these branches never run (**LEGACY**). `eventService`, `waitlistService` and `assignmentRequestService/assignmentService` are not imported by any UI component. The exception is `messageService` + `aiSupportService`, which the public Tangy Assistant **does** use (browser-local transcript, **PARTIALLY IMPLEMENTED**, see 01).

Build-time switches (vite.config.js)

Build-time guards in vite.config.js

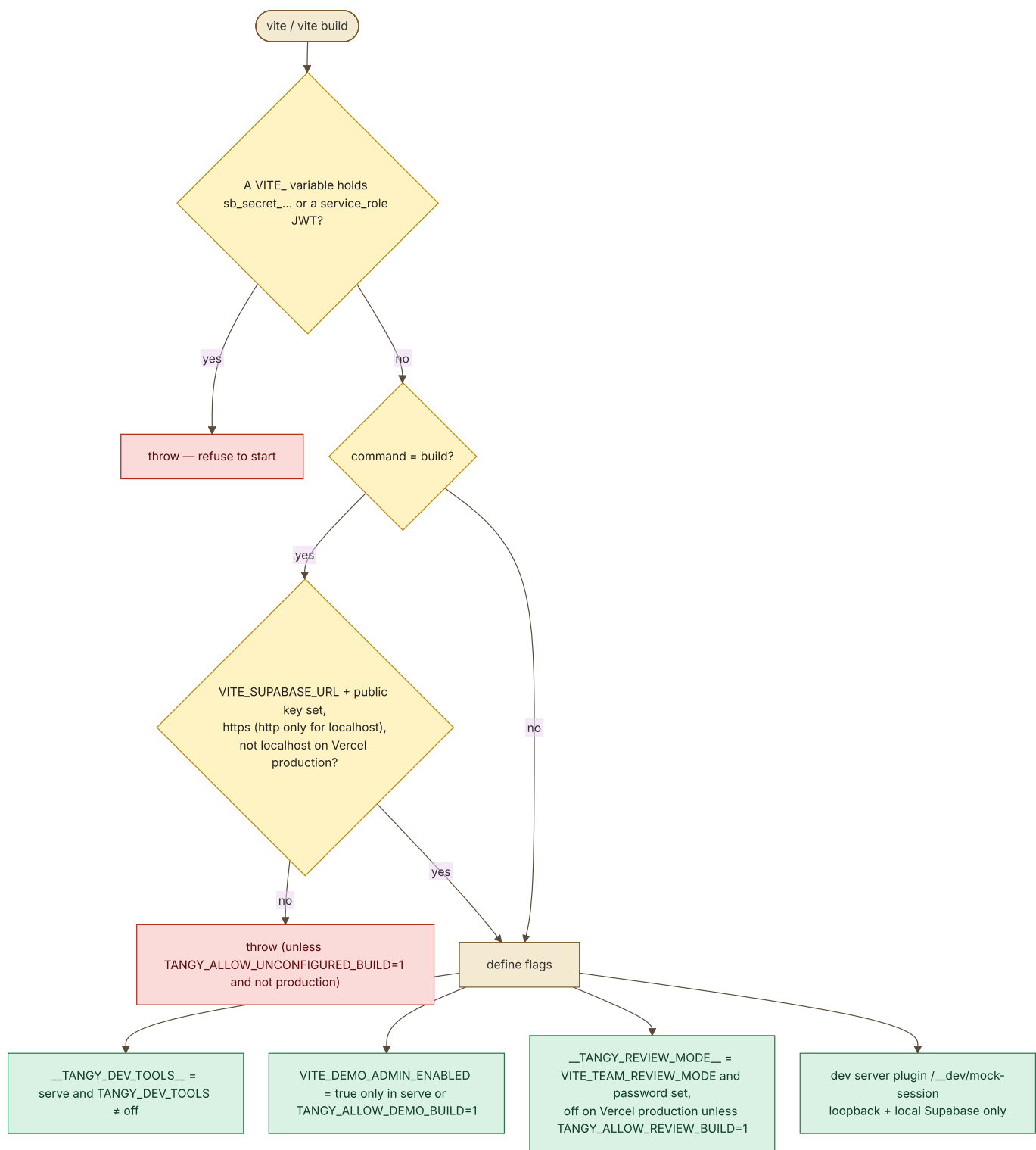


Diagram — Build-time guards in vite.config.js

05 — Authentication, Role and Permission Resolution

Supabase Auth is the only identity provider. Tangy never generates, stores or checks OTP codes itself. `EmailOtpAuth` calls `supabase.auth.signInWithOtp()` and `verifyOtp()`.

Implemented auth flows

Flow	Status	Where
Email OTP sign-up + sign-in (customers, partners, artists)	IMPLEMENTED	<code>src/components/auth/EmailOtpAuth.jsx</code> (<code>shouldCreateUser: allowSignup</code> , <code>verifyOtp({type:'email'})</code>), used by <code>UserLoginModal</code> , <code>/join</code> , <code>/join/login</code> , <code>/artist/login</code> , <code>/artist/apply</code> , <code>RequireAuthToApply</code> , <code>/invitation</code>
Console sign-in	IMPLEMENTED	<code>AdminLoginPanel</code> (<code>AdminGate.jsx</code>): OTP with <code>allowSignup=false</code> (existing accounts only); password fallback (<code>signInWithPassword</code>)
Set / change password	IMPLEMENTED	<code>/profile</code> and Patron settings: <code>supabase.auth.updateUser({ password })</code>
Password reset by email (<code>resetPasswordForEmail</code>)	NOT IMPLEMENTED	no call in <code>src/</code>
Social / OAuth sign-in	NOT IMPLEMENTED	—
Session persistence and refresh	IMPLEMENTED	<code>createClient(..., { auth: { persistSession: true, autoRefreshToken: true } })</code>
Role refresh while signed in	IMPLEMENTED	<code>UserAuthContext</code> re-reads <code>profiles.role</code> , <code>is_active</code> every 60 s, on window focus and on tab visibility
Console idle sign-out	IMPLEMENTED	<code>AdminShell</code> <code>useIdleTimeout</code> , setting <code>auth.admin_idle_timeout_minutes</code> (default 60; 0 disables)
Console sign-in audit	IMPLEMENTED	<code>AdminGate</code> → <code>log_auth_event('auth.login')</code> once per browser session
Invitation (console accounts)	IMPLEMENTED	<code>admin-invite-user</code> Edge Function + <code>create_account_invitation</code> / <code>invitation_preview</code> / <code>accept_account_invitation</code> (0030)
OTP email template contains <code>{{ .Token }}</code>	PRODUCTION CONFIGURATION / DOCUMENTED BUT NOT VERIFIED IN CODE	<code>supabase/README.md</code> §2b: dashboard setting; the repo says it "could not be verified against a live project"
Team review one-click login	DEMO / LOCAL ONLY	<code>/team-demo</code> , <code>signInWithPassword</code> with <code>VITE_TEAM_REVIEW_PASSWORD</code>
Dev role switcher	DEMO / LOCAL ONLY	<code>vite</code> dev server <code>/__dev/mock-session</code> (magic-link token for local accounts)

Complete authentication flowchart

Authentication → role → permission → enforcement

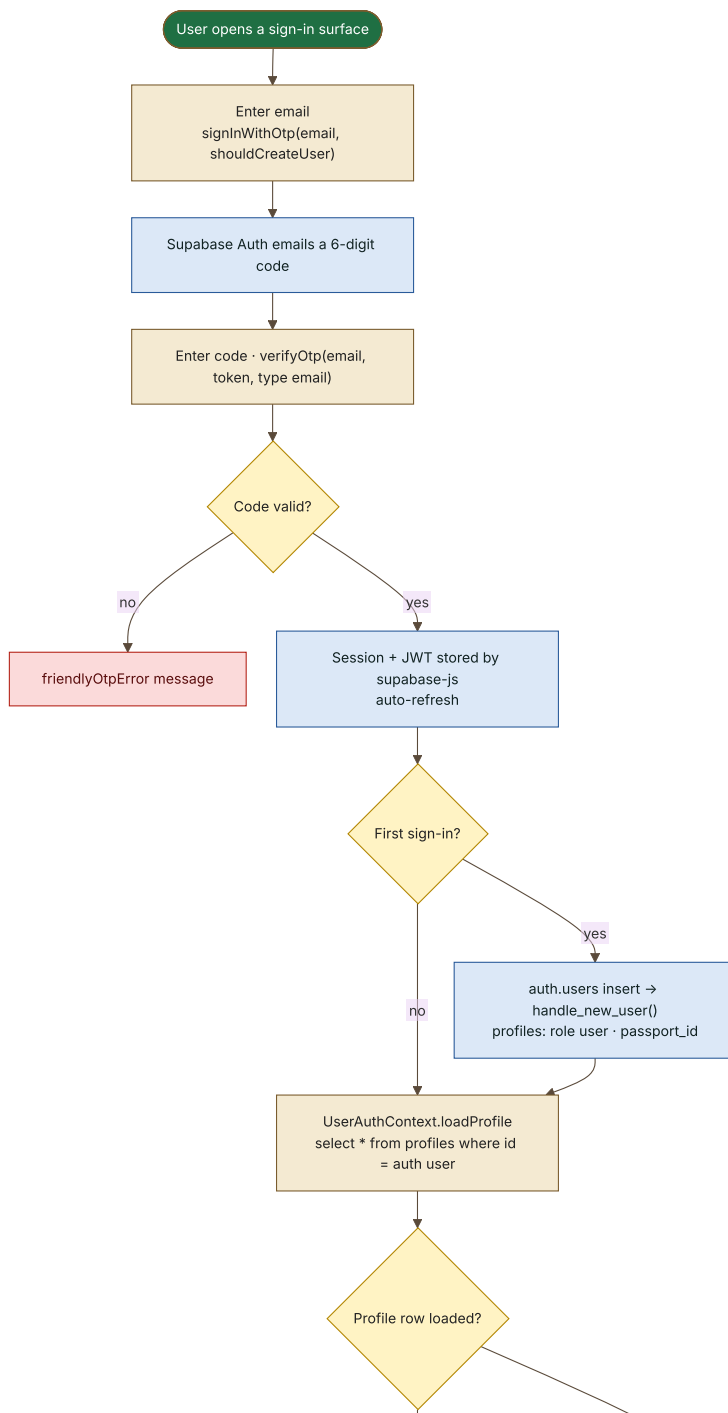


Diagram — Authentication → role → permission → enforcement (part 1 of 2)

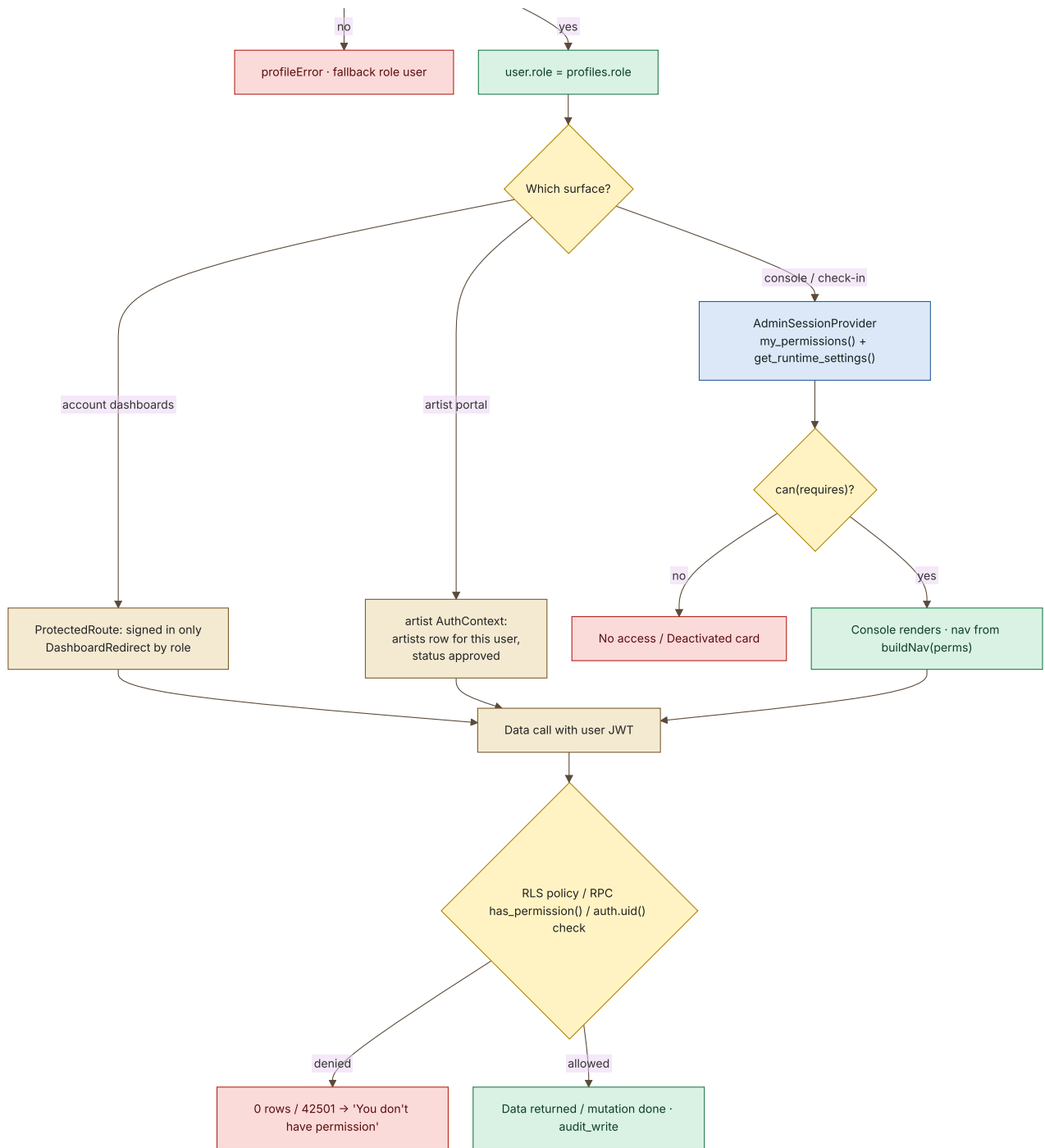


Diagram — Authentication → role → permission → enforcement (part 2 of 2)

Sign-in surfaces

Who signs in where

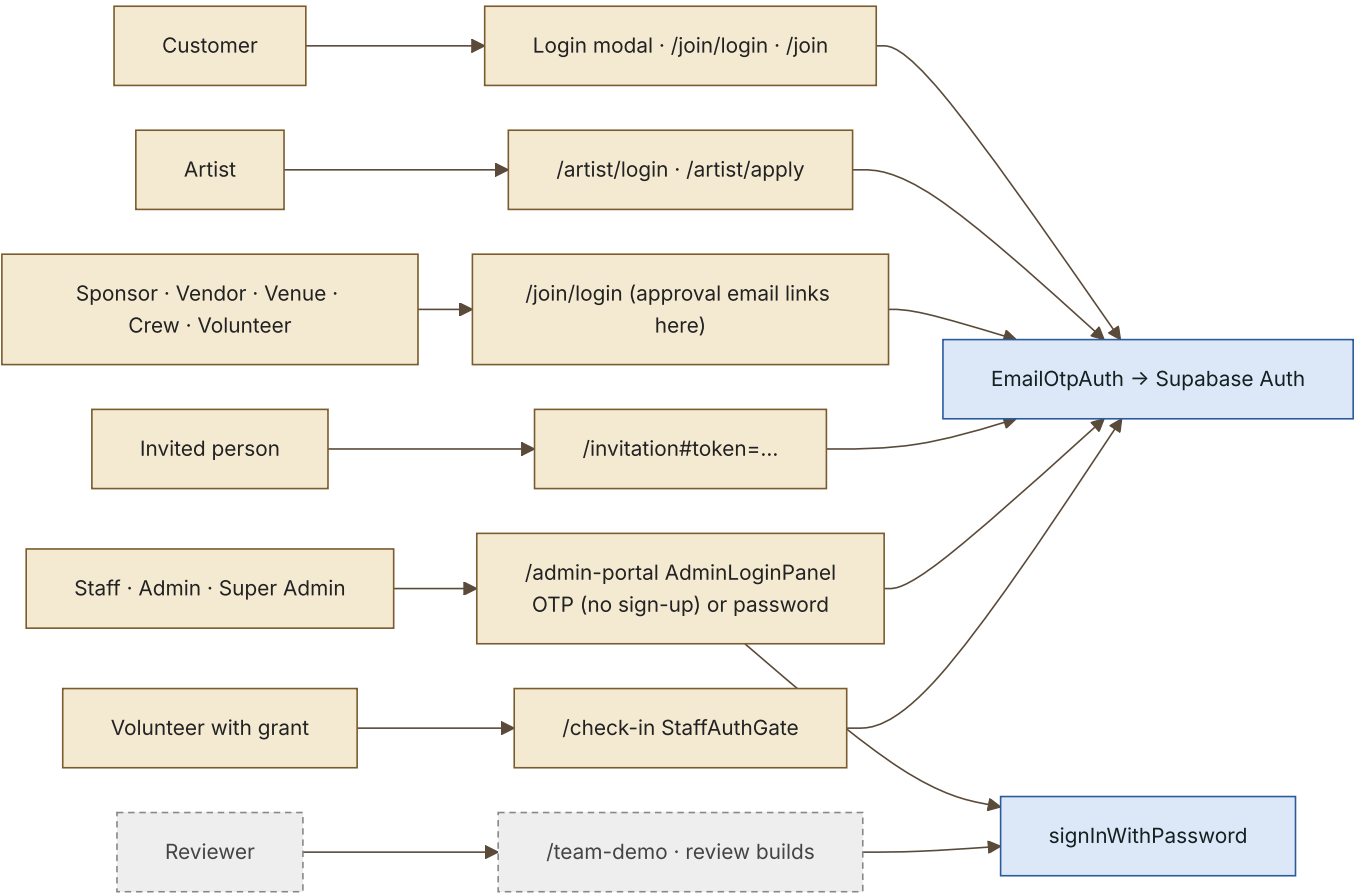


Diagram — Who signs in where

Invitation flow (console accounts)

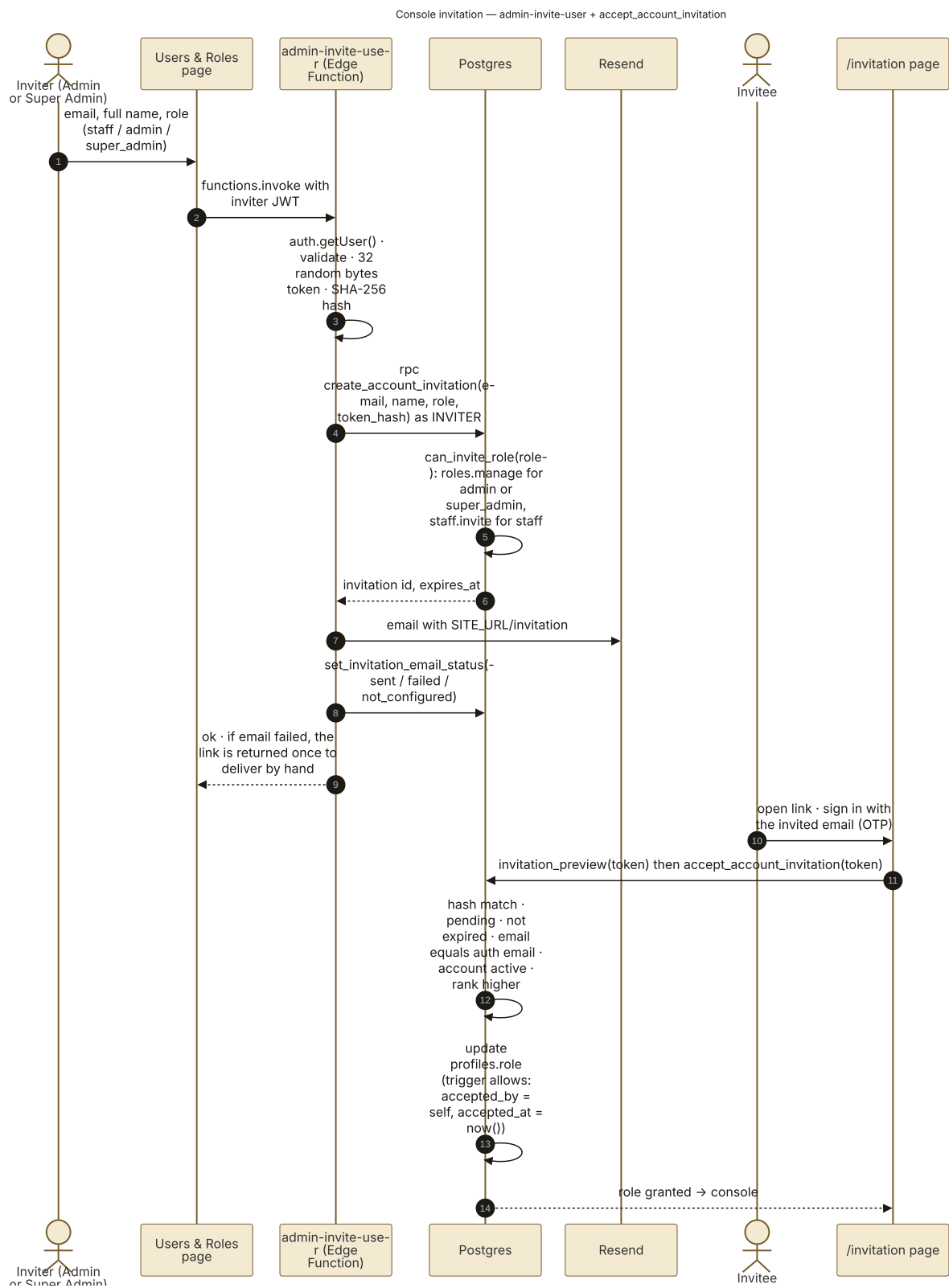


Diagram — Console invitation — admin-invite-user + accept_account_invitation

Session and role refresh

Session lifecycle

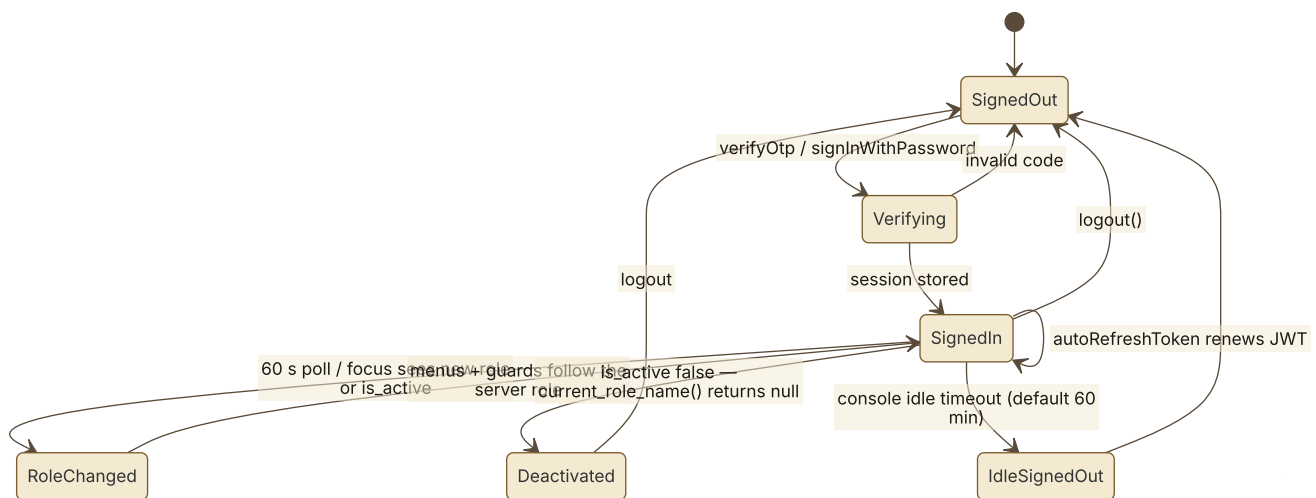


Diagram — Session lifecycle

Why the server role always wins: every RLS policy and RPC reads `profiles.role` through `current_role_name()` / `has_permission()` on each request. When a Super Admin changes someone's role, the database stops honouring the old role immediately. The 60-second client refresh only updates the menus.

Artist login specifics

The artist portal does **not** gate on `profiles.role`. `ArtistPortalShell` → artist `AuthContext` loads the `artists` row for `auth.uid()`:

- no row → redirect to `/artist/login?next=...`
- `status ≠ approved` → redirect to `/artist/application`
- `approved` → portal

Approval also sets `profiles.role = 'artist'` (`approve_artist_application`), so `/dashboard` redirects artists to `/artist/dashboard`.

Part II — Major Workflows

06 — Admin Architecture (the "Control Room") · 07 — Artist Architecture and Lifecycle · 08 — Artist Application Workflow (`/artist/apply`) · 09 — Event Lifecycle · 10 — Event ↔ Artist Workflow (Availability, Line-up, Booking Requests) · 11 — Customer Booking Workflow · 12 — Payment Architecture (Razorpay) · 13 — Ticket Architecture · 14 — Check-in Workflow · 15 — Waitlist Workflow · 16 — Notification Architecture · 17 — Email Architecture · 18 — Messaging Architecture · 19 — Content / CMS Architecture · 20 — Storage Architecture

06 — Admin Architecture (the "Control Room")

The console lives at `/admin-portal/*`: `src/admin/AdminApp.jsx`, with 54 routes. Old `/admin/*` links redirect here.

How every admin action is processed

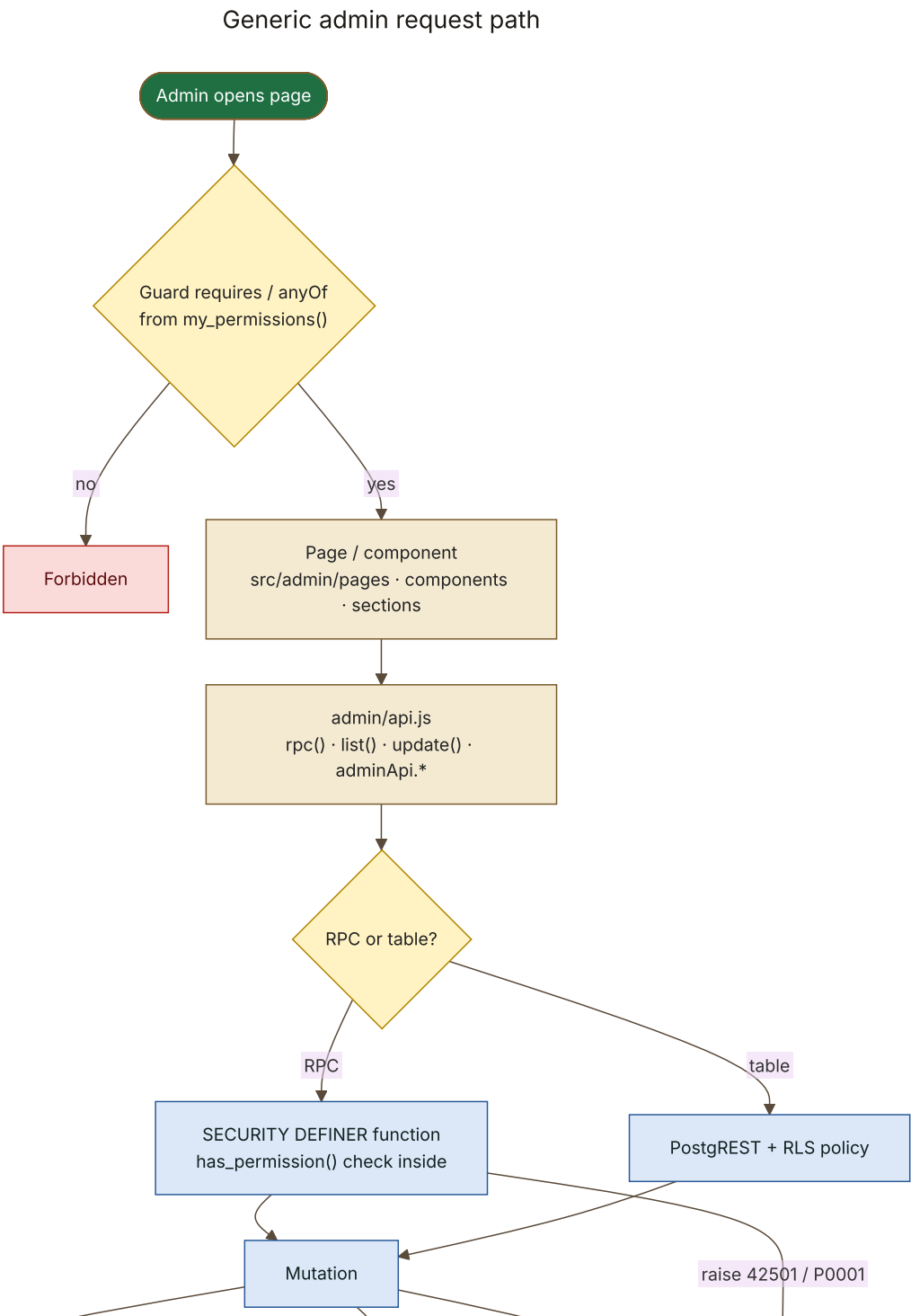


Diagram — Generic admin request path (part 1 of 2)

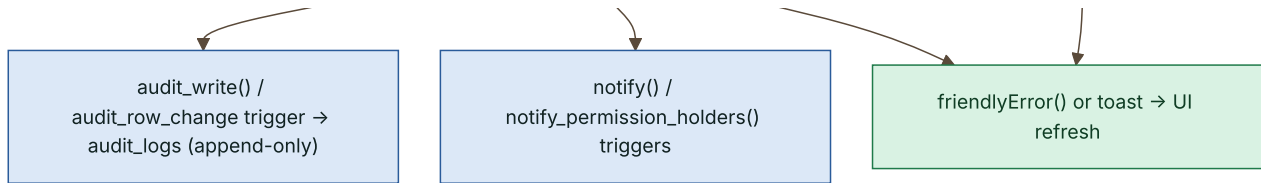


Diagram — Generic admin request path (part 2 of 2)

The UI never decides access. `rbac.js` says so: "Nothing here grants access — it only decides what to render, so a tampered client just sees buttons that fail." `admin/api.js update()` treats "0 rows updated" as forbidden, because RLS turns an unauthorized UPDATE into 0 rows.

Console map (permission → page → data)

Section	Route(s)	Permission (route guard)	Frontend	Service → RPC / table	Server checks, mutations, side effects
Dashboard	<code>/admin-portal</code>	<code>dashboard.view</code>	<code>DashboardPage</code>	<code>admin_dashboard_summary</code> , <code>staff_dashboard</code> , <code>admin_operations_overview</code> , <code>artist_day_summary</code> ; reads <code>bookings</code> , <code>event_requirements</code> , <code>event_tasks</code> , <code>access_requests</code> , <code>announcements</code> , <code>audit_logs</code> , <code>conversations</code>	Read-only. Kind chosen by <code>dashboardKind(perms)</code> : <code>super_admin</code> / <code>admin</code> / <code>staff</code>
Global search	header <code>⌘K</code>	(any console user)	<code>AdminShell</code>	<code>admin_search(p_query)</code>	Results scoped by permission in SQL
Applications	<code>/applications/:id</code>	<code>applications.view</code> (review: <code>applications.review</code>)	<code>ApplicationsPage</code>	<code>view</code> <code>applications_overview</code> ; <code>approve_*</code> / <code>reject_*</code> (artists, collaborations, crew_applications); then <code>send-approval-email</code>	Pending-only transitions, role provisioning (user→partner), <code>application_notifications</code> row, audit, applicant notification
Artist applications	<code>/artists/applications/:id</code>	<code>applications.view</code>	<code>ArtistApplicationsPage</code>	<code>review_artist_application</code> (<code>start_review</code> / <code>request_info</code> / <code>approve</code> / <code>reject</code>); signed URL for the video	See 08
Events	<code>/events</code> , <code>/events/new</code> , <code>/events/:id/:tab</code>	<code>events.view_all</code> (create: <code>events.manage</code>)	<code>EventsPage</code> , <code>EventDetailPage</code> , <code>EventForm</code> , <code>EventEditor</code> , <code>TicketTypesEditor</code> , <code>BookingFormEditor</code> , <code>EventOps</code> , <code>Team</code> , <code>Tasks</code>	<code>insert/update</code> <code>events</code> , <code>event_ticket_types</code> ; <code>save_event_lineup</code> , <code>update_event_artist</code> , <code>remove_event_artist</code> , <code>create_artist_request</code> , <code>manage_artist_request</code> , <code>artist_schedule_check</code> , <code>event_health</code> , <code>event_command_center</code> , <code>review_requirement</code> , <code>close_requirement</code> ; <code>event_requirements</code> , <code>event_documents</code> (+ <code>event_documents</code> bucket), <code>event_assignments</code> , <code>event_tasks</code> , <code>sponsor_deliverables</code>	<code>events_audit</code> , <code>events_notify_change</code> , <code>events_notify_publish</code> , <code>events_prevent_delete_with_bookings</code> , <code>events_validate_timezone</code> , <code>events_seed_ticket_type</code> , capacity → waitlist offers. See 09, 10
Calendar	<code>/calendar</code>	<code>events.view_all</code>	<code>CalendarPage</code>	<code>admin_calendar</code>	Read-only

Section	Route(s)	Permission (route guard)	Frontend	Service → RPC / table	Server checks, mutations, side effects
My Events / Event Info	/my-events[/:id], /event-info	events.view_assigned	MyEventsPage, StaffEventPage	event_assignments, events, venues, announcements, event_checkin_stats	Staff may update own assignment status
Bookings	/bookings[/:bookingId]	bookings.view_all	BookingsPage, components/Bookings.jsx	bookings, audit_logs, event_ticket_types; admin_cancel_booking, admin_record_refund, admin_cancel_ticket, admin_create_comp_booking; send-ticket-email (resend, force)	Cancel needs bookings.manage + reason, refuses if anyone checked in; refund needs bookings.manage + payments.view + Razorpay refund reference; comp booking checks capacity under lock and issues tickets; all audited; seat release → waitlist
Payments	/payments, /payments/webhooks	bookings.view_all + payments.view	PaymentsPage	bookings, payment_webhook_events	Read-only mirror of Razorpay state and webhook processing errors
Waitlist	/waitlist	bookings.view_all	WaitlistSection	waitlist; admin_offer_waitlist, admin_remove_waitlist_entry	Manual offer / removal, audited
Partner invoices	/invoices	payments.view + bookings.manage	InvoicesPage	insert/update partner_invoices	partner_invoices_notify, audit
Attendees	/attendees	attendees.view_all or attendees.view_assigned	components/Attendees.jsx	view_attendee_tickets (CSV export), my_checkin_events, check_in_ticket	Contacts only for those allowed to see them
QR Check-in	→ /check-in	checkin.perform	TangyWorldCheckInPage	see 14	
Check-in history	/check-ins	checkin.history	CheckinHistory	get_checkin_history	Read-only
Event tasks	/tasks	tasks.view_own or team.manage	components/Tasks.jsx	event_tasks, event_assignments	event_tasks_validate, guard_event_task_status, notifications
People	/people/:kind[/:id], /people/artists/:id[/:tab]	entities.manage	PeoplePage (EntityManager), ArtistDetailPage	artists, venues, sponsor_profiles, venue_profiles, vendor_profiles, crew_profiles, volunteer_profiles; artist_calendar, artist_availability_summary, start_private_artist_conversation, admin_start_partner_conversation	Private artist thread is Super Admin only
Reviews	/reviews[/:tab]	entities.manage	ReviewsPage	update_artist_media.status / sponsor_assets.status; signed URLs	guard_artist_media_status, stamp_asset_review, notifications media.reviewed / asset.reviewed

Section	Route(s)	Permission (route guard)	Frontend	Service → RPC / table	Server checks, mutations, side effects
Volunteers	/volunteers/:userId	volunteers.manage	VolunteersPage	volunteers_overview, grant_temporary_access, revoke_temporary_access, decline_access_request, volunteer_access_activity	access.grant checked in SQL; 30–720 min windows; audited
Team	/team	team.manage	TeamPage, components/Team.jsx	event_assignments, event_tasks, profiles	validate_event_assignment, notifications
Users & Roles	/users/:id	users.manage or staff.invite	UsersPage	profiles, audit_logs; admin_set_user_role, admin_set_user_active, list_account_invitations, revoke_account_invitation, admin-invite-user	roles.manage for role changes; last Super Admin protected; role.changed notification
Roles & Permissions	/roles	roles.manage	RolesPage	role_permissions; set_role_permission	Only admin/staff rows editable; audited
Messages	/messages/:conversationId	messages.manage	MessagesPanel (portal)	admin_conversations, conversation_messages, send_message, set_conversation_status, set_conversation_meta, mark_conversation_read; Realtime	Private artist threads hidden from non-participants (RLS 0034)
Content	/content/:section[:item]	content.manage / content.view / content.manage_sessions (+ area rights)	ContentPage, ContentCollections (TV, Diary, Gallery, Programmes), Announcements, MediaLibrary	tv_videos, diary_posts, gallery_albums, gallery_photos, programmes, programme_events, announcements; update_session_content; storage content-media	content_guard_publish (content.publish), audit triggers. See 19
Announcements (staff)	/announcements	announcements.view	StaffAnnouncementsPage	announcements	Read
Notifications	/notifications[:settings]	dashboard.view	NotificationsPanel	my_notifications, mark_notifications_read, my_notification_preferences, set_notification_preference; send_custom_notification (Super Admin)	
Reports	/reports	reports.view	ReportsPage	report_event_performance, report_revenue_by_month, report_applications, report_staff_activity, report_platform_activity	Read-only aggregates
Audit logs	/audit	audit.view	AuditLogsPage	audit_logs	Append-only table (trigger blocks update/delete)
Settings	/settings	settings.manage	SettingsPage	system_settings, role_permissions; update_system_setting	Audited (settings.updated)
Tangy AI	/ai	ai.use	AiPage	none	NOT IMPLEMENTED: aiService.js "NOT connected to any model yet"

Section	Route(s)	Permission (route guard)	Frontend	Service → RPC / table	Server checks, mutations, side effects
More operations	/ops/inbox, /ops/enquiries, /ops/contact, /ops/notifications, /ops/portals	operations.manage	InboxSection, EnquiriesSection, NotificationsSection, PortalsSection	support conversations (chatService), private_enquiries, contact_enquiries, email_outbox + application_notifications (delivery log), portal previews	Read / reply

Representative flows

Approve a partner / crew / artist application (generic Applications screen)

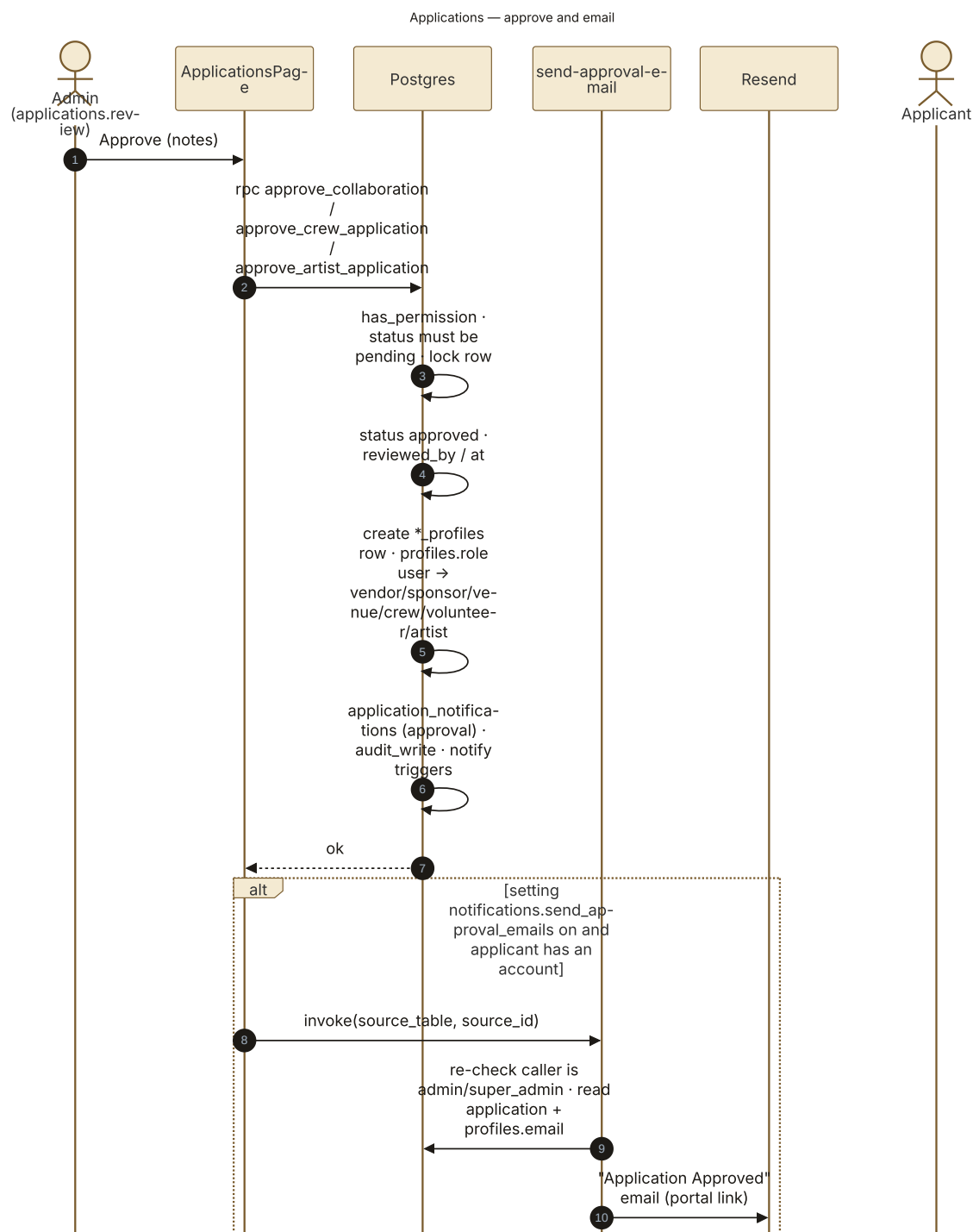


Diagram — Applications — approve and email (part 1 of 2)

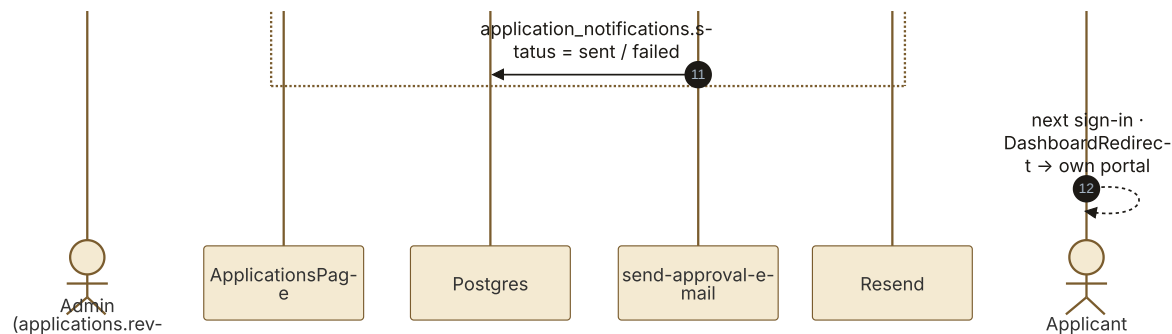


Diagram — Applications — approve and email (part 2 of 2)

Bookings drawer actions

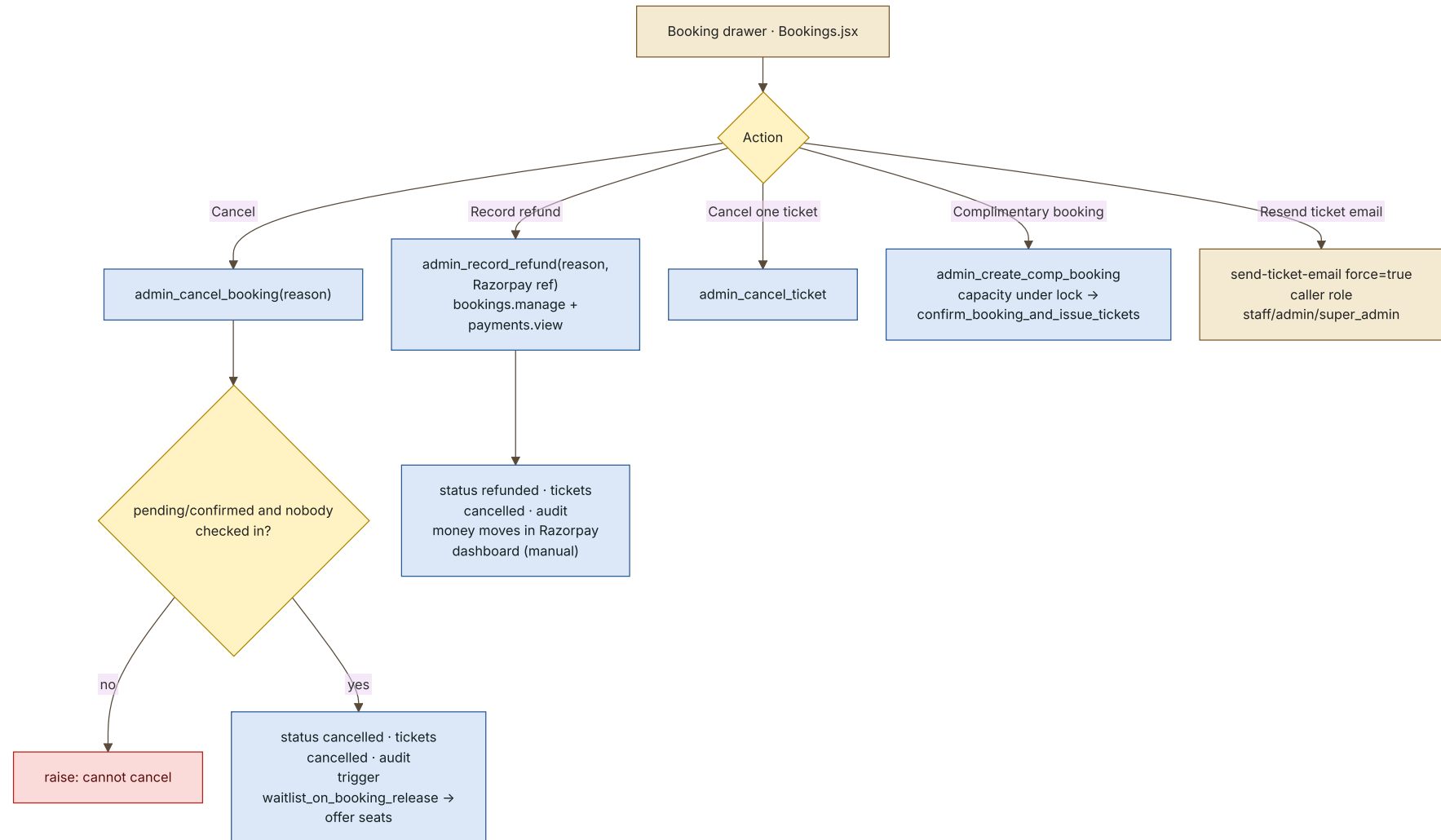


Diagram — Bookings drawer actions

Users & Roles — role change

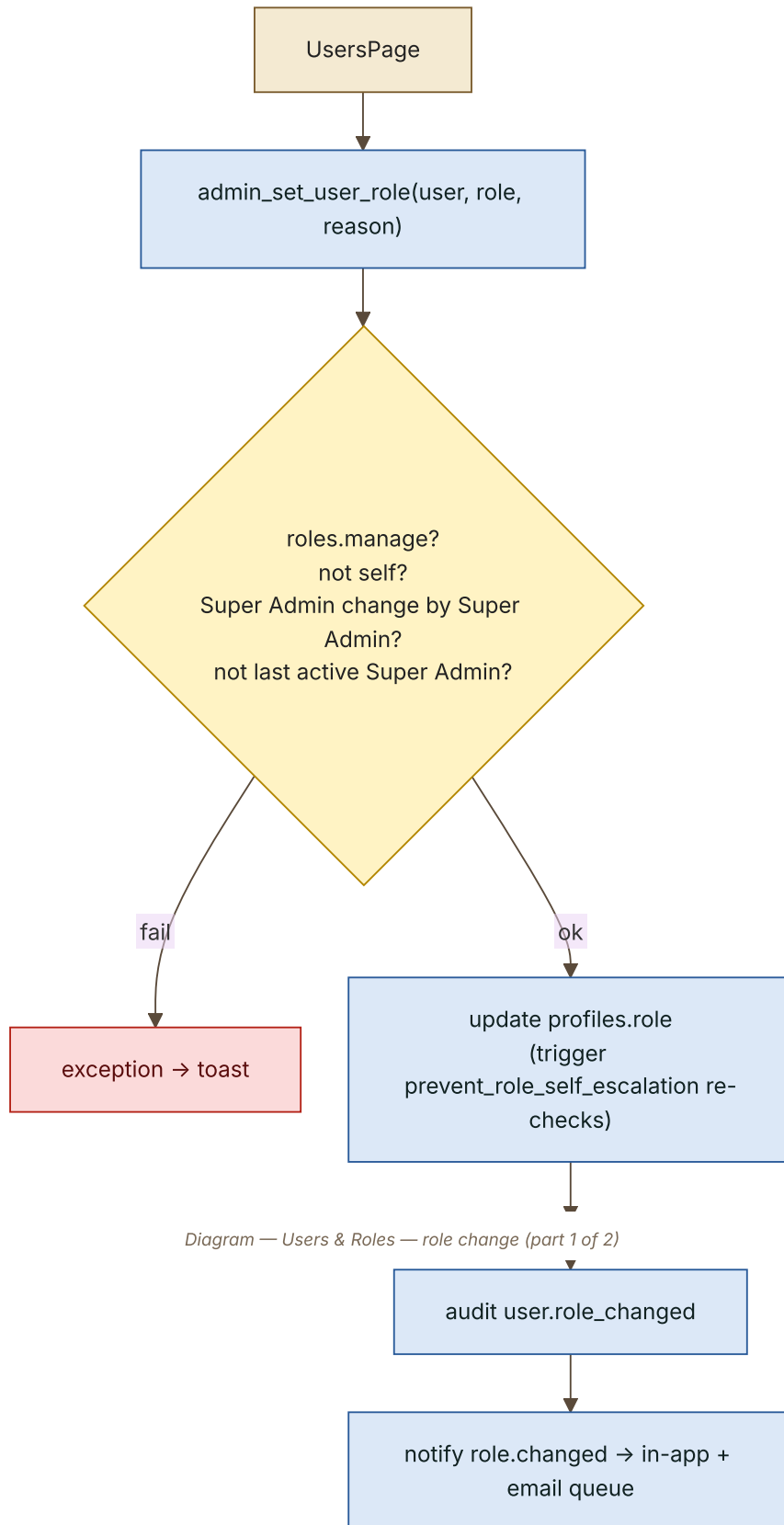


Diagram — Users & Roles — role change (part 1 of 2)

Diagram — Users & Roles — role change (part 2 of 2)

Volunteer temporary check-in access

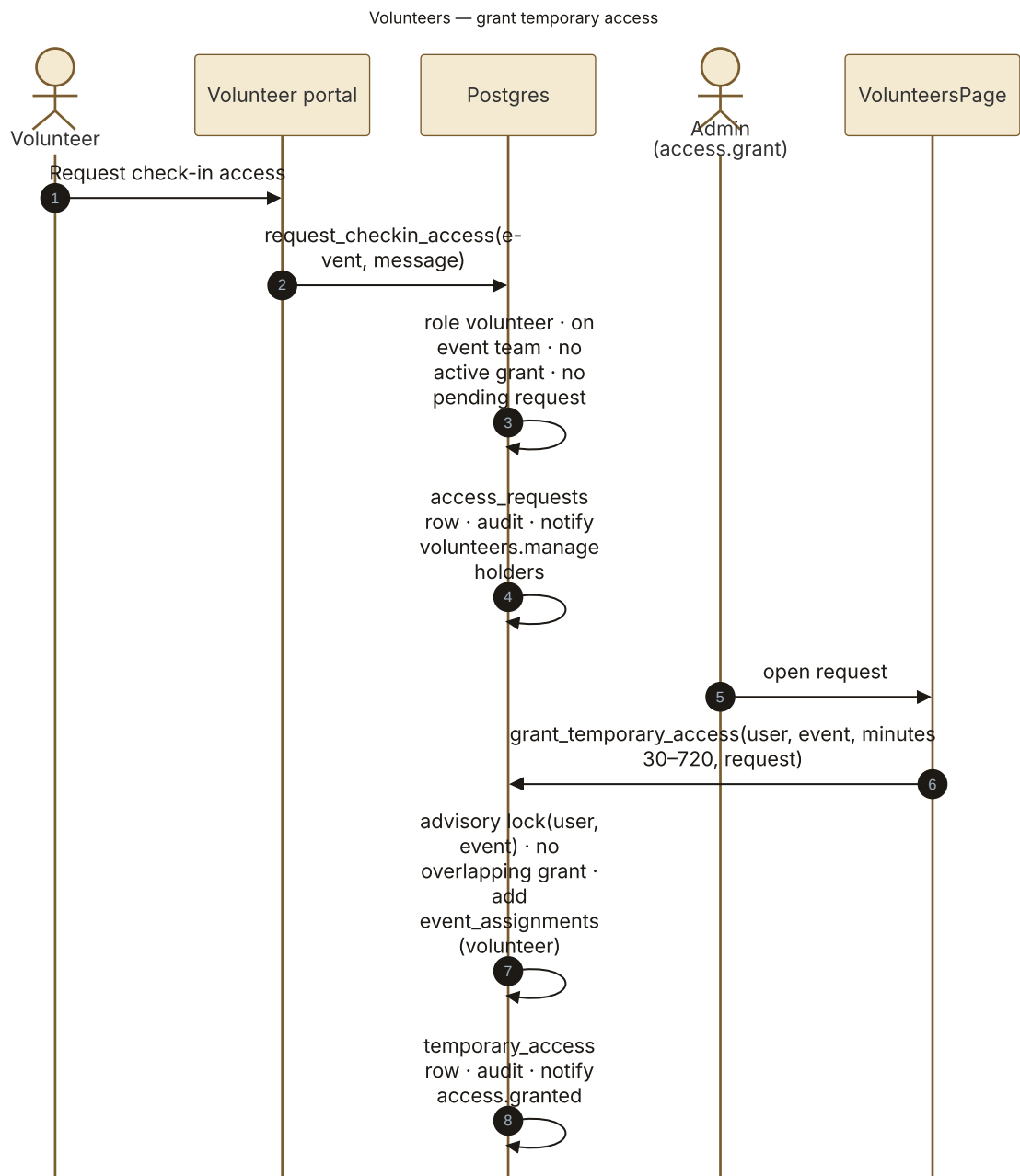


Diagram — Volunteers — grant temporary access (part 1 of 2)

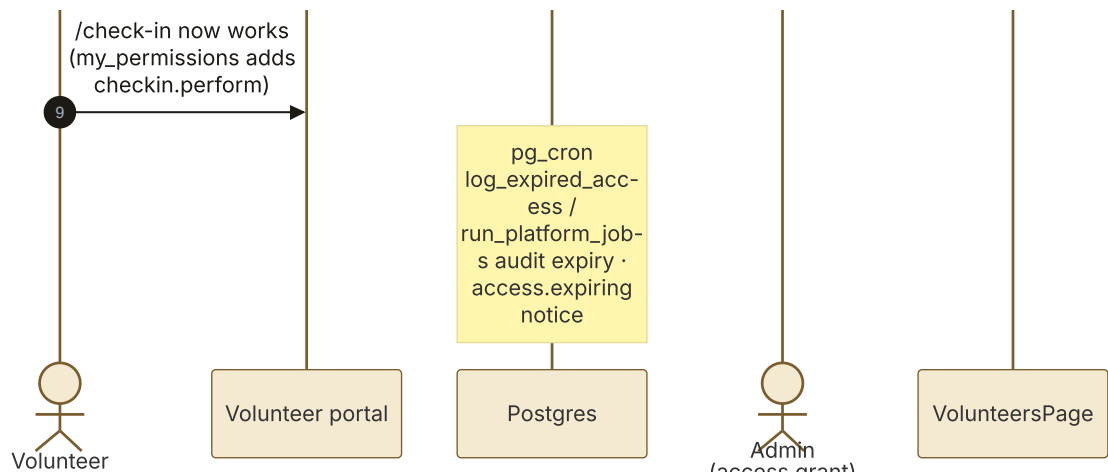


Diagram — Volunteers — grant temporary access (part 2 of 2)

07 — Artist Architecture and Lifecycle

Artist features come from migrations 0012 (availability), 0013 (media), 0014 (Spotify), 0020 (private profiles, profile completion, request details), 0022 (storage fixes), 0032 – 0034 (applications, portal, event workflow) and 0036 (documents bucket fix). The repository's own guide is docs/ARTIST_PORTAL.md.

Complete artist lifecycle (as implemented)

Artist lifecycle — discovery to future events

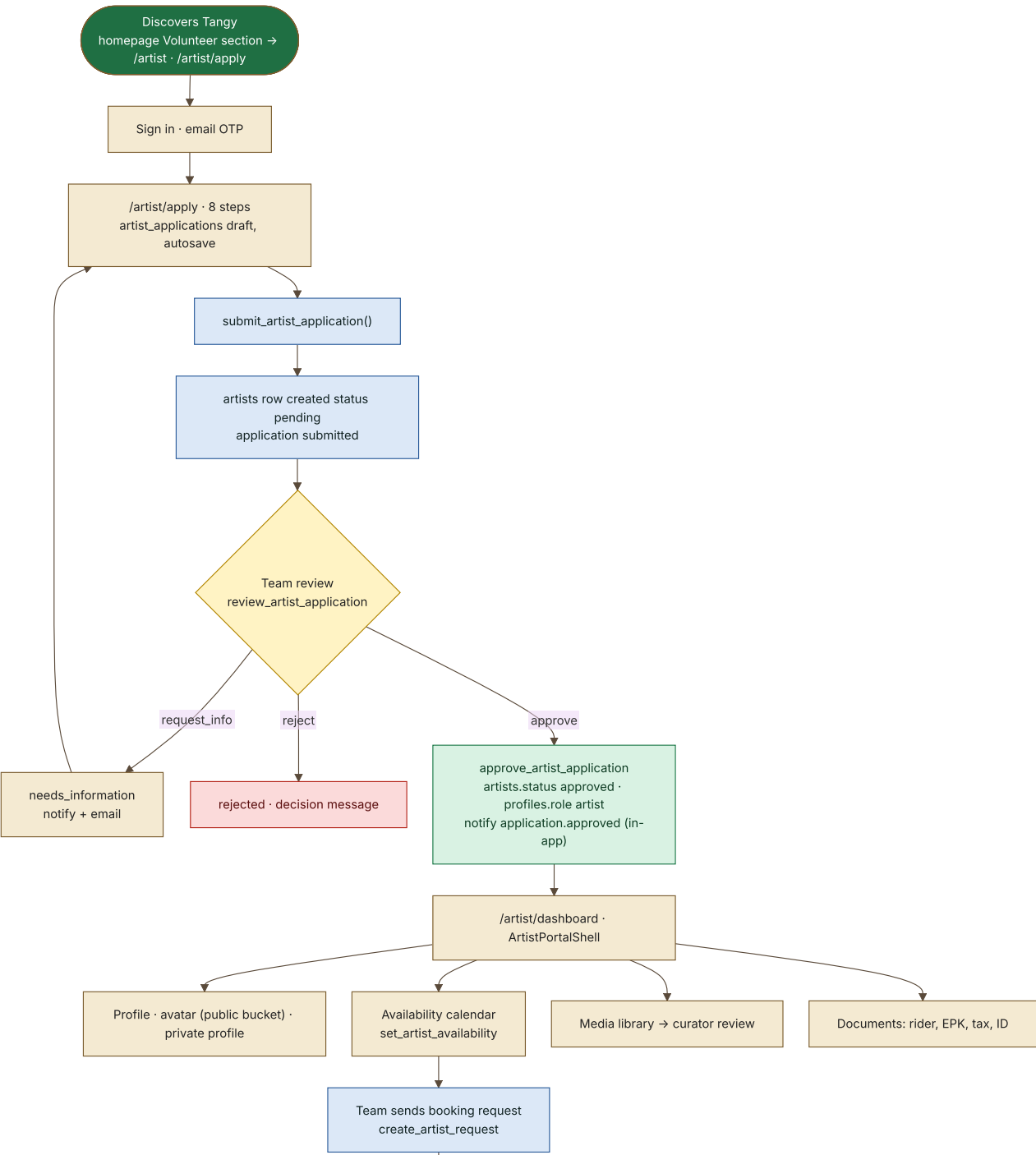


Diagram — Artist lifecycle — discovery to future events (part 1 of 2)

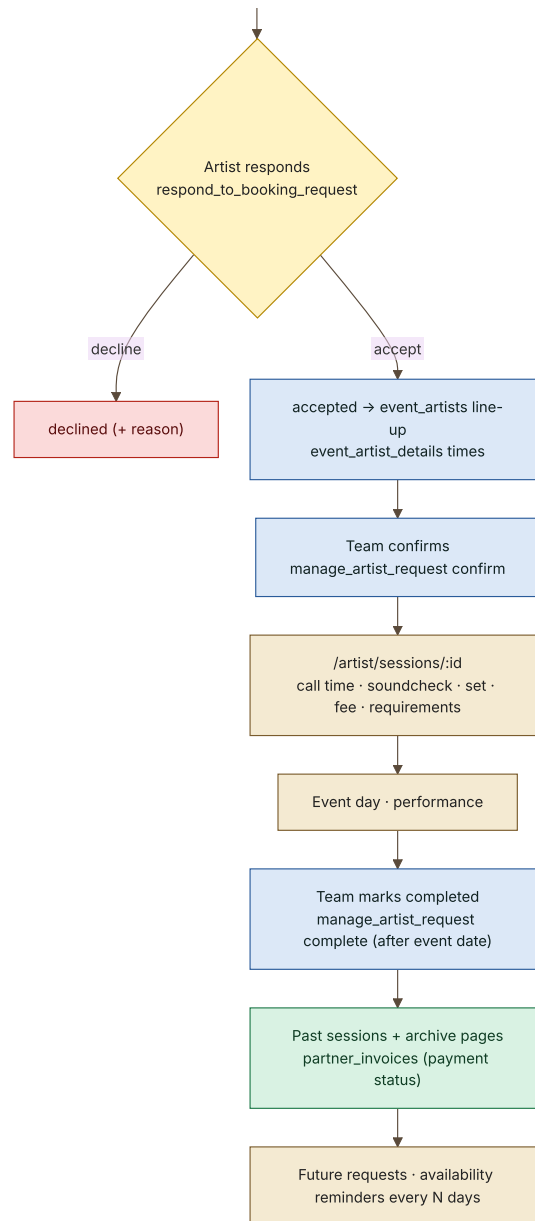


Diagram — Artist lifecycle — discovery to future events (part 2 of 2)

What each stage touches

Stage	Frontend	RPC / table	Storage	Notifications / email	Admin involvement	Security boundary
Apply (draft)	<code>artist/portal/pages/ApplyPage.jsx</code>	<code>insert/update artist_applications (artistApi.startApplication/saveApplication)</code>	<code>artist-media/applications/<uid>/...</code>	—	—	RLS: applicant edits own row only in <code>draft / needs_information</code> ; <code>guard_artist_application</code> blocks status/review fields
Submit	same	<code>submit_artist_application()</code>	—	<code>application.submitted</code> to self (in-app); on resubmit <code>application.resubmitted</code> to reviewers	—	Server validates required fields, video, consents

Stage	Frontend	RPC / table	Storage	Notifications / email	Admin involvement	Security boundary
Review	<code>admin/pages/ArtistApplicationPage.jsx</code>	<code>review_artist_application(start_review / request_info / approve / reject), application_reviews</code> (internal notes)	signed URL for video	<code>application.under_review</code> (in-app), <code>application.info_requested</code> (in-app + email), <code>application.approved / rejected</code> (in-app, critical)	<code>applications.review</code>	Internal notes separated from applicant-readable rows (0033)
Portal access	<code>ArtistPortalShell</code>	<code>artists</code> (own)	—	—	—	Approved <code>artists</code> row required
Profile	<code>artist/pages/ProfilePage.jsx</code> , <code>workspaceApi</code>	<code>artists</code> , <code>artist_private_profiles</code> , <code>artist_profile_completion()</code>	<code>artist-avatars/<artist id>/...</code> (public)	—	team can view	self update own row; status changes blocked by <code>prevent_artist_status_self_escalation</code>
Availability	<code>artist/portal/pages/AvailabilityPage.jsx</code>	<code>set_artist_availability()</code> , <code>artist_availability</code> , <code>artist_availability_summary()</code>	—	platform job <code>send_availability_reminders</code> (stale after <code>artists.availability_stale_days</code> , default 30)	team reads (<code>events.manage / entities.manage</code>)	Only approved artist; days with a confirmed performance keep their booking (<code>guard_artist_availability</code>)
Requests	<code>artist/portal/pages/RequestsPage.jsx</code>	<code>my_booking_requests()</code> , <code>mark_artist_request_viewed()</code> , <code>respond_to_booking_request()</code>	—	to the artist: <code>booking.requested</code> , <code>booking.confirmed</code> , <code>booking.cancelled</code> ; to the requesting admin: <code>booking.accepted / declined / expired</code>	team creates/confirm	Drafts invisible to the artist; per-artist-per-day advisory lock
Sessions	<code>artist/portal/pages/SessionsPage.jsx</code>	<code>event_artists</code> , <code>events</code> , <code>event_artist_details</code> , <code>event_requirements</code>	<code>event-documents</code> (audience-scoped)	<code>schedule.changed</code> , <code>event.*_changed</code> , <code>event.cancelled</code> , <code>event.reminder</code>	team edits line-up	<code>event_member_kind()</code>
Media	<code>artist/pages/MediaPage.jsx</code>	<code>artist_media</code> (status uploaded → <code>under_review</code> → <code>approved/rejected</code> , <code>archived</code>)	<code>artist-media/<artist id>/...</code> (private)	<code>media.submitted</code> , <code>media.reviewed</code>	Reviews page (<code>entities.manage</code>)	<code>guard_artist_media</code> : only curators approve/reject
Documents	<code>artist/portal/pages/InboxPages.jsx</code> (DocumentsPage)	<code>artist_documents</code>	<code>artist-documents/<artist id>/...</code> (private, 25 MB, pdf/images/docx)	—	team reads	own folder only (0036)

Stage	Frontend	RPC / table	Storage	Notifications / email	Admin involvement	Security boundary
Messages	InboxPages.jsx (MessagesPage)	my_conversations, conversation_messages, send_message, start_partner_conversation	—	message.new (email: generic text only)	Messages page; private Super Admin thread	RLS on participants
Payment / history	ArtistEventDrawer	partner_invoices (read)	—	invoice.issued / invoice.paid	Invoices page	own invoices

Portal shell

Artist portal — shell and pages

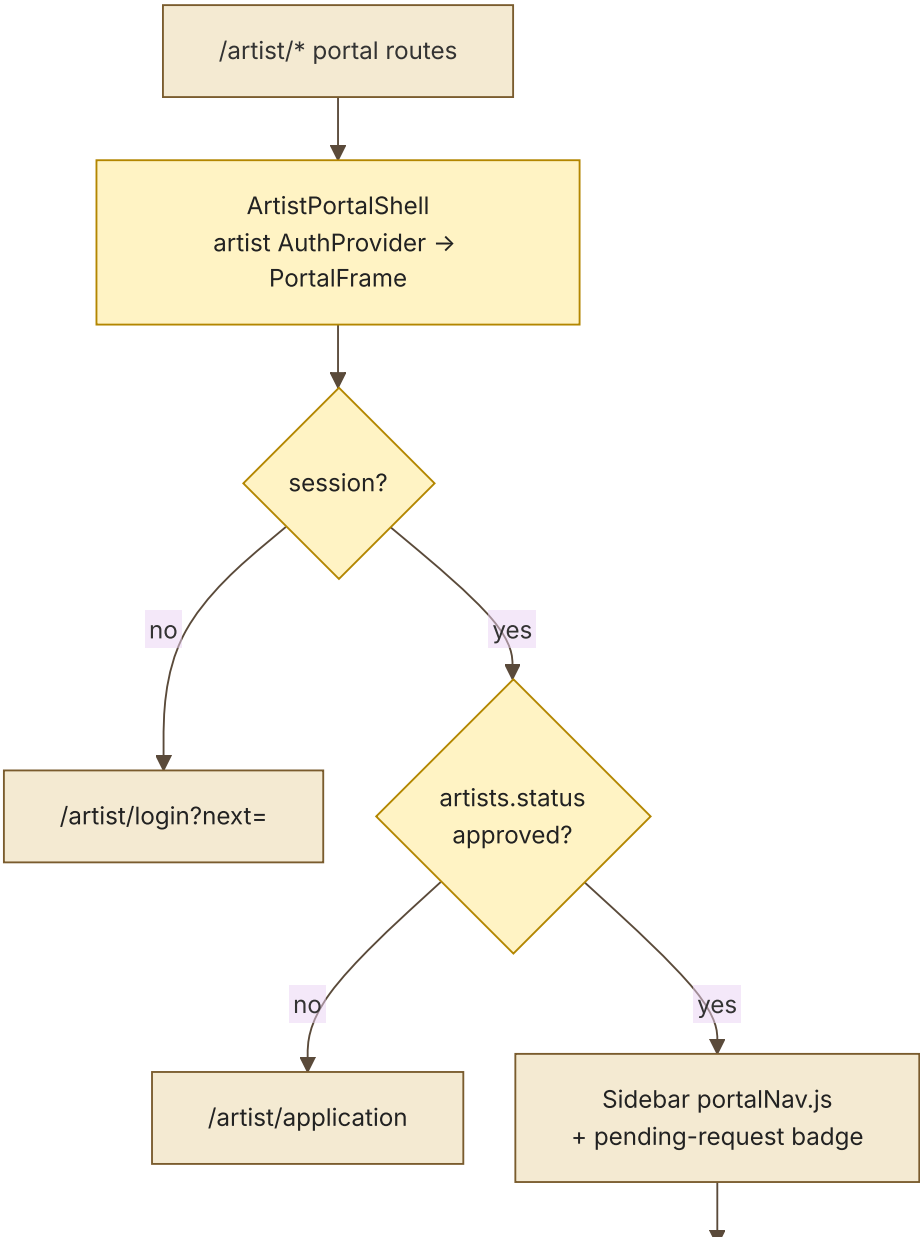


Diagram — Artist portal — shell and pages (part 1 of 2)

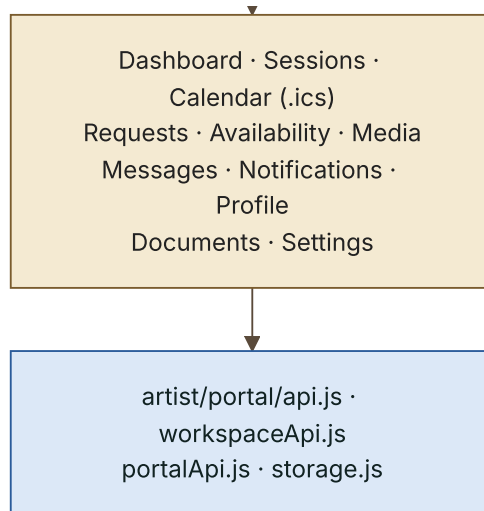


Diagram — Artist portal — shell and pages (part 2 of 2)

Artist approval email: a gap

When an artist is approved from the **Artist Applications** page (`review_artist_application` → `approve_artist_application`), the artist receives an **in-app** notification `application.approved`. Its catalogue entry has `email = false`, and that page does **not** invoke `send-approval-email`. The branded approval email is sent only when an application is approved from the generic **Applications** drawer (`ApplicationsPage`), which calls `send-approval-email` when `notifications.send_approval_emails` is on.

Status: **PARTIALLY IMPLEMENTED** for the artist-specific path. The artist is told in-app and by the dashboard redirect. An `application_notifications` row is created, so a "resend" from the Applications screen can still send it.

08 — Artist Application Workflow (/artist/apply)

Status: IMPLEMENTED (migrations 0032/0033; `src/artist/portal/pages/ApplyPage.jsx`, `ApplicationStatusPage.jsx`; admin `src/admin/pages/ArtistApplicationsPage.jsx`).

- One row per account in `artist_applications` (`user_id` is unique).
- Answers live in `data_jsonb` (< 200 KB), grouped by section: `about`, `artistry`, `experience`, `online`, `media`, `technical`, `availability`.
- The video, the consents and the review state are separate columns.

Step-by-step

#	Step ()	Main fields	Browser validation ()	Server validation on submit
1	About you	full name*, stage name*, email (prefilled), phone, WhatsApp, city*, country, website	name, stage name, city required; phone pattern	full name, stage name, city
2	Your artistry	artist type*, primary genre*, other genres, sub-genres, instruments, languages, sound/style, short bio*, long bio, years active, experience level	type, genre, short bio ≥ 40 chars	artist type, primary genre, short bio
3	Experience	years performing, number of shows, venues, festivals, cultural events, collaborations, performed with Tangy before (+ which event)	—	—
4	Online presence	Instagram, Spotify, YouTube, SoundCloud, website, other + self-reported audience numbers	https:// links (or @handle for Instagram)	—
5	Performance media	how the video is shared; primary video* (upload up to 50 MB or YouTube/Vimeo URL); title, place, date, type, description; optional 2nd video link; media consent*	video present; only YouTube/Vimeo links; consent	<code>video_url</code> or <code>video_storage_path</code> ; <code>media_consent</code> ; DB check constraint on <code>video_url</code> pattern
6	Technical & hospitality	format, set duration, technical requirements, own backline, rider notes, travelling from, members, accommodation, travel assistance, food, other	—	—
7	Availability	typical availability, preferred days, blocked dates, notice period, last-minute, travel radius, cities	—	—
8	Review	read-only summary with "edit" links; accuracy confirmation*	accuracy confirmed	<code>accuracy_confirmed</code>

Draft, autosave and submit flow

/artist/apply — draft, autosave, upload, submit

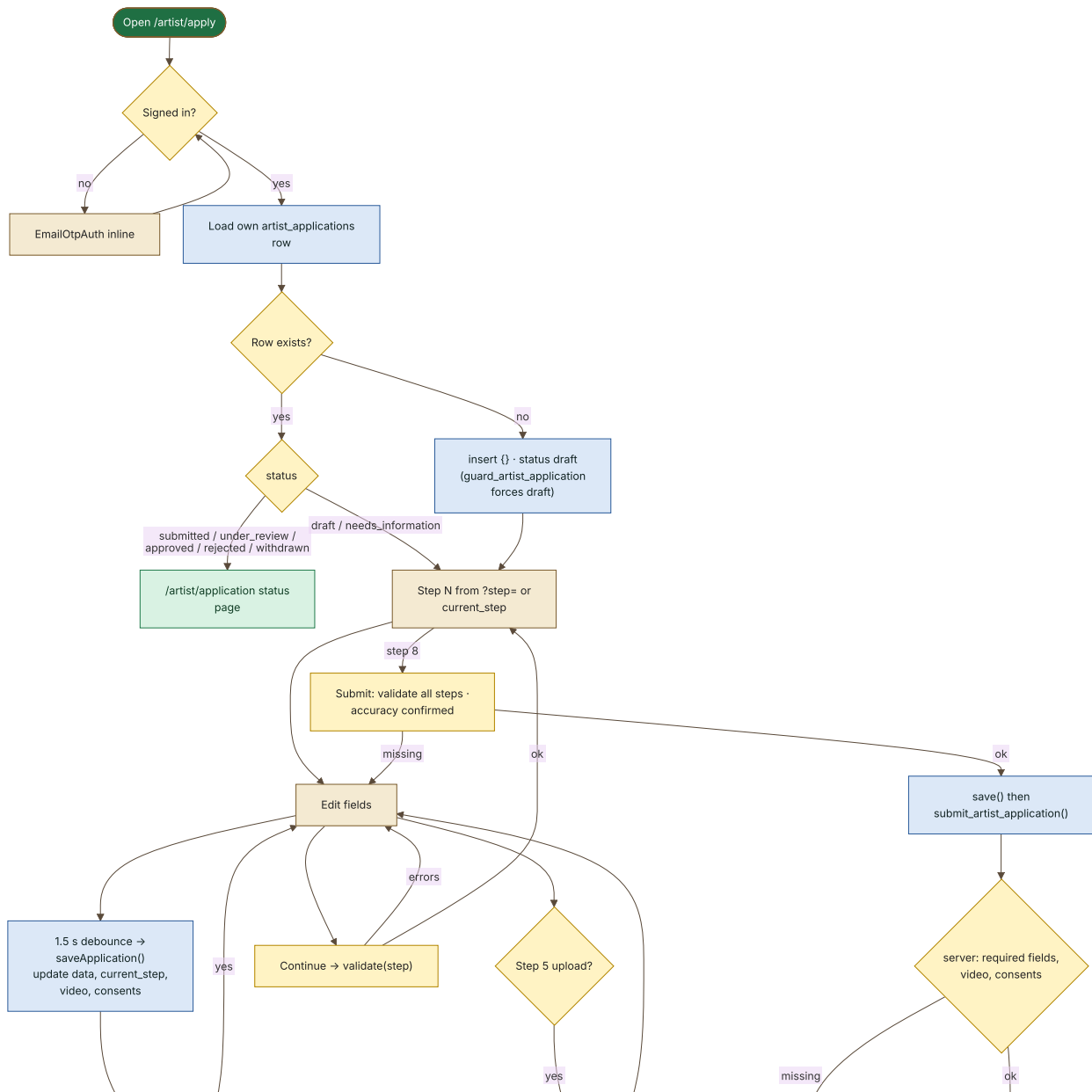


Diagram — /artist/apply — draft, autosave, upload, submit (part 1 of 2)

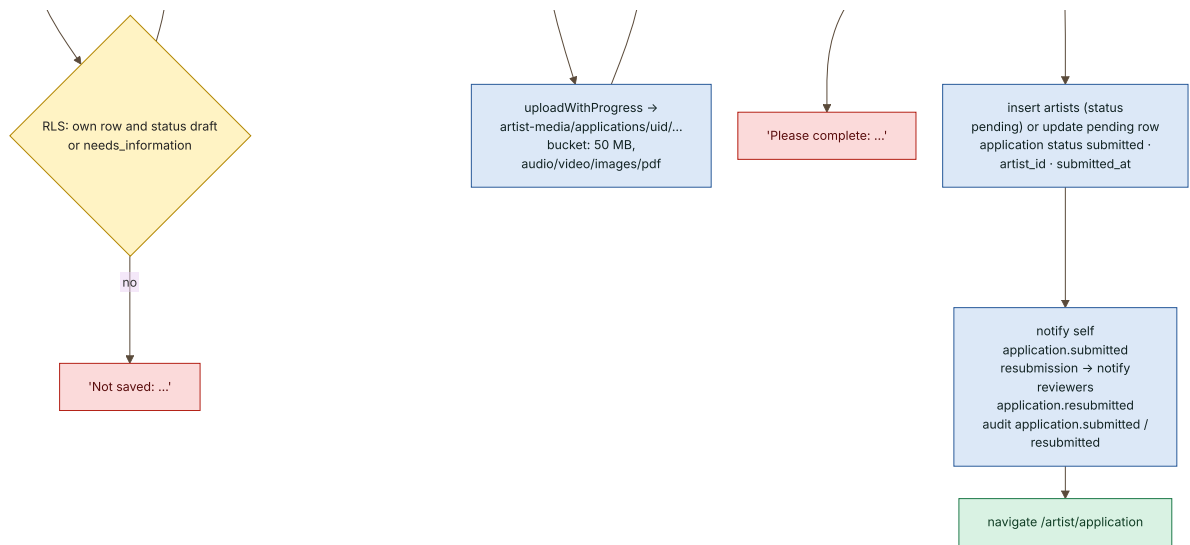


Diagram — /artist/apply — draft, autosave, upload, submit (part 2 of 2)

Application state machine

artist_applications.status

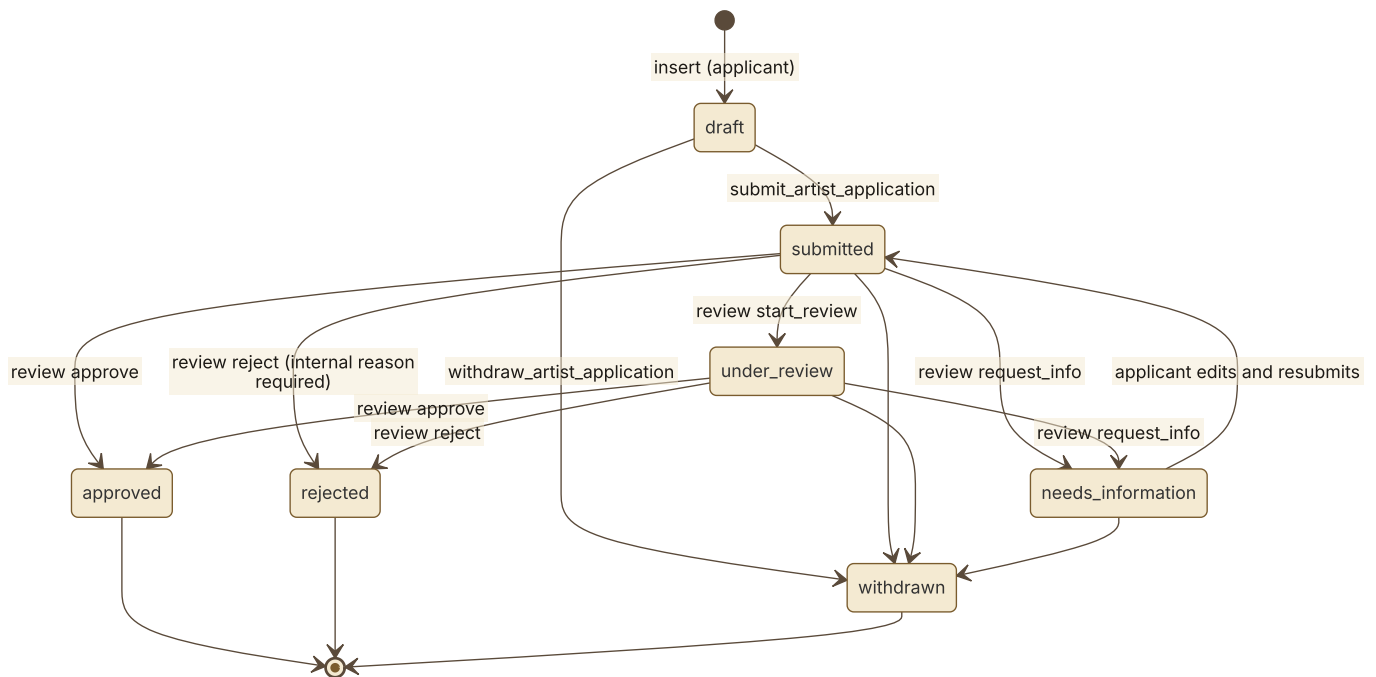


Diagram — artist_applications.status

`sync_artist_application_status` keeps the application in step when a decision is made on the `artists` row from the generic Applications screen.

Admin review flow

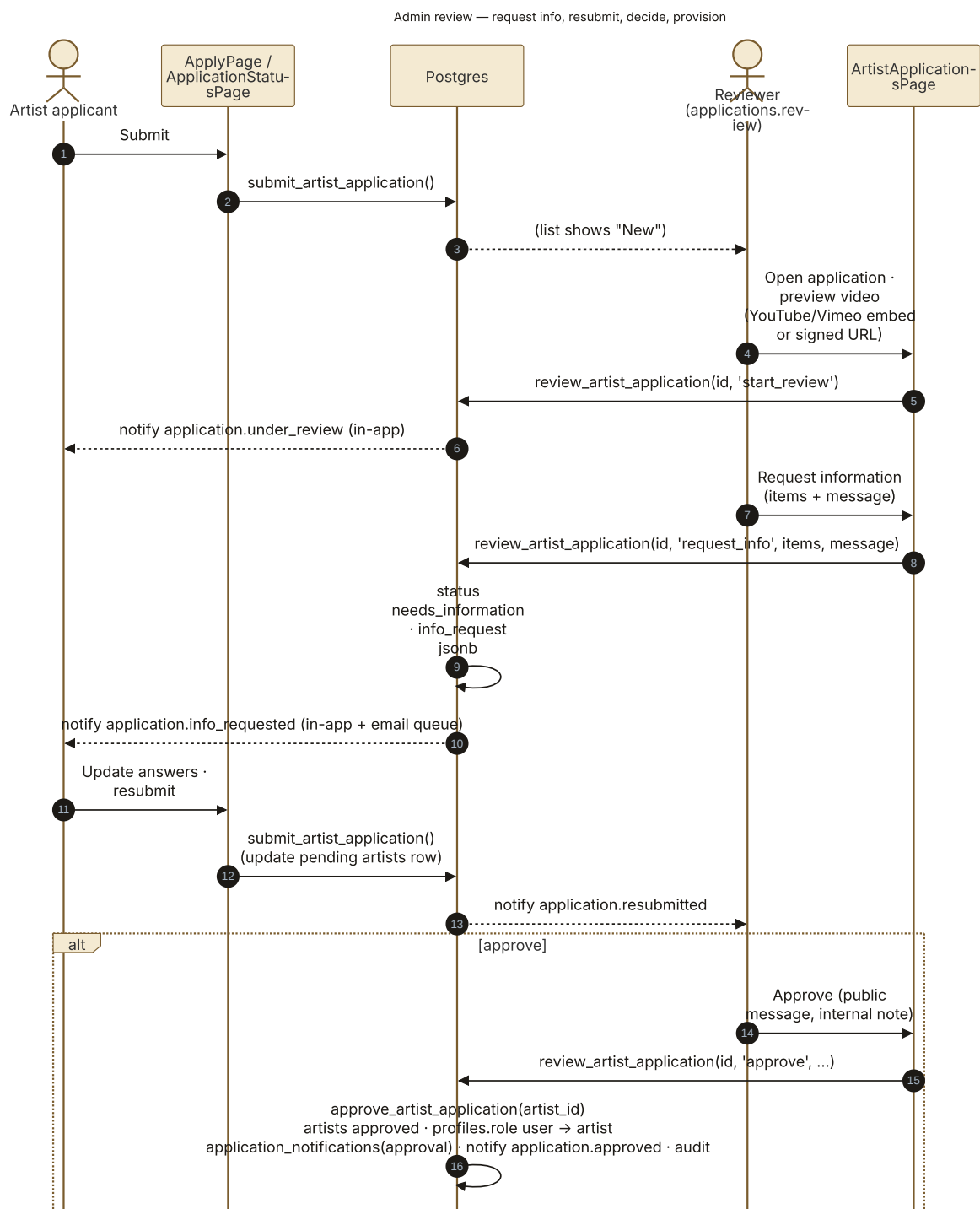


Diagram — Admin review — request info, resubmit, decide, provision (part 1 of 2)

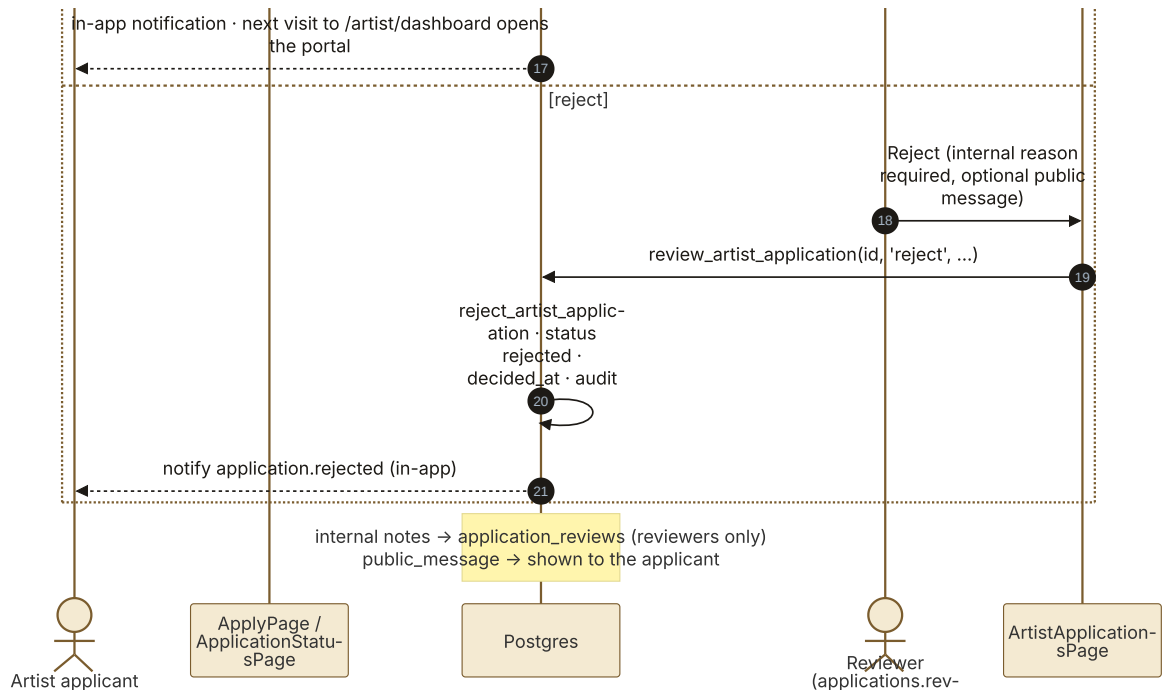


Diagram — Admin review — request info, resubmit, decide, provision (part 2 of 2)

Security boundaries

- **RLS:**
- The applicant reads their own row, and edits it only while it is `draft` or `needs_information`.
- Reviewers read with `applications.view`.
- **Trigger guard_artist_application:** for API callers it forces `status = draft` on insert and blocks every status and review field. Only the RPCs (which set `tangy.application_rpc = on`) can change them.
- **Uploads:** application uploads go to `artist-media/applications/<uid>/`. Only the applicant and `applications.view` holders can read them (0033 storage policies).
- **Approval email:** see the gap noted in 07. `application.info_requested` is emailed (catalogue `email = true`), while `approved` / `rejected` are in-app only on this path.

09 — Event Lifecycle

An "event" (UI: "session") is a row in `events`. Status values (convention since 0001, listed in `rbac.js`):

Status	UI label
draft	Draft
on-sale	Published · On sale
sold-out	Published · Sold out
past	Completed
cancelled	Cancelled

The UI's **lifecycle phase** (`eventPhase()` in `rbac.js`) is *derived from the date* and never stored: `draft` / `cancelled` / `upcoming` / `live` (today) / `completed`.

Event lifecycle — as implemented

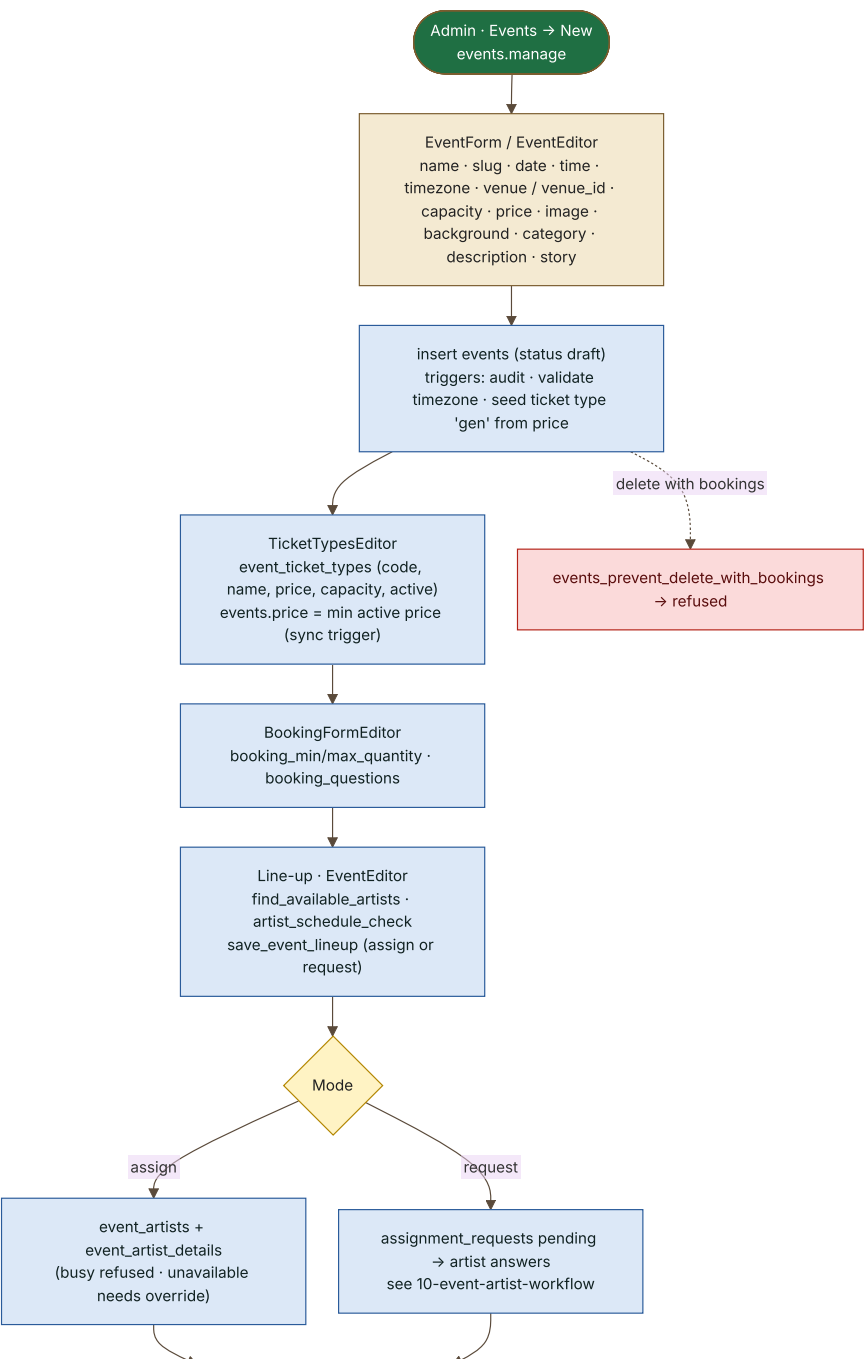


Diagram — Event lifecycle — as implemented (part 1 of 3)

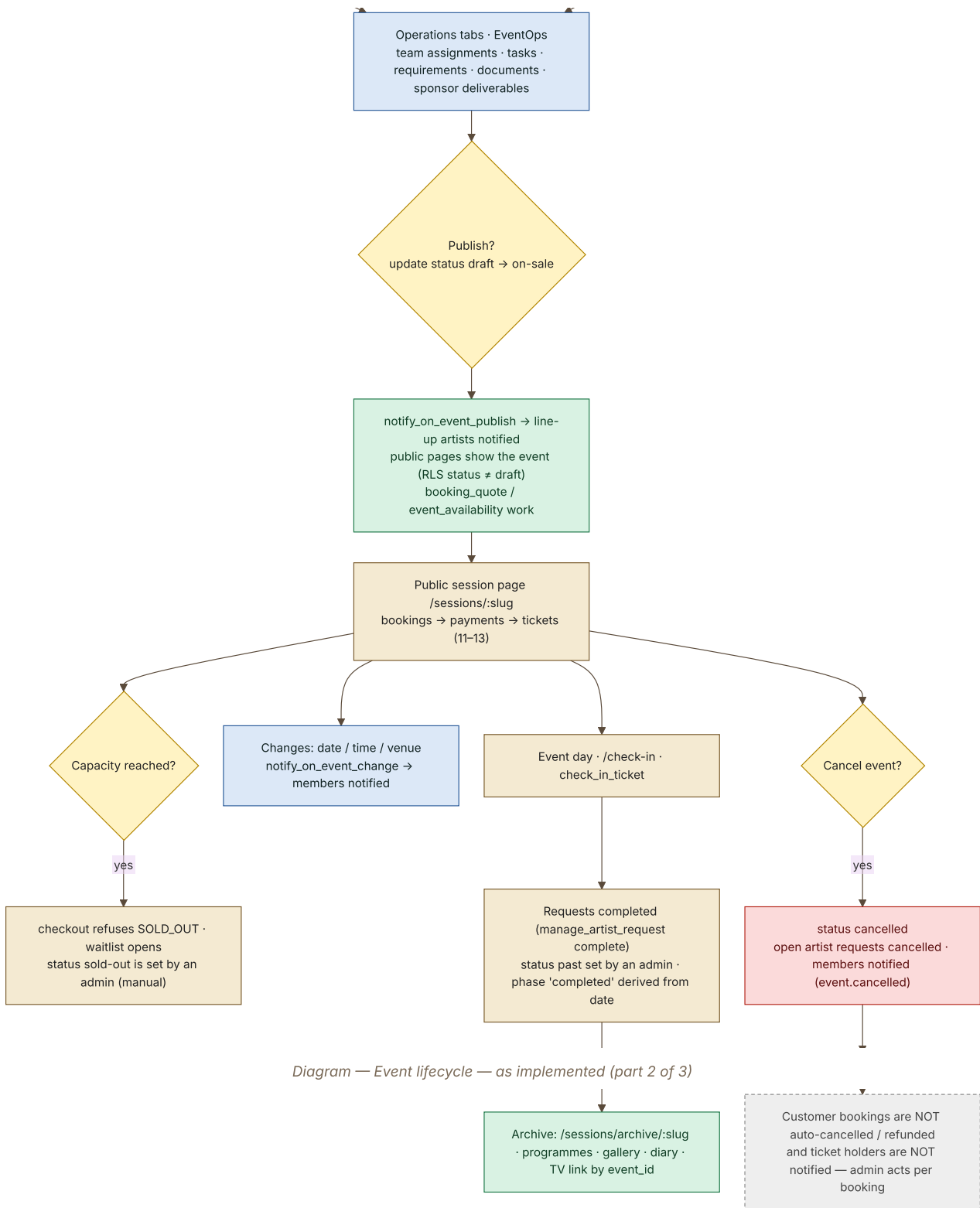


Diagram — Event lifecycle — as implemented (part 3 of 3)

Stage details

Stage	Frontend	Database	Side effects	Status
Create	admin/components/EventForm.js, EventEditor.js	insert events (events.manage via RLS is_admin() / policies)	events_audit, events_validate_timezone, events_seed_ticket_type	IMPLEMENTED

Stage	Frontend	Database	Side effects	Status
Venue	EventForm	venues (+ venue_profiles for partner venue hosts); events.venue_id, venue_partner_id	venue partner becomes an event member	IMPLEMENTED
Capacity	EventForm	events.capacity	raising capacity → events_capacity_to_waitlist → offer_waitlist_seats	IMPLEMENTED
Tickets	TicketTypesEditor.jsx	event_ticket_types; preview via booking_quote	event_ticket_types_sync_price, ticket_types_availability_signal	IMPLEMENTED
Booking form	BookingFormEditor.jsx	events.booking_questions, booking_min_quantity, booking_max_quantity	validated in create_pending_booking (booking_answers_error)	IMPLEMENTED
Artists / availability	EventDetailPage + ArtistAvailability.jsx	find_available_artists, artist_schedule_check, artist_day_status	—	IMPLEMENTED
Line-up	EventEditor / EventDetailPage	save_event_lineup, update_event_artist, remove_event_artist	notify_lineup_artist (only once published), event_artists_notify(_removed)	IMPLEMENTED
Request artist → accept → confirm	EventDetailPage	create_artist_request, respond_to_booking_request, manage_artist_request	see 10	IMPLEMENTED
Team / tasks / requirements / documents	EventOps.jsx, Team.jsx, Tasks.jsx	event_assignments, event_tasks, event_requirements, event_documents (+ event-documents bucket), sponsor_deliverables	notify triggers, validation triggers	IMPLEMENTED
Publish	status select in the editor	events.status draft → on-sale	events_notify_publish, public visibility via RLS	IMPLEMENTED
Sold out	status select	events.status = sold-out	Capacity enforcement is automatic in create_pending_booking; the status flag is manual. event_availability() reports sold_out from either flag or counts	PARTIALLY IMPLEMENTED (status not automatic)
Session content	Content → Sessions	update_session_content (description, story, image_url, tags, featured)	content.manage_sessions + content.edit	IMPLEMENTED
Event day	/check-in	check_in_ticket, event_checkin_stats	audit per check-in	IMPLEMENTED
Reminders	—	send_event_reminders (platform job)	members only (artists, assignees, venue and sponsor partners), N hours before start (notifications.event_reminder_hours, default 24). Ticket buyers do not get reminders. Customer reminders are NOT IMPLEMENTED	PARTIALLY IMPLEMENTED
Completion	Requests: manage_artist_request('complete') only after the event's local date	assignment_requests.completed_at	—	IMPLEMENTED; events.status = past is a manual flag

Stage	Frontend	Database	Side effects	Status
Archive	/sessions/archive, /sessions/archive/:slug, /archive/programmes	archiveService reads past events + linked content (event_id on gallery_albums, diary_posts, tv_videos; programme_events)	—	IMPLEMENTED
Cancel	status select	events.status = cancelled	notify_on_event_change: members notified; open requests cancelled. Bookings untouched	PARTIALLY IMPLEMENTED for customers
Delete	Events page	events_prevent_delete _with_bookings	refused when bookings exist	IMPLEMENTED

Event status state diagram

events.status (manual transitions, enforced side effects)

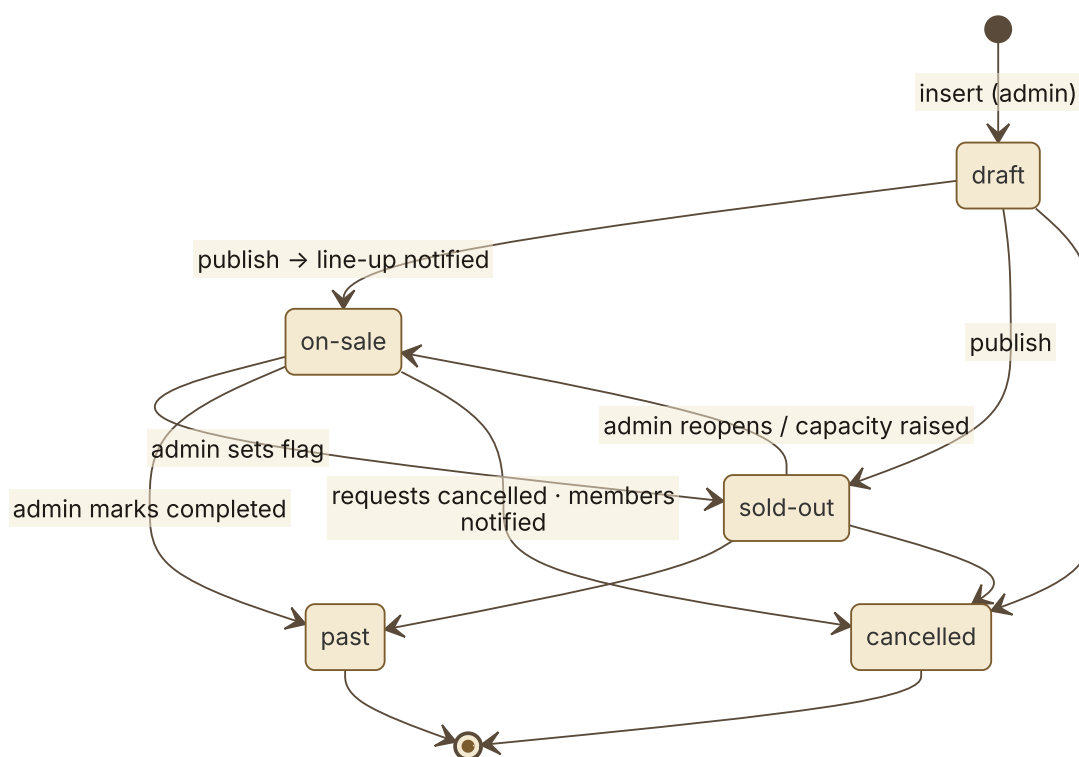


Diagram — events.status (manual transitions, enforced side effects)

Checkout accepts only `on-sale`. It also accepts `sold-out`, but only for the holder of a live waitlist offer (`razorpay-create-order`).

10 — Event ↔ Artist Workflow (Availability, Line-up, Booking Requests)

Source: migrations `0033_artist_portal.sql`, `0034_event_artist_workflow.sql` and the repository guide `docs/EVENT_WORKFLOW.md`. **Status: IMPLEMENTED.**

One source of truth for availability

`artist_day_status(artist, date, start, end, exclude_event)` (0034) is the single function that decides an artist's status for a date and optional time window. It combines three inputs:

- the artist's own calendar (`artist_availability`)
- confirmed line-ups (`event_artists` + `event_artist_details` times)
- pending requests (`assignment_requests` with status `pending`)

Computed status	Rule (checked in this order)
unavailable	Artist marked the day unavailable and the time windows overlap (or no window)
busy	A confirmed line-up slot on that date overlaps the window
tentative	Booked elsewhere that day (no overlap), or marked tentative, or a pending request exists, or available only for part of the window
available	Marked available covering the window
unknown ("No availability set")	Nothing recorded

A pending request **never counts as a booking** (busy). It only makes the day tentative. The artist stores only `available` / `tentative` / `unavailable` (`set_artist_availability`); `busy` and `unknown` are computed.

artist_day_status decision

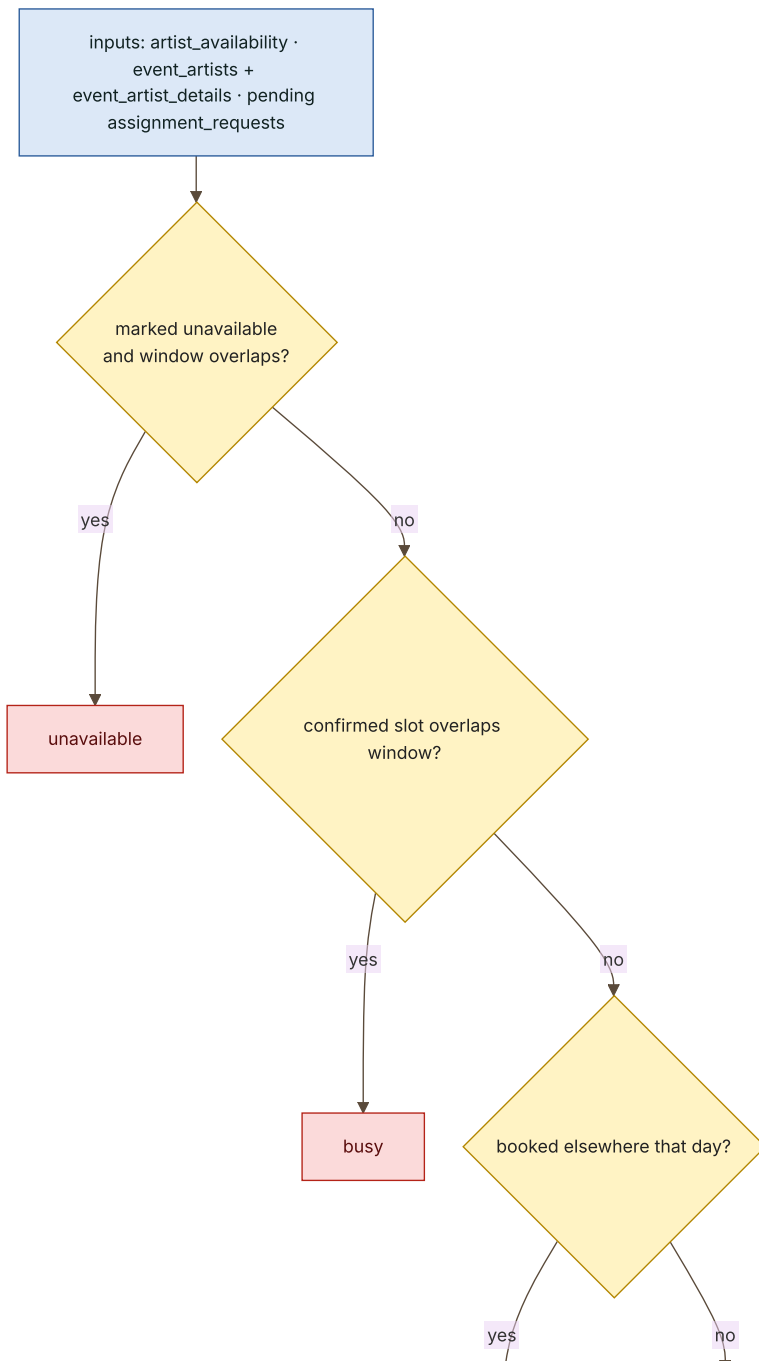


Diagram — `artist_day_status` decision (part 1 of 2)

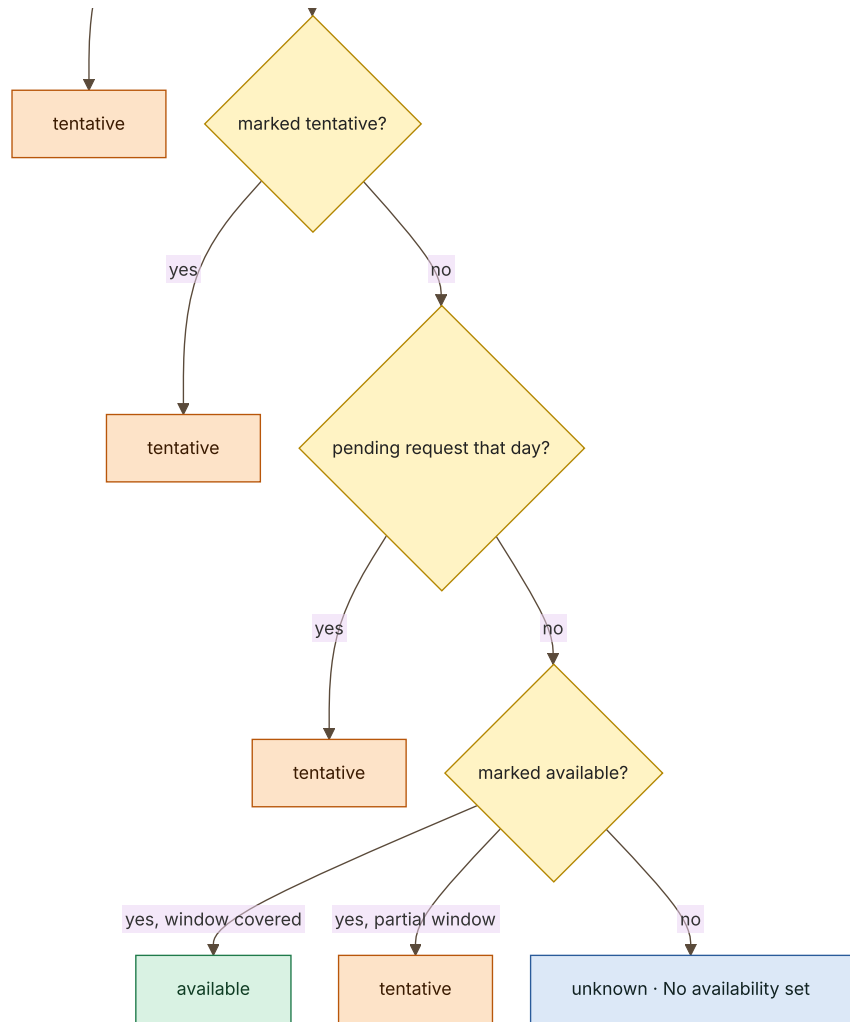


Diagram — artist_day_status decision (part 2 of 2)

The same data feeds:

- `find_available_artists()`: the admin's artist picker for an event date
- `artist_calendar()`: the shared calendar component used by the artist portal, the admin artist page and the event editor drawer
- `artist_availability_summary()`, `admin_calendar()`, `artist_day_summary()`
- `artist_schedule_check()`: pre-send check of other sessions that day

Artist availability workflow

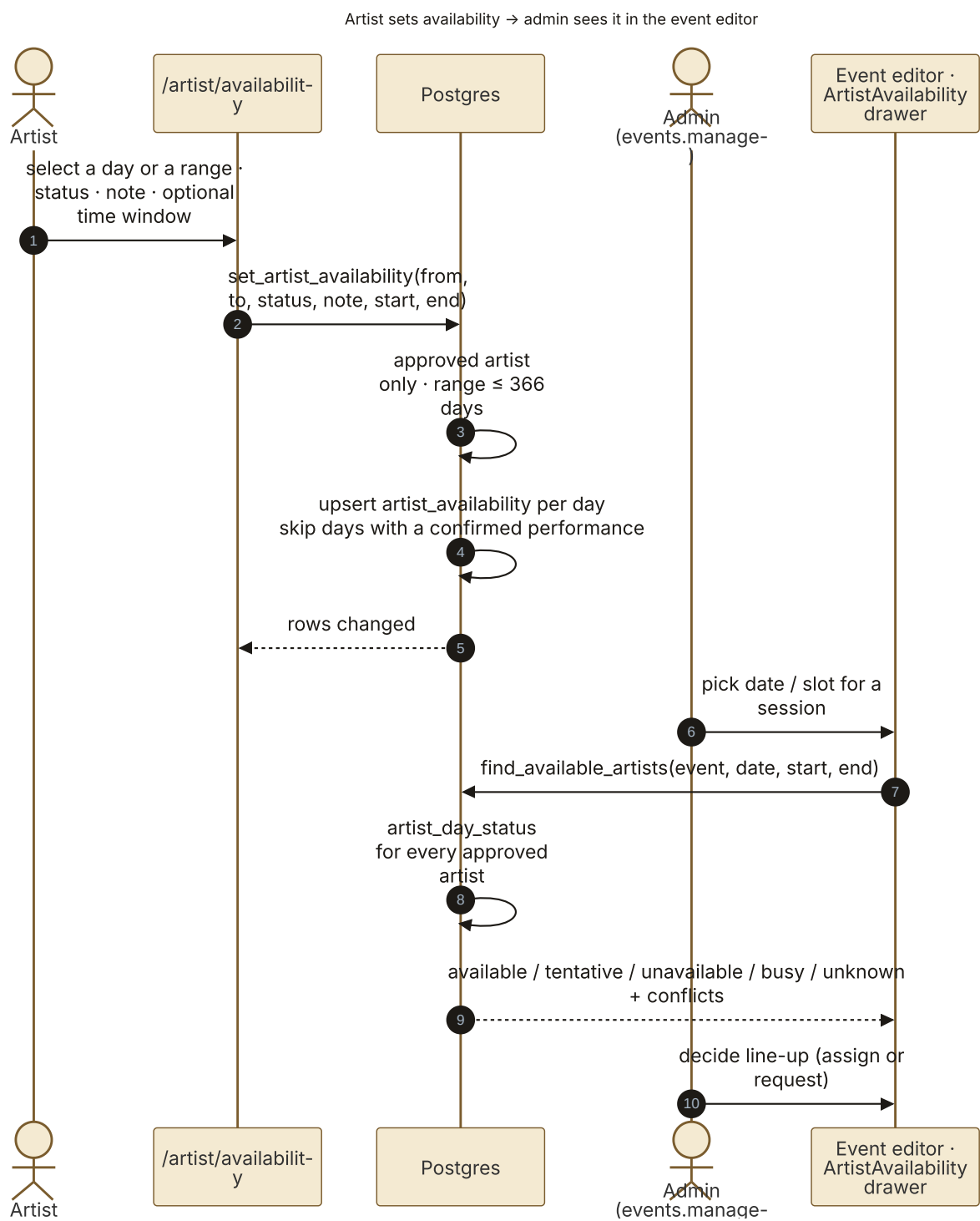


Diagram — Artist sets availability → admin sees it in the event editor

Event ↔ artist booking workflow

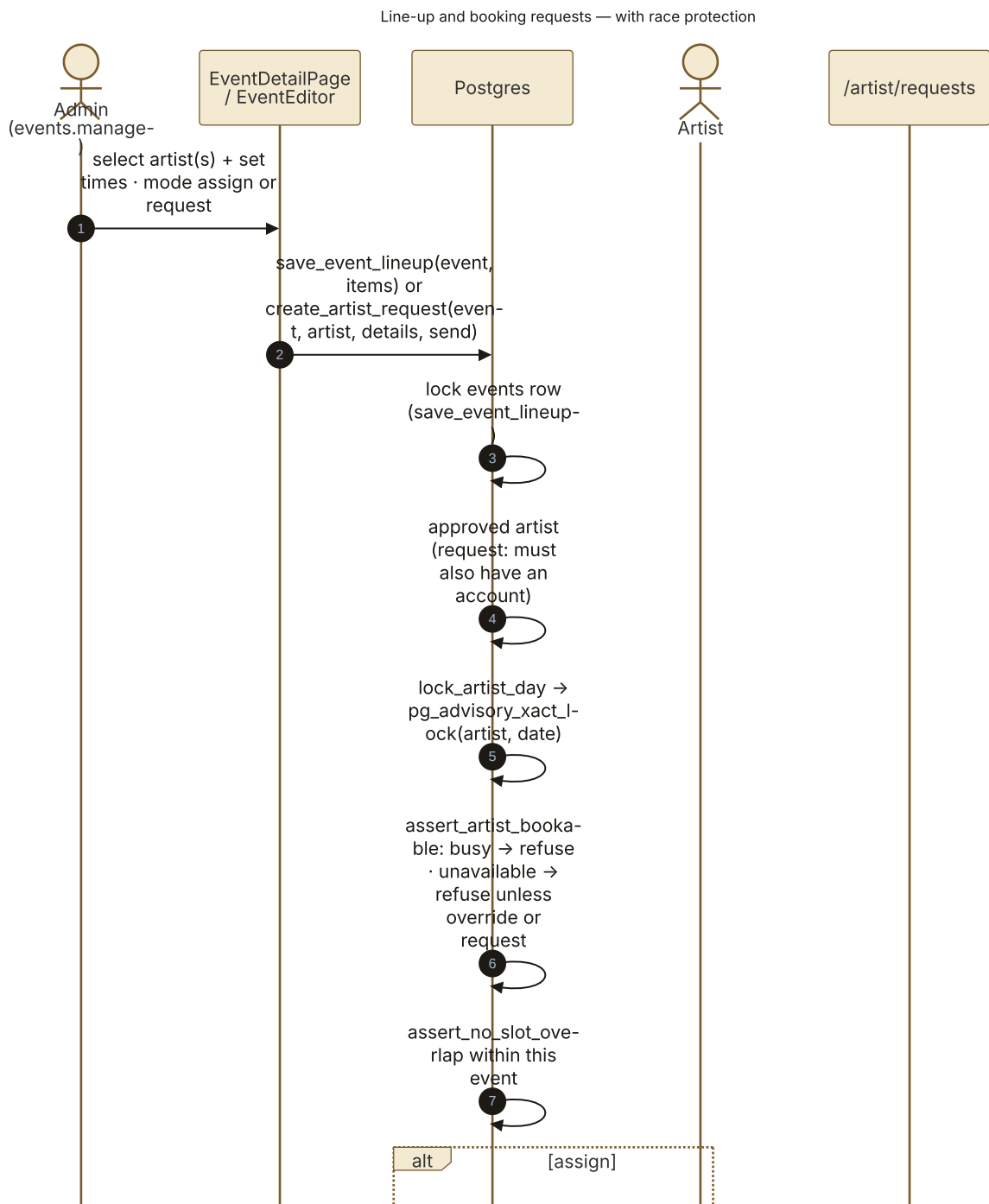


Diagram — Line-up and booking requests — with race protection (part 1 of 4)

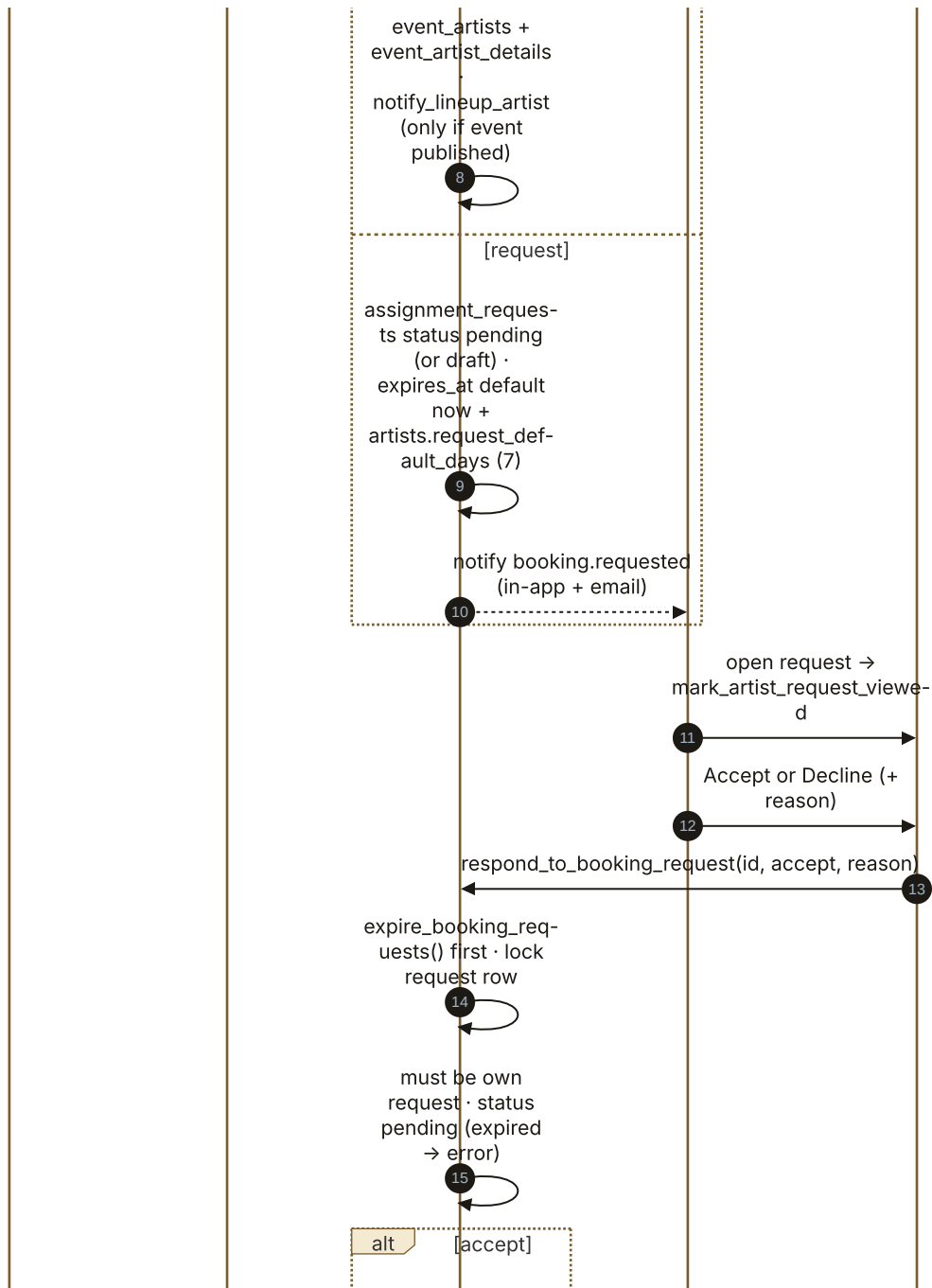


Diagram — Line-up and booking requests — with race protection (part 2 of 4)

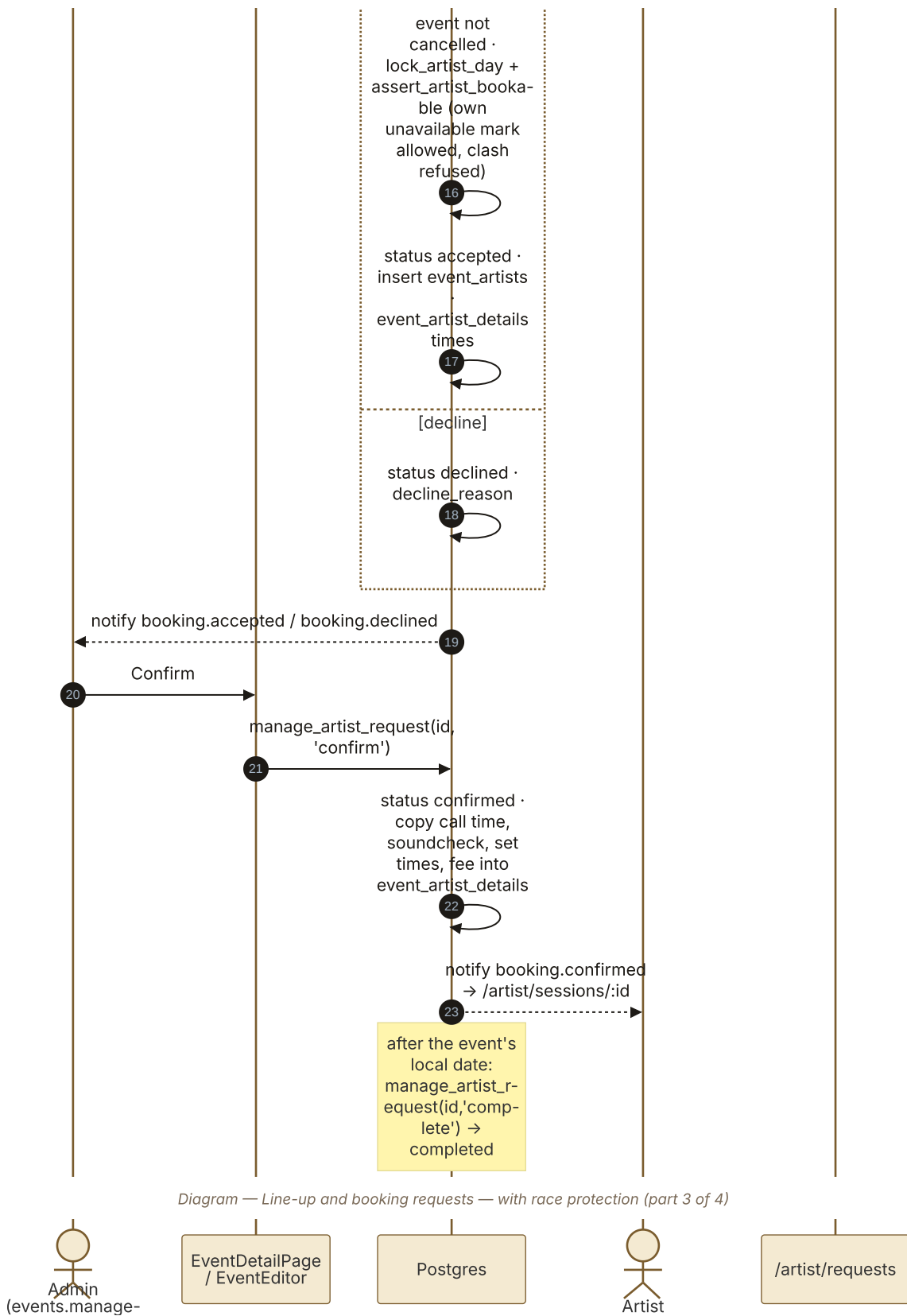


Diagram — Line-up and booking requests — with race protection (part 4 of 4)

Booking request states

assignment_requests.status (assignment_status enum)

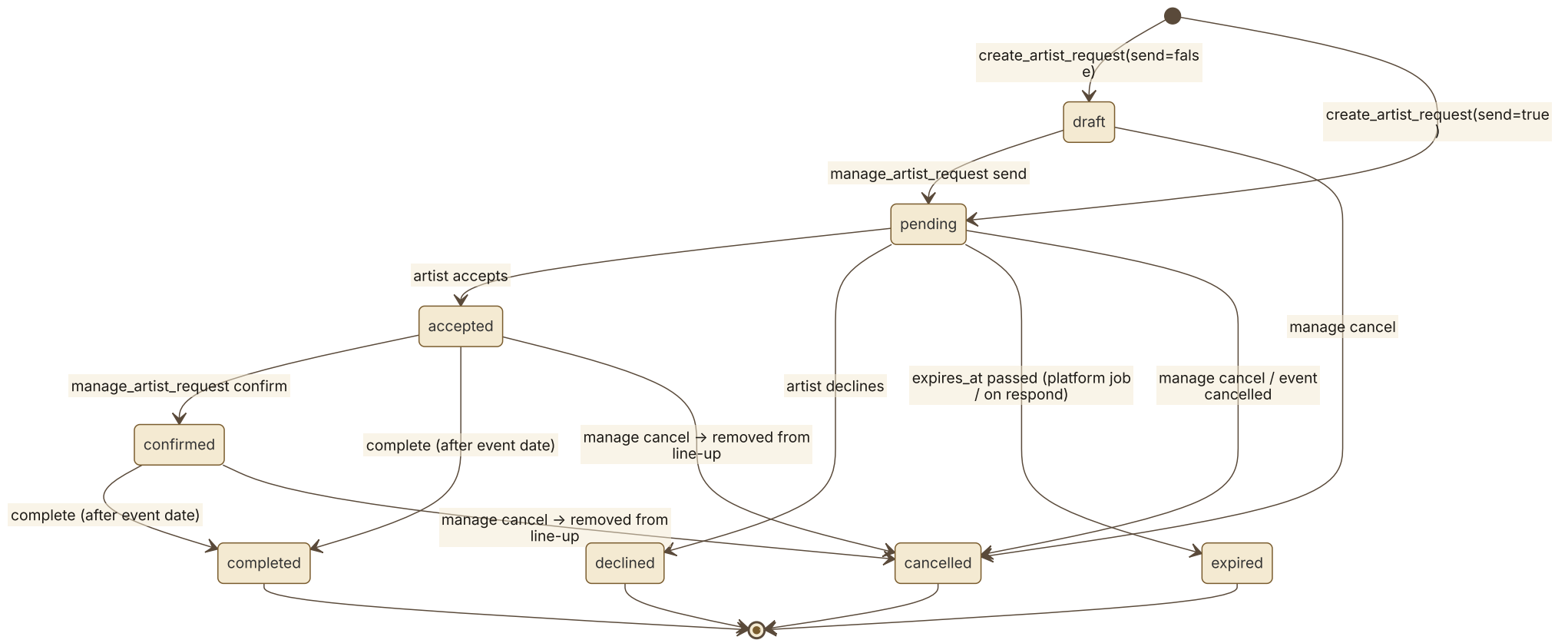


Diagram — assignment_requests.status (assignment_status enum)

Race-condition protection

Mechanism	Where	Protects against
<code>pg_advisory_xact_lock(hashtextextend('tangy.artist_day:'+artist+':'+date)) (lock_artist_day)</code>	<code>assert_artist_bookable</code> , used by <code>save_event_lineup</code> , <code>update_event_artist</code> , <code>create_artist_request</code> , <code>respond_to_booking_request</code>	Two admins (or an admin and the artist) double-booking the same artist on the same day
<code>select ... for update on events</code>	<code>save_event_lineup</code>	Concurrent line-up edits on one event
<code>for update</code> on the request row	<code>respond_to_booking_request</code> , <code>manage_artist_request</code>	Double answers
<code>assert_no_slot_overlap</code>	line-up / request	Overlapping sets inside one event
<code>guard_artist_availability</code> (0020)	<code>artist_availability</code> writes	Marking a day with a confirmed performance as unavailable

Legacy paths still in the code

- `create_assignment_request` / `respond_to_assignment_request` / `cancel_assignment_request` (0008) via `src/services/assignmentRequestService.js`: **LEGACY**. The service is not imported by any component. `respond_to_assignment_request` is still called *internally* by `respond_to_booking_request`.
- `create_booking_request` (0020) wrapper `adminApi.createBookingRequest`: **LEGACY / unused** by pages. `create_artist_request` (0033/0034) is the current path.

11 — Customer Booking Workflow

Status: **IMPLEMENTED** in code, verified locally by `e2e/checkout.mjs`, `e2e/groupcheckin.mjs` and `supabase/tests/pricing_settlement.test.sql`. Live payments need the Razorpay secrets (**PRODUCTION CONFIGURATION, not done**, `docs/OPERATIONS.md` §3).

Customer journey

Customer journey — session to check-in

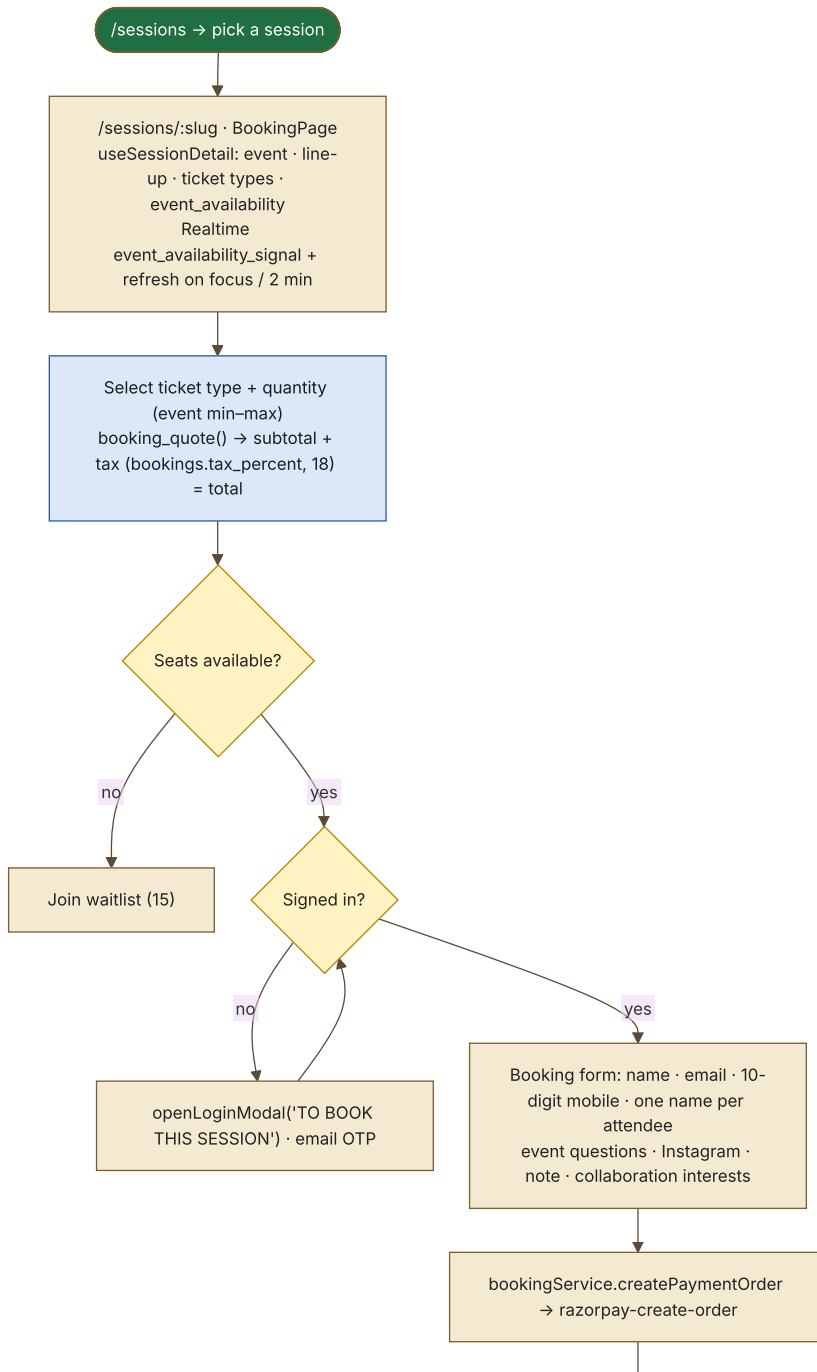


Diagram — Customer journey — session to check-in (part 1 of 3)

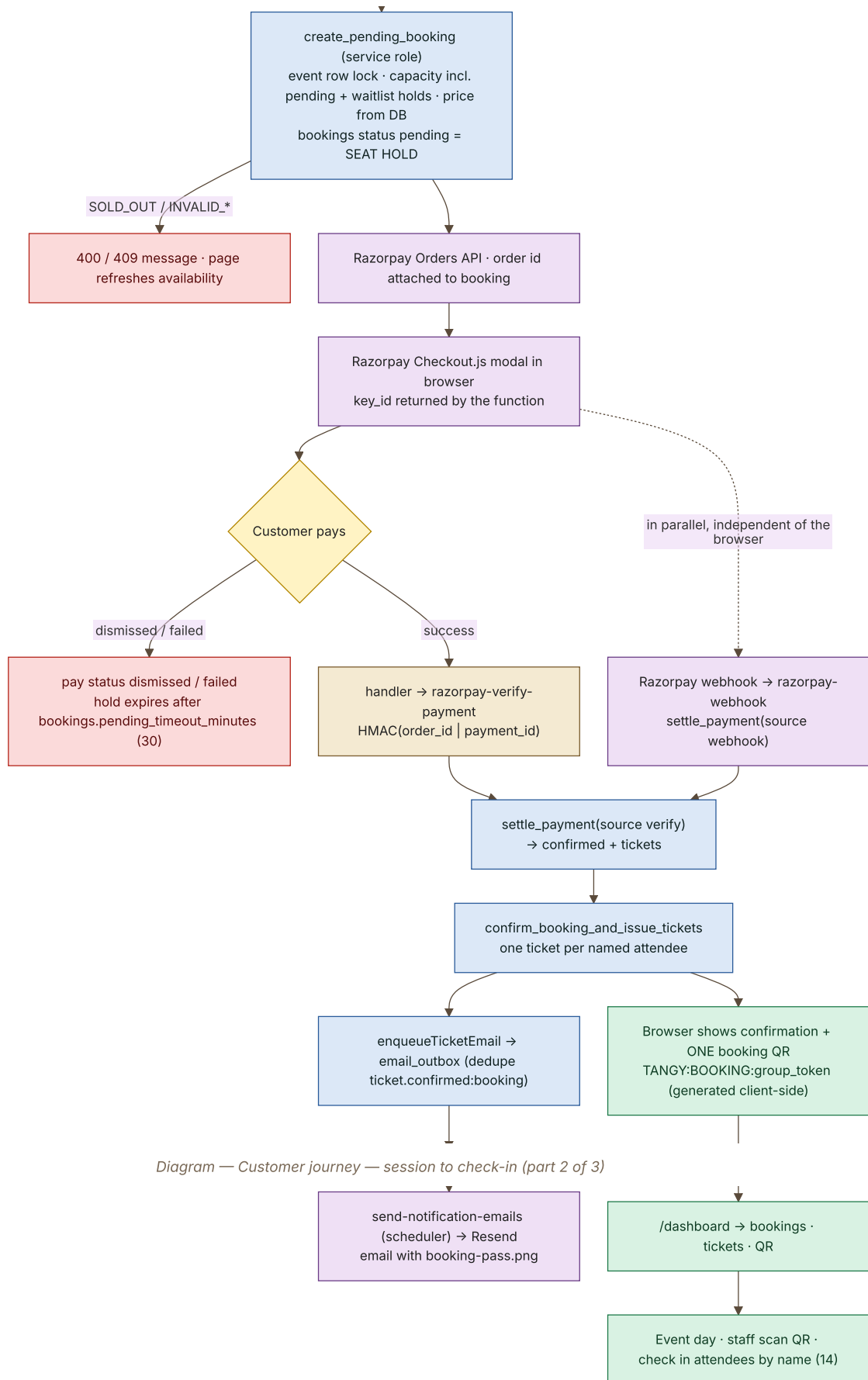


Diagram — Customer journey — session to check-in (part 2 of 3)

Diagram — Customer journey — session to check-in (part 3 of 3)

The two payment paths

	Browser verification path	Webhook path
Trigger	Razorpay Checkout <code>handler</code> callback in <code>BookingPage</code>	Razorpay → <code>POST /functions/v1/razorpay-webhook</code>
Function	<code>razorpay-verify-payment</code> (JWT required; caller must own the booking; order id must match)	<code>razorpay-webhook</code> (no JWT; X-Razorpay-Signature over the raw body)
Secret	<code>RAZORPAY_KEY_SECRET</code>	<code>RAZORPAY_WEBHOOK_SECRET</code>
Settles via	<code>settle_payment(order, payment, amount null, 'verify')</code>	<code>settle_payment(order, payment, amount_paise, 'webhook')</code> (amount checked)
Works if the browser closes	No	Yes
Source of truth	Fast path for the UI	Final source of truth (function header: <i>"this is the source of truth for payment status"</i>)

Both paths are **idempotent**:

- `settle_payment` returns `already_confirmed` for a second call.
- `confirm_booking_and_issue_tickets` issues tickets only if none exist.
- The ticket email is queued once per booking (unique `dedupe_key`).

What gets created

Step	Rows written
Order	<code>bookings</code> (status <code>pending</code> , <code>payment_status</code> default created, <code>registration_code</code> <code>TS-XXXXXXX</code> , <code>attendee_names[]</code> , <code>booking_answers</code> , <code>razorpay_order_id</code> , <code>group_token</code>)
Payment	<code>payment_webhook_events</code> (webhook path)
Settlement	<code>bookings.status = confirmed</code> , <code>payment_status = captured</code> , <code>razorpay_payment_id</code> , <code>razorpay_signature_verified = true</code> ; <code>tickets × quantity</code> (<code>ticket_number</code> <code>TS-...-01</code> , random 20-byte hex <code>token</code> , <code>attendee_name</code>)
Email	<code>email_outbox</code> row (<code>ticket.confirmed</code>) → later <code>bookings.ticket_email_status = sent / failed</code>
Audit	<code>audit_logs</code> (<code>booking.*</code> system audit via <code>bookings_system_audit</code> , <code>payment.*</code> on review / late)
Availability	<code>event_availability_signal</code> version bump (trigger) → Realtime → open session pages refresh
Waitlist	If the booking came from an offer: <code>waitlist.status = converted</code> , <code>booking_id</code> set

What the customer sees vs what the admin sees

	Customer	Admin
During checkout	Ticket type, live remaining seats, quote incl. tax, form validation, Razorpay modal	—
After payment	Confirmation, named attendees, one booking QR, email (when configured)	Bookings list: confirmed, payment captured, ticket email status
Payment problem	"We received your payment, but these seats are no longer available..." (409 <code>review</code>)	Notification <code>payment.review</code> to <code>payments.view</code> holders; booking <code>payment_status = needs_review</code> ; audit <code>payment.needs_review</code>
Later	<code>/dashboard</code> bookings + tickets + QR; passport	Cancel, record refund, cancel ticket, resend email, comp booking

Not implemented for customers

- **Self-service cancellation or refund: NOT IMPLEMENTED.** Only `admin_cancel_booking` / `admin_record_refund`. Refunds themselves are issued **manually in the Razorpay dashboard** (webhook mirrors `refund.processed` / `refund.created` amounts).
- **Reminder emails to ticket holders: NOT IMPLEMENTED** (`send_event_reminders` targets event members only).
- **Automatic notification when an event is cancelled: NOT IMPLEMENTED** for ticket holders.

12 — Payment Architecture (Razorpay)

Code: IMPLEMENTED. Production: PRODUCTION CONFIGURATION, NOT DONE. Still to do: Razorpay keys and webhook secret as Edge Function secrets, the webhook URL configured in the Razorpay dashboard, and test-mode verification (docs/OPERATIONS.md §3, §3a).

Components

Component	File	Notes
Checkout UI	src/pages/BookingPage.jsx	Loads https://checkout.razorpay.com/v1/checkout.js on demand; retries with the same details reuse the same order (orderRef fingerprint)
Client service	src/lib/bookingService.js	createPaymentOrder , verifyPayment , sendTicketEmail (all functions.invoke)
Order creation	supabase/functions/razorpay-create-order	JWT check → validation → create_pending_booking (service role) → Razorpay Orders API
Checkout verification	supabase/functions/razorpay-verify-payment	JWT + ownership + HMAC → settle_payment('verify')
Webhook	supabase/functions/razorpay-webhook	HMAC on raw body → dedupe in payment_webhook_events → settle_payment('webhook') / failure / refund mirror
Pricing	booking_quote() (0026)	Ticket price × quantity + tax at bookings.tax_percent (default 18)
Seat hold	create_pending_booking() (latest 0027)	Event row lock; capacity = confirmed + pending + live waitlist holds
Settlement	settle_payment() (latest 0027)	The only place a paid booking is confirmed
Hold expiry	expire_stale_bookings() (0020)	Pending > bookings.pending_timeout_minutes (default 30) → expired
Crypto	_shared/crypto.ts	hmacSha256Hex , timingSafeEqual , requireSecret (≥ 8 chars, never "undefined")

End-to-end architecture

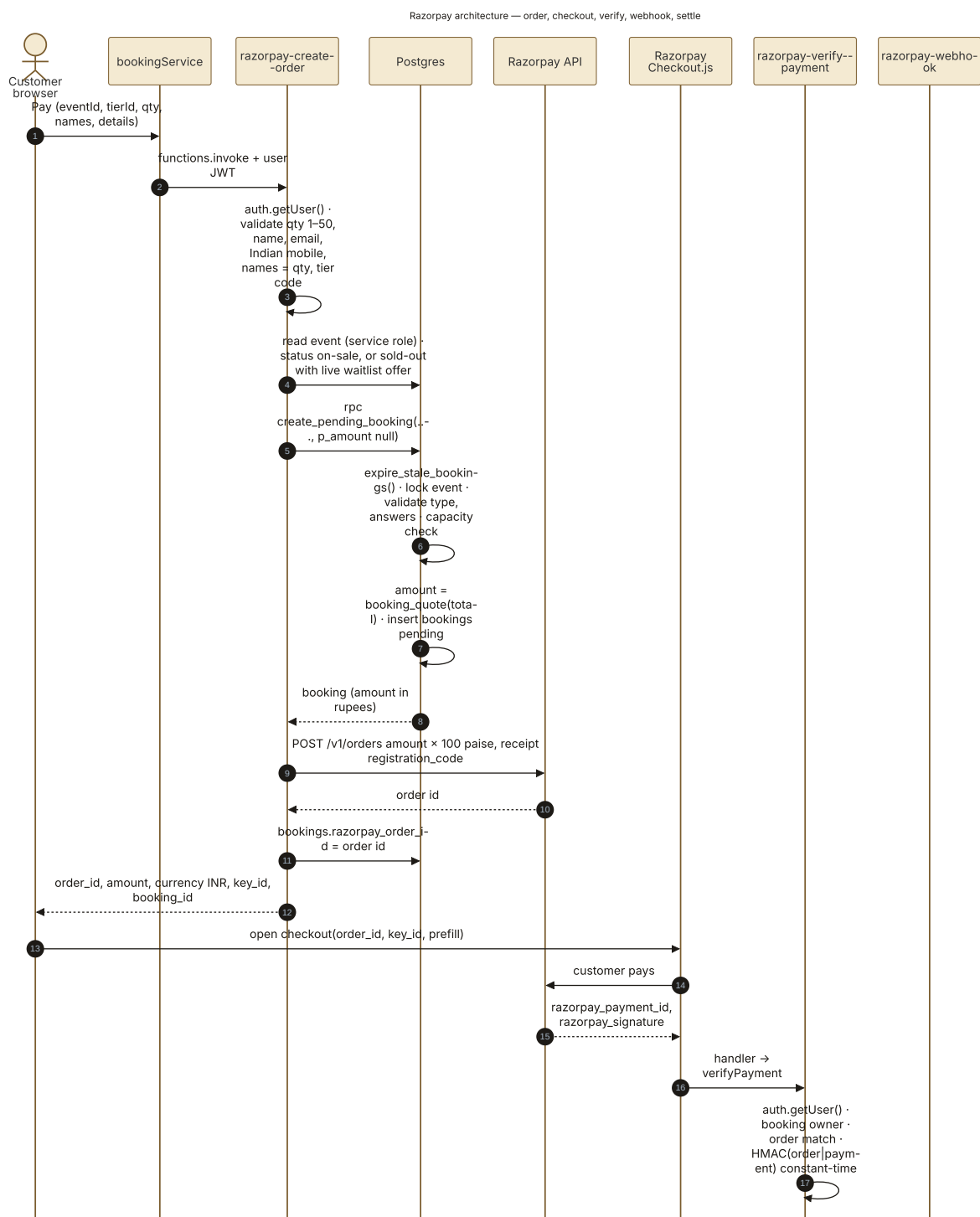


Diagram — Razorpay architecture — order, checkout, verify, webhook, settle (part 1 of 2)

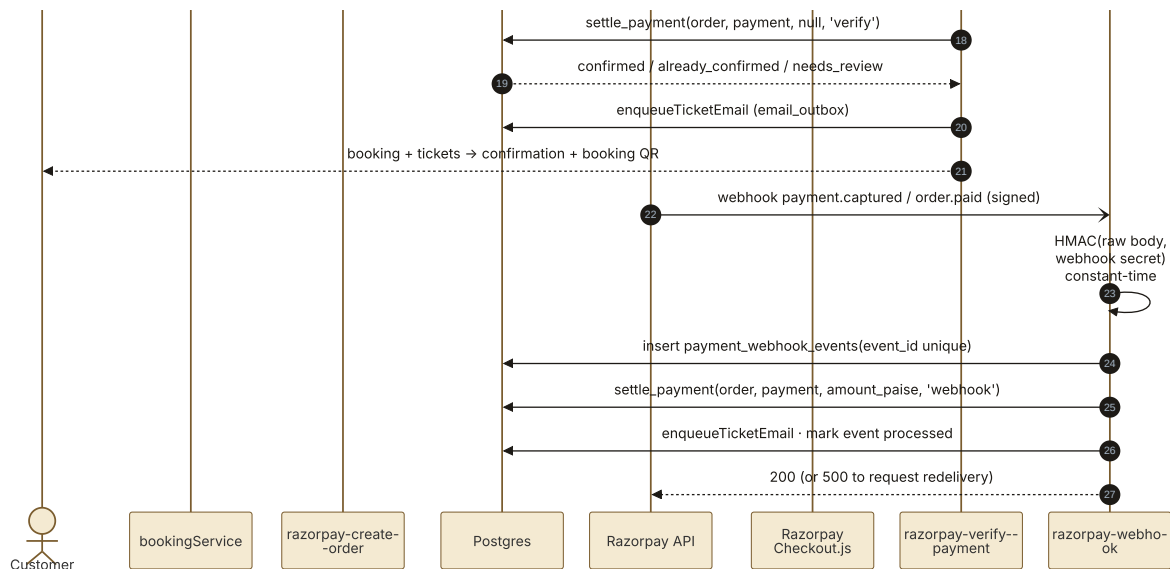


Diagram — Razorpay architecture — order, checkout, verify, webhook, settle (part 2 of 2)

`settle_payment` is the **final source of truth**, because the webhook runs whether or not the browser completes. Both paths converge on it, under row locks on the event and the booking.

settle_payment decision tree

settle_payment(order, payment, amount, source)

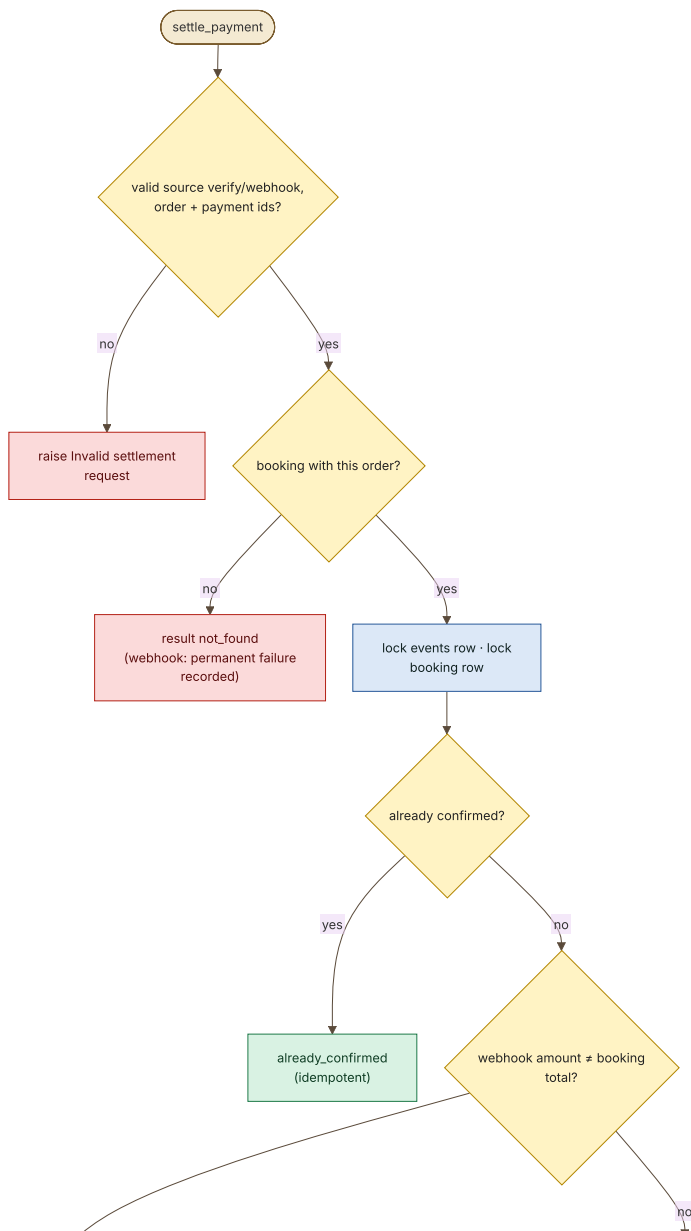


Diagram — `settle_payment(order, payment, amount, source)` (part 1 of 2)

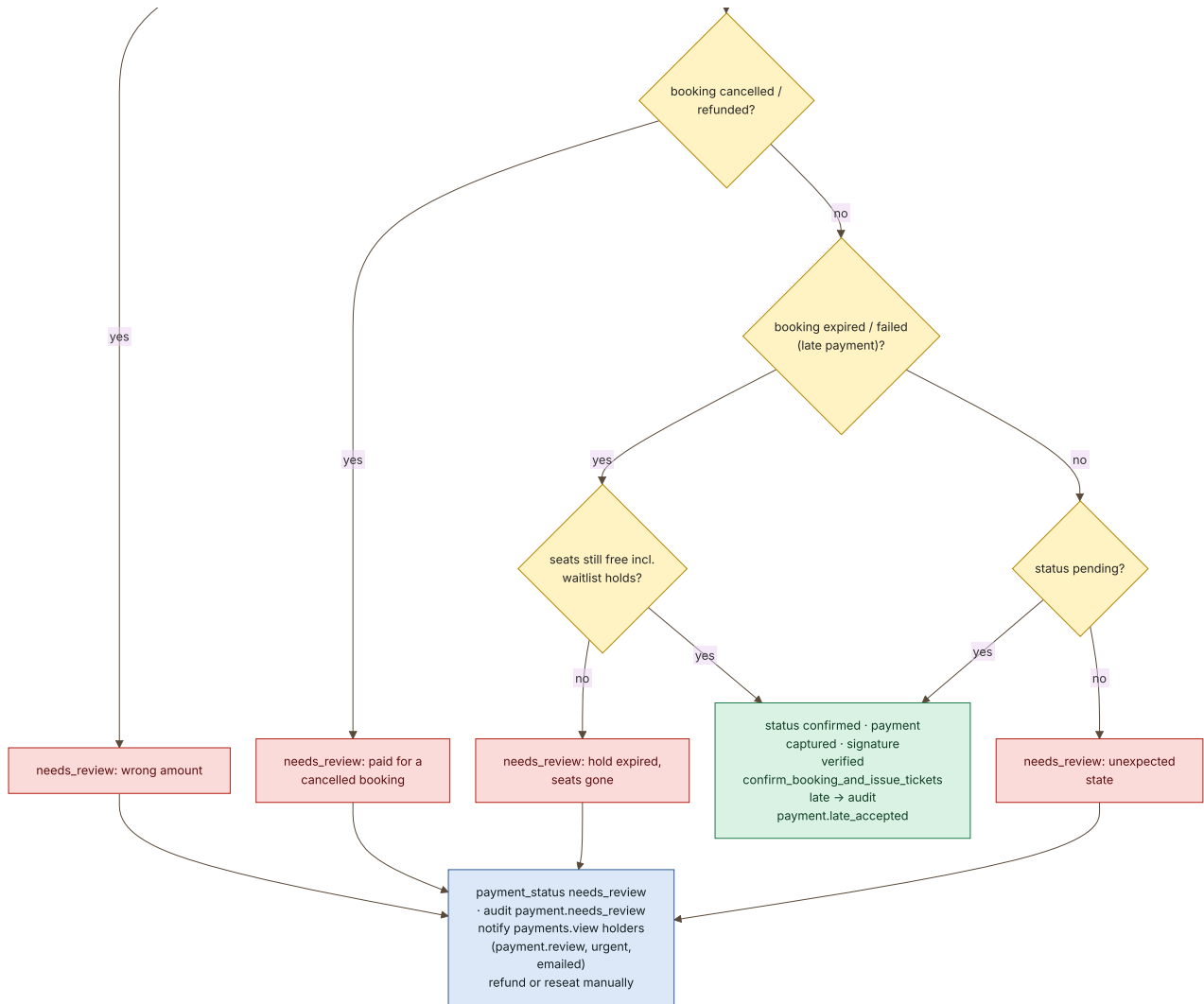


Diagram — `settle_payment(order, payment, amount, source)` (part 2 of 2)

Special cases (all supported by code)

Case	What happens	Where
Failed payment	Checkout <code>payment.failed</code> → UI error. Webhook <code>payment.failed</code> → booking <code>failed</code> / <code>payment_status failed</code> only while still pending (never overwrites confirmed). Seats are released, and the <code>waitlist_on_booking_release</code> trigger offers them onwards	<code>BookingPage</code> , <code>razorpay-webhook</code>
Payment succeeds, browser closes	The webhook settles and queues the ticket email; the customer sees the booking in <code>/dashboard</code>	<code>razorpay-webhook</code>
Duplicate webhook	<code>payment_webhook_events.event_id (<event>:<payment id>)</code> is unique. A duplicate of a processed event → <code>200 ok (duplicate)</code> . A duplicate of a recorded-but-unfinished event is processed now	<code>razorpay-webhook</code>
Invalid signature	<code>400 Invalid signature</code> , nothing recorded	<code>razorpay-webhook</code> , <code>razorpay-verify-payment</code>
Missing secret	<code>create-order: 503</code> and the hold is released (<code>status failed</code>). <code>verify: 503</code> . webhook: <code>503</code>	functions
Razorpay order API fails	<code>502</code> ; the booking is marked <code>failed</code> so the seats are freed	<code>razorpay-create-order</code>
Late payment (paid after the 30-min hold expired)	Accepted if seats are still free (audit <code>payment.late_accepted</code>), else <code>needs_review</code> . A second trigger <code>flag_late_payment</code> audits and alerts on any expired → confirmed change	<code>settle_payment</code> , <code>flag_late_payment</code>
Wrong amount	Webhook amount (paise) ≠ booking total → <code>needs_review</code>	<code>settle_payment</code>

Case	What happens	Where
Refund	Issued manually in the Razorpay dashboard. Webhook <code>refund.processed</code> / <code>refund.created</code> mirrors <code>refunded_amount</code> and <code>payment_status</code> <code>refunded</code> / <code>partially_refunded</code> . The admin records the refund (<code>admin_record_refund</code> with the Razorpay reference), which sets the booking <code>refunded</code> and cancels tickets	<code>razorpay-webhook</code> , 0017
Partial refund	Mirrored as <code>payment_status = partially_refunded</code> (amount refunded < total)	<code>razorpay-webhook</code>
Retry / redelivery	A transient DB error returns <code>500</code> so Razorpay retries. A permanent failure is stored (<code>record_webhook_failure</code> → <code>processing_error</code> + <code>payment.webhook_failed</code> alert) and acknowledged with <code>200</code>	<code>razorpay-webhook</code>
Idempotency	settle (<code>already_confirmed</code>), ticket issuance (exists check), ticket email (dedupe key), webhook events (unique id)	—
Hold expiry	<code>expire_stale_bookings</code> (every 5 min via <code>run_platform_jobs</code> , and before every new checkout): pending, unpaid, older than 30 min → <code>expired</code> , customer notified (<code>booking_payment.expired</code> , in-app), audit <code>booking.expired</code> , seats → waitlist	0020
<code>payment.authorized</code>	Status mirror only (<code>payment_status</code> <code>authorized</code>)	<code>razorpay-webhook</code>
Manual capture accounts	DOCUMENTED BUT NOT VERIFIED IN CODE. The code never calls Razorpay's capture API; the runbook requires automatic capture	<code>docs/OPERATION S.md</code> §3a

Payment status values

Column	Values
<code>bookings.status</code> (enum <code>booking_status</code>)	<code>pending</code> , <code>confirmed</code> , <code>cancelled</code> , <code>refunded</code> , <code>expired</code> , <code>failed</code>
<code>bookings.payment_status</code>	<code>created</code> , <code>authorized</code> , <code>captured</code> , <code>failed</code> , <code>refunded</code> , <code>partially_refunded</code> , <code>not_required</code> , <code>needs_review</code> (constraint 0026)
<code>bookings.source</code>	<code>online</code> (checkout), <code>complimentary</code> (admin comp booking)

13 — Ticket Architecture

Status: **IMPLEMENTED** (0016 tickets, 0017 fixes, 0023 named attendees + group QR).

Model

- A **booking** holds the buyer, the quantity and the named attendees (`attendee_names[]`, collected *before* payment). It also holds an opaque `group_token` : 20 random bytes, hex, default value set in 0023.
- **Tickets** exist only after settlement: one `tickets` row per attendee. Each has a `ticket_number` (`<registration_code>-01, -02 ...`), its own random 20-byte hex `token`, a tier, an `attendee_name` and `status` `valid` / `checked_in` / `cancelled`.
- **Attendance** is one `checkins` row per ticket (`unique(ticket_id)`). It records who checked the ticket in, when, how (`qr` / `manual`), notes, and a `batch_id` for group check-ins.

Ticket pipeline

Booking → settlement → tickets → QR → email → check-in

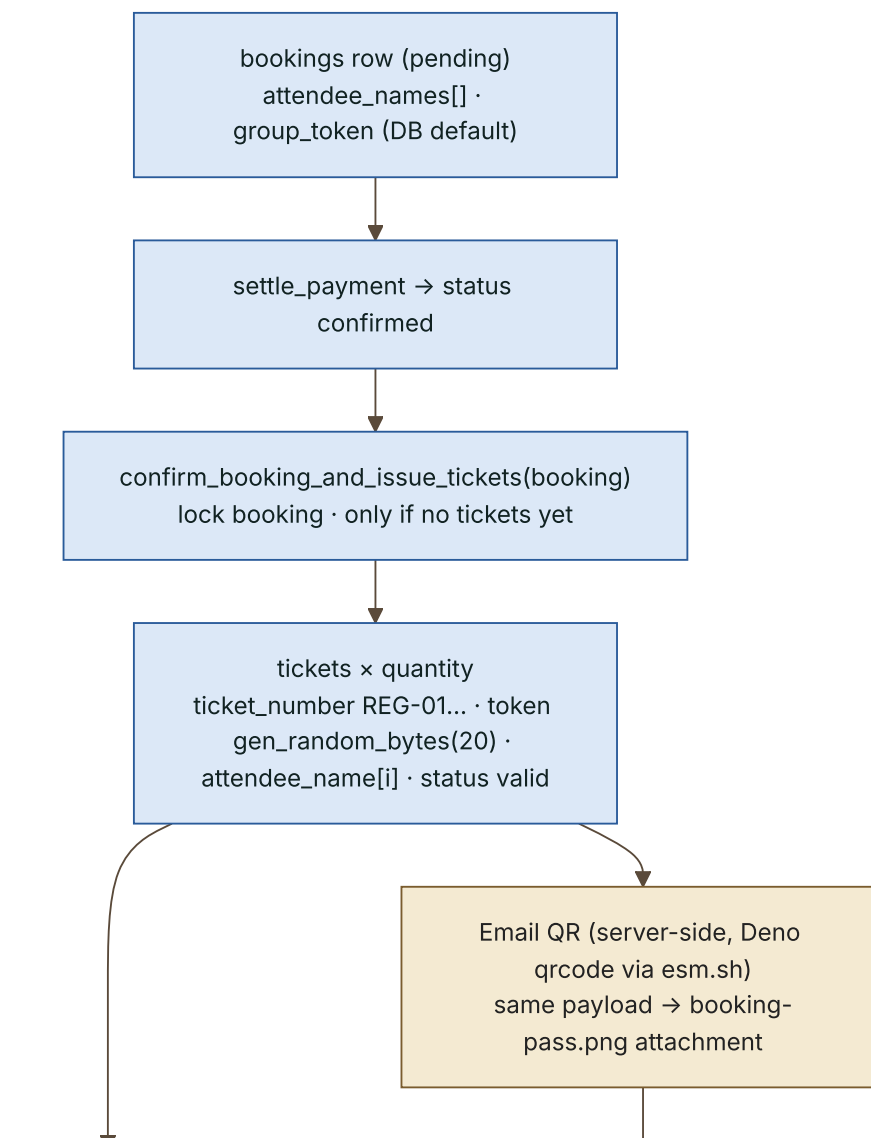


Diagram — Booking → settlement → tickets → QR → email → check-in (part 1 of 2)

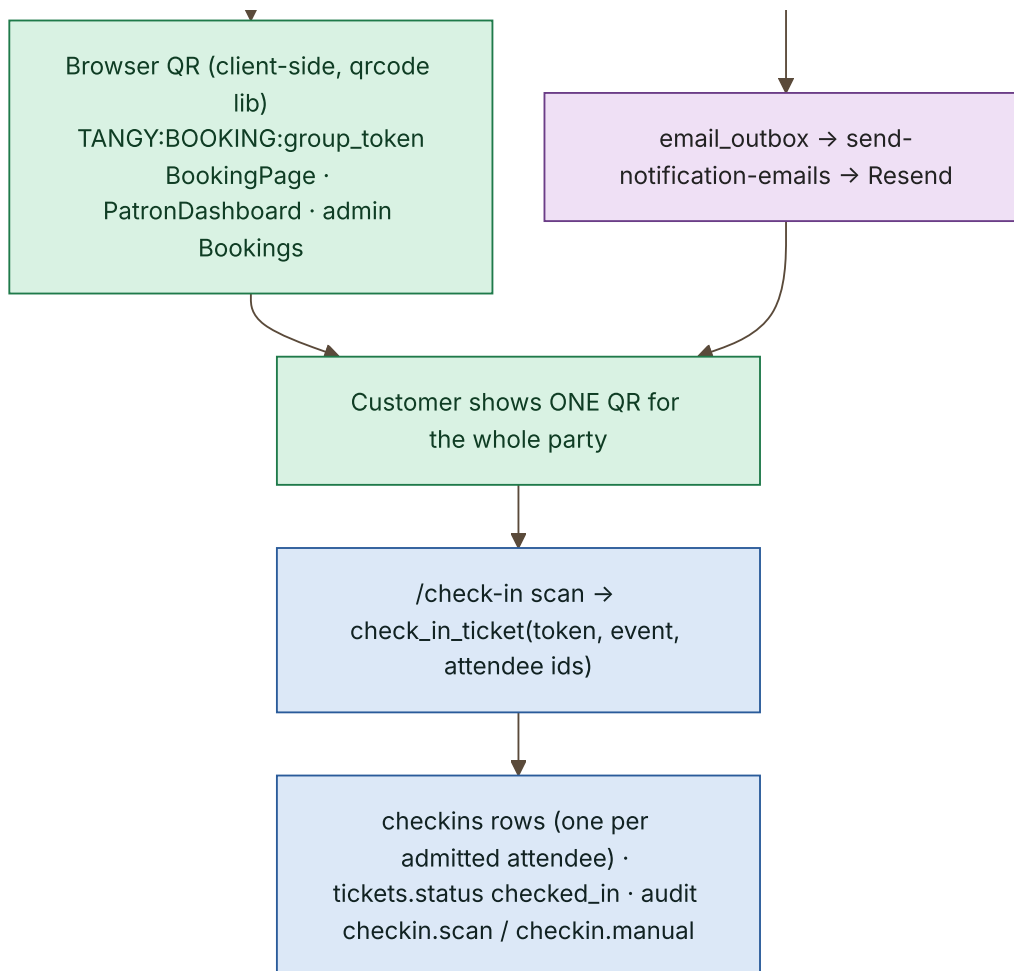


Diagram — Booking → settlement → tickets → QR → email → check-in (part 2 of 2)

Where QR codes are generated, and what is inside

Payload	Generated where	Contains	Used for
TANGY:BOOKING: <group_token>	Client-side with <code>src/lib/qr.js</code> (<code>qrcode.toDataURL</code>) in <code>BookingPage</code> (after confirmation), <code>PatronDashboard</code> , <code>admin Bookings.jsx</code> ; server-side in <code>_shared/ticketEmail.ts</code> (email attachment)	Only the opaque booking token: no ids, names or contact details	Group check-in: staff see the attendee list and tick who is present
TANGY:TICKET: <token>	Still accepted by <code>check_in_ticket</code> (issued per ticket since 0016)	Opaque ticket token	Single-attendee check-in (legacy-compatible)
Bare token typed by hand	—	Either token	Server resolves which kind it is

No QR exists before payment is confirmed. `finalizeConfirmedBooking` only renders it from a server-confirmed booking.

Where ticket data comes from

Consumer	Source
Customer dashboard	<code>bookings</code> + embedded <code>tickets(*)</code> + <code>events(...)</code> (RLS: own rows) via <code>bookingService.getMyBookings</code>
Confirmation screen	<code>razorpay-verify-payment</code> response (<code>booking</code> , <code>tickets</code>)
Ticket email	<code>buildTicketEmail</code> reads <code>tickets</code> , <code>events</code> , <code>profiles.email</code> with the service role; the recipient is the account email , never the typed attendee email
Admin attendees list / CSV	view <code>attendee_tickets</code> (ticket + booking + event + check-in + payment status)
Scanner	<code>check_in_ticket</code> / <code>booking_attendees_state</code> responses

Ticket states

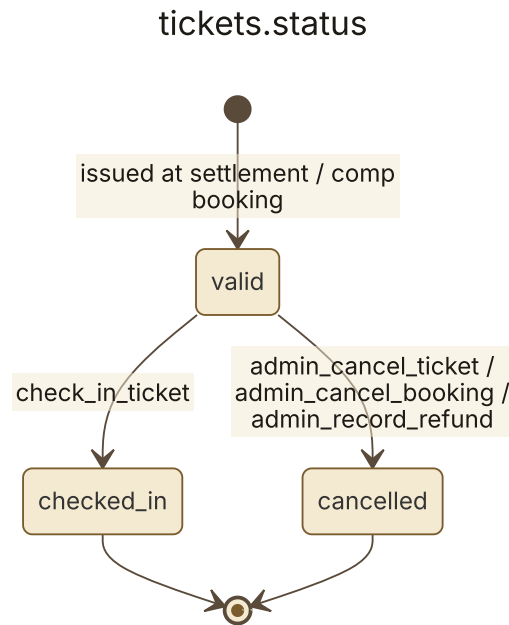


Diagram — tickets.status

`admin_cancel_booking` refuses when any ticket on the booking is already `checked_in`.

14 — Check-in Workflow

Status: IMPLEMENTED. `src/admin/TangyWorldCheckInPage.jsx` (route `/check-in`), `src/lib/checkinService.js`, RPC `check_in_ticket` (latest in 0023), `can_checkin_event` / `has_active_access` (0018). Verified locally by `e2e/groupcheckin.mjs`, `e2e/staff.mjs` (mobile QR via a fake camera, `e2e/make-cam.mjs`) and `supabase/tests/named_group_checkin.test.sql`.

Who can check in

Who	How the right is granted	Scope
Super Admin, Admin	<code>checkin.perform</code> + <code>events.view_all</code>	every event
Staff	<code>checkin.perform</code> + <code>is_assigned_to_event(event)</code>	assigned events only (otherwise <code>not_assigned</code>)
Volunteer	live <code>temporary_access</code> row (<code>has_active_access</code>), added to <code>my_permissions()</code> as <code>checkin.perform</code>	that event, for the granted window (30 min – 12 h)
Manual (typed code)	allowed unless setting <code>checkin.allow_manual</code> is false	—

Check-in flow

Check-in part 1 — sign-in, scan and authorization

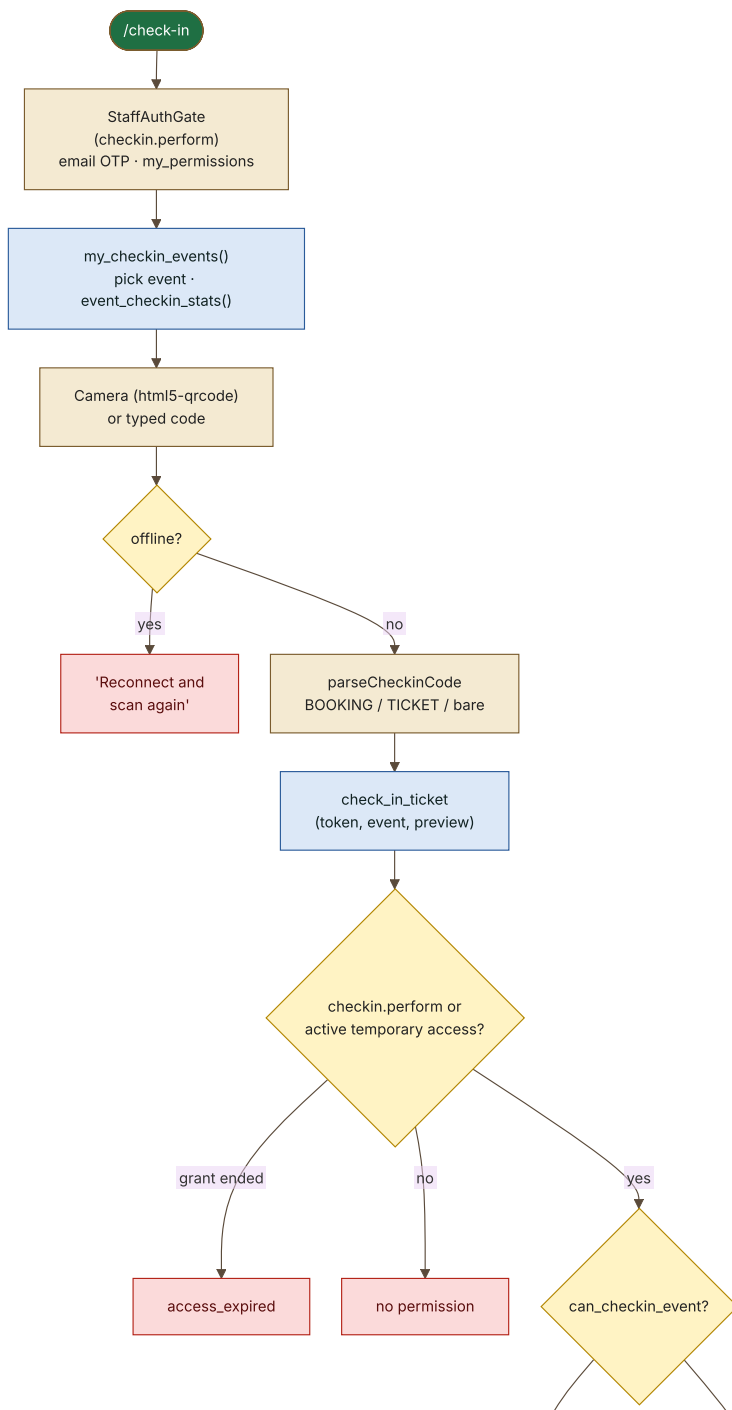


Diagram — Check-in part 1 — sign-in, scan and authorization (part 1 of 2)

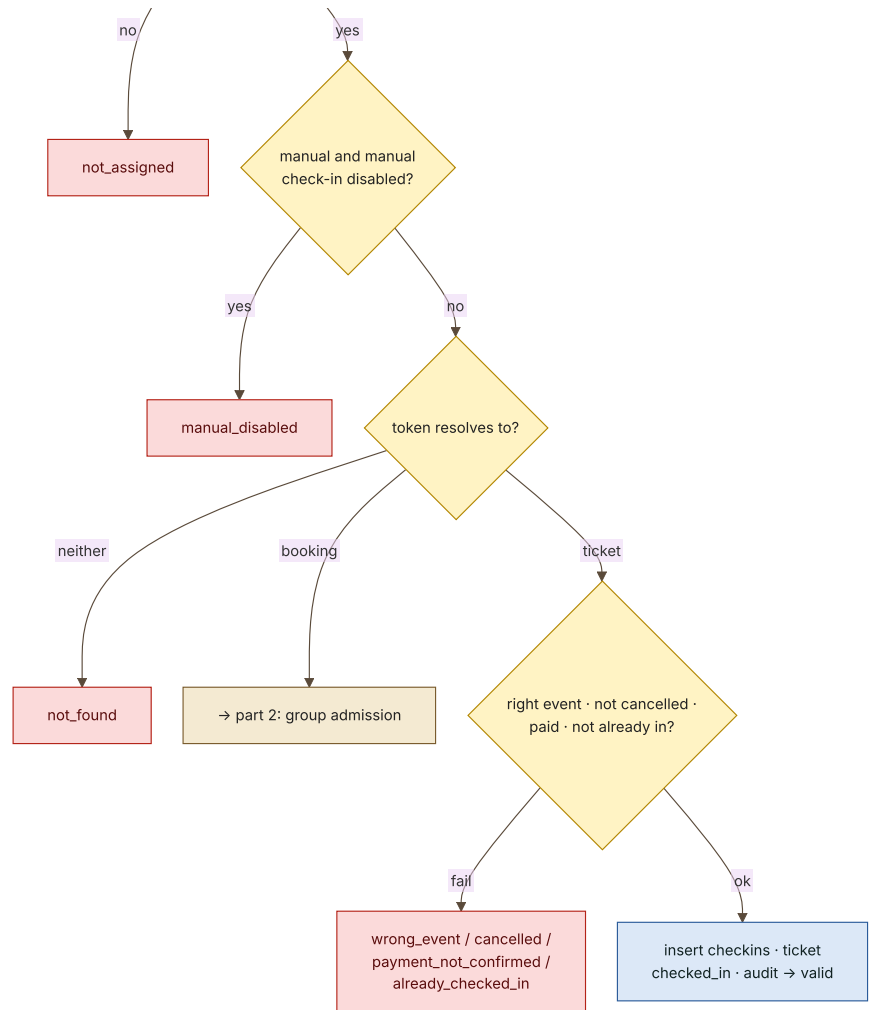


Diagram — Check-in part 1 — sign-in, scan and authorization (part 2 of 2)

Check-in part 2 — group admission by name

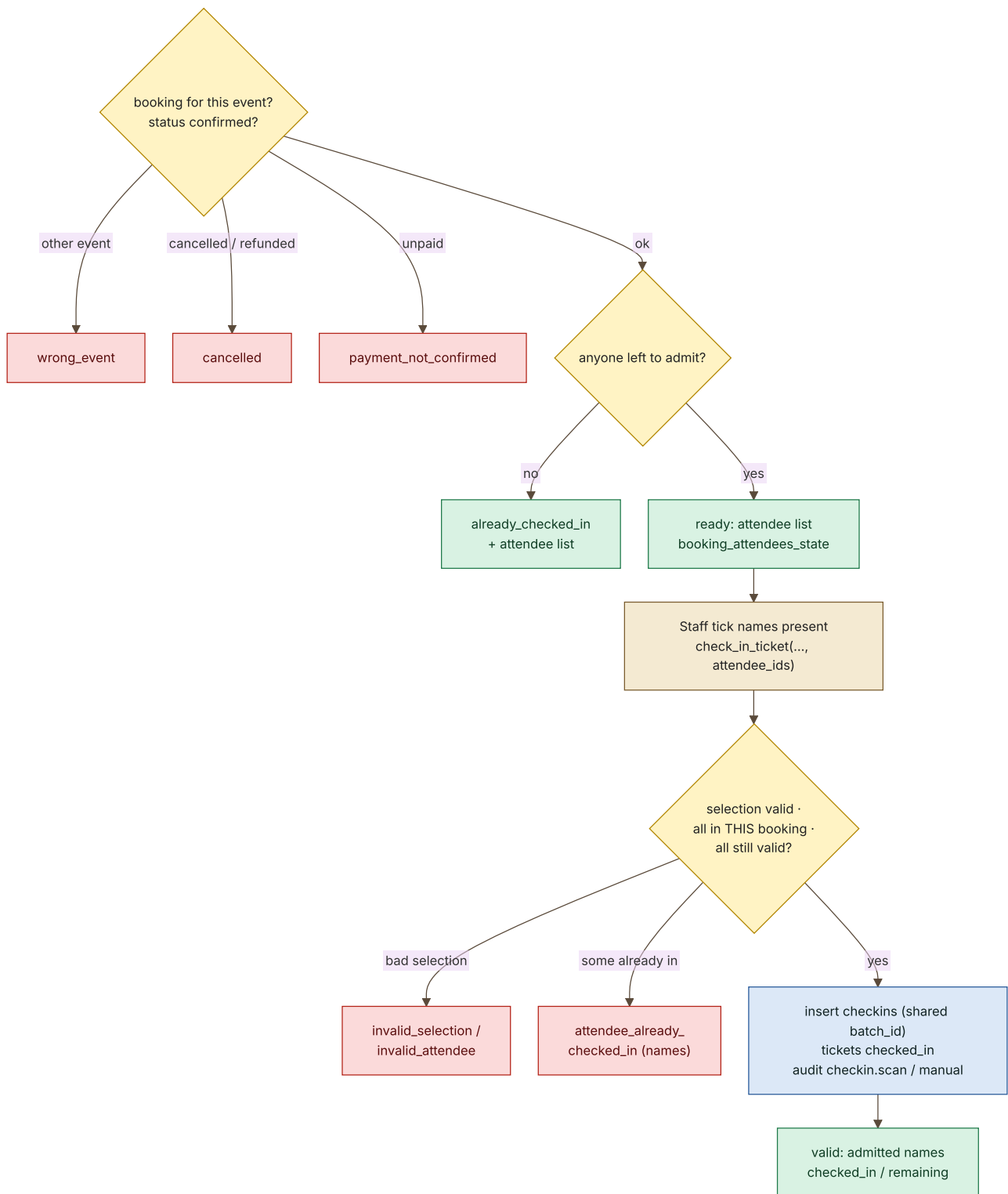


Diagram — Check-in part 2 — group admission by name

The browser never decides validity, counts or writes `checkins`. It only renders the `result` the RPC returns (comment at the top of `checkinService.js`).

Concurrency

- `check_in_ticket` locks the ticket (for update), the booking (for update) and exactly the selected attendee tickets (for update).
- A forged or foreign attendee id is missing from the locked set, so the whole request is refused.

- `checkins` has `unique(ticket_id)`, so a double scan cannot create two rows.

Group check-in

One QR per booking (0023):

1. Scanning shows every named attendee with their state.
2. Staff admit the people present now. Each admission is one atomic batch (`batch_id`).
3. Late arrivals are admitted later from the same QR (partial check-in).

`booking_checkin_history` returns the batches for the admin booking drawer.

History and audit

- `get_checkin_history(event, mine, limit, offset)` feeds `/admin-portal/check-ins` (`checkin.history`).
- Every admission writes `audit_logs` with `via = role` or `via = temporary_access`.
- Expired volunteer grants are logged by `log_expired_access()` (platform job) as `access.expired`.

15 — Waitlist Workflow

Status: IMPLEMENTED (0027 server-authoritative waitlist, 0029 allocation policy). Frontend:

`src/components/booking/WaitlistPanel.jsx` (session page), `WaitlistDirectory.jsx` (`/sessions/waitlist`), Patron dashboard waitlist tab, admin `WaitlistSection`. Tests: `supabase/tests/waitlist.test.sql`, `e2e/waitlist.mjs`, `scripts/test-concurrency.sh` (offer race).

Rules (from the 0027 header)

- Join only when signed in, and only when the session is sold out. If seats are actually free, `join_waitlist` refuses with `SEATS_AVAILABLE` and tells the person to book directly.
- Order is by `queue_no`. Allocation setting `waitlist.allocation`:
- `strict_order` (default): never skip the head of the queue
- `first_fit`: offer to the earliest party that fits
- Seats held by live offers count against capacity **everywhere**: `create_pending_booking`, `event_availability`, `settle_payment`. A released seat can only be claimed by the person it was offered to.
- Every offer decision runs under the **event row lock**, the same lock checkout takes, so two people can never be offered the same seat.
- Legacy anonymous rows (no account) are listed for the team but never auto-offered.

Workflow

Waitlist — join, offer, convert, expire

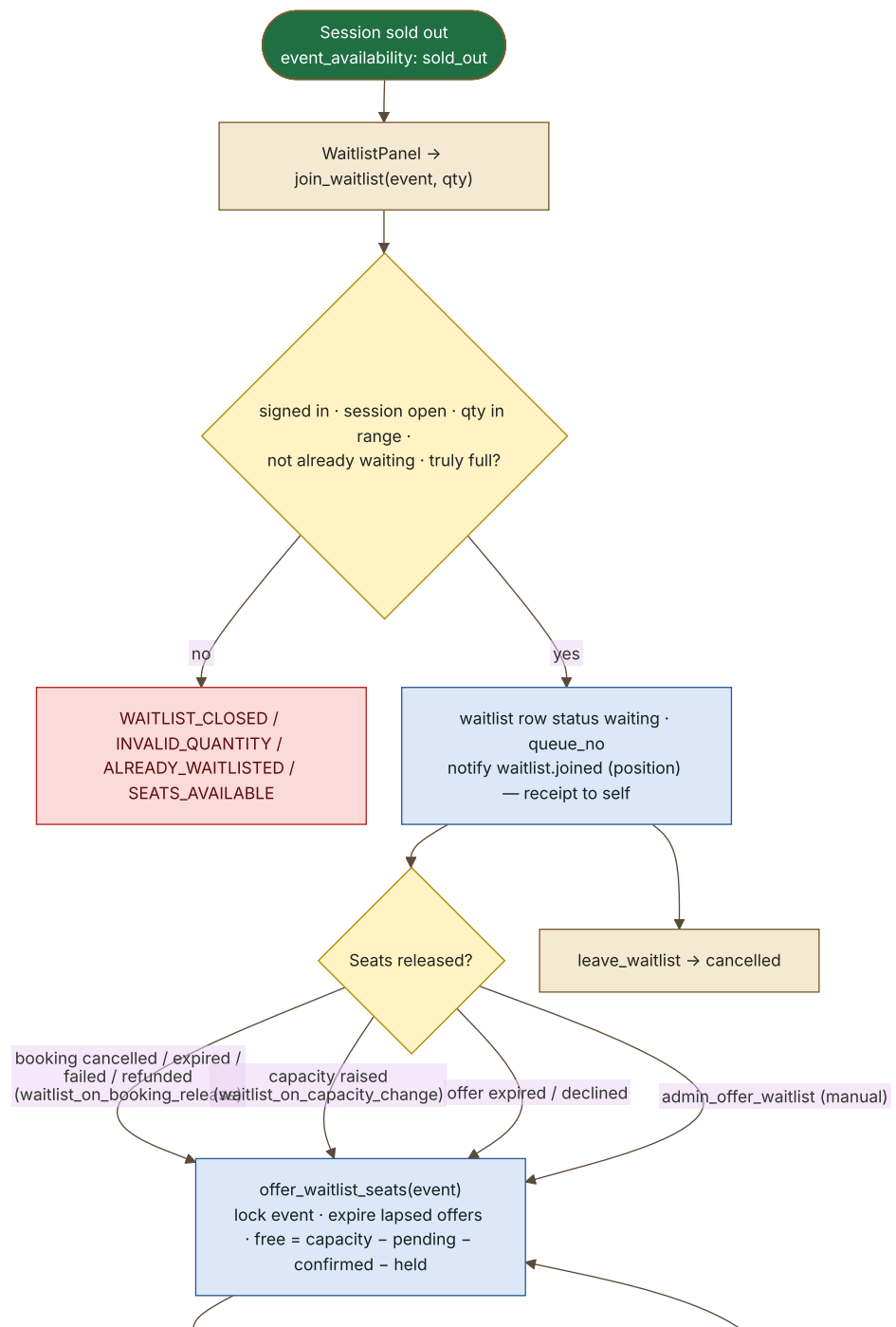


Diagram — Waitlist — join, offer, convert, expire (part 1 of 2)

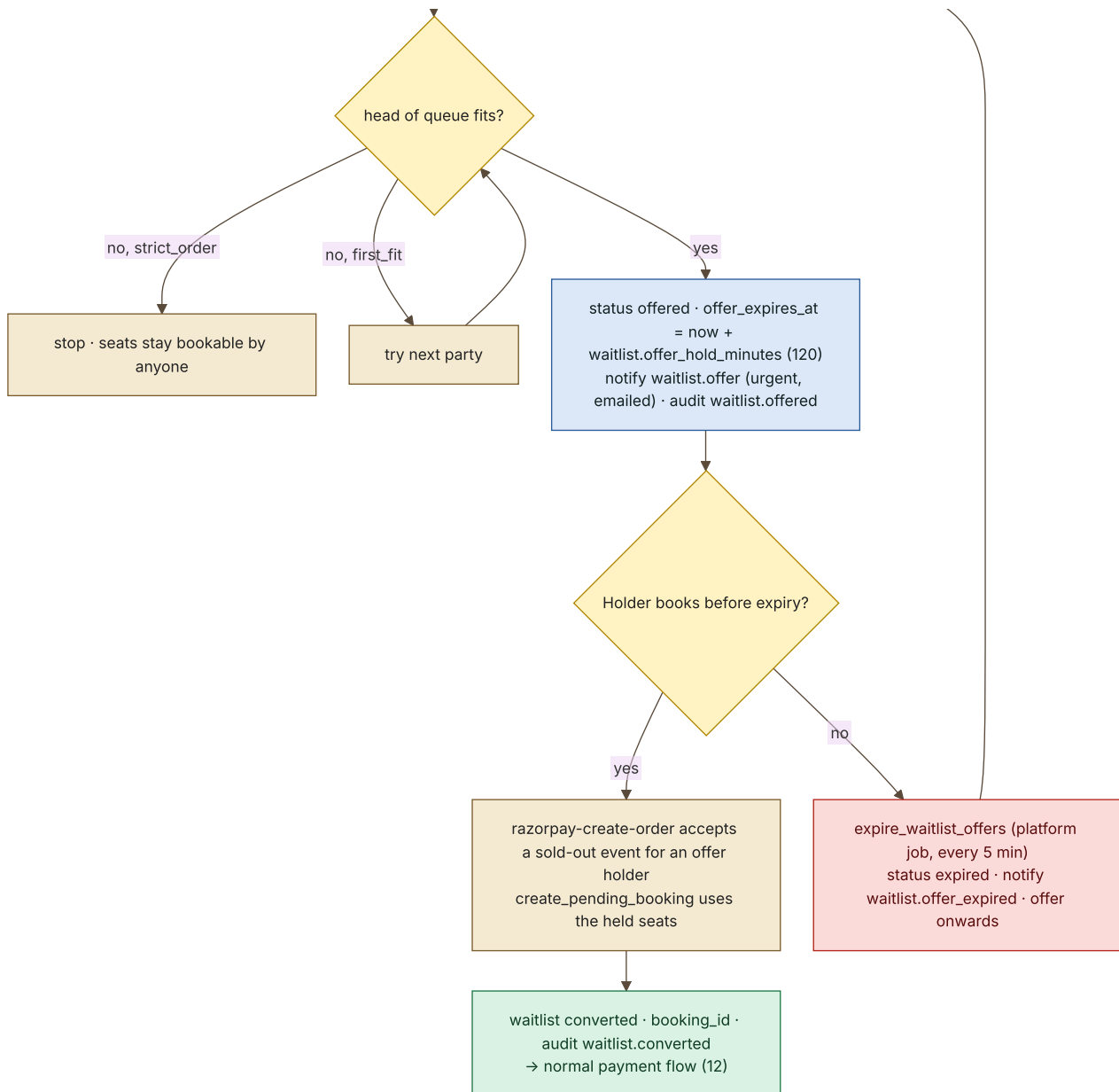


Diagram — Waitlist — join, offer, convert, expire (part 2 of 2)

States

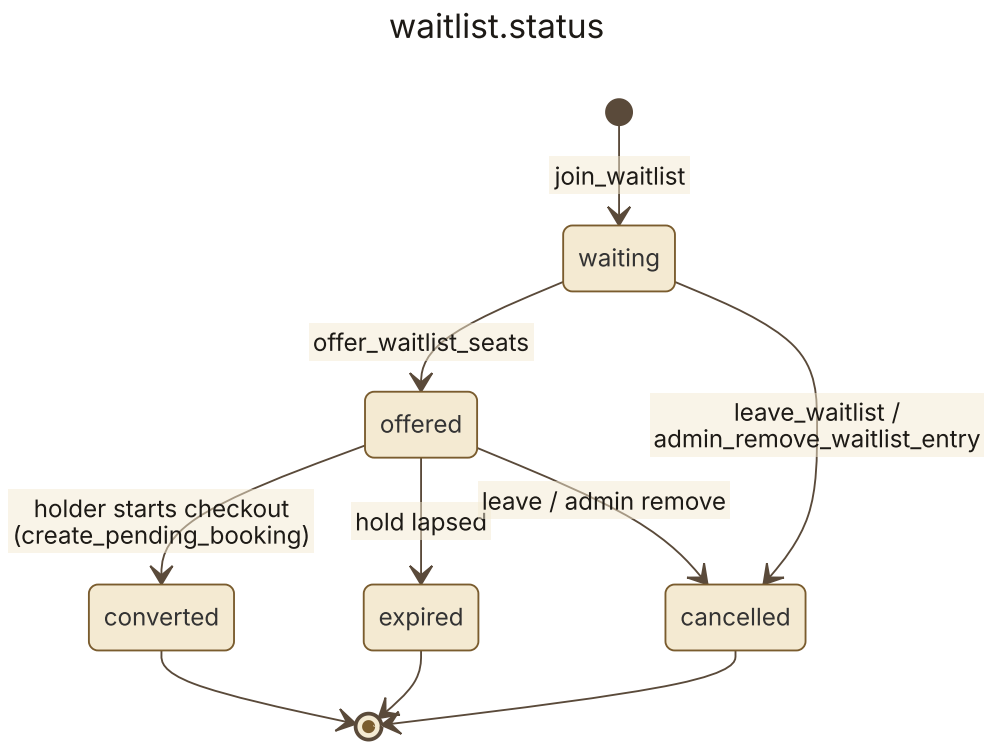


Diagram — waitlist.status

Scheduler behaviour

`run_platform_jobs()` (pg_cron, every 5 minutes) calls `expire_waitlist_offers()`. It expires lapsed offers (for update skip locked), notifies the holders, then calls `offer_waitlist_seats()` for every event that still has waiting parties. `offer_waitlist_seats()` also expires lapsed offers for its own event at the start of each pass, so seats flow on even between scheduler runs whenever a booking changes.

Who sees what

	Customer	Admin
Position	<code>my_waitlist()</code> (position, offer expiry) on the session page and dashboard	<code>/admin-portal/waitlist:</code> queue per event, offer / remove (audited)
Notifications	<code>waitlist.joined</code> , <code>waitlist.offer</code> , <code>waitlist.offer_expired</code> (critical in-app + emailed). Conversion is recorded as audit <code>waitlist.converted</code> only (no notification is sent for it)	—
RLS	<code>waitlist: self read own</code> (user_id) and <code>self read own by verified email</code> (JWT email, 0035)	<code>bookings.view_all / is_admin()</code> read

16 — Notification Architecture

Status: IMPLEMENTED in the database (0018 notifications, 0020 preferences + email outbox, 0021 canonical links, 0025 receipts, 0034 workflow types). In-app delivery uses Supabase Realtime (`notifications` is in the `supabase_realtime` publication). Email delivery needs the email drain and Resend (**PRODUCTION CONFIGURATION**).

The single entry point: `notify()`

`notify(user, type, title, body, link, event, priority)` (latest in 0025) does the following:

1. Skips notifying a person about their **own** action, except the receipts `application.received` and `waitlist.joined`.
2. Skips inactive accounts.
3. Looks up `notification_meta(type)`, giving `category`, `pref` key, `priority`, `email` flag and `critical` flag (0034 catalogue, 54 types).
4. **In-app**: inserts into `notifications` if the type is `critical` **or** the user's preferences allow `in_app` for that pref key.
5. **Email**: inserts into `email_outbox` if the type's `email` flag is true **and** preferences allow `email` (critical does *not* override an email opt-out). Message text is never emailed (`message.new` gets a generic body). The dedupe key is one email per thread per 15 min for chat, otherwise one per identical notice per hour, with `on conflict (dedupe_key) do nothing`.
6. Links are rewritten to the current console paths (`canonical_console_link`, `/admin/...` → `/admin-portal/...`).

`notify_permission_holders(permission, ...)` fans out to every active account whose role has that permission.

notify() — from event to inbox and email

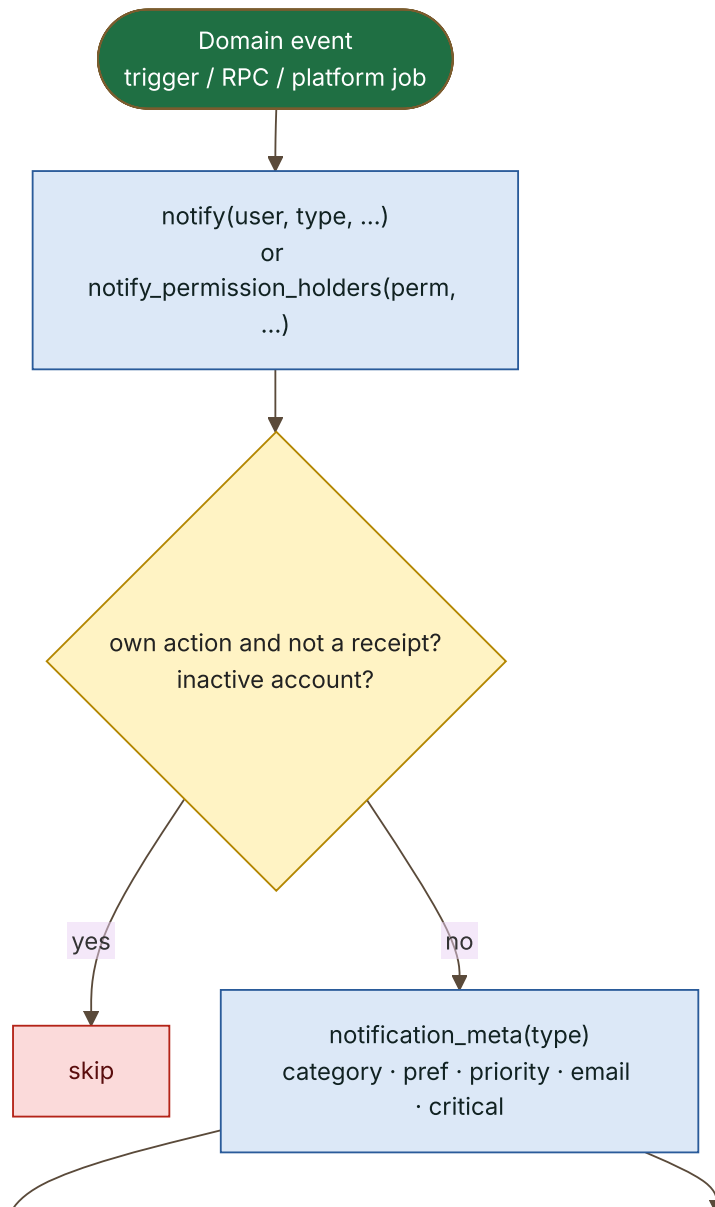


Diagram — notify() — from event to inbox and email (part 1 of 2)

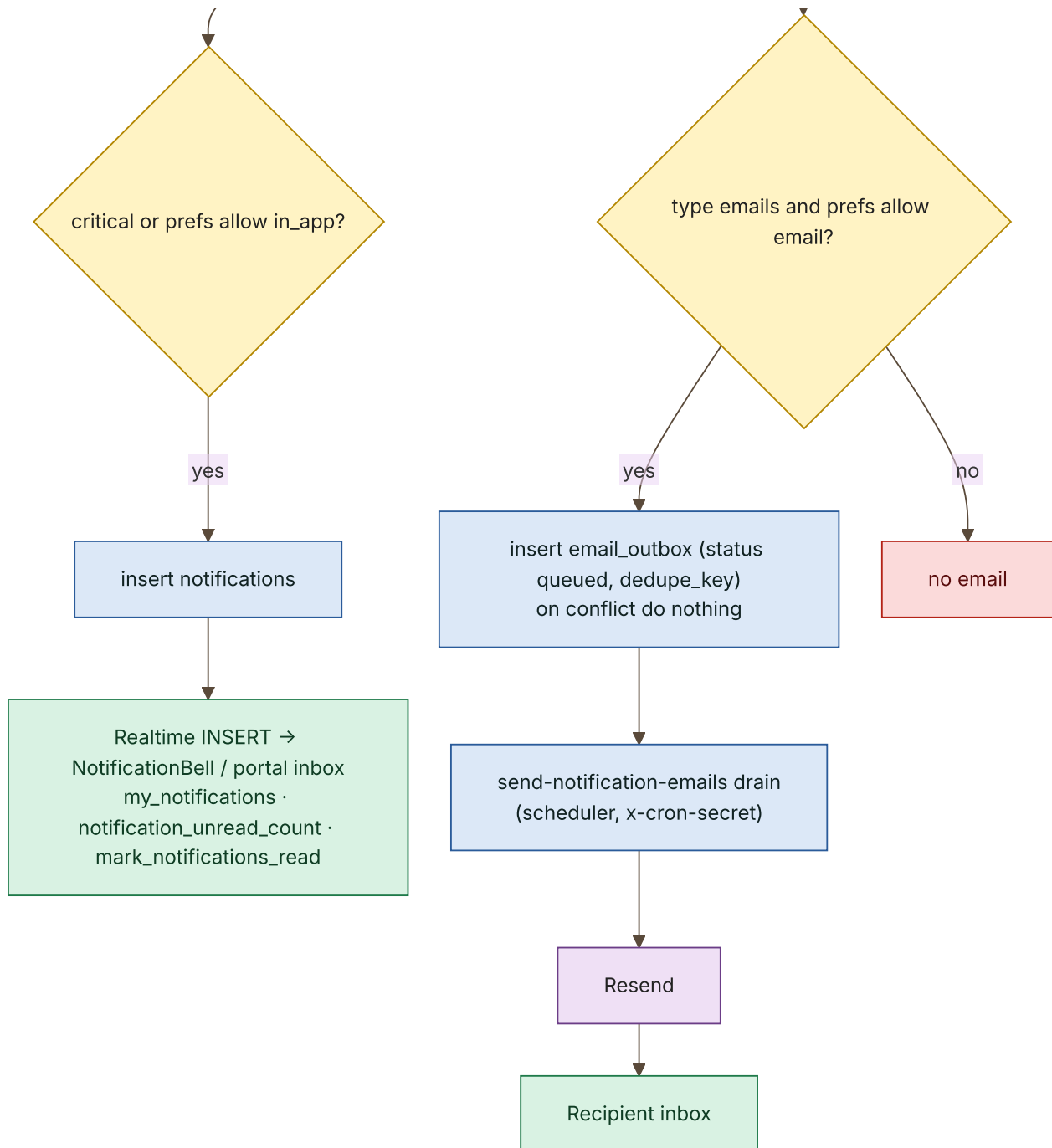


Diagram — notify() — from event to inbox and email (part 2 of 2)

Preferences

- `notification_preferences (user_id, email_enabled, prefs jsonb)`.
- `prefs` per key: `booking_requests`, `schedule_changes`, `event_updates`, `event_reminders`, `requirement_requests`, `requirement_reviews`, `messages`, `announcements`, `document_updates`, `payment_updates`, `tasks`, `access`, `applications`. Each has `in_app` and `email` booleans; the default is allowed.
- UI: `src/portal/NotificationPreferences.jsx` (portals, artist settings, admin notification settings) → `my_notification_preferences()`, `set_notification_preference()`.
- `email_enabled = false` turns off all email for that user.

Notification triggers (where `notify` is called)

Source (function / trigger)	Type(s)	Recipient
<code>notify_on_application</code> (applications inserted)	<code>application.new</code>	<code>applications.review</code> holders

Source (function / trigger)	Type(s)	Recipient
notify_on_application_decision	application.approved / application.rejected	applicant
notify_on_enquiry_received (0025)	application.received (receipt) + enquiry.new	submitter + applications.review holders
submit_artist_application / review_artist_application	application.submitted, .under_review, .info_requested, .resubmitted	applicant / reviewers
approve_artist_application	application.approved	artist
notify_on_booking_request (0034)	booking.requested, .confirmed, .cancelled → artist; .accepted / .declined / .expired → requester	artist / admin
notify_lineup_artist, notify_on_event_publish	assignment.new ("You're on the line-up", only once published)	artist
notify_on_event_artist_removed	assignment.removed	artist
notify_on_artist_schedule, notify_on_assignment_schedule	schedule.changed	artist / assignee
notify_on_event_change (0034)	event.date_changed / .time_changed / .venue_changed / .cancelled / .updated	line-up artists + event members
notify_on_assignment	assignment.new	assignee (vendor, crew, volunteer, staff...)
notify_on_task, notify_overdue_tasks (job)	task.assigned, task.overdue	assignee; team.manage holders
notify_on_requirement	requirement.requested / .submitted / .reviewed	partner / events.manage holders / partner
notify_on_document	document.added	audience of the document
notify_on_media_review	media.submitted / media.reviewed	entities.manage / artist
notify_on_sponsor_asset	asset.submitted / asset.reviewed	entities.manage / sponsor
notify_on_invoice	invoice.issued / invoice.paid	partner
notify_on_announcement	announcement.published	members targeted by the announcement
on_message_created	message.new	other participants / messages.manage holders (support threads)
request_checkin_access, grant_temporary_access, revoke_temporary_access, decline_access_request, notify_expiring_access (job)	access.requested, .granted, .revoked, .declined, .expiring	admins / volunteer
join_waitlist, offer_waitlist_seats, expire_waitlist_offers	waitlist.joined, .offer, .offer_expired	customer
expire_stale_bookings (job)	booking_payment.expired	customer
settle_payment, flag_late_payment, record_webhook_failure	payment.review, payment.late, payment.webhook_failed	payments.view holders
send_event_reminders (job)	event.reminder	event members (not ticket buyers)
send_availability_reminders (job)	availability.reminder	artists with stale calendars
admin_set_user_role	role.changed	the user
send_custom_notification (Super Admin)	admin.message	one chosen user

Where users see notifications

Surface	Component	Data
Admin header bell	<code>src/portal/NotificationBell.jsx</code> in <code>AdminShell</code>	<code>notification_unread_count</code> , <code>my_notifications</code> , Realtime inserts
Admin notifications page	<code>/admin-portal/notifications[/settings]</code>	same + preferences
Partner portals	<code>PartnerPortal</code> notifications tab	<code>my_notifications</code> , <code>mark_notifications_read</code>
Artist portal	<code>/artist/notifications</code>	same
Email	Resend via the outbox	see 17-email-architecture.md
Admin delivery log	<code>/admin-portal/ops/notifications</code> (<code>NotificationsSection</code>)	<code>email_outbox</code> (status, attempts, last_error), <code>application_notifications</code>

17 — Email Architecture

All email leaves through **one module**: `supabase/functions/_shared/email.ts`. It is the only place email secrets are read. Provider is chosen by `EMAIL_PROVIDER`:

Provider	Behaviour	Environment
resend (default)	POST <code>https://api.resend.com/emails</code> with <code>RESEND_API_KEY</code> ; supports attachments	production (needs the key and a verified domain: PRODUCTION CONFIGURATION, not done , <code>docs/OPERATIONS.md §2</code>)
mailpit	POST <code>\$MAILPIT_URL/api/v1/send</code>	LOCAL ONLY
log	Logs recipient + subject only, reports "sent"	development
disabled / missing key	<code>notConfigured</code> : nothing is sent and nothing is marked failed ; queued emails wait	—

The four email paths

Email	Trigger	Sender	Idempotency
Ticket confirmation (with <code>booking-pass.png</code> QR)	<code>settle_payment</code> confirmed a booking → <code>enqueueTicketEmail</code> in <code>razorpay-verify-payment</code> and <code>razorpay-webhook</code>	<code>email_outbox</code> row of type <code>ticket.confirmed</code> → <code>send-notification-emails</code> drain → <code>sendTicketRow</code>	unique <code>dedupe_key</code> = <code>ticket.confirmed:<booking id></code> ; <code>bookings.ticket_email_status</code> (pending / sent / failed)
Notification emails (54-type catalogue, those with <code>email = true</code>)	<code>notify()</code>	<code>email_outbox</code> → <code>send-notification-emails</code> → <code>notificationHtml</code>	<code>dedupe_key</code> per user + type + link + title hour (15-min bucket for chat)
Application approved (branded)	Admin approves on the generic Applications screen	browser → <code>send-approval-email</code> directly (not queued)	<code>application_notifications</code> (<code>source_table</code> , <code>source_id</code> , <code>'approval'</code>) unique row; <code>status</code> sent/failed; <code>force</code> to resend
Console invitation	Users & Roles → invite	browser → <code>admin-invite-user</code> directly (not queued)	Invitation row; <code>email_status</code> sent / failed / <code>not_configured</code> ; link returned once if email fails

Ticket email: the actual implementation

Since commit `cbd1151` ("*ticket email is queued server-side on settlement, not sent by the browser*"), the ticket email no longer depends on the customer's browser.

Ticket email — queue, dedupe, drain, fallbacks

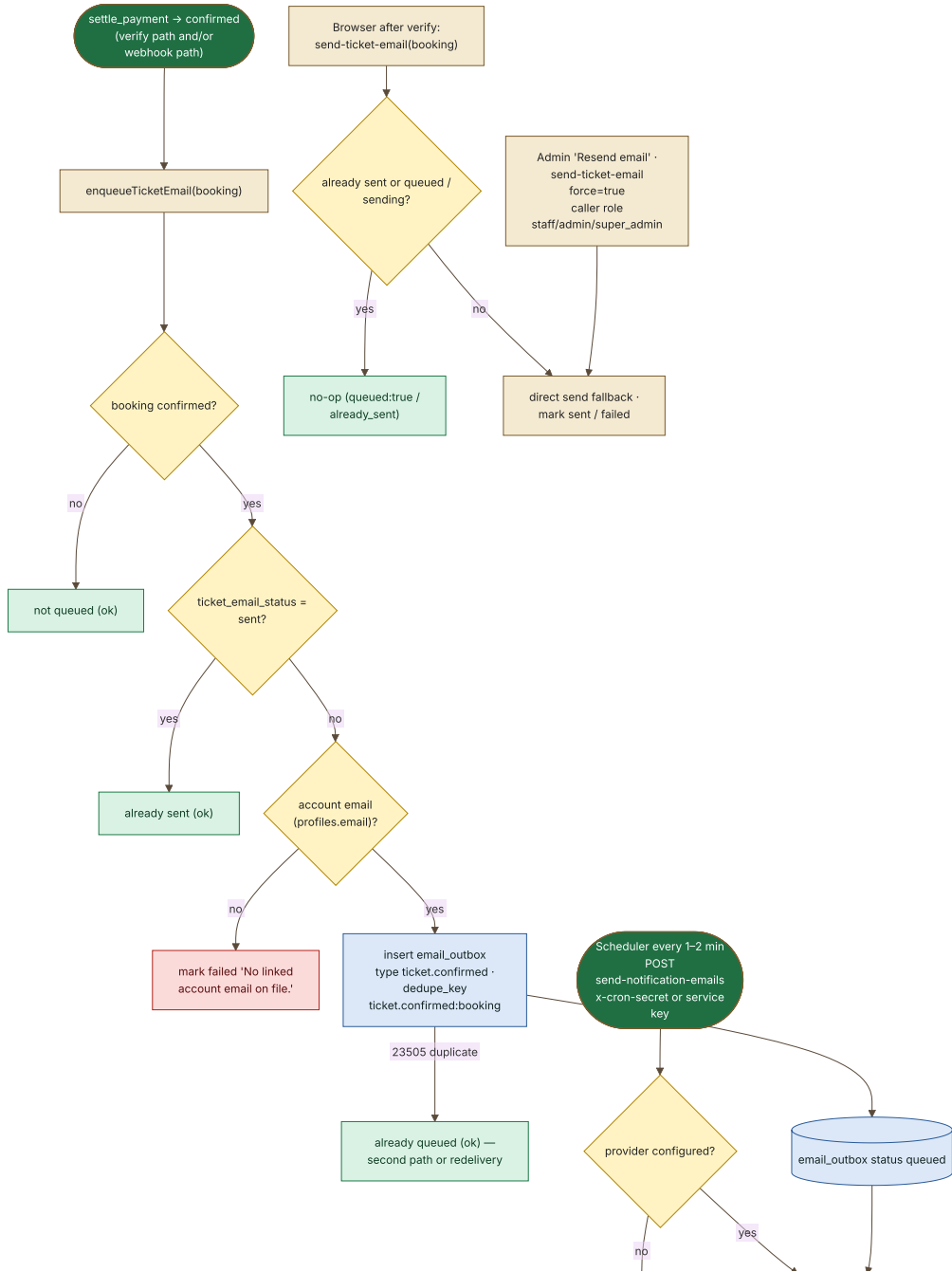


Diagram — Ticket email — queue, dedupe, drain, fallbacks (part 1 of 2)

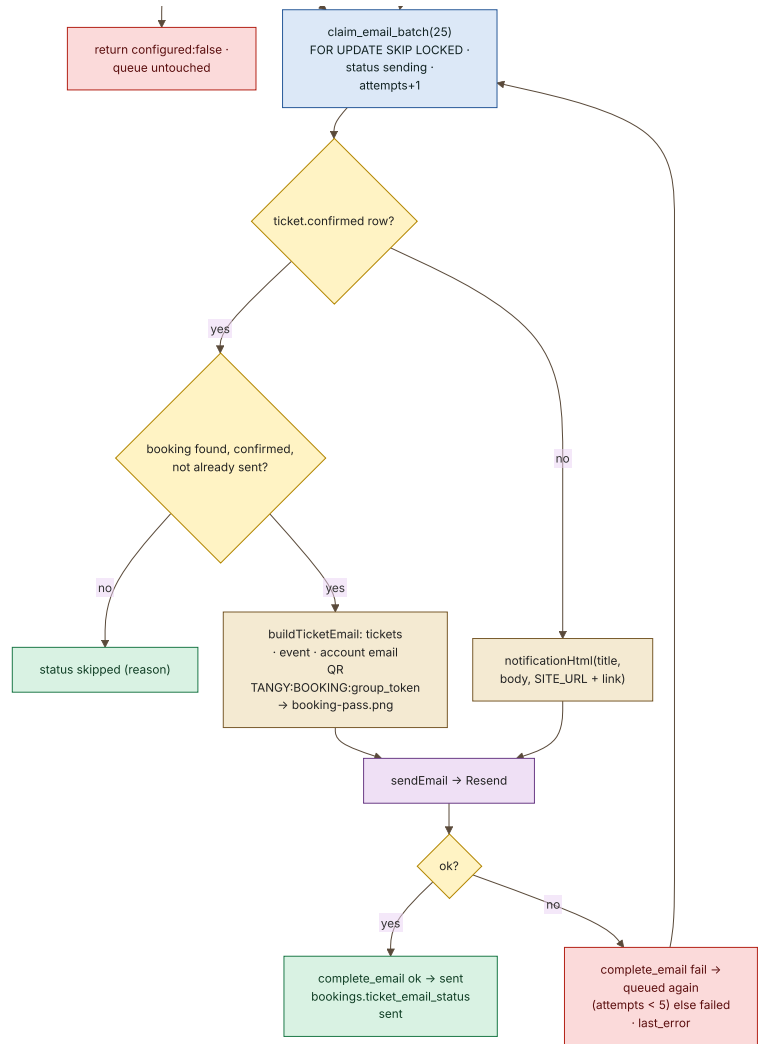


Diagram — Ticket email — queue, dedupe, drain, fallbacks (part 2 of 2)

Cases

Case	Behaviour
Browser closed right after paying	Webhook path queues the email; drain sends it
Both verify and webhook ran	Same <code>dedupe_key</code> : one row, one email
Provider not configured	Drain returns <code>{configured:false}</code> ; rows stay <code>queued</code> (not failed) and go out once configured
Provider error	Retried by later drain runs, up to 5 attempts, then <code>failed</code> with <code>last_error</code> (visible in Admin → Email delivery)
Stuck <code>sending</code> row	Reclaimed after 10 minutes (<code>claim_email_batch</code>)
Already emailed (admin resend or fallback first)	Drain marks the queued row <code>skipped</code> ("Ticket email already sent.")
Admin resend	<code>send-ticket-email</code> with <code>force:true</code> always sends; only <code>bookings.ticket_email_*</code> changes, never payment or tickets
No account email	<code>ticket_email_status = failed</code> with the reason; nothing queued

Notification email drain

`send-notification-emails` (`--no-verify-jwt`):

1. Accepts only `Authorization: Bearer <service role key>` or `x-cron-secret: <CRON_SECRET>`. It never accepts an end-user JWT.
2. Claims up to 25 rows.
3. Renders each one: ticket rows through `sendTicketRow`, everything else through `notificationHtml`. The action label comes from `ACTION_LABEL[type]` and the link is `SITE_URL + link`.

4. Calls `complete_email` for each row.

Response: `{claimed, sent, failed, skipped}`.

Scheduling the drain is PRODUCTION CONFIGURATION. No migration schedules it. `docs/OPERATIONS.md` §2a Step 5 asks the operator to create a Supabase Cron / external HTTP job every 1–2 minutes. Locally: `scripts/run-jobs.sh emails`.

Approval email

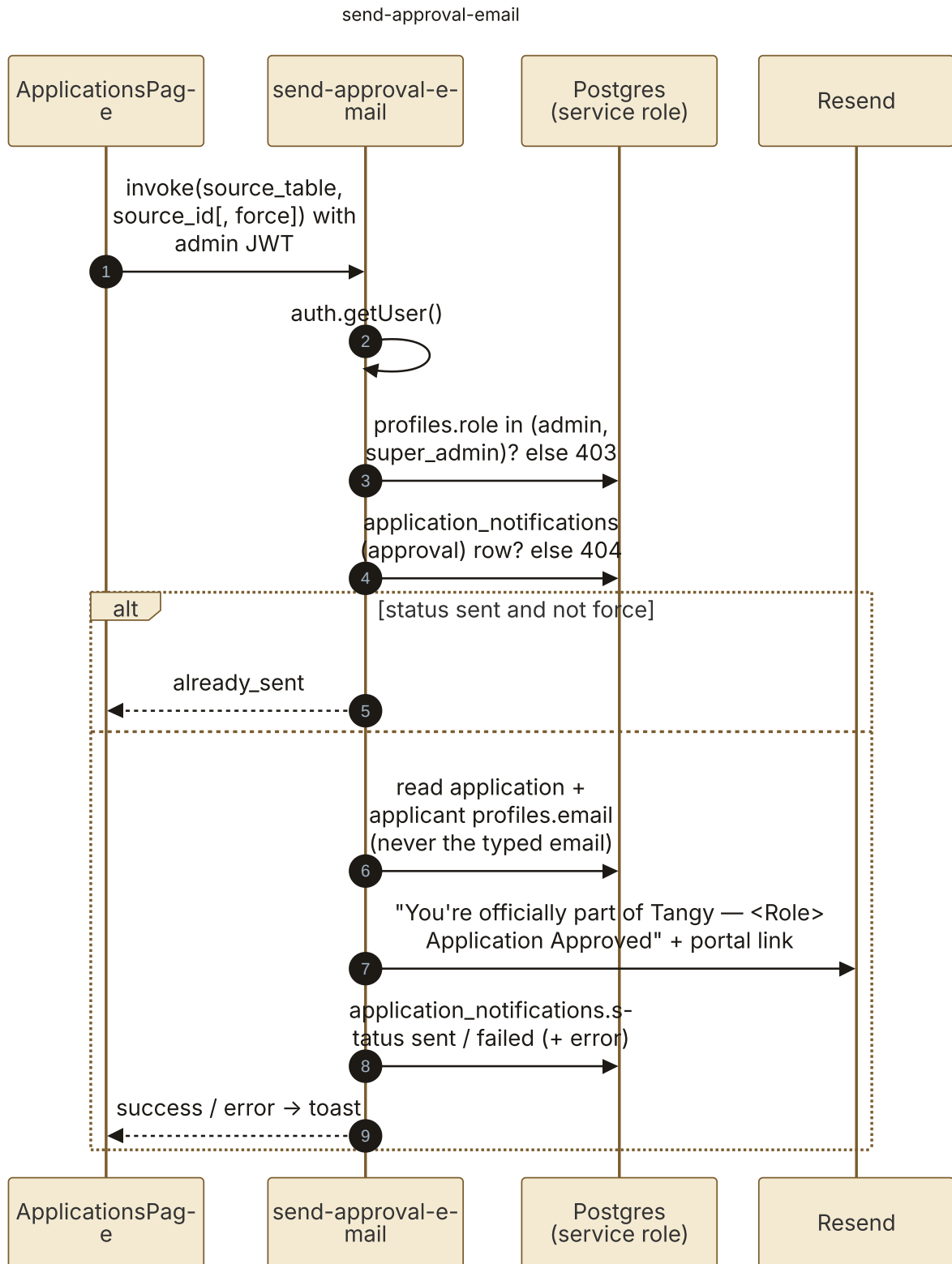


Diagram — `send-approval-email`

What never goes into an email

- message text: `message.new` gets *"You have a new message on Tangy..."*
- payment details or booking answers (`docs/OPERATIONS.md` §2)
- user-typed attendee emails: the recipient is always the **account** email from `profiles`

18 — Messaging Architecture

Status: IMPLEMENTED (0008 base, 0018 partner threads, 0020 priority/meta, 0034 private artist threads and RLS tightening). Tests: `supabase/tests/messaging_security.test.sql`, `e2e/portals.mjs`, `e2e/platform.mjs`.

The repository states plainly: messaging is **not end-to-end encrypted**. Protection is HTTPS, authentication, authorization and RLS (0034 header; the artist messages page says so too).

Data model

Messaging tables

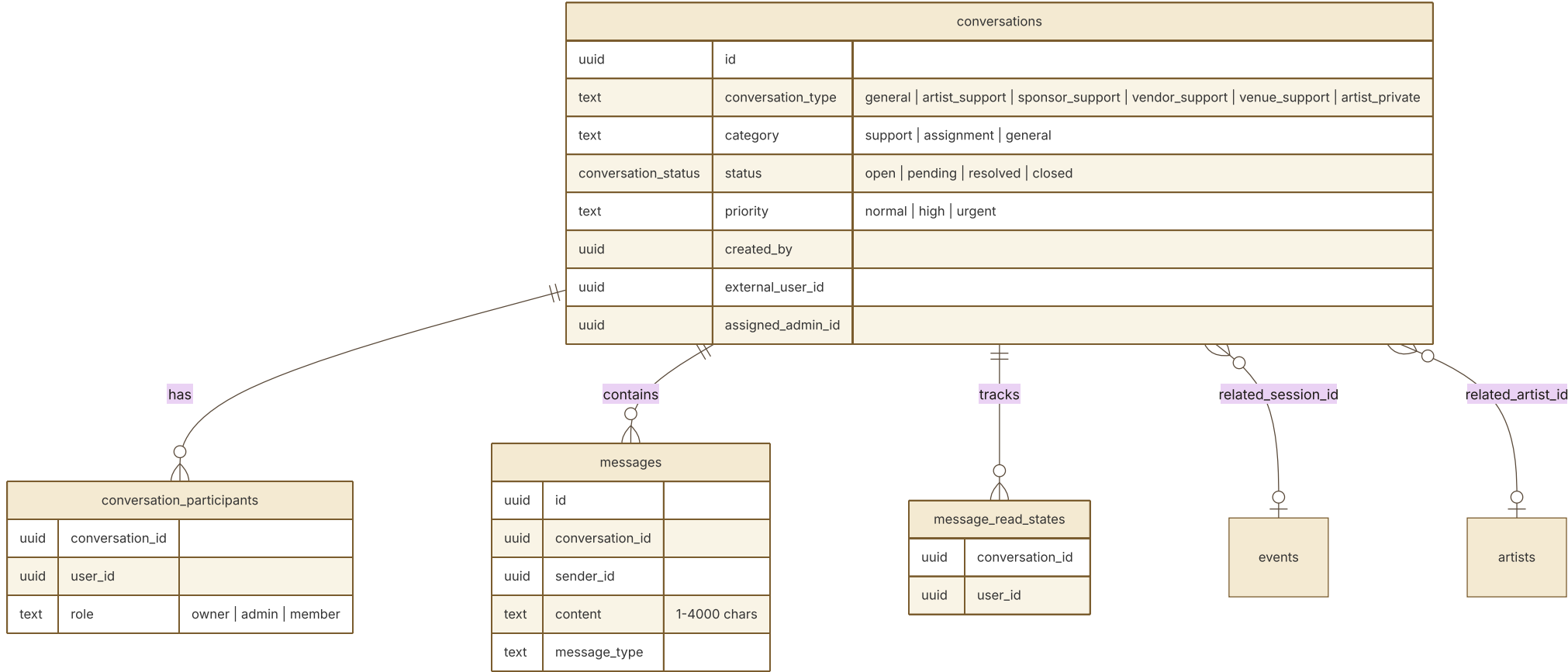


Diagram — Messaging tables

Conversation types and who can read or write

Type	Started by	Participants	Who else can read	Notes
<code>general</code> (category support)	Any signed-in user, via Tangy Assistant → "Request an agent" (<code>get_or_create_support_conversation</code> , 0008)	creator (+ assigned admin)	admin-level (<code>is_staff_or_admin()</code>) = admin / super_admin) through RLS	Admin "Website support inbox" (<code>/admin-portal/ops/inbox</code> , <code>InboxSection</code> , <code>chatService</code>)
<code>artist_support</code> , <code>sponsor_support</code> , <code>vendor_support</code> , <code>venue_support</code>	Partner from their portal (<code>start_partner_conversation</code> , optionally tied to an event they are part of) or admin (<code>admin_start_partner_conversation</code>)	partner owner + admins who reply (auto-added as <code>admin</code> participant by <code>send_message</code>)	<code>messages.manage</code> holders	Admin Messages page (<code>MessagesPanel</code> , <code>admin_conversations</code>)
<code>artist_private</code>	Super Admin only (<code>start_private_artist_conversation</code> , 0034)	that Super Admin + the artist	nobody else: not other admins, staff or partners	Excluded from staff/admin read in every policy and function
category assignment	legacy 0008 assignment requests	—	—	LEGACY

RLS (final state, 0034)

Table	Policy	Rule
conversations	participant or staff read	<code>is_participant(id)</code> or (<code>is_staff_or_admin()</code> and <code>conversation_type <> 'artist_private'</code>)
conversations	creator or staff insert	creator may insert only <code>general</code> with no <code>external_user_id</code> ; admins any non-private type
conversations	staff/admin update	admin-level, and for private threads only a participant
messages	participant or staff read	<code>is_participant(conversation_id)</code> or (<code>is_staff_or_admin()</code> and not <code>is_private_conversation(...)</code>)
messages	participant or staff insert	<code>sender_id = auth.uid()</code> and same visibility rule
conversation_participants	—	populated only inside RPCs (0008 comment); <code>guard_conversation_admin_participant</code> (0035): only the team joins as <code>admin</code>

Partner ↔ team flow

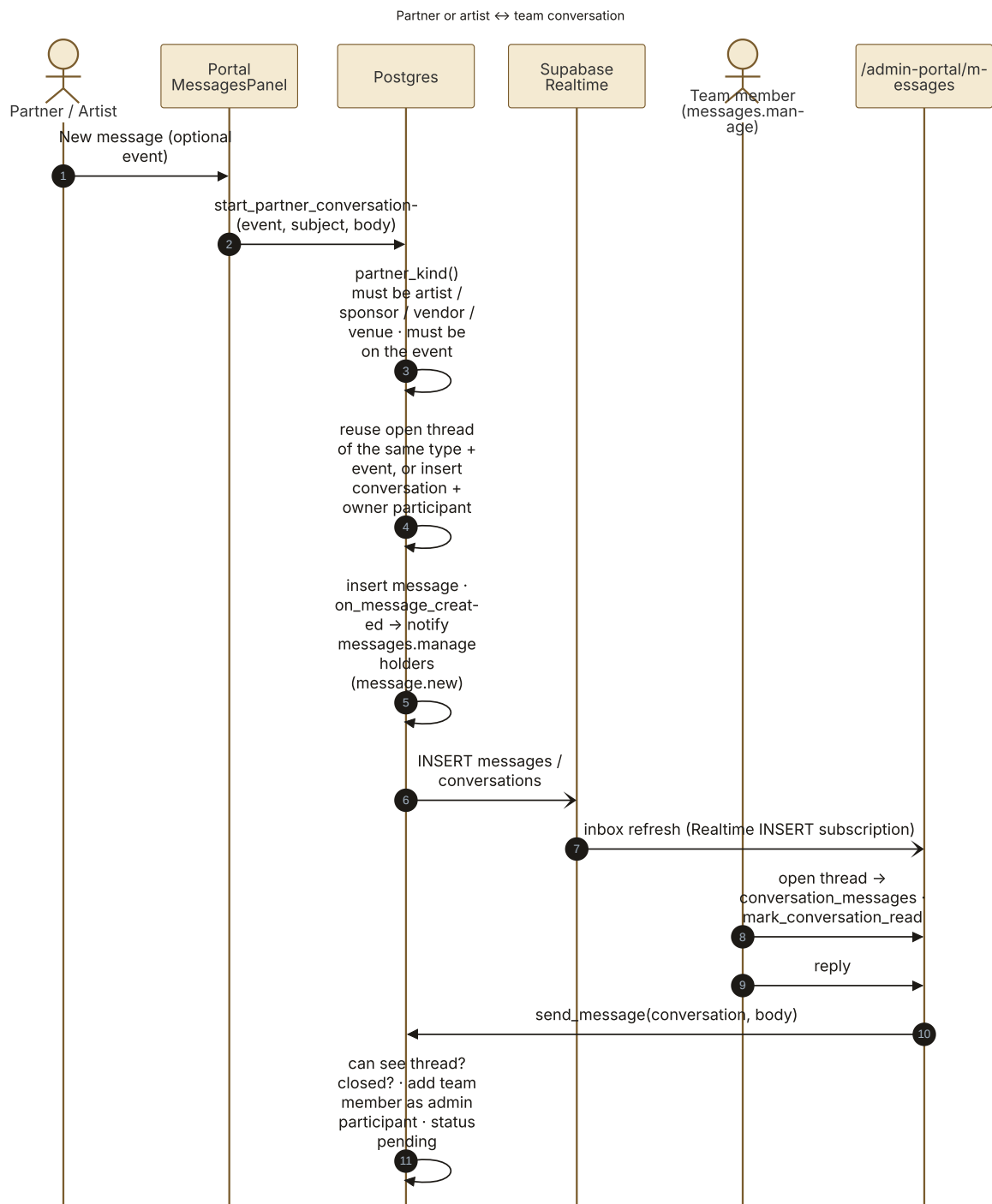


Diagram — Partner or artist ↔ team conversation (part 1 of 2)

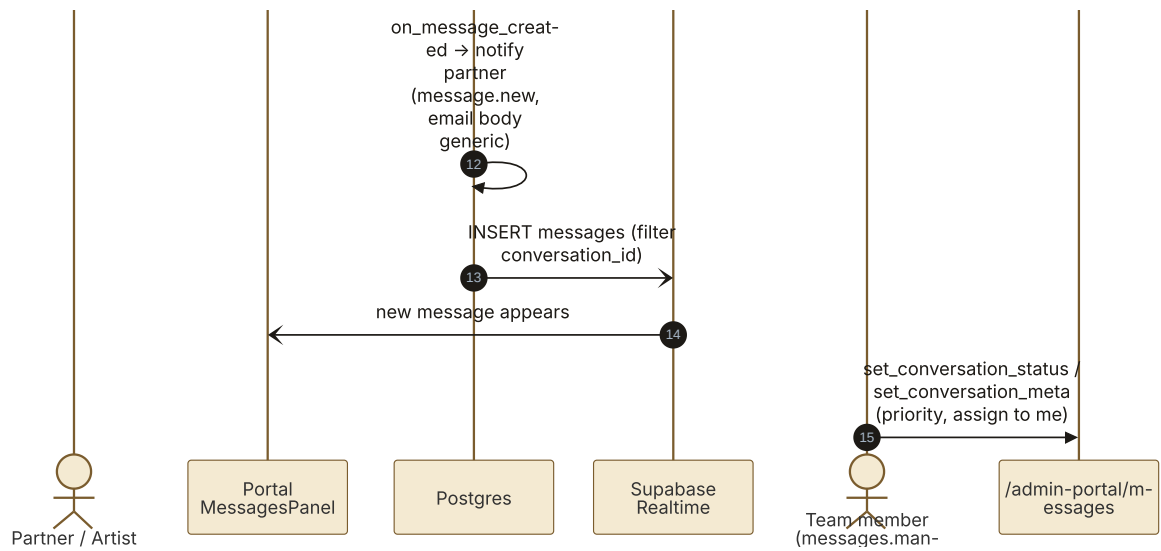


Diagram — Partner or artist ↔ team conversation (part 2 of 2)

Customer ↔ team (website support)

Tangy Assistant escalation → support conversation

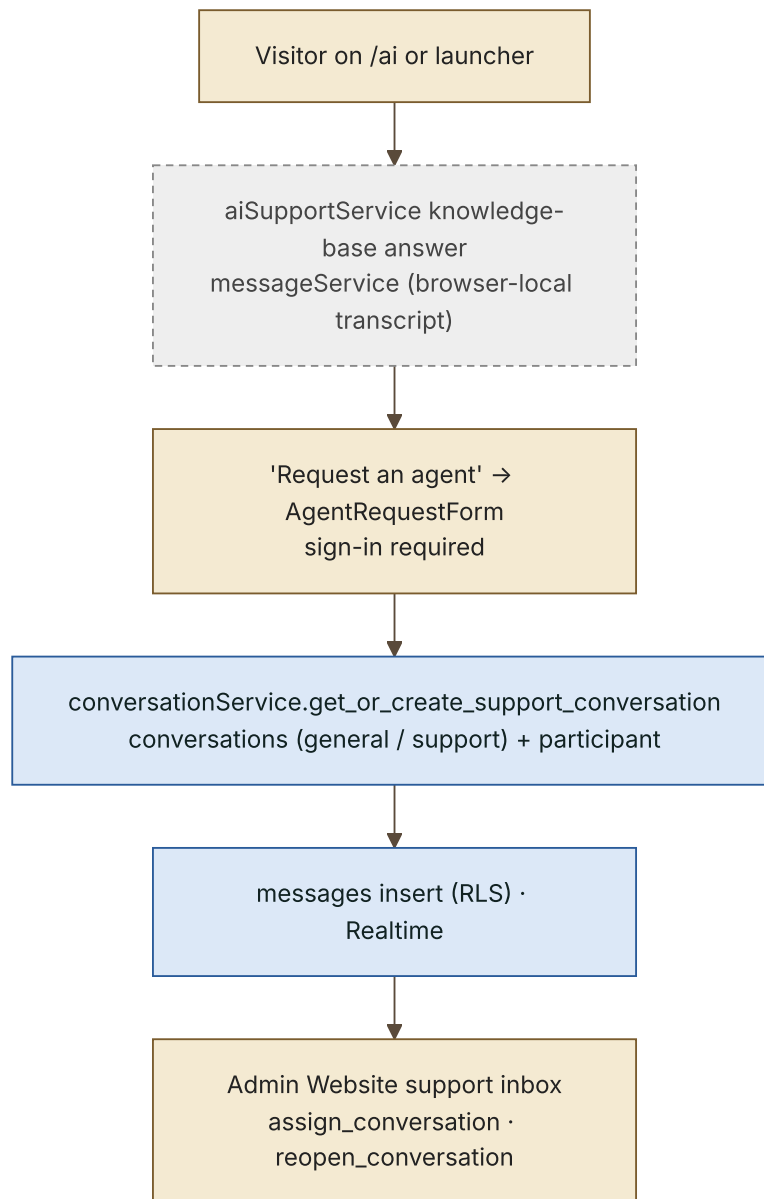


Diagram — Tangy Assistant escalation → support conversation

Private Super Admin ↔ artist

`start_private_artist_conversation(artist_user, subject, body):`

- Super Admin only, and only for an artist with a portal account.
- Reuses the open private thread if there is one.
- Both participants are added. The thread is invisible to every other console user (RLS, `is_private_conversation`, `can_manage_conversation` excludes it).

The entry point is the admin `ArtistDetailPage`.

Notifications and email

`on_message_created` (latest 0034) notifies the other participants, or `messages.manage` holders for partner and support threads, with `message.new`. Notification links go straight to the conversation. Email copies are throttled to one per thread per 15 minutes and never contain the message text.

19 — Content / CMS Architecture

Status: IMPLEMENTED.

Migration	What it added
0028	CMS tables + permissions
0029	private <code>content-media</code> + signed URLs
0031	programmes + session history
0017 / 0018 / 0020	announcements

Before 0028, Tangy TV lived in each admin's `localStorage` and the diary, gallery and artists were hard-coded in `src/data/mockData.js` (0028 header).

Content types

Type	Table(s)	Statuses	Admin UI	Public UI	Permission
Tangy TV	<code>tv_videos</code>	draft / published / archived (+ <code>published_at</code>)	Content → Tangy TV (<code>TvManager</code>)	<code>/tv</code> , <code>/tv/:slug</code> , homepage TV	<code>content.*</code> + <code>content.manage_tv</code>
Diary	<code>diary_posts</code>	draft / published / archived	Content → Diary (<code>DiaryManager</code>)	<code>/diary</code> , <code>/diary/:slug</code> , Tangy Diary section	<code>content.*</code> + <code>content.manage_diary</code>
Gallery	<code>gallery_albums</code> , <code>gallery_photos</code>	album: draft / published / archived	Content → Gallery (<code>GalleryManager</code>)	<code>/gallery</code> , <code>/gallery/:album</code> , <code>/gallery/archive</code>	<code>content.*</code> + <code>content.manage_media</code>
Programmes (seasons)	<code>programmes</code> , <code>programme_events</code>	draft / published / archived	Content → Sessions area (<code>ProgrammesManager</code>)	<code>/archive/programmes[:slug]</code>	<code>content.manage_sessions</code>
Session copy	<code>events</code> (description, story, image_url, tags, featured)	follows event status	Content → Sessions (<code>update_session_content</code>)	session pages, archive	<code>content.manage_sessions</code> + <code>content.edit</code>
Artist pages	<code>artists</code> (public profile fields, slug <code>artists_assignment_slug</code>) → view <code>public_artists</code>	artist <code>approved</code>	Content → Artists / People	<code>/artists</code> , <code>/artists/:slug</code> , <code>/artist</code> directory	<code>entities.manage</code> / artist self
Media library	storage <code>content-media</code>	private; public only if referenced by published content	Content → Media library (<code>MediaLibrary</code>)	via signed URLs	<code>content.*</code> + <code>content.manage_media</code>
Announcements	<code>announcements</code>	draft / scheduled / published / expired / archived; <code>audience</code> all / guest / patron / artist / staff; <code>character</code> ; <code>publish_at</code> ; <code>target_team</code>	Content → Announcements (<code>Announcements.jsx</code> , <code>content.manage</code>)	Character pop-up (<code>AnnouncementCharacterOverlay</code> via <code>announcementService</code>); staff <code>/admin-portal/announcements</code> ; portal announcements (<code>portal_announcements</code>)	<code>content.manage</code>

Publishing workflow

Content — draft → publish → public → archive

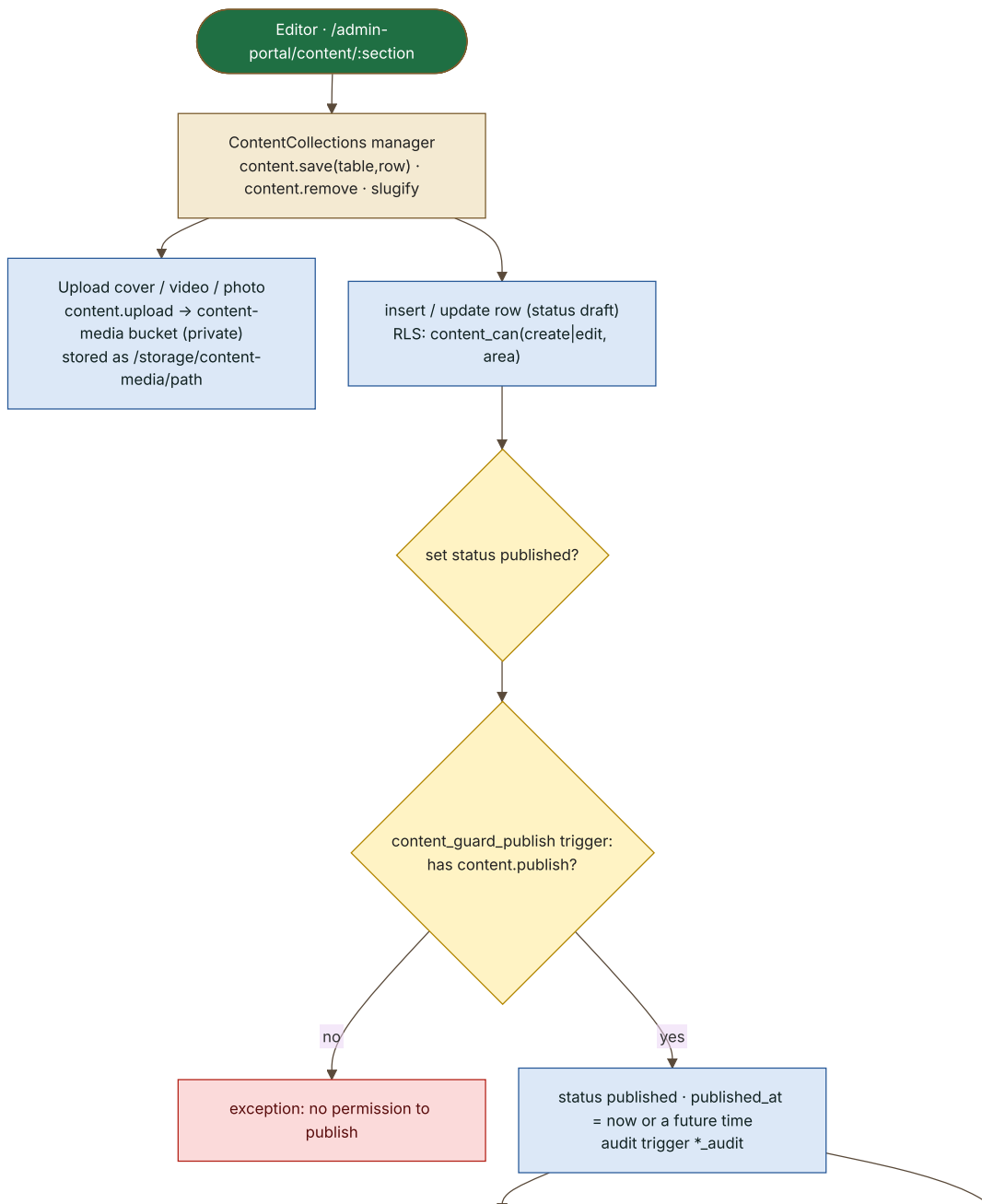


Diagram — Content — draft → publish → public → archive (part 1 of 2)

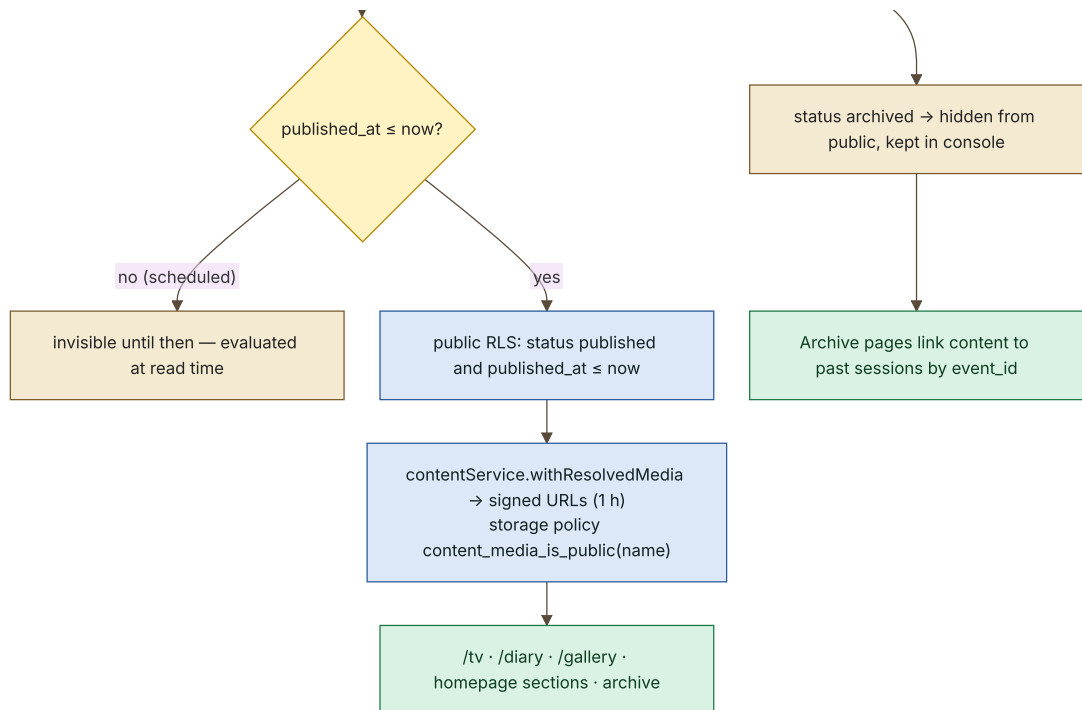


Diagram — Content — draft → publish → public → archive (part 2 of 2)

Review step: there is **no separate review status** for TV, diary, gallery or programmes. "Review" means a holder of `content.view` sees drafts in the console, and publishing needs `content.publish`. Announcements have a `scheduled` status. Artist media and sponsor assets **do** have a review queue (see [20](#) and [06](#) Reviews).

Scheduled publishing

No job is needed. Visibility is `status = 'published' and published_at <= now()`, evaluated at read time (`docs/OPERATIONS.md` §1).

Public read path

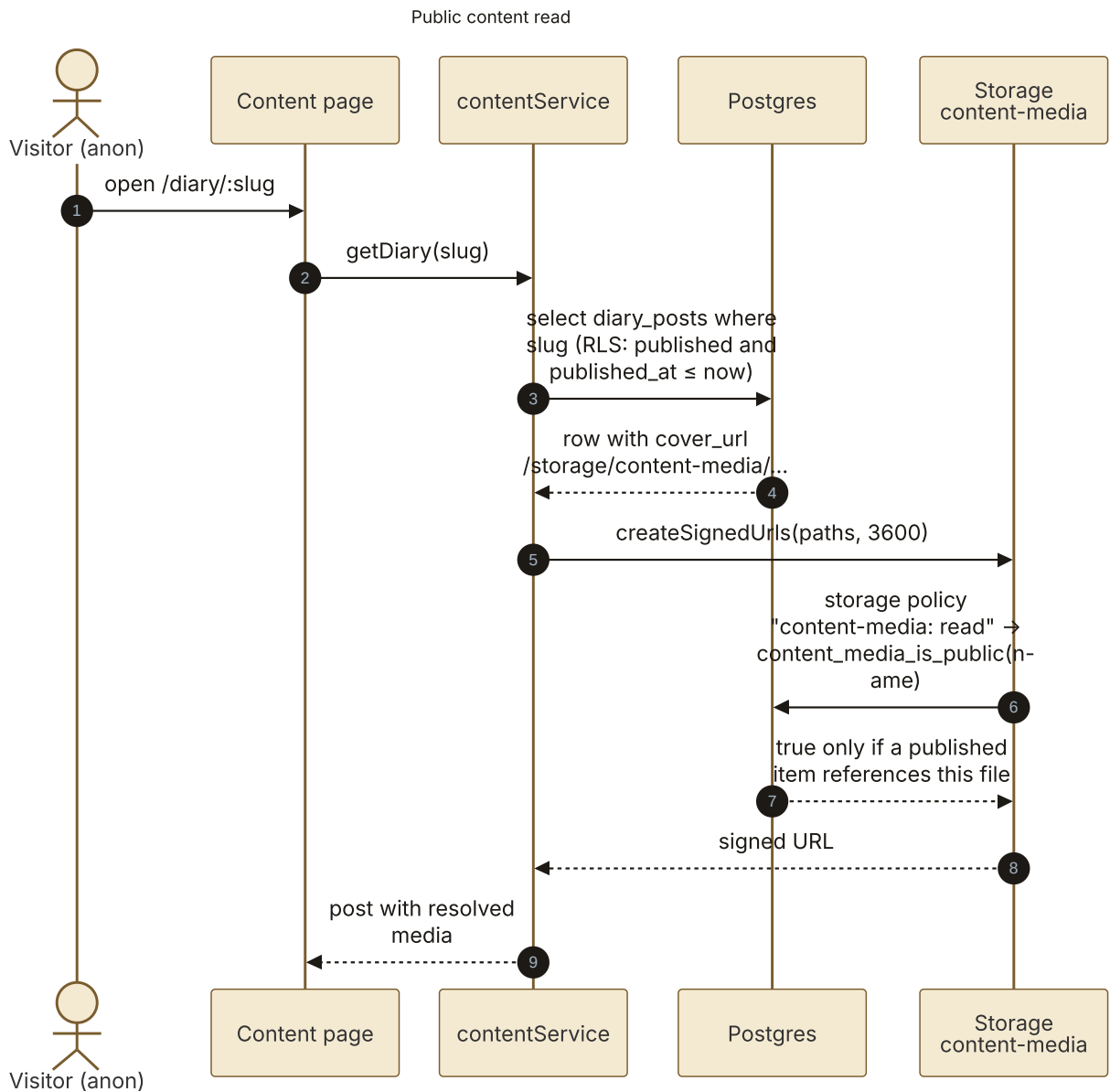


Diagram — Public content read

Archive

`src/lib/archiveService.js` builds the archive from past `events` (`date < today`) joined with:

- `event_artists` + `public_artists`
- `gallery_albums`, `diary_posts` and `tv_videos` linked by `event_id`
- `programmes` / `programme_events`

Pages: `/sessions/archive`, `/sessions/archive/:slug`, `/archive/programmes[:slug]`, plus the static museum pages (`/archive/museum-timeline`, `/archive/past-memories`, `/archive/contact-sheets`).

20 — Storage Architecture

Six Supabase Storage buckets, all created by migrations. No environment variables are needed (`.env.example`). Client helpers are in `src/lib/storage.js` :

- `uploadWithProgress()` posts to the Storage REST API with the **user's own JWT**, so bucket policies decide.
- `signedUrl()` creates short-lived signed URLs (default 10 min).
- Client size cap: 25 MB by default; artist uploads use 50 MB.

Buckets at a glance

Bucket	Public?	Size / types (bucket limits)	Path convention	Created / last changed
artist-avatars	public	5 MB · jpeg/png/webp/gif/avif	<artist id>/...	0013, 0022, 0029
artist-media	private	50 MB · audio/*, mp4/webm/quicktime, images, pdf	<artist id>/... and applications/<user id>/...	0013, 0020, 0022, 0029, 0033
artist-documents	private	25 MB · pdf, jpeg/png/webp, docx	<artist id>/...	0033, 0036
event-documents	private	25 MB · pdf, images, txt/csv, Office docs, zip	events/... (team) and requirements/<requirement id>/... (partners)	0020, 0029
sponsor-assets	private	25 MB · images incl. svg, pdf, postscript/illustrator, zip, octet-stream	<sponsor user id>/...	0020, 0029
content-media	private since 0029 (was public in 0028)	50 MB · jpeg/png/webp/gif/avif, mp4/webm	free paths; stored in rows as /storage/content-media/<path>	0028, 0029

Per-bucket access (final policies)

Storage buckets — who can upload / read / delete

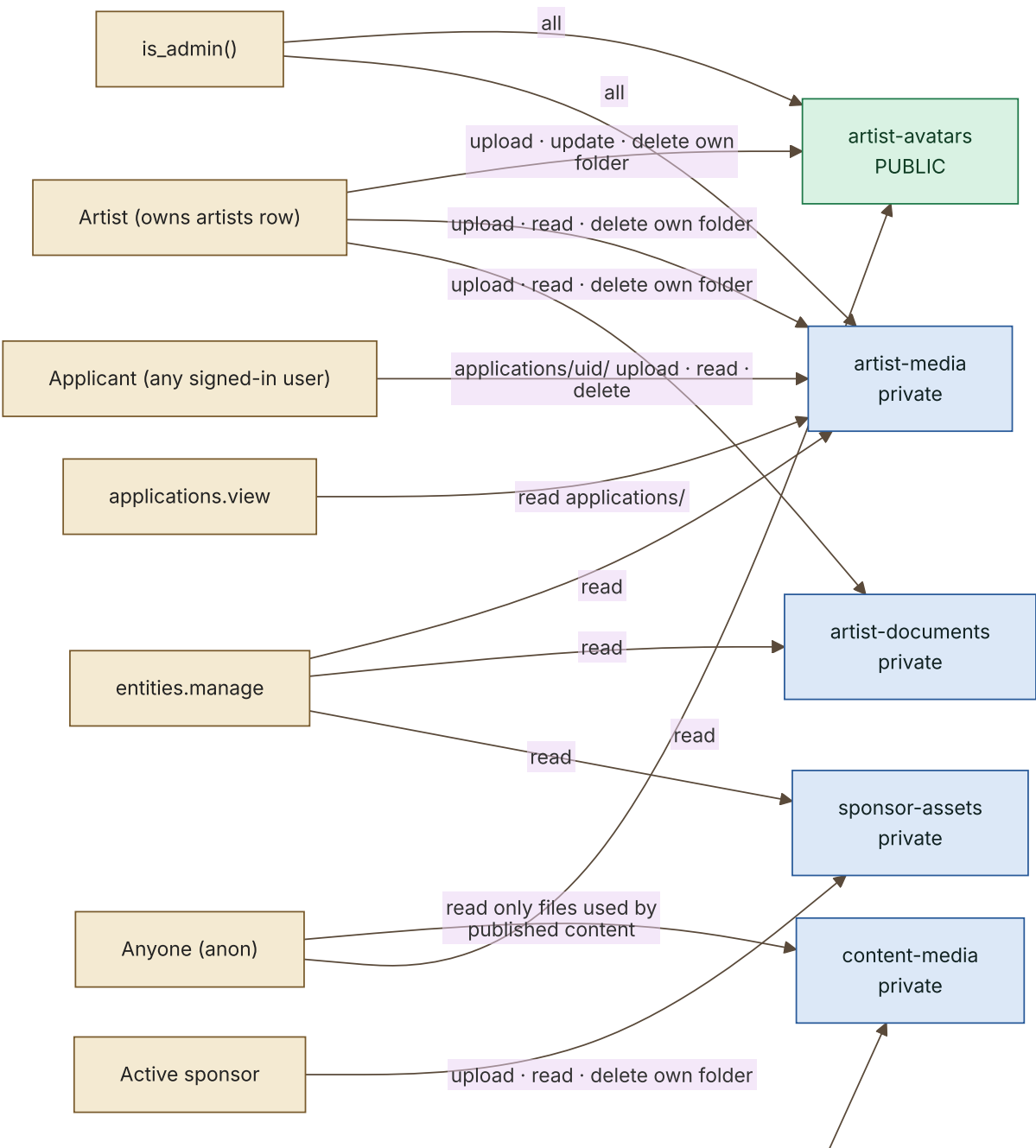


Diagram — Storage buckets — who can upload / read / delete (part 1 of 2)

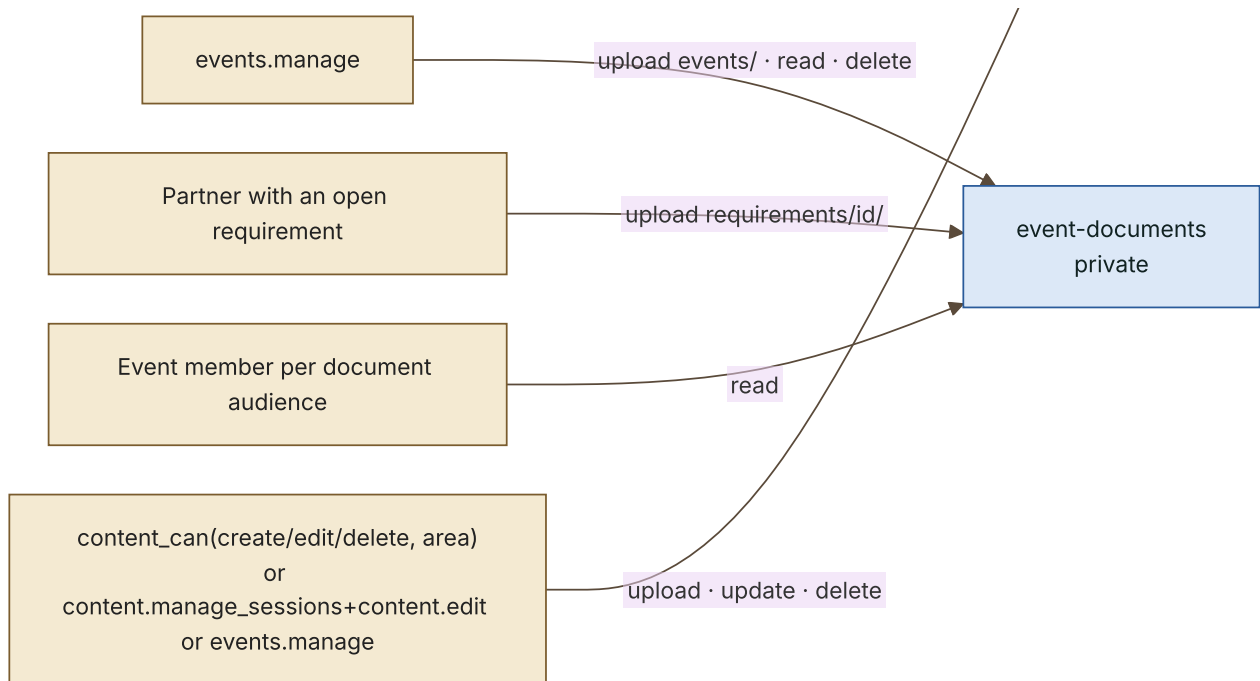


Diagram — Storage buckets — who can upload / read / delete (part 2 of 2)

Bucket	Upload	Read	Delete	Approval dependency	DB relationship
artist-avatars	the artist whose <code>artists.id</code> is the first folder	public; owner via policy (for upsert)	owner; <code>is_admin()</code>	none	<code>artists.avatar_url</code>
artist-media	owner artist (first folder = own <code>artists.id</code>); applicant under <code>applications/<own uid>/</code>	owner, <code>is_admin()</code> , <code>entities.manage</code> (curators), <code>applications.view</code> (application uploads)	owner / applicant; admin	<code>artist_media.status</code> uploaded → under_review → approved / rejected (curators only, <code>guard_artist_media</code>)	<code>artist_media.storage_path</code> , <code>artist_applications.video_storage_path</code>
artist-documents	owner artist	owner, <code>entities.manage</code>	owner	none	<code>artist_documents.storage_path</code> (kind rider / epk / tax / payment / identity / other)
event-documents	<code>events/...</code> needs <code>events.manage</code> ; <code>requirements/<id>/...</code> needs the requirement's own user while status is <code>requested</code> / <code>changes_requested</code> (<code>can_write_event_file</code>)	<code>events.manage</code> , or a non-expired <code>event_documents</code> row whose audience matches the caller (<code>user</code> , <code>members</code> , or a member kind), or the requirement owner (<code>can_read_event_file</code>)	<code>events.manage</code>	requirement review (<code>review_requirement</code>)	<code>event_documents</code> , <code>event_requirements</code>
sponsor-assets	active <code>sponsor</code> into <code><own uid>/</code>	owner, <code>entities.manage</code>	owner, <code>entities.manage</code>	<code>sponsor_assets.status</code> submitted → approved / <code>changes_requested</code> (Reviews page)	<code>sponsor_assets</code>
content-media	content editors by area	editors; anon only if <code>content_media_is_public(name)</code> : the file is referenced by published TV / diary / gallery / non-draft event	editors (<code>content.can('delete', ...)</code>)	publish rights (<code>content.publish</code>)	<code>tv_videos</code> , <code>diary_posts</code> , <code>gallery_albums</code> , <code>gallery_photos</code> , <code>events.image_url</code>

Upload flows

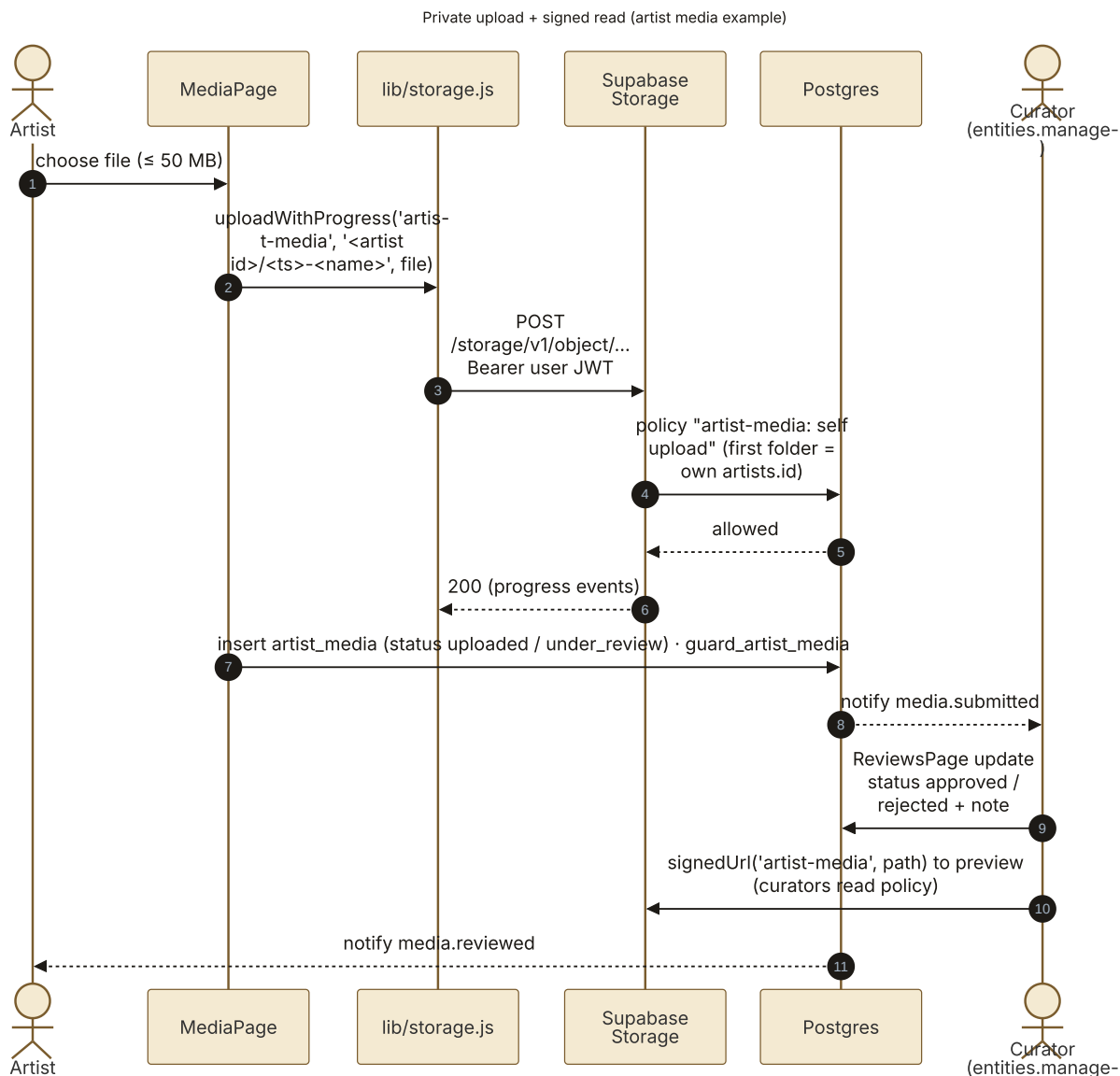


Diagram — Private upload + signed read (artist media example)

Flow	Page	Bucket / path	Notes
Artist media	src/artist/pages/MediaPage.jsx	artist-media/<artist id>/...	review by curators
Artist documents	src/artist/portal/pages/InboxPages.jsx (Documents)	artist-documents/<artist id>/...	0036 fixed the ownership check
Artist application video	src/artist/portal/pages/ApplyPage.jsx	artist-media/applications/<user id>/...	reviewers preview with a 15-min signed URL
Avatar	src/artist/pages/ProfilePage.jsx	artist-avatars (public URL)	
Sponsor assets	src/portal/PartnerExtras.jsx	sponsor-assets/<uid>/...	review on the Reviews page
Event documents (team)	src/admin/components/Event0ps.jsx	event-documents/events/...	audience-scoped reads
Requirement responses (partners)	src/portal/PortalSections.jsx	event-documents/requirements/<id>/...	only while the requirement is open
CMS media	src/admin/components/MediaLibrary.jsx , contentService.upload	content-media/...	1-hour signed URLs for visitors

Documentation drift: `.env.example` still describes `content-media` as public. Migration 0029 made it private, and `docs/OPERATIONS.md` §5 and `src/lib/storage.js` reflect that.

Part III — Database & Backend

21 — Database Architecture · 22 — Database Function / RPC Map · 23 — Edge Function Architecture · 24 — Scheduled Job Architecture

21 — Database Architecture

Supabase Postgres, schema `public`, built by 36 ordered migrations (`supabase/migrations/0001–0036`). Extensions:

- `pgcrypto`: `gen_random_bytes`, `digest`
- `pg_cron`: when available
- Realtime publication `supabase_realtime`, containing `conversations`, `messages`, `notifications` and `event_availability_signal`

Domain map

Database domains (61 tables, 3 views)

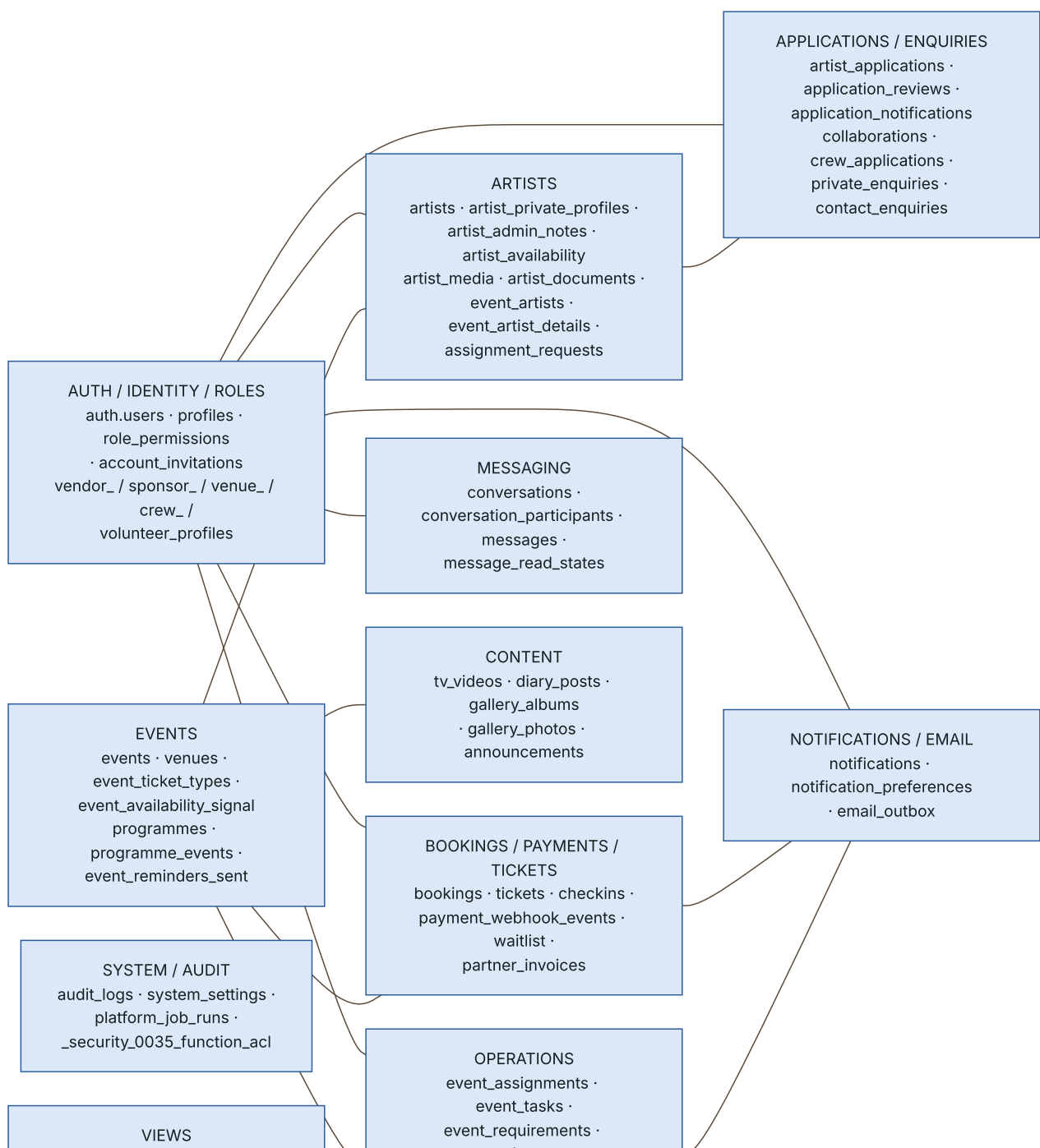


Diagram — Database domains (61 tables, 3 views) (part 1 of 2)

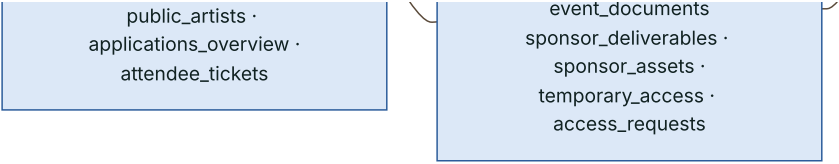


Diagram — Database domains (61 tables, 3 views) (part 2 of 2)

Core relationships

Identity, events and bookings

ER — identity, events, bookings, tickets, waitlist

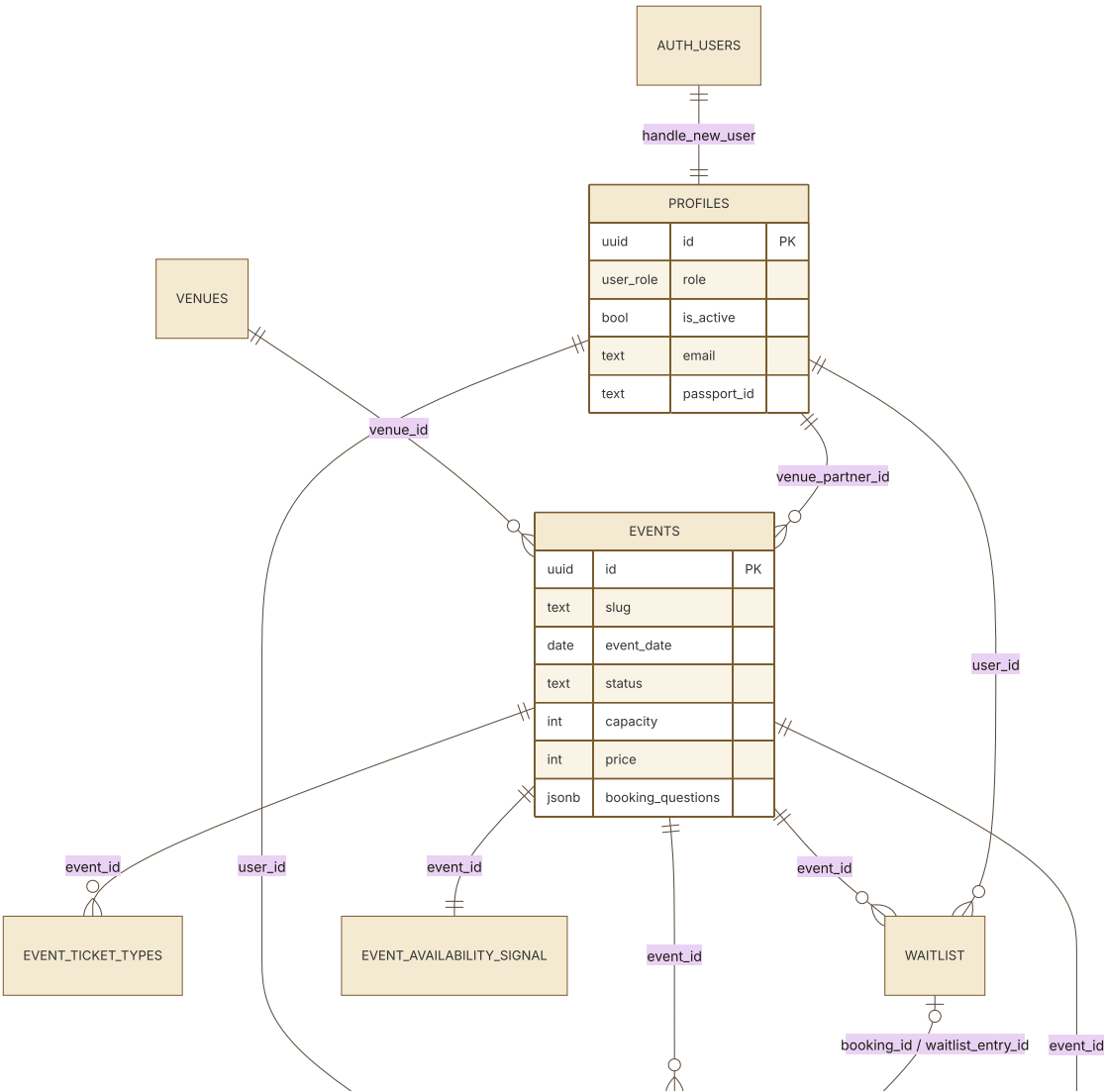


Diagram — ER — identity, events, bookings, tickets, waitlist (part 1 of 2)

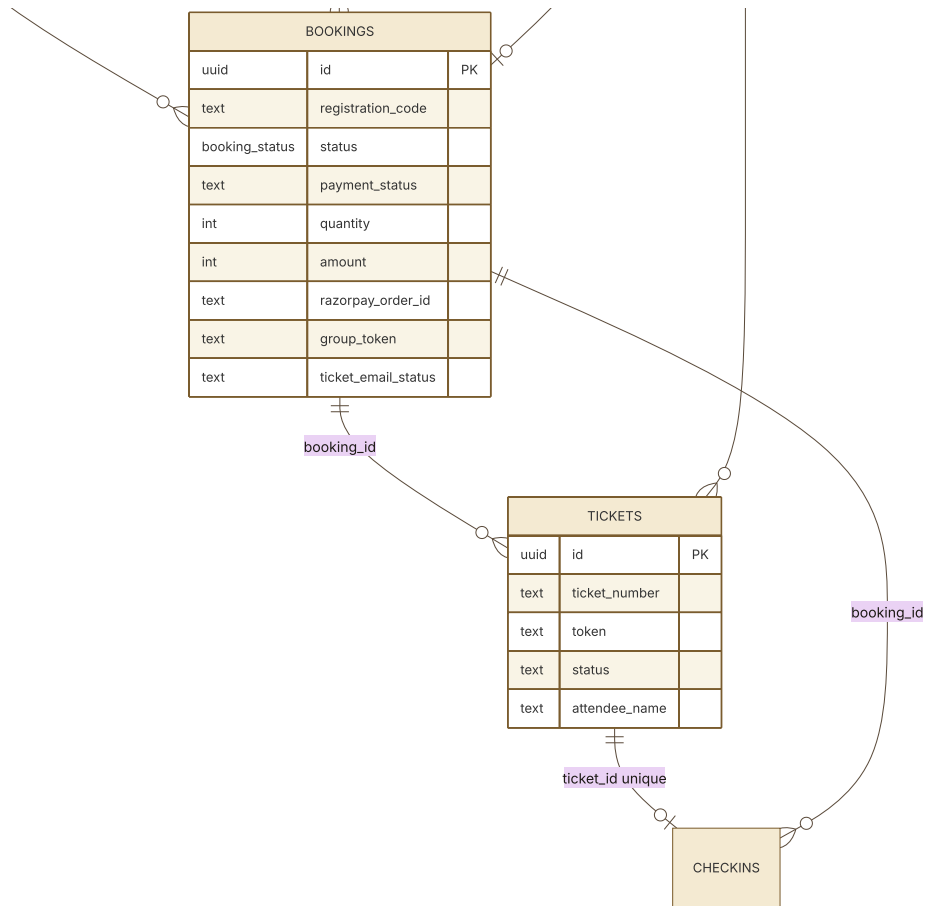


Diagram — ER — identity, events, bookings, tickets, waitlist (part 2 of 2)

Artists and line-ups

ER — artists and applications

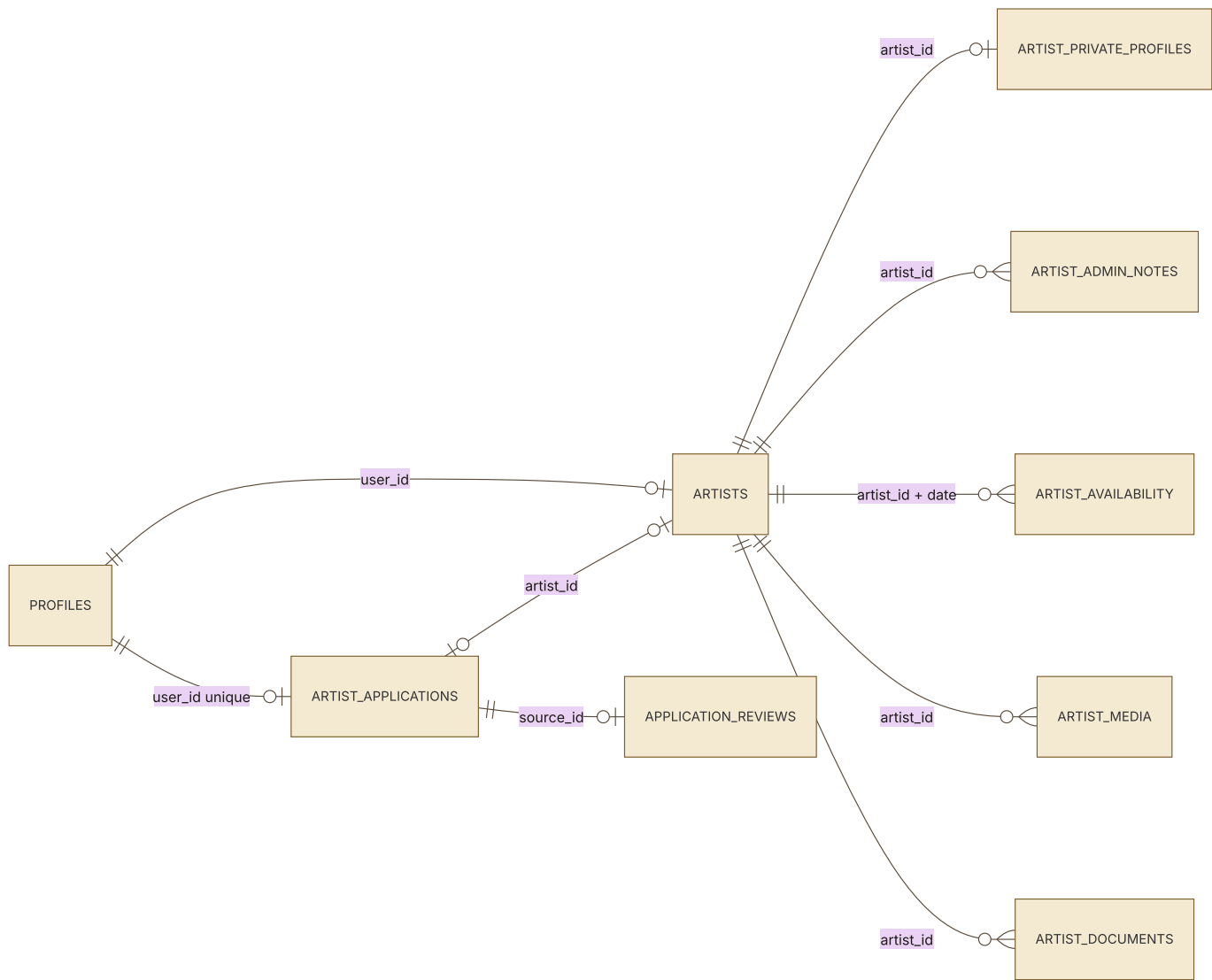


Diagram — ER — artists and applications

ER — line-ups and booking requests

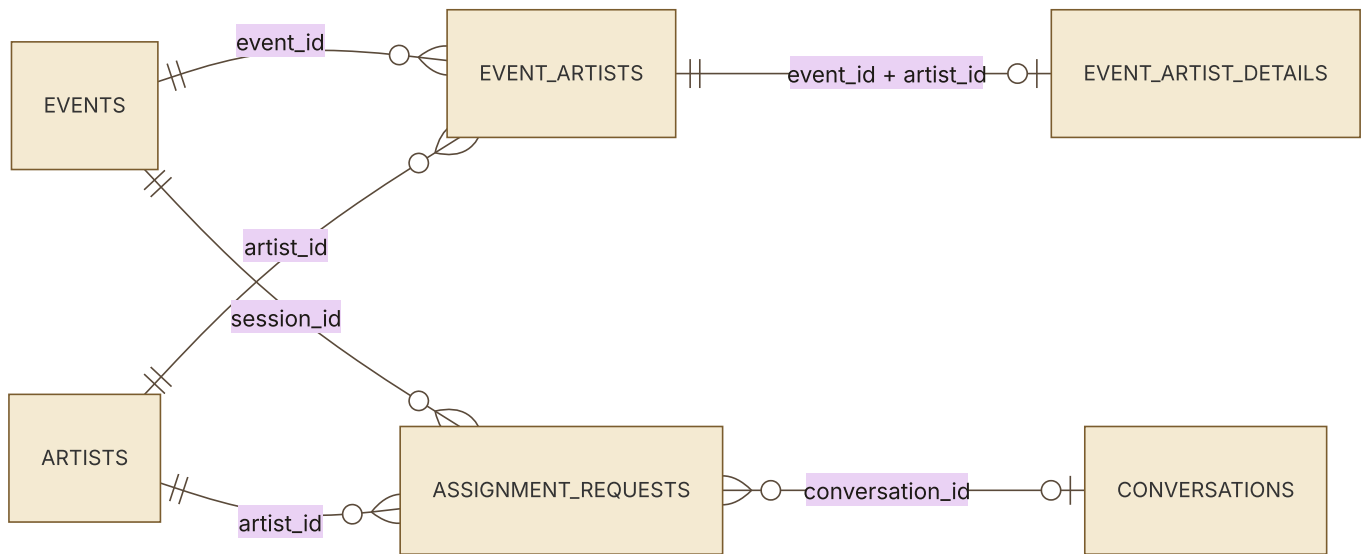


Diagram — ER — line-ups and booking requests

Operations and partners

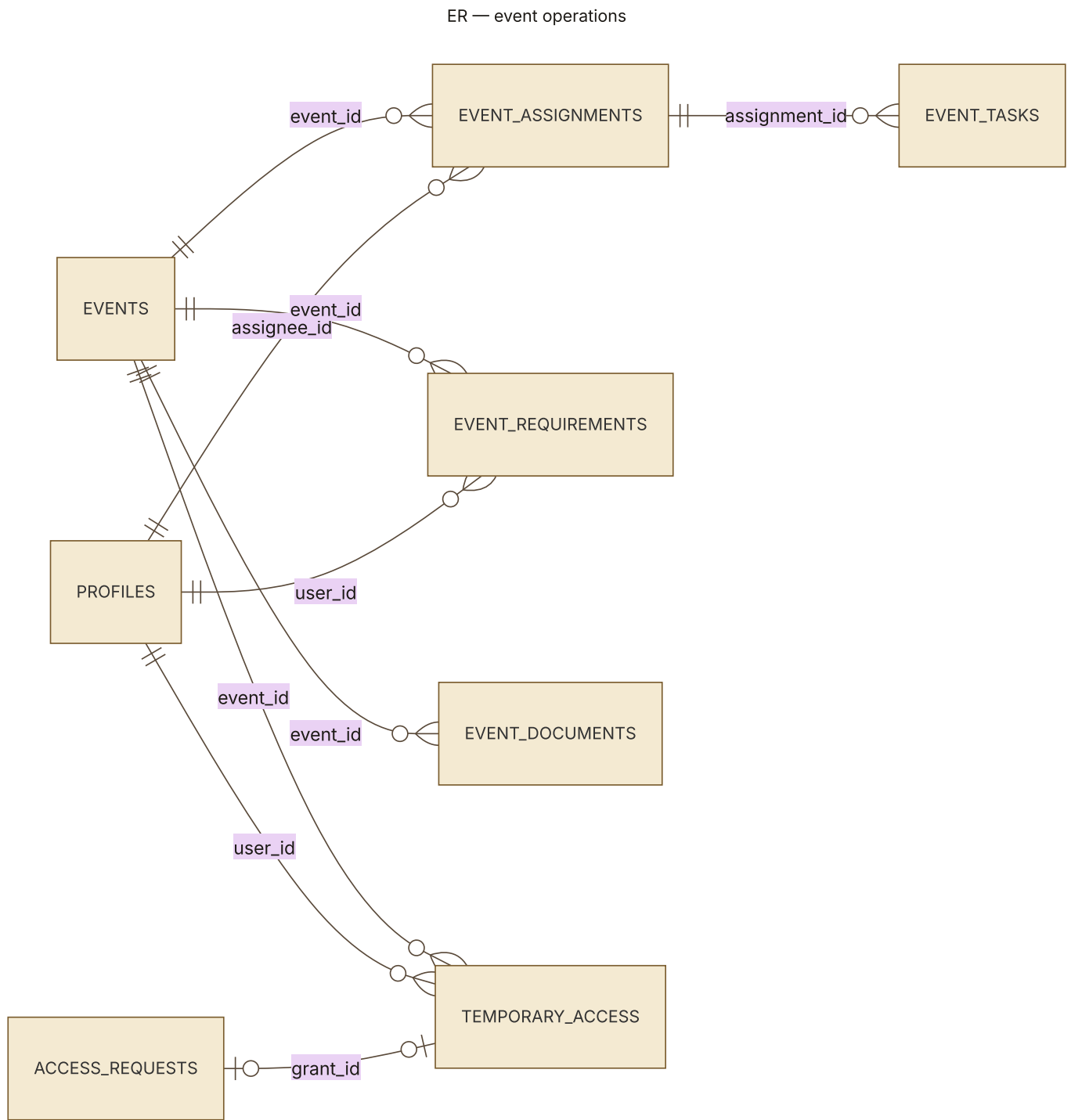


Diagram — ER — event operations

ER — partners and sponsors

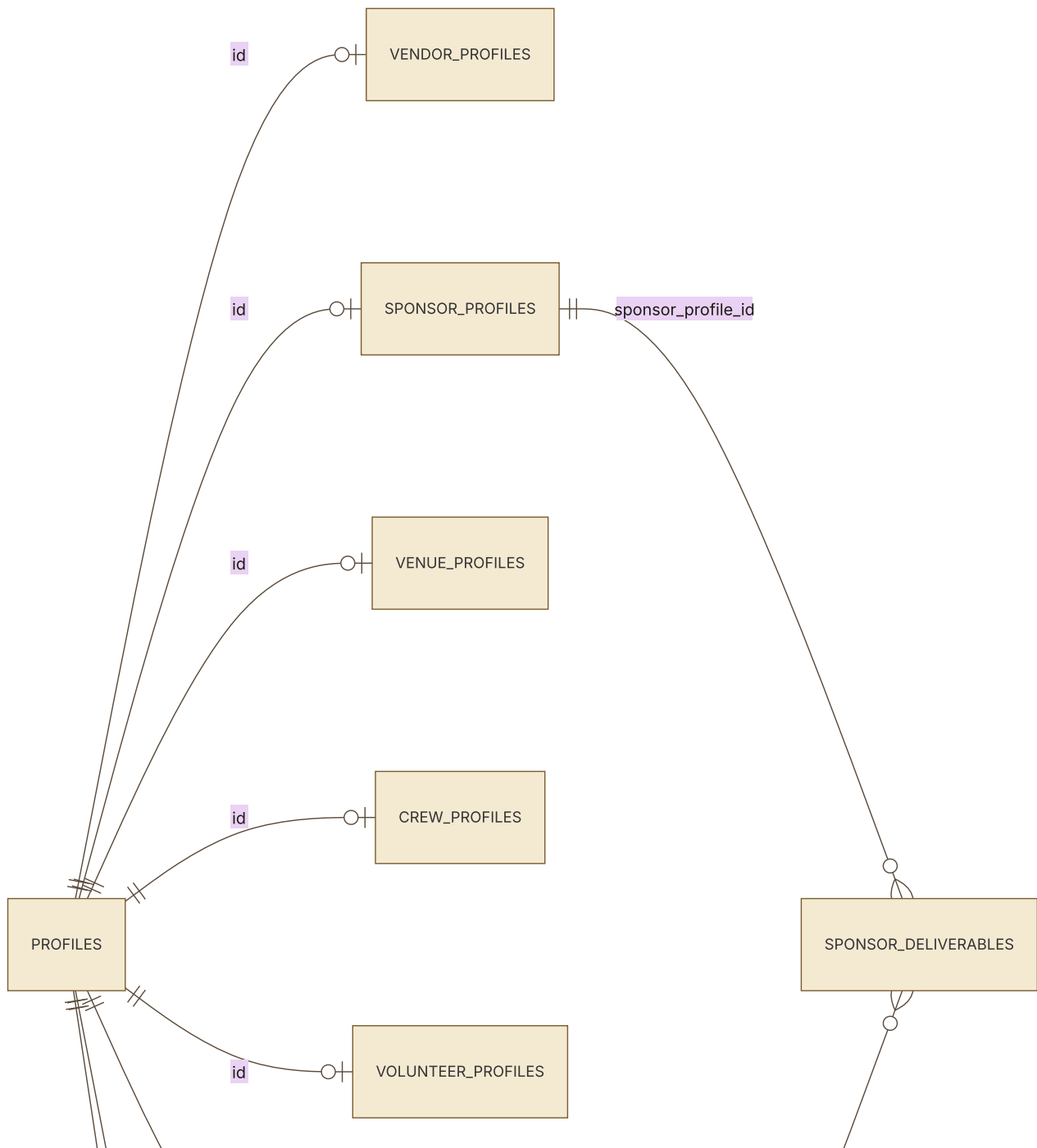


Diagram — ER — partners and sponsors (part 1 of 2)

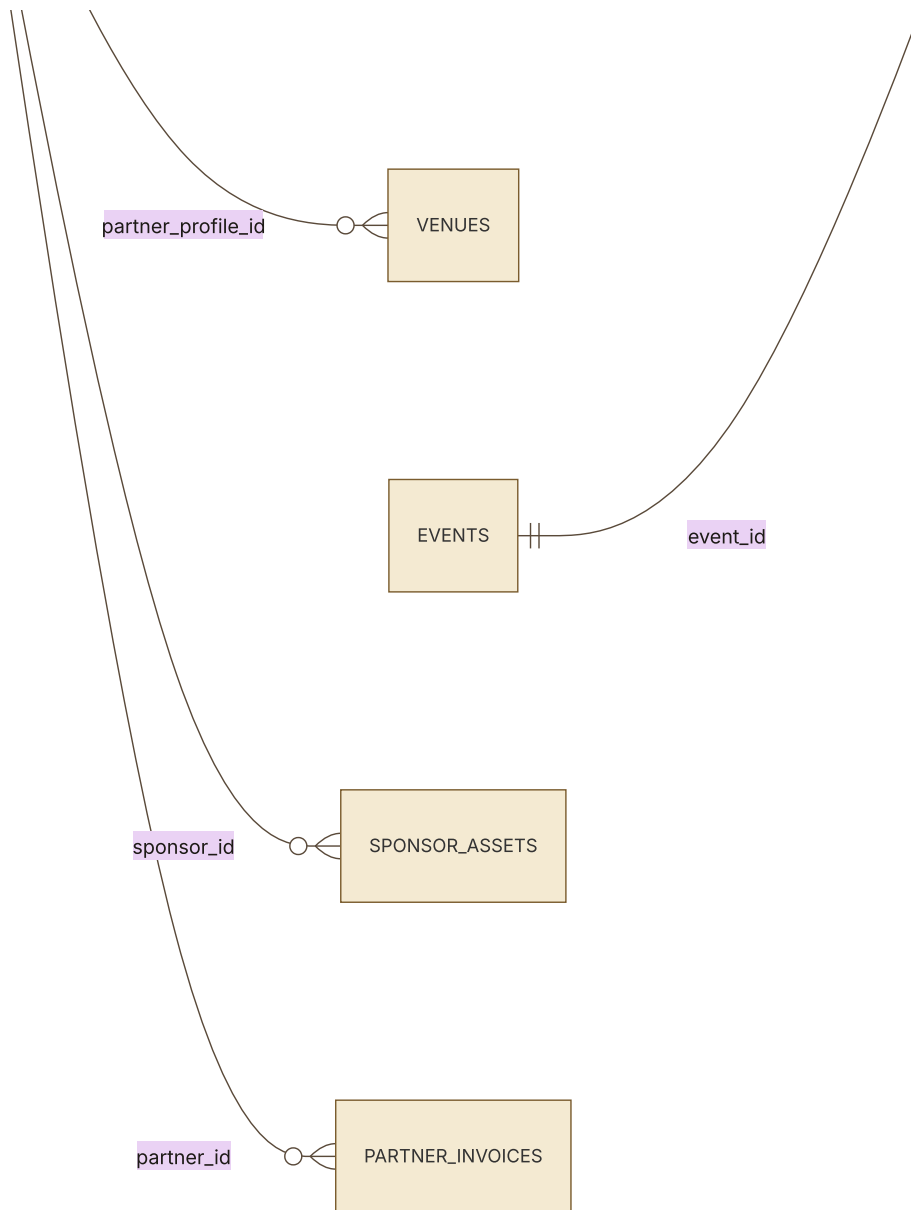


Diagram — ER — partners and sponsors (part 2 of 2)

Table catalogue (important tables)

Table	Purpose	Who reads (RLS)	Who writes	Important triggers / functions
profiles	Account, role, active flag, passport	self; <code>is_admin()</code>	self (non-identity fields); RPCs	<code>handle_new_user</code> , <code>prevent_role_self_escalation</code> , <code>guard_profile_identity</code> , <code>sync_profile_email</code>
role_permissions	Role → permission matrix	console roles	migrations; <code>set_role_permission</code> (Super Admin)	<code>has_permission</code> , <code>my_permissions</code>
events	Sessions	public if status ≠ draft; assigned staff; event members; admin	admin (<code>is_admin()</code>), <code>update_session_content</code>	<code>events_audit</code> , <code>events_notify_change</code> , <code>events_notify_publish</code> , <code>events_prevent_delete_with_bookings</code> , <code>events_seed_ticket_type</code> , <code>events_capacity_to_waitlist</code> , <code>events_availability_signal</code> , <code>events_validate_timezone</code>
event_ticket_types	Per-event prices / capacities	public (active, non-draft event); events.manage	events.manage	<code>sync_event_from_price</code> , availability signal

Table	Purpose	Who reads (RLS)	Who writes	Important triggers / functions
bookings	Seat hold / purchase	self; bookings.view_all	no client insert; create_pending_booking (service role), admin RPCs, Edge Functions	bookings_release_to_waitlist, bookings_flag_late_payment, bookings_system_audit, bookings_availability_signal, bookings_touch_payment
tickets	One per attendee	self; bookings.view_all	confirm_booking_and_issue_tickets only	—
checkins	Attendance	self (own booking); bookings.view_all	check_in_ticket only	unique(ticket_id)
waitlist	Queue + offers	self (user_id / verified JWT email); bookings.view_all; admin	join_waitlist, leave_waitlist, offer functions	waitlist_availability_signal
payment_webhook_events	Razorpay event log	is_admin()	razorpay-webhook (service role)	unique event_id
artists	Artist profile / application record	public if approved (view public_artists); self; entities.manage / events.view_all	self (profile), submit_artist_application, approvals	prevent_artist_status_self_escalation, artists_slug, artists_sync_application, artists_guard_duplicate
artist_applications	8-step application	self; applications.view	self while draft / needs_information; RPCs	guard_artist_application
artist_availability	Artist calendar	self; events.manage / entities.manage	self; set_artist_availability	guard_artist_availability
assignment_requests	Booking requests to artists	artist (non-draft); admin	RPCs	assignment_requests_notify, guard_assignment_request_artist
event_artists / event_artist_details	Line-up / set details	public line-up via views; members; team	save_event_lineup, update_event_artist, respond_to_booking_request, manage_artist_request	notify + audit triggers
collaborations / crew_applications	Partner / crew / volunteer applications	self; admin	self insert (signed in); approvals	duplicate and start guards, receipt and decision notifications, private-notes move
event_assignments, event_tasks, event_requirements, event_documents	Event operations	assignee / members / team	team; partners respond / submit	validation + notify + audit triggers
temporary_access, access_requests	Volunteer check-in grants	own; volunteers.manage	RPCs	guard_temporary_access
notifications	In-app inbox	own	notify() only	Realtime
email_outbox	Email queue	settings.manage	notify(), enqueueTicketEmail (service role)	claim_email_batch, complete_email
conversations, messages, ...	Messaging	participants; admin-level except private	RPCs / RLS inserts	on_message_created, guard_conversation_admin_participant
content tables	CMS	public when published and due	content editors	content_guard_publish, audit triggers

Table	Purpose	Who reads (RLS)	Who writes	Important triggers / functions
audit_logs	Who did what	audit.view	audit_write() , audit_row_change triggers, system inserts	audit_logs_append_only (blocks update / delete)
system_settings	Runtime settings	console (exposed keys via get_runtime_settings)	update_system_setting (settings.manage)	audited
platform_job_runs	Scheduled job log	operations.manage	run_platform_jobs	advisory lock

Views

View	Purpose
public_artists (0020)	Safe public subset of approved artists
applications_overview (0017/0033)	Union of artist / collaboration / crew applications for the admin list. Review notes come from application_reviews (null for applicants)
attendee_tickets (0016/0024)	Ticket + booking + event + check-in + payment status for attendee lists and CSV

Key settings (system_settings , seeded by migrations)

Key	Default	Used by
bookings.pending_timeout_minutes	30	expire_stale_bookings
bookings.tax_percent	18	booking_quote
waitlist.offer_hold_minutes	120	offer_waitlist_seats
waitlist.allocation	strict_order	offer_waitlist_seats
notifications.event_reminder_hours	24	send_event_reminders
notifications.send_approval_emails	true	ApplicationsPage
auth.admin_idle_timeout_minutes	60	AdminShell
checkin.allow_manual	(true if unset)	check_in_ticket
artists.availability_stale_days	30	send_availability_reminders
artists.request_default_days	7 (fallback)	create_artist_request

22 — Database Function / RPC Map

About 228 SQL functions exist. This map covers the ones **called by the frontend or the Edge Functions**, plus the internal functions those depend on. Every listed RPC is `SECURITY DEFINER` with `set search_path = public` and checks the caller itself, unless noted.

Grant model:

- **Service-role only** (revoke ... from public, anon, authenticated; grant ... to service_role):
`create_pending_booking`, `confirm_booking_and_issue_tickets`, `settle_payment`, `expire_stale_bookings`,
`record_webhook_failure`, `claim_email_batch`, `run_platform_jobs`, and `audit_write` / `notify` /
`notify_permission_holders` for direct calls.
- **Callable by anonymous visitors:** `booking_quote`, `event_availability`, `invitation_preview`,
`content_media_is_public`.
- Migration 0035 revoked `EXECUTE` from anon/authenticated on every remaining definer function that does not check its caller.

RPC call graph — main callers

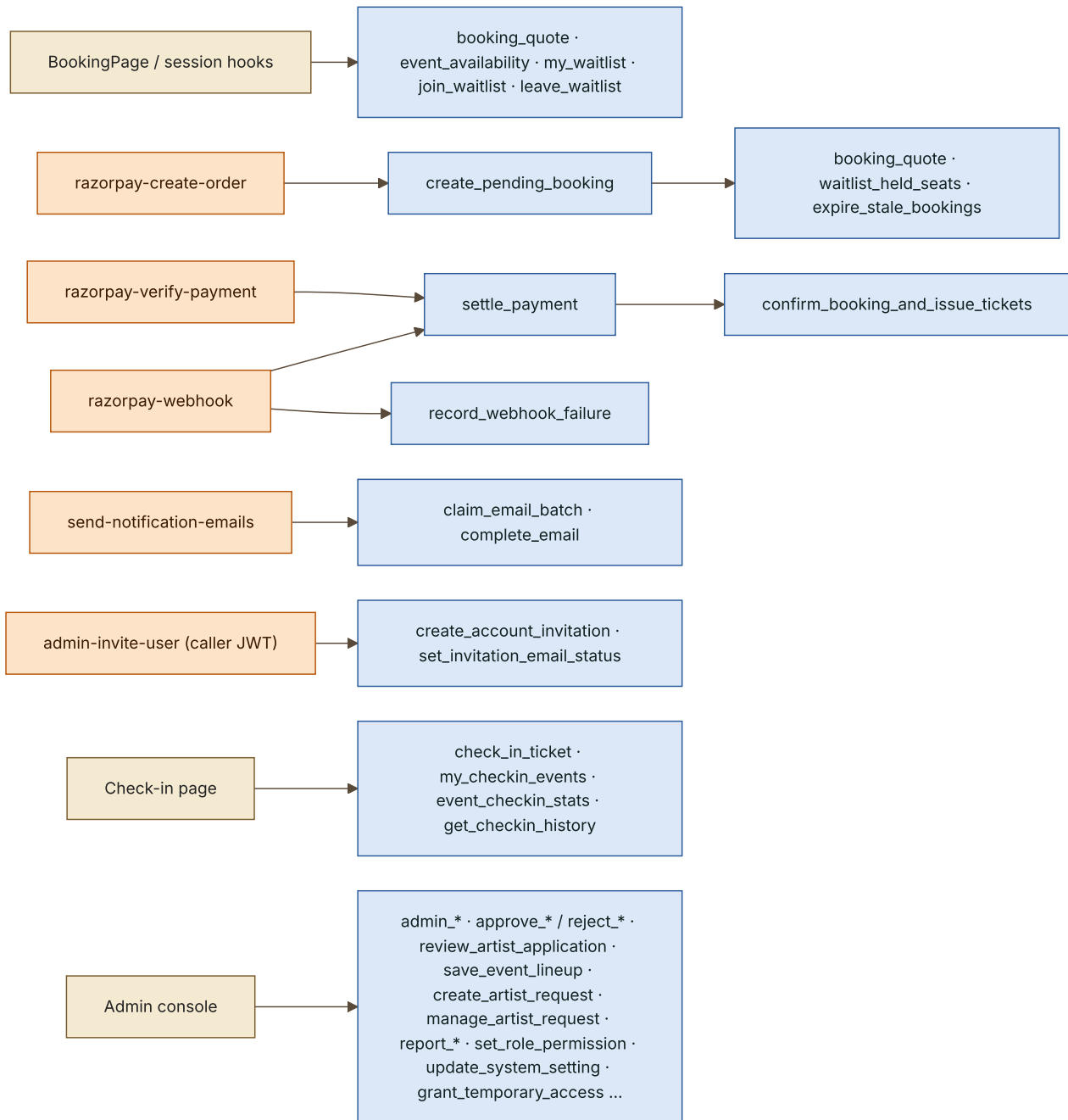


Diagram — RPC call graph — main callers (part 1 of 2)

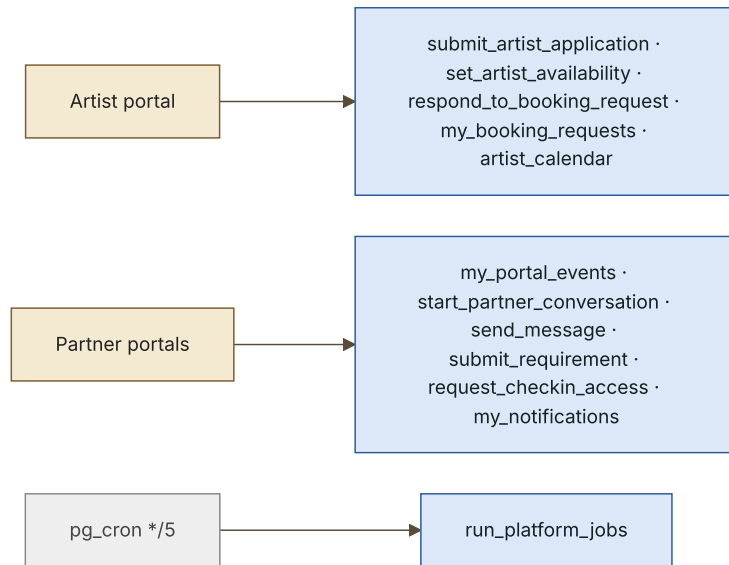


Diagram — RPC call graph — main callers (part 2 of 2)

AUTH / RBAC

RPC	Caller	Validation	Tables	Returns	Security
my_permissions()	AdminSession	—	role_permissions, temporary_access	text[] (role perms + checkin.perform while a grant is live)	inactive → no role
get_runtime_settings()	AdminSession	—	system_settings (exposed)	jsonb	console
log_auth_event(action)	AdminGate / AdminShell	de-duplicated, console accounts only	audit_logs	void	—
admin_set_user_role(user, role, reason)	UsersPage	roles.manage · not self · Super Admin rules · last Super Admin	profiles, audit_logs	void	notify role.changed
admin_set_user_active(user, active, reason)	UsersPage	roles.manage (via trigger) · not self	profiles	void	audited
set_role_permission(role, perm, on)	RolesPage	roles.manage · only admin/staff · not roles.manage	role_permissions	void	audited
create_account_invitation(...)	admin-invite-user (as inviter)	can_invite_role · email · rank · hash format	account_invitations	invitation row	audited
invitation_preview(token) / accept_account_invitation(token)	InvitationPage	hash match · pending · expiry · email = auth email · active · rank	account_invitations, profiles	preview / role granted	preview is anon-callable
list_account_invitations, revoke_account_invitation	UsersPage	invite rights	account_invitations	rows / void	

BOOKING / PAYMENT

RPC	Caller	Validation	Tables	Returns	Security
booking_quote(event, type, qty)	session page, TicketTypesEditor, create_pending_booking	qty 1-50 · active type · event not draft	event_ticket_types, system_settings	jsonb subtotal / tax / total	anon-callable

RPC	Caller	Validation	Tables	Returns	Security
<code>event_availability(event)</code>	session page, waitlist directory	—	events, bookings, event_ticket_types, waitlist	jsonb remaining, sold_out, per type	anon-callable
<code>create_pending_booking(...)</code>	razorpay-create-order	names = qty · event lock · quantity range · ticket type + its capacity · answers (<code>booking_answers_error</code>) · instagram / notes / collab interests · capacity incl. pending + waitlist holds	bookings, waitlist (converted), audit_logs	bookings row	service role only
<code>settle_payment(order, payment, amount, source)</code>	razorpay-verify-payment, razorpay-webhook	see 12 decision tree	events, bookings, tickets, audit_logs, notifications	jsonb result	service role only
<code>confirm_booking_and_issue_tickets(booking)</code>	settle_payment, admin_create_comp_booking	lock booking · idempotent	bookings, tickets	setof tickets	service role only (0017 security fix)
<code>record_webhook_failure(event, error)</code>	razorpay-webhook	—	payment_webhook_events, notifications	void	service role
<code>admin_cancel_booking, admin_record_refund, admin_cancel_ticket, admin_create_comp_booking</code>	Bookings.jsx	bookings.manage (+ payments.view for refunds) · reason / reference · state checks	bookings, tickets, audit_logs	void / booking	audited

WAITLIST

RPC	Caller	Validation	Tables	Returns
<code>join_waitlist(event, qty)</code>	WaitlistPanel	signed-in active · event open · qty range · not duplicate · truly full	waitlist, notifications	position
<code>leave_waitlist(event)</code>	WaitlistPanel, dashboard	own entry	waitlist	—
<code>my_waitlist()</code>	session page, dashboard, directory	own	waitlist	entries + offer expiry
<code>admin_offer_waitlist, admin_remove_waitlist_entry</code>	WaitlistSection	bookings.manage-level	waitlist, audit_logs	—
<code>offer_waitlist_seats(event) / expire_waitlist_offers()</code>	triggers / platform job	event lock · allocation policy	waitlist, notifications, audit_logs	count

CHECK-IN

RPC	Caller	Validation	Tables	Returns
<code>check_in_ticket(token, event, method, notes, attendee_ids, preview)</code>	checkinService, Attendees.jsx	permission or temporary access · <code>can_checkin_event</code> · manual setting · token type · event · status · selection	tickets, bookings, checkins, audit_logs	jsonb result (13 outcomes)
<code>my_checkin_events()</code>	check-in page	role / assignment / grant	events	events list
<code>event_checkin_stats(event)</code>	check-in, event pages	permission	tickets, checkins	counts

RPC	Caller	Validation	Tables	Returns
<code>get_checkin_history(...)</code> , <code>booking_checkin_history(booking)</code>	history pages, booking drawer	checkin.history / bookings access	checkins	rows
<code>request_checkin_access</code> , <code>grant_temporary_access</code> , <code>revoke_temporary_access</code> , <code>decline_access_request</code> , <code>my_checkin_access</code> , <code>volunteers_overview</code> , <code>volunteer_access_activity</code>	volunteer portal / VolunteersPage	volunteer role · access.grant · durations 30–720 min · advisory lock	access_requests, temporary_access, event_assignments, audit_logs	rows

EVENTS / ARTISTS

RPC	Caller	Validation	Tables	Returns
<code>find_available_artists(event, date, start, end)</code>	ArtistAvailability.jsx	events.manage / entities.manage	artists + <code>artist_day_status</code>	artists with status + conflicts
<code>artist_schedule_check(...)</code>	EventDetailPage	events.manage / entities.manage	as above	jsonb
<code>save_event_lineup(event, items)</code>	EventEditor, EventDetailPage	events.manage · event lock · approved artist · <code>assert_artist_bookable</code> (advisory lock) · <code>assert_no_slot_overlap</code>	event_artists, event_artist_details, assignment_requests	jsonb added / requested
<code>update_event_artist</code> , <code>remove_event_artist</code>	EventDetailPage	events.manage · same checks	same	—
<code>create_artist_request(event, artist, details, send)</code>	EventDetailPage	events.manage · approved artist with account · bookable · no overlap	assignment_requests, audit_logs	id
<code>manage_artist_request(id, action, reason)</code>	EventDetailPage	events.manage · state machine	assignment_requests, event_artist_details, event_artists	—
<code>respond_to_booking_request(id, accept, reason)</code>	artist Requests	own request · pending · bookable	assignment_requests, event_artists, event_artist_details	—
<code>my_booking_requests()</code> , <code>mark_artist_request_viewed(id)</code>	artist portal	own, non-draft	assignment_requests	rows
<code>set_artist_availability(...)</code> , <code>artist_availability_summary</code> , <code>artist_calendar</code> , <code>admin_calendar</code> , <code>artist_day_summary</code>	artist portal / admin	approved artist / team	artist_availability (+ line- ups, requests)	rows / jsonb
<code>event_health(event)</code> , <code>event_command_center(event)</code>	EventDetailPage, EventOps	team	many	jsonb
<code>review_requirement</code> , <code>close_requirement</code> , <code>submit_requirement</code>	EventOps / portals	events.manage / own requirement	event_requirements	—
<code>update_session_content(event, fields)</code>	Content → Sessions	content.manage_sessions + content.edit · whitelisted fields	events	events row

APPLICATIONS

RPC	Caller	Validation	Tables	Returns
<code>submit_artist_application()</code> , <code>withdraw_artist_application()</code>	<code>ApplyPage</code> , status page	own · status · required fields	artist_applications, artists, notifications, audit_logs	row
<code>review_artist_application(id, action, items, message, internal)</code>	<code>ArtistApplicationsPage</code>	applications.review · state machine · reason for reject	artist_applications, artists, profiles, application_reviews	row
<code>approve_artist_application</code> , <code>approve_collaboration</code> , <code>approve_crew_application</code> (+ <code>reject_*</code>)	<code>ApplicationsPage</code> (and the RPC above)	applications.review · pending only	application tables, *_profiles, profiles.role, application_notifications, event_assignments (volunteer), audit_logs	void

MESSAGING / NOTIFICATIONS

RPC	Caller	Validation	Tables
<code>get_or_create_support_conversation</code> , <code>assign_conversation</code> , <code>reopen_conversation</code>	<code>chatService</code>	signed in / admin	conversations, participants
<code>start_partner_conversation</code> , <code>admin_start_partner_conversation</code> , <code>start_private_artist_conversation</code>	portals / admin	partner kind + event membership / messages.manage / Super Admin	conversations, participants, messages
<code>my_conversations</code> , <code>admin_conversations</code> , <code>conversation_messages</code> , <code>send_message</code> , <code>mark_conversation_read</code> , <code>set_conversation_status</code> , <code>set_conversation_meta</code>	<code>portalApi</code> , <code>MessagesPanel</code>	participant / messages.manage · private excluded · length 1–4000 · closed check	conversations, messages, read states
<code>my_notifications</code> , <code>notification_unread_count</code> , <code>mark_notifications_read</code> , <code>my_notification_preferences</code> , <code>set_notification_preference</code> , <code>portal_announcements</code>	bell, portals	own	notifications, notification_preferences, announcements
<code>send_custom_notification(user, ...)</code>	<code>CustomNotification.jsx</code>	Super Admin	notifications, email_outbox
<code>claim_email_batch(limit)</code> , <code>complete_email(id, ok, error)</code>	<code>send-notification-emails</code>	service role	email_outbox

REPORTS / OPERATIONS

RPC	Caller	Notes
<code>admin_dashboard_summary</code> , <code>staff_dashboard</code> , <code>admin_operations_overview</code> , <code>admin_search</code>	Dashboard, header search	permission-scoped aggregates
<code>report_event_performance</code> , <code>report_revenue_by_month</code> , <code>report_applications</code> , <code>report_staff_activity</code> , <code>report_platform_activity</code>	<code>ReportsPage</code> , <code>EventDetailPage</code>	reports.view
<code>update_system_setting(key, value)</code>	<code>SettingsPage</code>	settings.manage · audited
<code>run_platform_jobs(source)</code>	<code>pg_cron / scripts/run-jobs.sh</code>	service role · advisory lock · run log

23 — Edge Function Architecture

Seven Deno functions in `supabase/functions/`, plus shared modules in `_shared/`:

Shared module	Purpose
<code>cors.ts</code>	origin allow-list
<code>crypto.ts</code>	HMAC-SHA256, constant-time compare, <code>requireSecret</code>
<code>email.ts</code>	provider module: resend / mailpit / log / disabled
<code>ticketEmail.ts</code>	build, queue and mark the ticket email; QR via <code>esm.sh/qrcode</code>

Each function imports `@supabase/supabase-js@2` from `https://esm.sh`.

Deployment is PRODUCTION CONFIGURATION, not done. There is no `supabase/config.toml`. Deploy flags are documented in `docs/OPERATIONS.md` §3a:

- `razorpay-webhook` and `send-notification-emails` deploy with `--no-verify-jwt`.
- The other five keep the gateway JWT check.

Master diagram

Edge Functions — callers, auth, data, external services

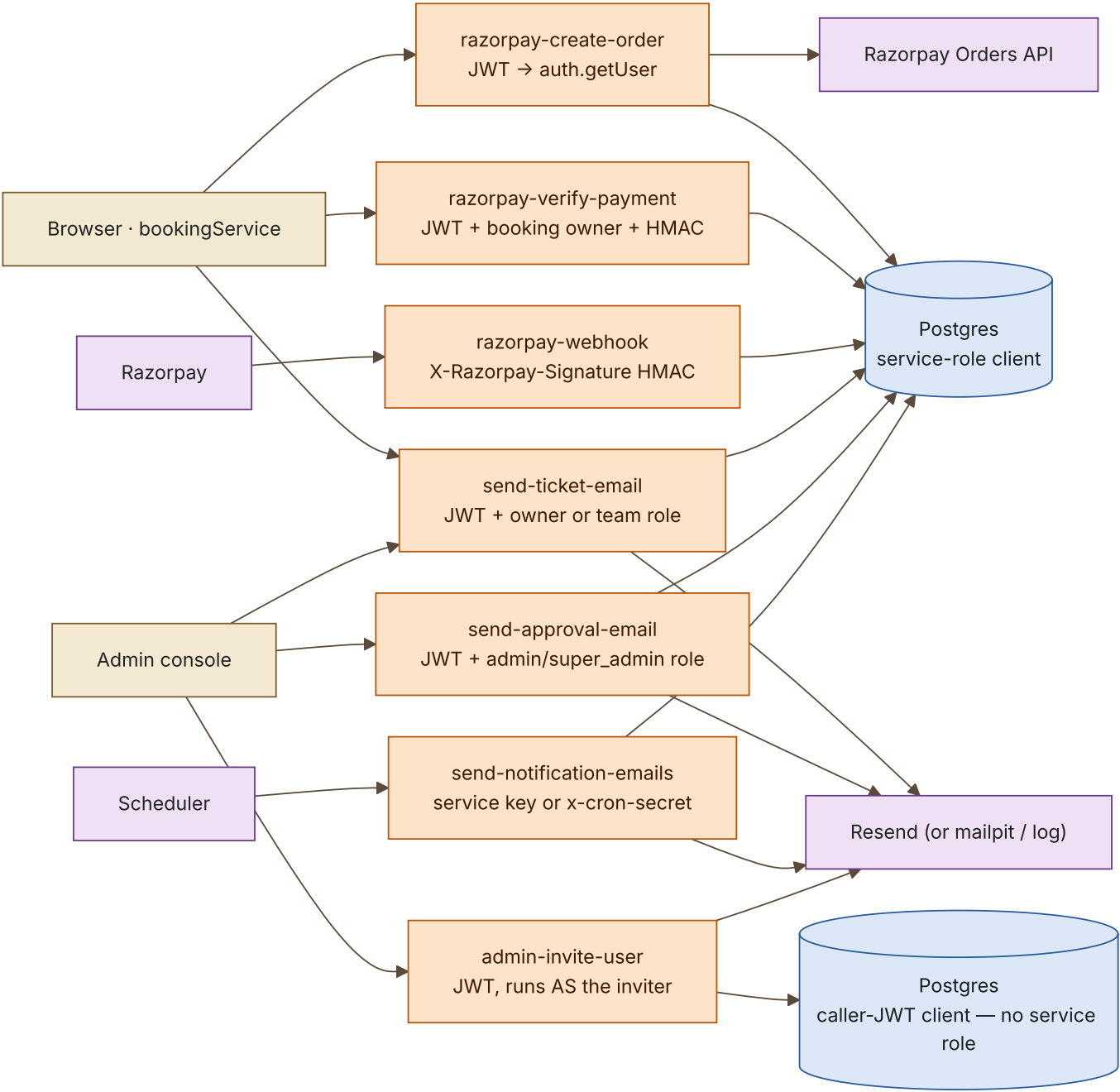


Diagram — Edge Functions — callers, auth, data, external services

Function by function

razorpay-create-order

Caller	Browser (bookingService.createPaymentOrder)
Auth	Gateway JWT check + authClient.auth.getUser() → 401 Sign in required.
Input	eventId, quantity, tierId, attendeeName, attendeeEmail, attendeePhone, attendeeNames[], details{answers, instagram, note, collabInterests, collabNote}

Validation	qty integer 1–50; name 1–120; email regex; Indian mobile <code>^[6-9]\d{9}\$</code> after stripping +91/0; names = qty, each 1–120; tier <code>^[a-z][a-z0-9_]{0,31}\$</code> ; details is an object; event exists; status <code>on-sale</code> , or <code>sold-out</code> with a live waitlist offer; event min/max quantity
Database calls	service role: read <code>events</code> , read <code>waitlist</code> ; rpc <code>create_pending_booking</code> ; update <code>bookings.razorpay_order_id</code> ; on failure update <code>bookings.status = failed</code>
External	POST <code>https://api.razorpay.com/v1/orders</code> (Basic auth key id : secret), amount in paise, receipt = registration code
Output	<code>{order_id, amount, currency 'INR', key_id, booking_id}</code>
Failure	400 validation / <code>INVALID_*</code> ; 404 event; 409 not on sale / <code>SOLD_OUT</code> ; 503 Razorpay not configured (hold released); 502 order API failed (hold released); 500 otherwise

razorpay-verify-payment

Caller	Browser after the Razorpay Checkout success callback
Auth	JWT + <code>getUser</code> ; booking must belong to the caller (403); order id must match (400)
Input	<code>booking_id, razorpay_order_id, razorpay_payment_id, razorpay_signature</code>
Logic	Already confirmed and verified → return the tickets (idempotent, re-queues the email). Otherwise <code>requireSecret('RAZORPAY_KEY_SECRET')</code> (503 if missing) → <code>HMAC_SHA256(order_id + " " + payment_id)</code> → <code>timingSafeEqual</code> (400 on mismatch) → <code>rpc settle_payment(source 'verify')</code> → <code>enqueueTicketEmail</code>
Output	<code>{success, booking, tickets}</code> ; 409 <code>{review:true}</code> when <code>needs_review</code>
Failure	500 "Could not confirm booking — contact support with your payment ID" if settlement errors

razorpay-webhook

Caller	Razorpay servers (no CORS, no JWT; <code>--no-verify-jwt</code>)
Auth	<code>RAZORPAY_WEBHOOK_SECRET</code> (503 if missing); HMAC of the raw body vs <code>X-Razorpay-Signature</code> (400 on mismatch)
Events handled	<code>payment.captured</code> / <code>order.paid</code> → <code>settle_payment(source 'webhook', amount)</code> + <code>enqueueTicketEmail</code> ; <code>payment.authorized</code> → mirror; <code>payment.failed</code> → pending booking <code>failed</code> ; <code>refund.processed</code> / <code>refund.created</code> → <code>refunded_amount, payment_status</code>
Idempotency	<code>payment_webhook_events.event_id = "<event>:<payment id>"</code> unique; duplicate of a processed event → <code>200 ok (duplicate)</code> ; recorded but unfinished → processed now
Failure	Retryable DB errors → <code>500</code> (Razorpay redelivers); permanent (<code>not_found</code>) → <code>record_webhook_failure + 200</code> ; no stable id → <code>200 skip</code>
Repo caveat	The header says: verify event names and payload fields against a live Razorpay payload before going live. DOCUMENTED BUT NOT VERIFIED IN CODE

send-ticket-email

Caller	Browser after a verified payment (best effort); admin "Resend email" (<code>force: true</code>)
Auth	JWT; caller is the booking owner, or <code>profiles.role</code> in <code>staff, admin, super_admin</code> (role-string check, not <code>has_permission</code>)
Logic	Booking must be confirmed (409). Without <code>force</code> : already sent → <code>already_sent</code> ; already queued / sending / sent in the outbox → <code>queued</code> (no second copy). Otherwise build and send directly, then mark <code>ticket_email_status</code>
External	Resend; <code>esm.sh/qrcode</code> for the QR attachment

send-notification-emails

Caller	Scheduler (Supabase Cron HTTP / external) every 1–2 min: PRODUCTION CONFIGURATION ; locally <code>scripts/run-jobs.sh emails</code>
Auth	Authorization: Bearer <code><service role key></code> or <code>x-cron-secret: CRON_SECRET</code> ; anything else 403
Logic	Provider not configured → <code>{configured:false}</code> and the queue is untouched. Otherwise <code>claim_email_batch(25)</code> → per row: ticket rows rendered from the booking (<code>sendTicketRow</code>), others with <code>notificationHtml</code> → <code>complete_email</code>
Output	<code>{claimed, sent, failed, skipped}</code>

send-approval-email

Caller	ApplicationsPage via notificationService.sendApprovalEmail
Auth	JWT + caller profiles.role in admin, super_admin (403)
Input	source_table (collaborations, crew_applications, artists), source_id, optional force
Logic	Requires the application_notifications approval row (404 otherwise); skips if already sent unless forced; recipient = applicant's account email; portal link per role
Output / state	application_notifications.status sent / failed

admin-invite-user

Caller	UsersPage → adminApi.inviteUser
Auth	JWT; all DB work runs under the inviter's JWT (no service role); SITE_URL required (503)
Logic	Validate; random 32-byte token → SHA-256 hash → create_account_invitation (the DB decides who may invite which role, 42501 → 403); email the SITE_URL/invitation#token=... link; set_invitation_email_status
Output	{ok, invitation_id, expires_at, existing_account, email_status} + invite_url once if email failed

Shared CORS behaviour (_shared/cors.ts)

- The response echoes **the request's own origin only if it is on the allow-list**: ALLOWED_ORIGINS, else SITE_URL. It is never *.
- With neither set, only http://localhost:* / http://127.0.0.1:* are allowed, so a deployment that forgot SITE_URL **fails closed**.
- Methods POST, OPTIONS; Vary: Origin.
- Applied to the browser-called functions. The webhook and email drain don't need it.

24 — Scheduled Job Architecture

Schedulers in the repository

Scheduler	What	Cadence	Defined where	Status
pg_cron job tangy- platform-jobs	<code>select public.run_pla tform_jobs()</code>	<code>*/5 * * *</code> * (every 5 min)	0020: <code>create extension if not exists pg_cron;</code> inside a <code>do</code> block that only runs if <code>pg_cron</code> is available, otherwise it raises a notice to schedule externally	IMPLEMENTED (auto-created when the extension exists). Production confirmation is an operator step (<code>docs/OPERATIONS.md §2a Step 1</code>)
pg_cron job tangy-log- expired-access	<code>select public.log_exp ired_access()</code>	<code>*/5 * * *</code> *	0018, only if <code>pg_cron</code> was already installed at that point	Conditional. The same work also runs inside <code>run_platform_jobs</code> , so it is redundant when present
Email drain	<code>POST /functions/v1/ send- notification- emails with x- cron-secret</code>	every 1–2 min (runbook)	Not in code: Supabase Cron HTTP job or an external scheduler	PRODUCTION CONFIGURATION, not done
Opportunistic runs	<code>expire_stale_b ookings()</code>	before every checkout	<code>create_pending_booking</code>	IMPLEMENTED
	<code>expire_booking _requests()</code>	before every artist response	<code>respond_to_booking_request</code>	IMPLEMENTED
	<code>offer_waitlist _seats()</code>	on every seat release / capacity change	triggers	IMPLEMENTED
Local manual run	<code>scripts/run- jobs.sh [emails]</code>	on demand	script	DEMO / LOCAL ONLY

run_platform_jobs()

run_platform_jobs — single-flight job runner (every 5 minutes)

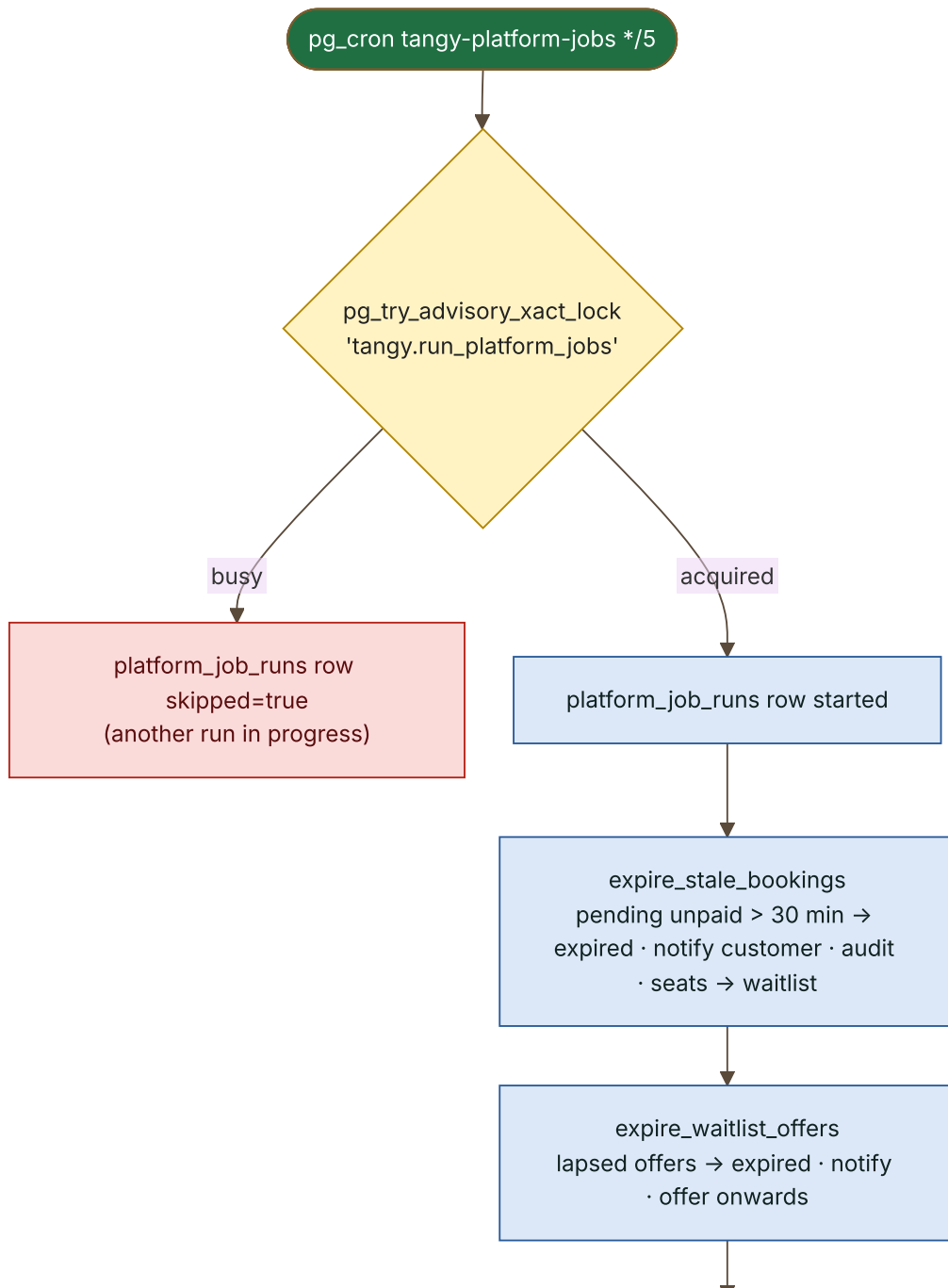


Diagram — run_platform_jobs — single-flight job runner (every 5 minutes) (part 1 of 3)

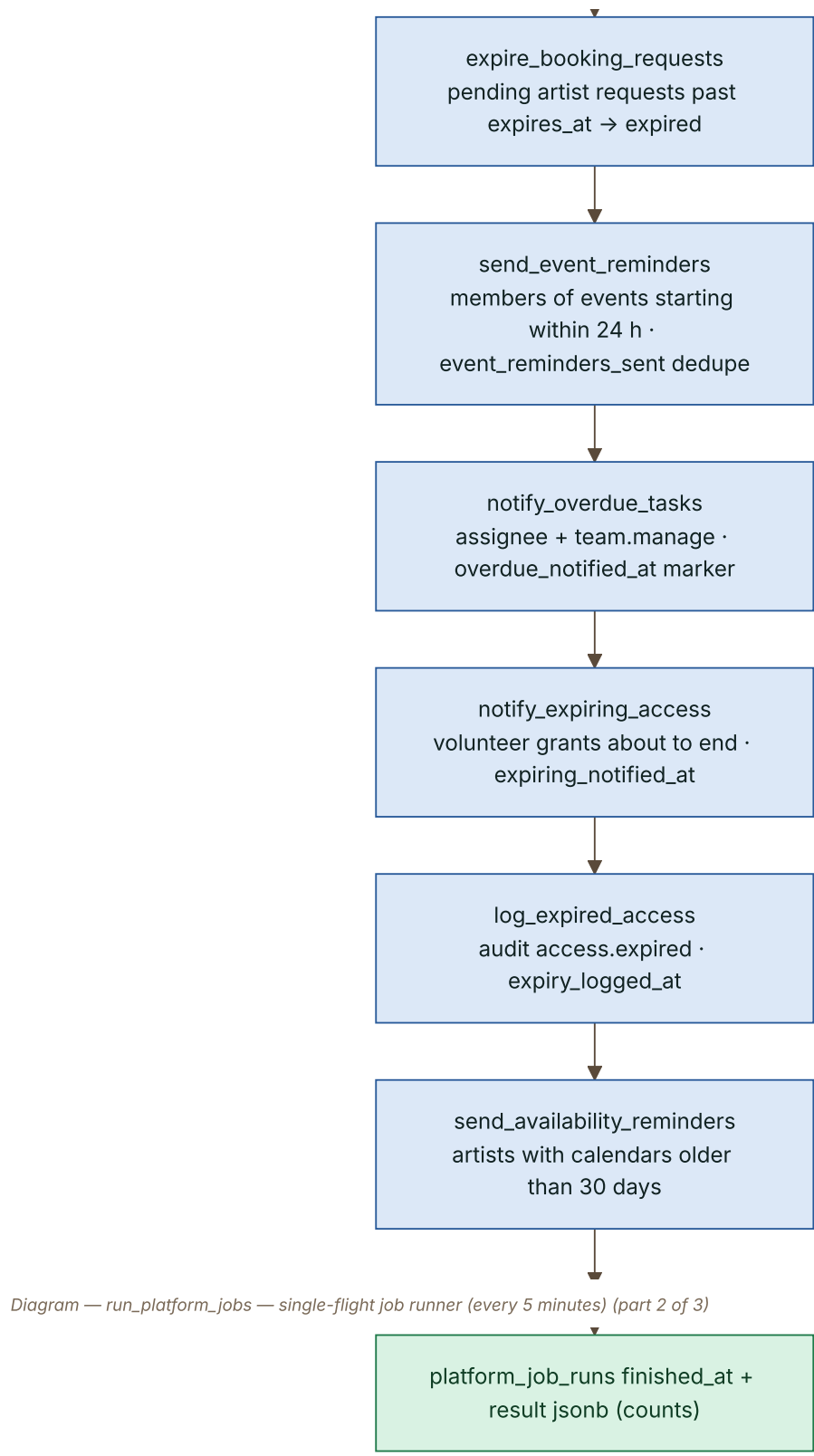


Diagram — run_platform_jobs — single-flight job runner (every 5 minutes) (part 3 of 3)

Job	Tables affected	Notifications	Email	Audit / log
expire_stale_bookings	bookings → expired	booking_payment.expired	no (catalogue email = false)	booking.expired
expire_waitlist_offers	waitlist → expired, new offers	waitlist.offer_expired, waitlist.offer	yes	waitlist.offered
expire_booking_requests	assignment_requests → expired	booking.expired to the requester (trigger)	no	—

Job	Tables affected	Notifications	Email	Audit / log
send_event_reminders	event_reminders_sent	event.reminder	yes	—
notify_overdue_tasks	event_tasks.overdue_notified_at	task.overdue	no	—
notify_expiring_access	temporary_access.expiring_notified_at	access.expiring	no	—
log_expired_access	temporary_access.expiry_logged_at	—	—	access.expired
send_availability_reminders	none (dedupe = an availability.reminder notification in the last N days)	availability.reminder	no	—
every run	platform_job_runs	—	—	run log readable with operations.manage

Every job is **idempotent**: it touches only eligible rows and stamps markers, uses `for update skip locked` where rows are claimed, and the runner holds a transaction-scoped advisory lock. Tests: `supabase/tests/media_realtime_jobs.test.sql` (run once, run twice, expired and already-processed records) and `scripts/test-concurrency.sh` (overlapping runs).

Email drain

Email drain — operator-scheduled HTTP job

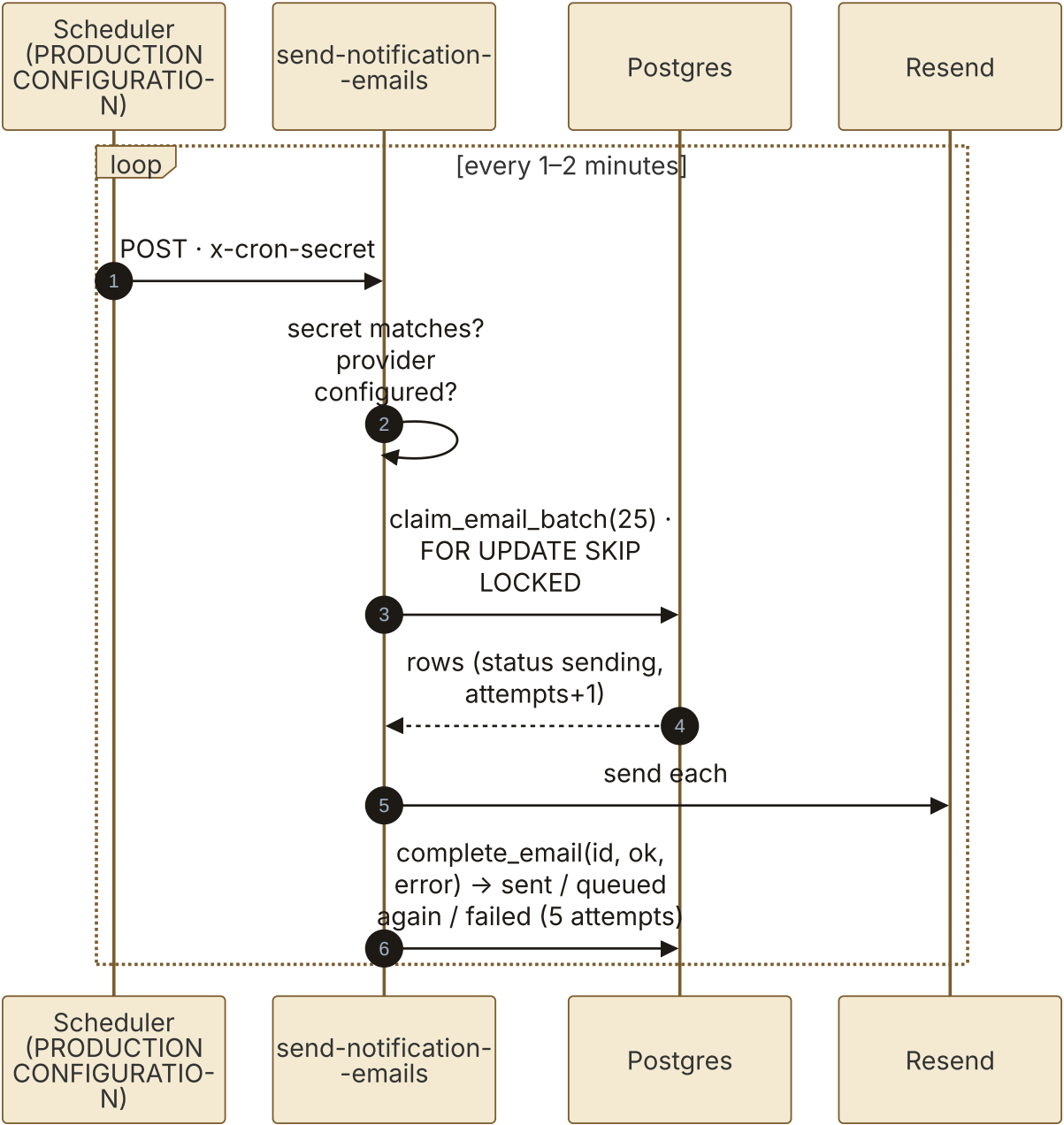


Diagram — Email drain — operator-scheduled HTTP job

Part IV — Security

25 — Security Architecture

25 — Security Architecture

Layered model

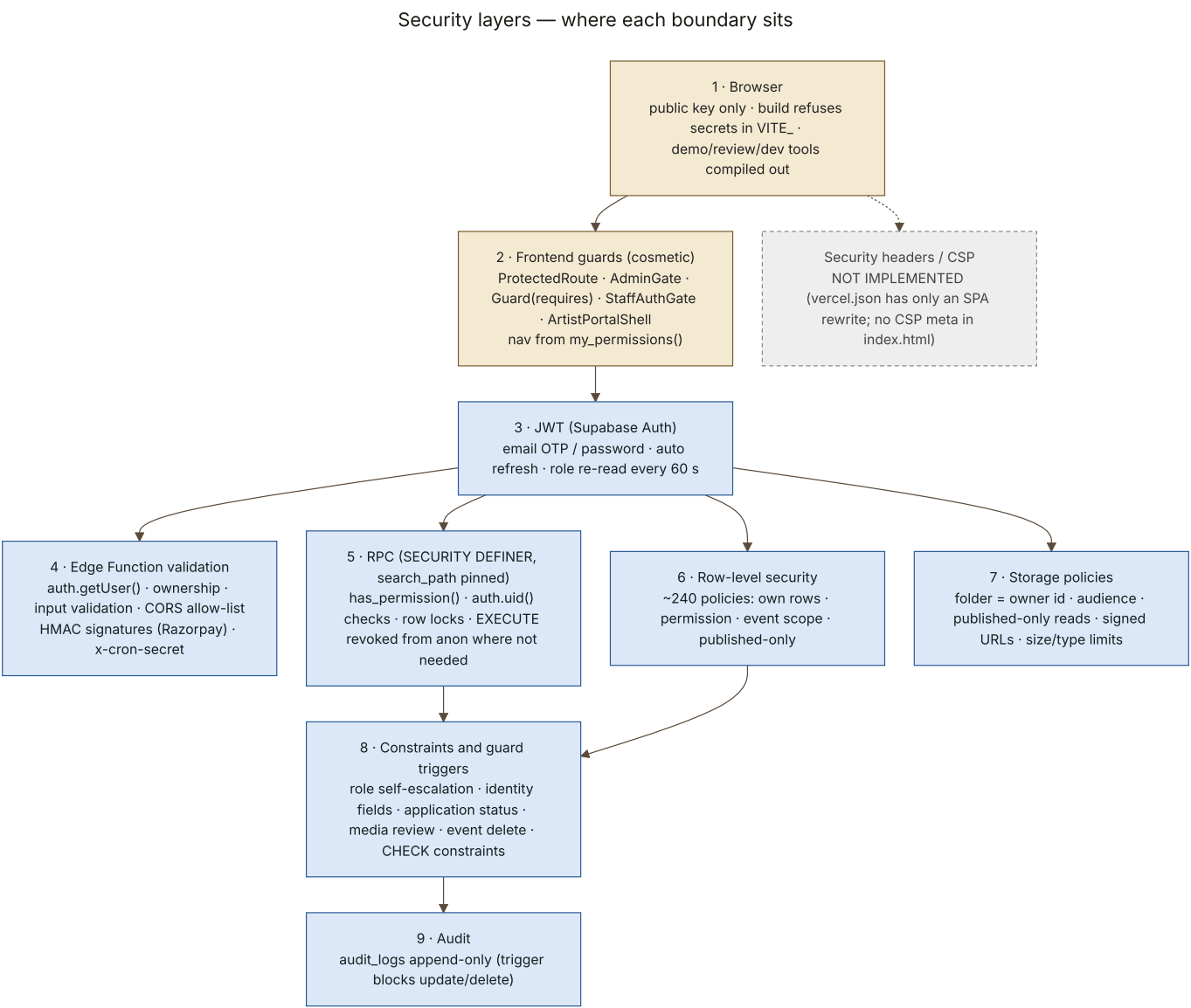


Diagram — Security layers — where each boundary sits

The rule stated across the code: **hiding a button is not security; the database refuses the data.** (`AdminApp.jsx` Guard comment, `rbac.js`, `0002_rls.sql` header.)

Keys and secrets

Item	Where it lives	Who can see it	Enforcement
Supabase publishable / anon key	<code>VITE_SUPABASE_PUBLISHABLE_KEY</code> (or <code>VITE_SUPABASE_ANON_KEY</code>) → browser bundle	public by design	RLS makes it safe
Supabase service-role / secret key	injected into Edge Functions as <code>SUPABASE_SERVICE_ROLE_KEY</code> ; local dev server only (<code>SUPABASE_SERVICE_ROLE_KEY</code> , never <code>VITE_</code>)	server only	<code>vite.config.js</code> <code>assertNoSecretInBrowserEnv</code> refuses <code>sb_secret_...</code> , a service_role JWT, or a value equal to the service key in any <code>VITE_</code> var
<code>RAZORPAY_KEY_ID</code>	Edge Function secret; returned to the browser in the order response (public id)	public	—

Item	Where it lives	Who can see it	Enforcement
RAZORPAY_KEY_SECRET	Edge Function secret	server	<code>requireSecret</code> (never the literal <code>"undefined"</code>)
RAZORPAY_WEBHOOK_SECRET	Edge Function secret (different from the key secret)	server	503 without it
RESEND_API_KEY, EMAIL_FROM, EMAIL_REPLY_TO	Edge Function secrets	server	only read in <code>_shared/email.ts</code>
CRON_SECRET	Edge Function secret + scheduler	server	header <code>x-cron-secret</code>
SITE_URL, ALLOWED_ORIGINS	Edge Function secrets	—	CORS allow-list + email links
VITE_TEAM_REVIEW_PASSWORD	review deployment env	ships in that bundle: demo accounts only	refused on Vercel production unless <code>TANGY_ALLOW_REVIEW_BUILD=1</code>
<code>.env</code> , <code>.env.local</code> , <code>.env.*.local</code> , <code>.env.review.local</code>	developer machines	gitignored	—

No secret value appears in the repository. Production secrets are **not yet set** (`docs/OPERATIONS.md`).

Role security

- `profiles.role` changes only through the paths below, all policed by `prevent_role_self_escalation` (0030):
- approvals (user → partner/artist, `applications.review`)
- `admin_set_user_role` (`roles.manage`)
- accepting an invitation for your own email
- SQL / service role (bootstrap)
- Nobody changes their own role or active flag. Only a Super Admin grants or removes Super Admin. The last active Super Admin cannot be demoted.
- A deactivated account resolves to **no role** (`current_role_name()` requires `is_active`), so every permission check fails immediately.
- `is_staff_or_admin()` now means **admin-level only** (0017 security fix 2). Staff get narrow, event-scoped access.
- `guard_profile_identity` (0035): users cannot change their own `email`, `passport_id`, `member_since` or `created_at`. The account email is the recipient of tickets and approvals, and authorizes waitlist reads.
- Internal review notes moved off applicant-readable rows into `application_reviews` (0033).
- `artist_availability` is no longer publicly readable (0033).

Payment security

- Prices** come only from the database (`booking_quote`); the client can choose a ticket type and quantity, never a price.
- Overselling** is prevented by the event row lock + capacity check in `create_pending_booking` (service role only).
- Confirmation** happens only after HMAC-SHA256 verification, with constant-time comparison, of either:
 - the checkout signature `order_id|payment_id` with the key secret, or
 - the webhook raw body with the webhook secret.
- `confirm_booking_and_issue_tickets` was callable by any signed-in user before 0017. That would have issued tickets without payment. It is now service-role only.
- Admin booking edits** go only through audited RPCs. No direct status flips are possible (0017 security fix 4).

Storage security

Private by default, with ownership by path, audience-scoped reads, published-only anonymous reads and short-lived signed URLs. See 20. The bucket size and type limits were added in 0029.

Function privileges (0035 §7)

PostgreSQL grants `EXECUTE` to `PUBLIC` on new functions by default, and Supabase adds anon/authenticated. Migration 0035 found **151 SECURITY DEFINER functions callable without signing in**. It revoked them, except:

- functions that check the caller themselves
- intentionally public ones: `event_availability`, `booking_quote`, `invitation_preview`
- RLS helpers that policies evaluate

A preflight (`supabase/preflight/0035_phase1_security_gaps_preflight.sql`) and a test (`phase1_security_gaps.test.sql`) cover this.

CORS

Browser-called Edge Functions echo only an allow-listed origin and never `*`. They fail closed when `SITE_URL` / `ALLOWED_ORIGINS` are missing (loopback only). Every one of them also requires a JWT, so CORS is defence in depth.

Other controls

Control	Where
Console idle sign-out (default 60 min)	<code>AdminShell</code>
Console sign-in audit	<code>log_auth_event</code>
Append-only audit log	<code>audit_logs_append_only</code> trigger
Safe error messages (no SQL / RLS internals)	<code>admin/api.js friendlyError</code> , <code>UserAuthContext.friendlySignInError</code> , Edge Functions return fixed messages
Messaging privacy	RLS; private Super Admin ↔ artist threads; no message text in emails; not E2EE (stated)
Enquiries / applications require an account; choosing a path never grants a role	0025
Duplicate application guards	<code>guard_duplicate_application</code> , <code>*_guard_duplicate</code> triggers
Demo admin mode grants nothing (no Supabase session)	<code>DemoAdminContext.jsx</code>
Dev mock-session middleware: loopback only + local Supabase only	<code>vite.config.js devMockSession</code>

Gaps

Item	Status
CSP / security headers (HSTS, X-Frame-Options, ...)	NOT IMPLEMENTED in the repo (<code>vercel.json</code> contains only a rewrite)
Rate limiting for OTP / checkout beyond Supabase defaults	NOT DETERMINABLE FROM REPOSITORY (Supabase Auth project settings)
Payment capture mode (must be automatic)	DOCUMENTED BUT NOT VERIFIED IN CODE
OTP email template contains the code	PRODUCTION CONFIGURATION
<code>send-ticket-email</code> / <code>send-approval-email</code> authorize by <code>profiles.role</code> string rather than <code>has_permission()</code>	Inconsistent with the permission model elsewhere (an edited permission matrix does not affect these two checks)

Part V — Environments & Deployment

26 — Local / Demo Architecture · 27 — Deployment Architecture · 28 — Environment Architecture

26 — Local / Demo Architecture

Everything in this document is **LOCAL ONLY / DEMO ONLY / NOT PRODUCTION**, except where a "review" deployment is named. Review deployments use a disposable hosted Supabase project, also never production.

Four separate "demo" mechanisms

Mechanism	What it is	Real Supabase session?	Gate
Local stack + demo dataset	Docker Supabase, every migration, <code>scripts/demo-data.sh seed</code> , Mailpit for OTP codes	yes (local)	script refuses non-localhost URLs
Dev role switcher	<code>npm run dev</code> → <code>/admin-portal</code> opens <code>DevRoleSelector</code> ; signs into local accounts via the dev-server middleware <code>/__dev/mock-session</code> , or uses mock identities	yes with a local stack; mock otherwise	<code>__TANGY_DEV_TOOLS__</code> true only for <code>vite serve</code> ; middleware only for loopback + localhost Supabase
Demo Admin mode	<code>/demo-admin</code> , <code>/demo-admin/control-room</code> , <code>/demo/:role</code> : role dashboards rendered with bundled demo data (<code>src/pages/demoAdmin/*</code>)	no (an in-memory flag; grants nothing, RLS blocks everything)	<code>VITE_DEMO_ADMIN_ENABLED</code> , compiled to false in builds unless <code>TANGY_ALLOW_DEMO_BUILD=1</code>
Team review mode	<code>/team-demo</code> : one-click password sign-in to 9 demo accounts in a hosted review project	yes (review project)	<code>VITE_TEAM_REVIEW_MODE</code> + <code>VITE_TEAM_REVIEW_PASSWORD</code> ; refused on Vercel production unless <code>TANGY_ALLOW_REVIEW_BUILD=1</code>

Local stack

Local development stack

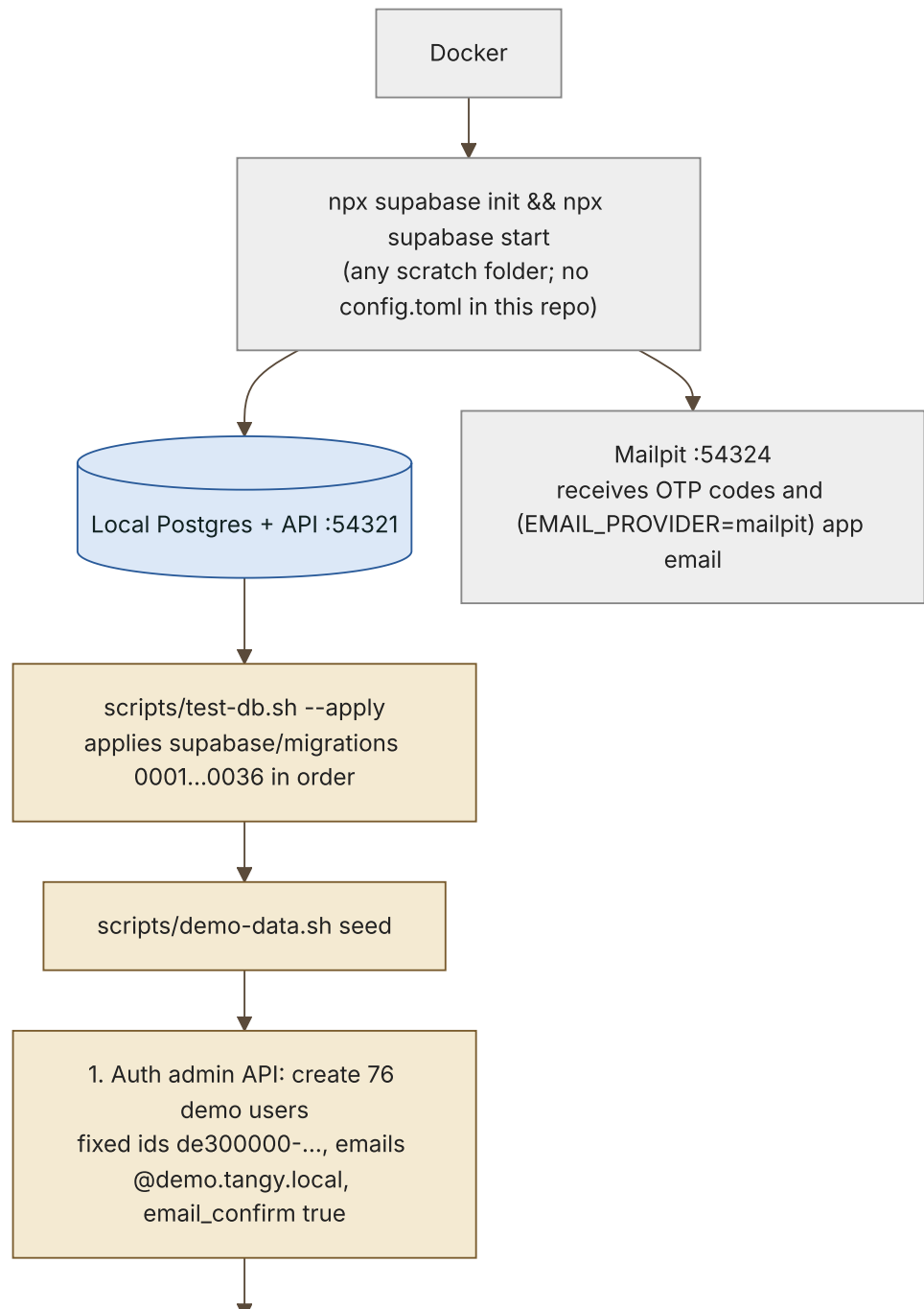


Diagram — Local development stack (part 1 of 3)

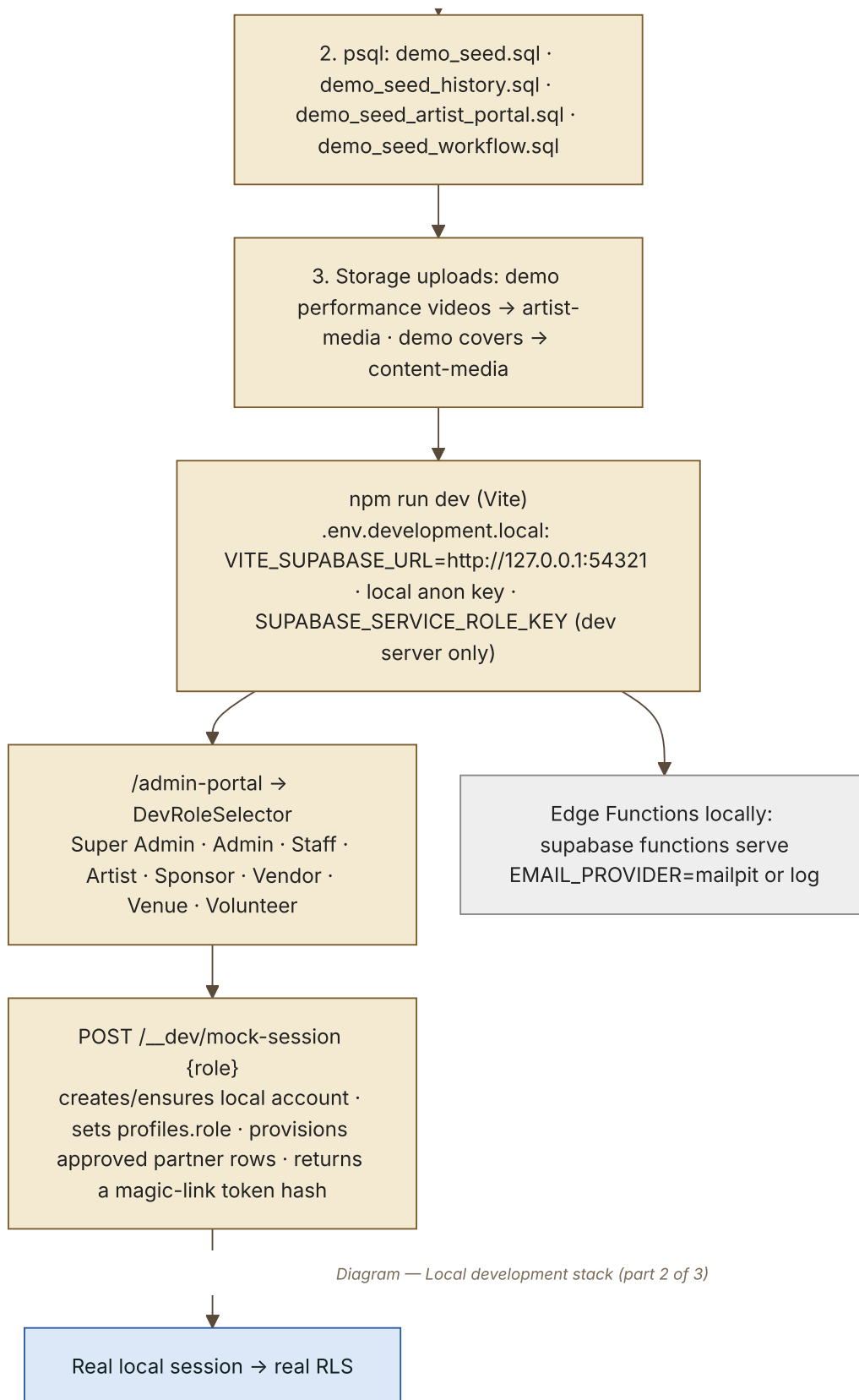


Diagram — Local development stack (part 2 of 3)

Diagram — Local development stack (part 3 of 3)

How demo data is generated and removed

Step	Command / file	What it does
Guard	scripts/demo-data.sh	Refuses unless SUPABASE_URL is http://127.0.0.1:* / http://localhost:* ; DEMO_TARGET=review requires .env.review.local and an https://<ref>.supabase.co URL

Step	Command / file	What it does
Accounts	Auth admin API POST /auth/v1/admin/users	76 accounts with deterministic ids and @demo.tangy.local emails (422 = already exists)
Data	supabase/demo/demo_seed*.sql	35 sessions (26 past, 2022–2026), 27 artists, 8 programmes, 22 albums / 88 photos, 15 TV records, 21 diary posts, 12 announcements, partners, 15 volunteers, 40 bookings, 129 tickets, 15 waitlist entries, 12 artist applications in every state, 16 booking requests (docs/OPERATIONS.md §8)
Media	Storage REST uploads	demo videos and covers from public/media/demo and two real empty-venue photos
Identify	—	every record has a de300000-... id, @demo.tangy.local email, "(demo)" names, example.com links
Remove	scripts/demo-data.sh remove	demo_remove.sql + deletes the demo storage objects + deletes every demo auth user
Status	scripts/demo-data.sh status	counts what's loaded
Review project	DEMO_TARGET=review scripts/demo-data.sh migrate · seed · review-logins	applies migrations to the review project, loads the same data, sets the shared review password on the 9 Team Demo accounts

Demo accounts (from the seed note and src/config/teamReview.js)

Role	Email
Super Admin	director@demo.tangy.local
Admin / Manager	ops@demo.tangy.local
Staff	desk@demo.tangy.local
Artist	ananya.rao@demo.tangy.local
Sponsor	saffron.tea@demo.tangy.local
Vendor	kulhad.chai@demo.tangy.local
Venue host	farah@demo.tangy.local
Volunteer (live check-in access)	aisha@demo.tangy.local
Customer (holds a waitlist offer)	meera.kulkarni@demo.tangy.local

Locally, sign in with the email; the one-time code arrives in Mailpit (http://127.0.0.1:54324).

Team review deployment

Team review deployment (disposable, never production)

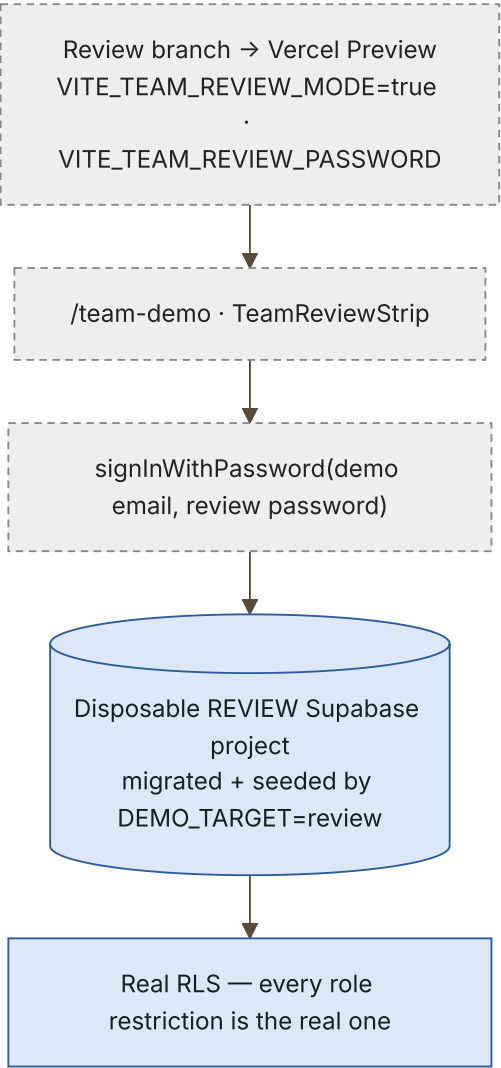


Diagram — Team review deployment (disposable, never production)

Test tooling (local only)

Tool	Purpose
<code>scripts/test-db.sh</code>	runs <code>supabase/tests/*.test.sql</code> (20 suites), each in a rolled-back transaction
<code>scripts/test-fresh-db.sh</code>	migrations on an empty database
<code>scripts/test-production-bootstrap.sh</code>	validates the production bootstrap package (checksums, 35 files, inventory, all suites)
<code>scripts/test-migration-safety-0035.sh</code>	0035 preflight + safety
<code>scripts/test-concurrency.sh</code>	last-seat race, offer race, overlapping job runs across two sessions
<code>scripts/test-edge-functions.mjs</code> , <code>test-edge-shared.mjs</code> , <code>test-ticket-email.mjs</code> , <code>test-email-config.mjs</code>	Edge Function / email behaviour
<code>e2e/*.mjs</code> (Playwright, <code>e2e/run.sh</code>)	browser suites: OTP login, RBAC for three console roles, direct API authorization attempts, checkout, group check-in, waitlist, portals, artist portal, invitations, routing sweep, responsive / mobile; QR via a fake camera (<code>make-cam.mjs</code>)
<code>scripts/route-inventory.mjs</code>	generates <code>docs/ROUTES.md</code>

27 — Deployment Architecture

The repository proves the **frontend build is designed for Vercel**:

- `vercel.json` SPA rewrite
- `vite.config.js` reads `VERCEL_ENV`
- commit history: "Fix Vercel 404s on direct route navigation" (16 Aug), "build on Vercel's Linux — lowercase archive folder, lockfile for npm 10" (2 Oct)

The **backend deployment is not done**. `docs/OPERATIONS.md` §3a: "Nothing here has been deployed yet: the repository has no `supabase/config.toml`, no linked project ref, no CI and no deploy script, so every step below is an operator action."
`supabase/production-bootstrap/README.md`: "Nothing here has been applied to production."

Deployment model

Deployment — what is code, what is operator action

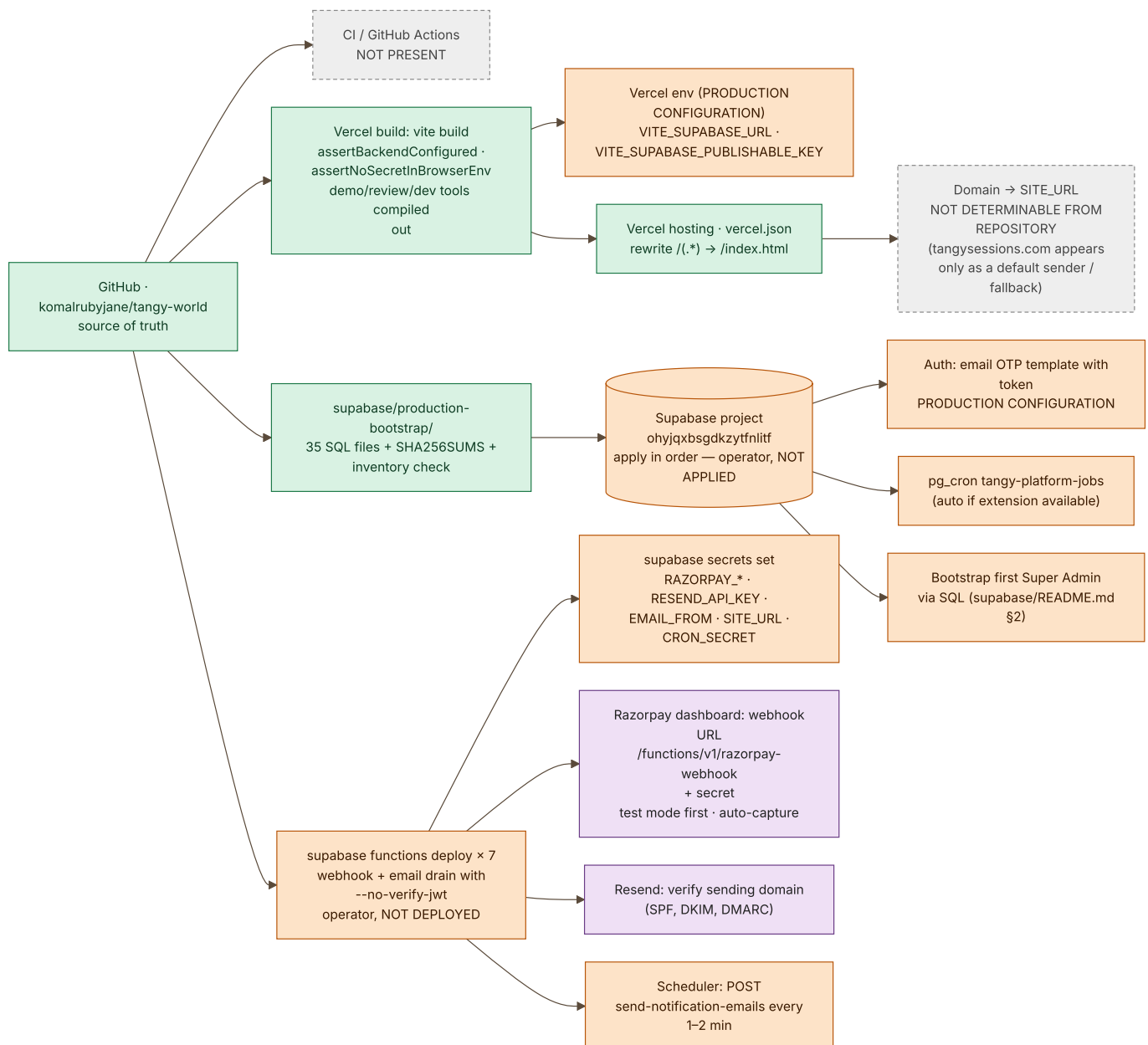


Diagram — Deployment — what is code, what is operator action

Hosting references, labelled

Reference	Evidence	Label
Vercel	<code>vercel.json</code> ; <code>VERCEL_ENV</code> checks in <code>vite.config.js</code> ; deploy-fix commits; review deploys as "Vercel Preview" (<code>docs/EVENT_WORKFLOW.md</code>)	CURRENT (frontend hosting target)
Vercel Preview + disposable review Supabase project	<code>teamReview.js</code> , <code>docs/EVENT_WORKFLOW.md</code> "Team review deployment"	PREVIEW
Supabase hosted project <code>ohyjxbsgdtkzytfnlitf</code>	<code>docs/OPERATIONS.md</code> §2a/§3a, webhook URL	CURRENT target, NOT APPLIED
Local Supabase (Docker)	<code>supabase/README.md</code> , <code>e2e/README.md</code> , scripts	LOCAL
<code>tangy-world.html</code> (legacy static page at the repo root)	<code>App.jsx</code> comment: it collides with a <code>/tangy-world</code> route in dev	LEGACY
Azure, Netlify, Cloudflare, GitHub Pages	not found	NOT USED

Ordered production checklist (from the runbooks)

1. Apply `supabase/production-bootstrap/*` in order to the empty project; verify with `checks/production_inventory.readonly.sql` (`scripts/test-production-bootstrap.sh` validates the package locally).
2. Create the first Super Admin by SQL (`supabase/README.md` §2). Invite the rest through the console.
3. Configure the Auth email OTP template (`{{ .Token }}`).
4. Confirm `pg_cron` / `tangy-platform-jobs` exists, or schedule `run_platform_jobs()` externally every 5 min.
5. Set Edge Function secrets; deploy the 7 functions with the documented JWT flags.
6. Razorpay: webhook URL + secret (`payment.captured`, `order.paid`, `payment.failed`, `payment.authorized`, `refund.processed`, `refund.created`), auto-capture, test keys first.
7. Resend: verify the domain; set `EMAIL_FROM`; schedule the email drain.
8. Vercel: set `VITE_SUPABASE_URL` + `VITE_SUPABASE_PUBLISHABLE_KEY`; deploy; set `SITE_URL` to the canonical origin.
9. Verify in test mode: a booking, the ticket email, an approval email, the email delivery log, the webhook log.

None of these steps can be confirmed as done from the repository.

28 — Environment Architecture

Environments

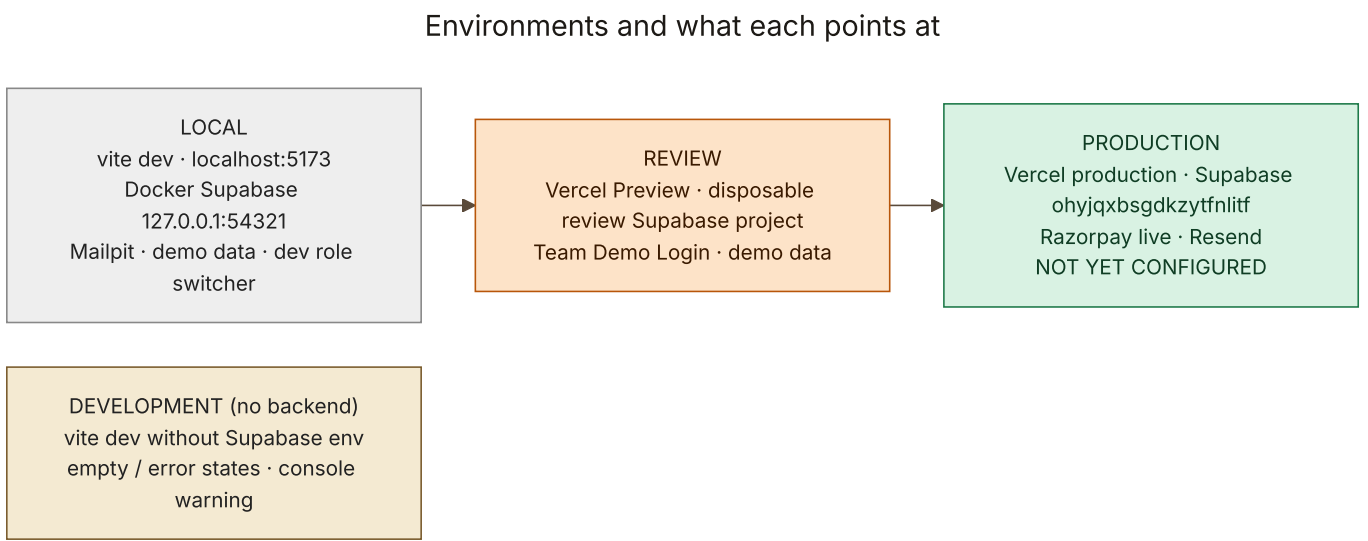


Diagram — Environments and what each points at

Per-environment matrix

	LOCAL	DEVELOPMENT (no backend)	REVIEW	PRODUCTION
Frontend URL	<code>http://localhost:<vite port></code>	same	Vercel Preview URL (not in repo)	canonical origin: NOT DETERMINABLE FROM REPOSITORY
Supabase project	local stack (<code>npx supabase start</code>)	none (<code>supabase client is null</code>)	disposable hosted review project (<code>.env.review.local</code> , gitignored)	<code>ohyjxbsgdkzytfnlitf</code> (runbooks)
Database	migrations via <code>scripts/test-db.sh --apply</code> ; demo seed	—	<code>DEMO_TARGET=review</code> <code>scripts/demo-data.sh migrate + seed</code>	<code>supabase/production-bootstrap/</code> (no demo seed); not applied
Edge Functions	<code>supabase functions serve (+ -env-file)</code>	—	deployed to the review project (operator)	to deploy (§3a)
Razorpay mode	none locally (E2E sends HMAC-signed fake webhooks with <code>RAZORPAY_WEBHOOK_SECRET_LOCAL</code>)	—	test keys (if configured)	test first, then live (<code>rzp_live_...</code>)
Email provider	<code>mailpit</code> or <code>log</code>	—	project's own email / Resend	<code>resend</code> (needs <code>RESEND_API_KEY</code> , verified domain)
Auth email (OTP)	Mailpit inbox	—	project email	Supabase Auth email with the token template
Demo / dev tools	dev role switcher (<code>__TANGY_DEV_TOOLS__</code>), demo admin possible	dev switcher with mock identities	Team Demo Login (<code>__TANGY_REVIEW_MODE__</code>)	all compiled out
Scheduler	<code>scripts/run-jobs.sh [emails]</code>	—	<code>pg_cron</code> if available	<code>pg_cron</code> + email drain scheduler

Environment variables by consumer

Variable	Consumer	Local	Review	Production
VITE_SUPABASE_URL	browser bundle	http://127.0.0.1:54321	review project https URL	production https URL (Vercel)
VITE_SUPABASE_PUBLISHABLE_KEY / VITE_SUPABASE_ANON_KEY	browser	local anon key	review publishable key	production publishable key
SUPABASE_SERVICE_ROLE_KEY	Vite dev server only (/__dev/mock-session)	local service key	—	— (never in Vercel)
VITE_DEMO_ADMIN_ENABLED	build flag	optional	unset	unset (forced false)
VITE_TEAM_REVIEW_MODE , VITE_TEAM_REVIEW_PASSWORD	build flags	—	set	unset (refused on production unless TANGY_ALLOW_REVIEW_BUILD=1)
TANGY_DEV_TOOLS , TANGY_ALLOW_UNCONFIGURED_BUILD , TANGY_ALLOW_DEMO_BUILD , TANGY_ALLOW_REVIEW_BUILD	vite.config.js	dev overrides	—	unset
VITE_RAZORPAY_KEY_ID	listed in .env.example , not read by the app	—	—	—
RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET , RAZORPAY_WEBHOOK_SECRET	Edge Functions	local test values	review	production secrets
EMAIL_PROVIDER , RESEND_API_KEY , EMAIL_FROM (RESEND_FROM_EMAIL), EMAIL_REPLY_TO , MAILPIT_URL	_shared/email.ts	mailpit / log	resend	resend
SITE_URL , ALLOWED_ORIGINS	CORS + email links + invites	unset → loopback CORS	review URL	canonical origin
CRON_SECRET	send-notification- emails	local value	review	production
SUPABASE_URL , SUPABASE_ANON_KEY , SUPABASE_SERVICE_ROLE_KEY (functions)	Edge Functions	injected	injected	injected
SUPABASE_DB_URL , SUPABASE_SECRET_KEY , TEAM_REVIEW_PASSWORD	scripts/demo- data.sh (.env.review.local)	—	set	never

Secret values are intentionally absent from this document and from the repository.

Part VI — Failure Flows & Data Flow

29 — Failure and Recovery Flows · 30 — Data Flow Diagrams

29 — Failure and Recovery Flows

Each flow shows the **normal path**, the **error path** and the **recovery path** as implemented.

Payments

Payment failures — failed, browser closed, duplicate, bad signature

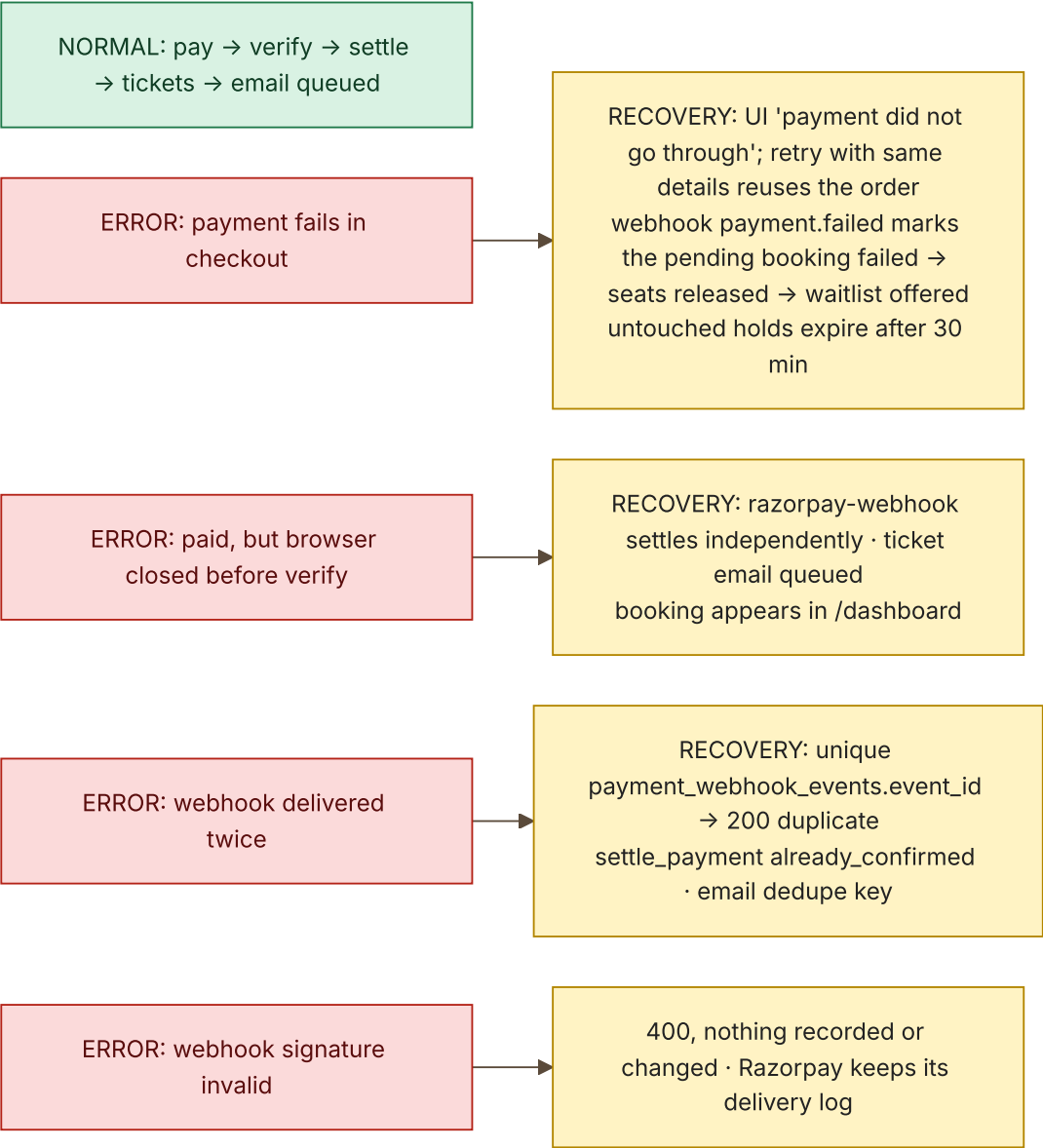


Diagram — Payment failures — failed, browser closed, duplicate, bad signature (part 1 of 2)

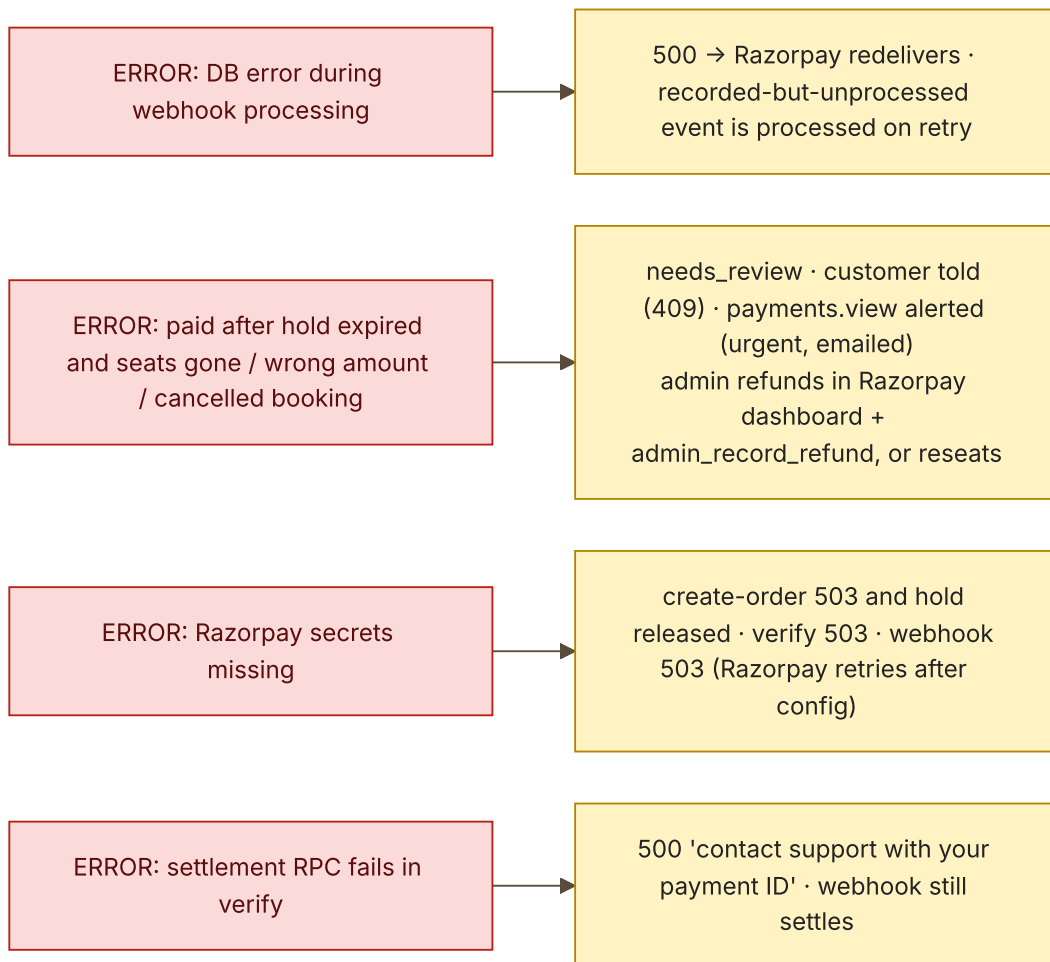


Diagram — Payment failures — failed, browser closed, duplicate, bad signature (part 2 of 2)

Email failures

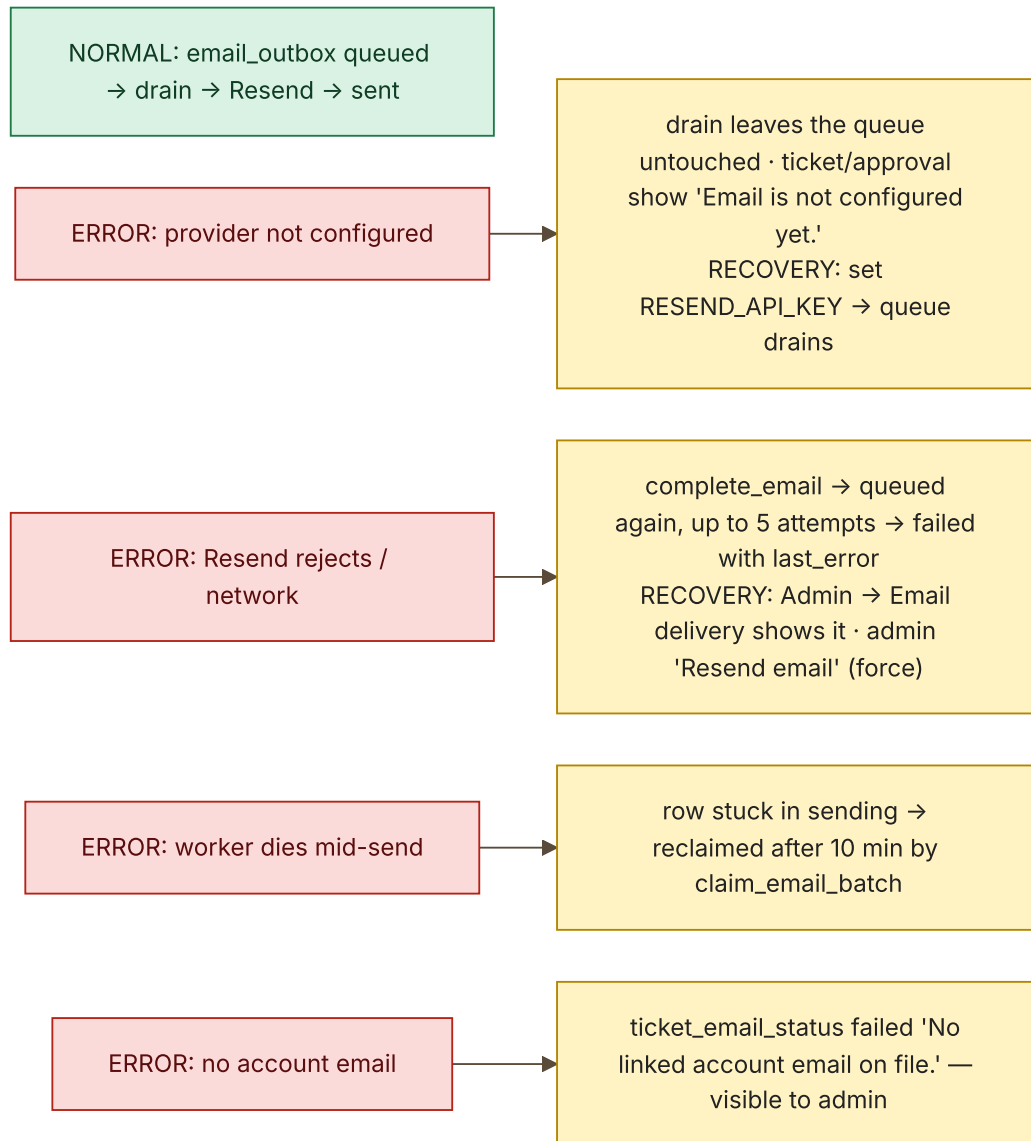


Diagram — Email failures (part 1 of 2)

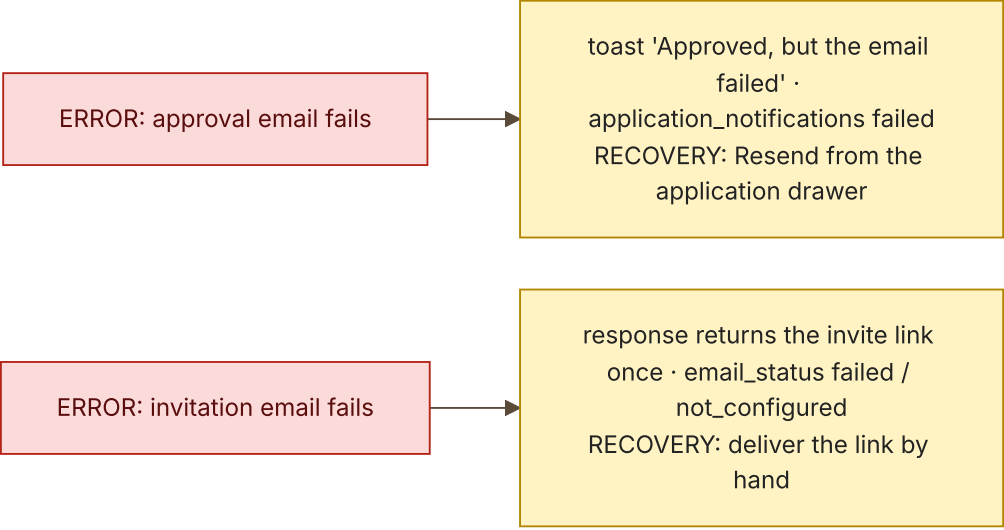


Diagram — Email failures (part 2 of 2)

Artist workflow

Artist request failures

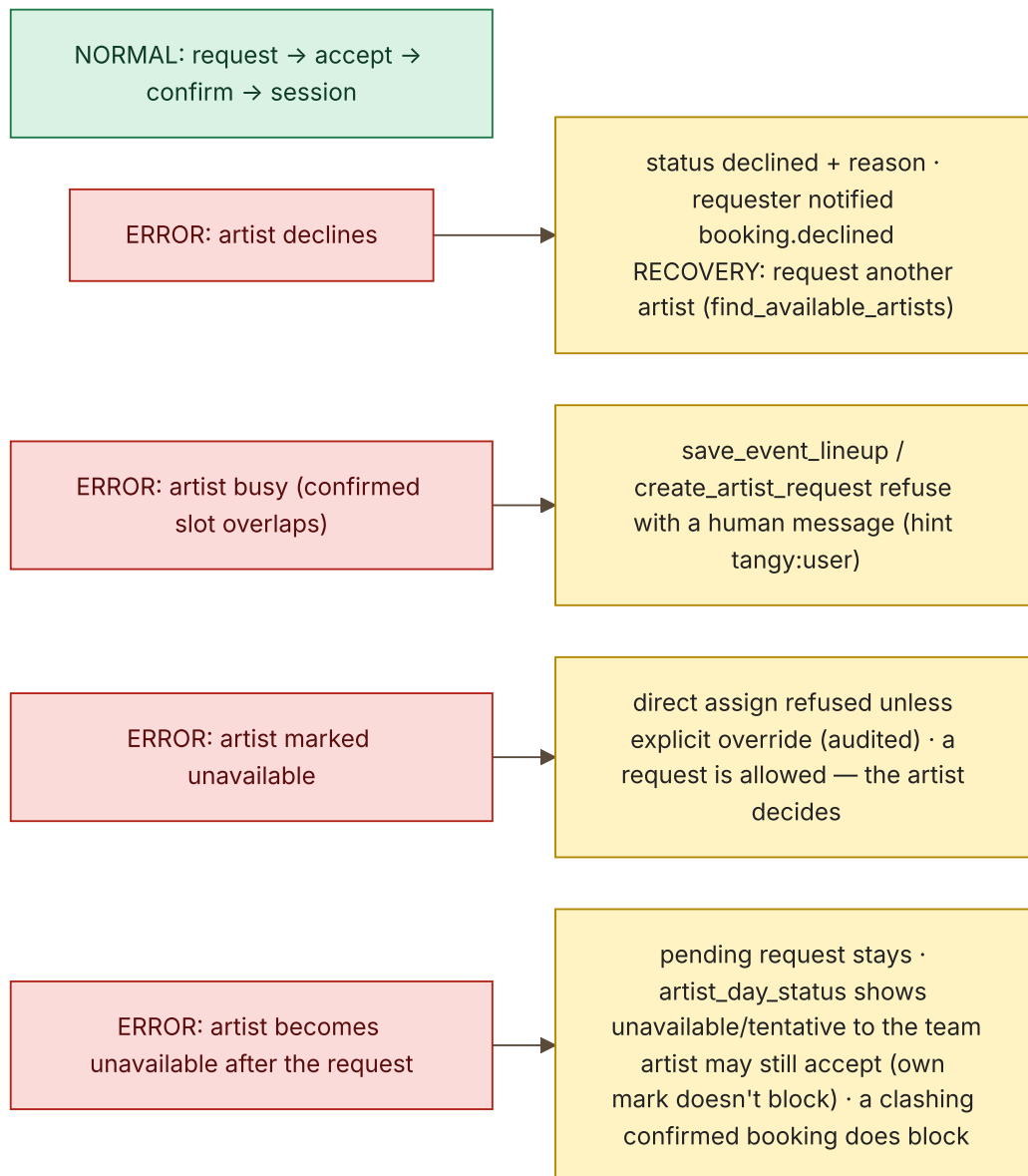


Diagram — Artist request failures (part 1 of 2)

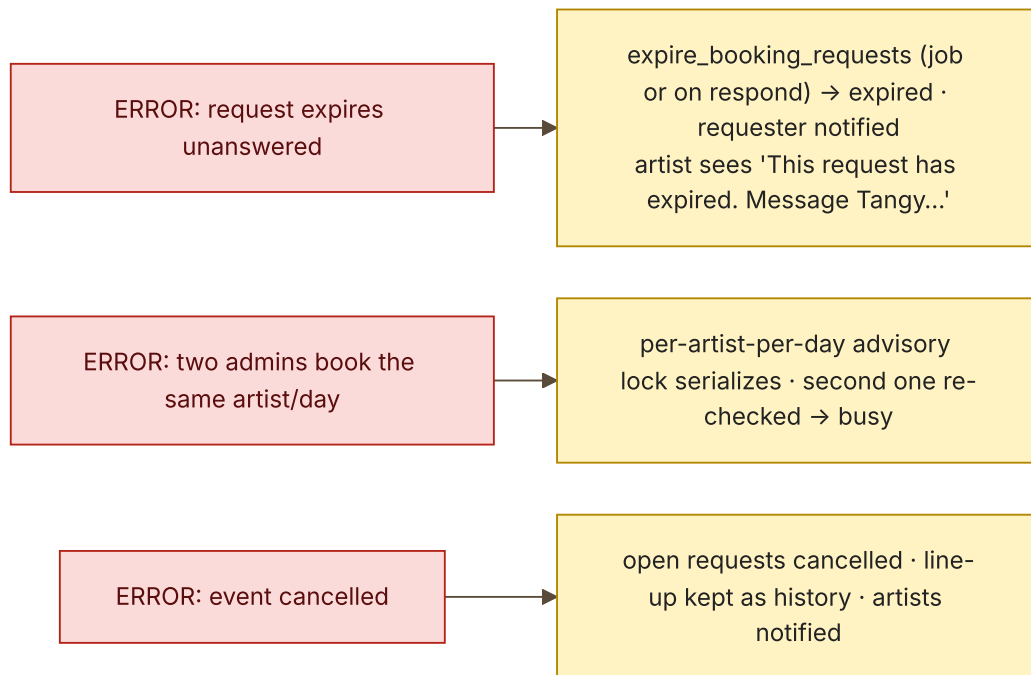


Diagram — Artist request failures (part 2 of 2)

Waitlist, uploads, authorization, sessions, roles, jobs

Failure	Error path	Recovery path
Waitlist offer expires	hold lapses (default 120 min)	<code>expire_waitlist_offers</code> → <code>expired</code> , holder notified, seats offered to the next party; the person can rejoin
Two people race for the last seat	concurrent checkouts	event row lock in <code>create_pending_booking</code> ; the loser gets <code>SOLD_OUT</code> (409) and the page refreshes availability (tested by <code>scripts/test-concurrency.sh</code>)
Upload rejected	storage policy, size or type limit	<code>uploadWithProgress</code> maps to "You don't have permission to upload here." / "That file is too large." / "A file with that name already exists."; client caps 25 MB (50 MB for artist media / video)
Media rejected by curator	<code>artist_media.status = rejected</code> + note	artist notified (<code>media.reviewed</code>); can resubmit (<code>rejected</code> → <code>under_review</code> allowed by <code>guard_artist_media</code>)
Unauthorized admin action	route guard → Forbidden; RPC raises 42501 / exception; RLS update → 0 rows	<code>friendlyError</code> → "You don't have permission to do that."; nothing changes; no partial write
Direct API attack (forged REST / RPC call)	RLS + definer checks + revoked EXECUTE	denied (e2e suites include direct API authorization attempts)
Session expires	JWT refresh fails / signed out elsewhere	<code>onAuthStateChange</code> → <code>user = null</code> → <code>ProtectedRoute</code> redirects to <code>/join/login?next=...</code> ; Edge Functions answer 401 "Sign in required."; console idle timeout signs out with a reason on the login card
Role changes while signed in	server stops honouring the old role immediately	client re-reads <code>role</code> , <code>is_active</code> every 60 s / on focus → menus, guards and dashboard follow; <code>role.changed</code> notification
Account deactivated	<code>current_role_name()</code> → null	console shows "Account deactivated"; every RPC / RLS check fails
Volunteer access ends mid-shift	<code>check_in_ticket</code> → <code>access_expired</code>	<code>access.expiring</code> warning beforehand; admin grants a new window
Scheduled job step errors	<code>run_platform_jobs()</code> runs as one transaction: an error aborts the run and its log row	next 5-minute run retries (all steps idempotent); checkout calls <code>expire_stale_bookings</code> itself; <code>pg_cron</code> 's own run history records the failure (platform feature)
Overlapping job runs	two schedulers fire	advisory lock: the second is recorded as <code>skipped</code>
Supabase not configured in a build	—	<code>vite build</code> fails (cannot ship a bundle with no backend); in dev the app shows empty / error states

Failure	Error path	Recovery path
Profile row missing after sign-in	profiles select fails	profileError shown; user treated as user ; console explains "profile could not be loaded"
Booking form invalid	client + Edge Function + create_pending_booking validation	specific 400 messages (INVALID_DETAILS , INVALID_QUANTITY , INVALID_TICKET_TYPE , INVALID_ATTENDEE_NAMES)

30 — Data Flow Diagrams

Each diagram shows where a piece of data **originates**, where it is **stored** and where it **ends up**.

1. Authentication data

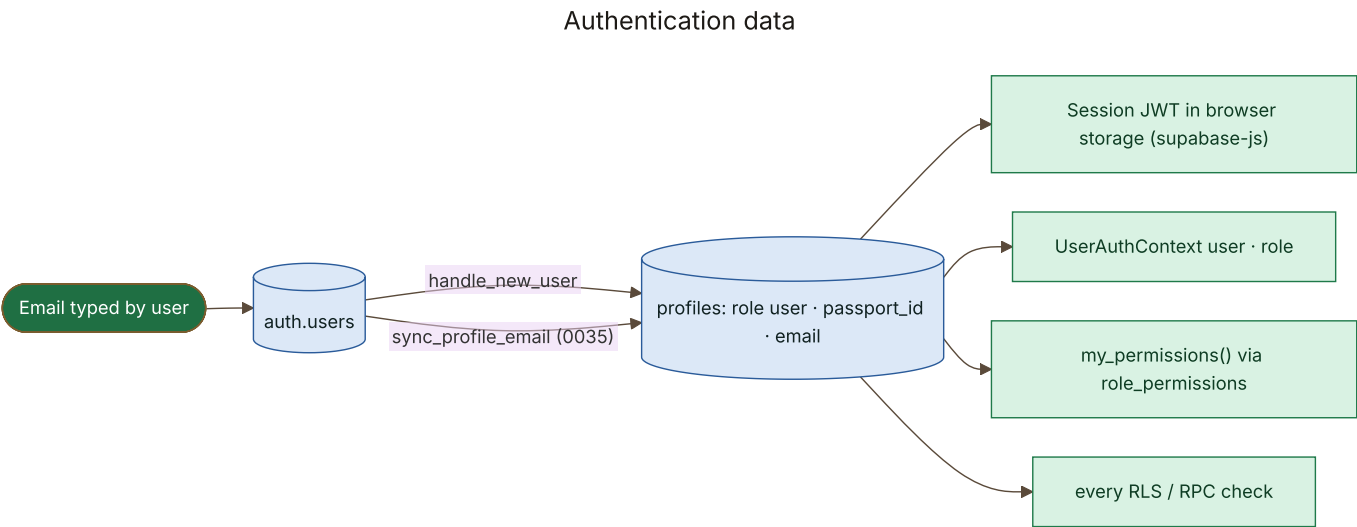


Diagram — Authentication data

2. Event data

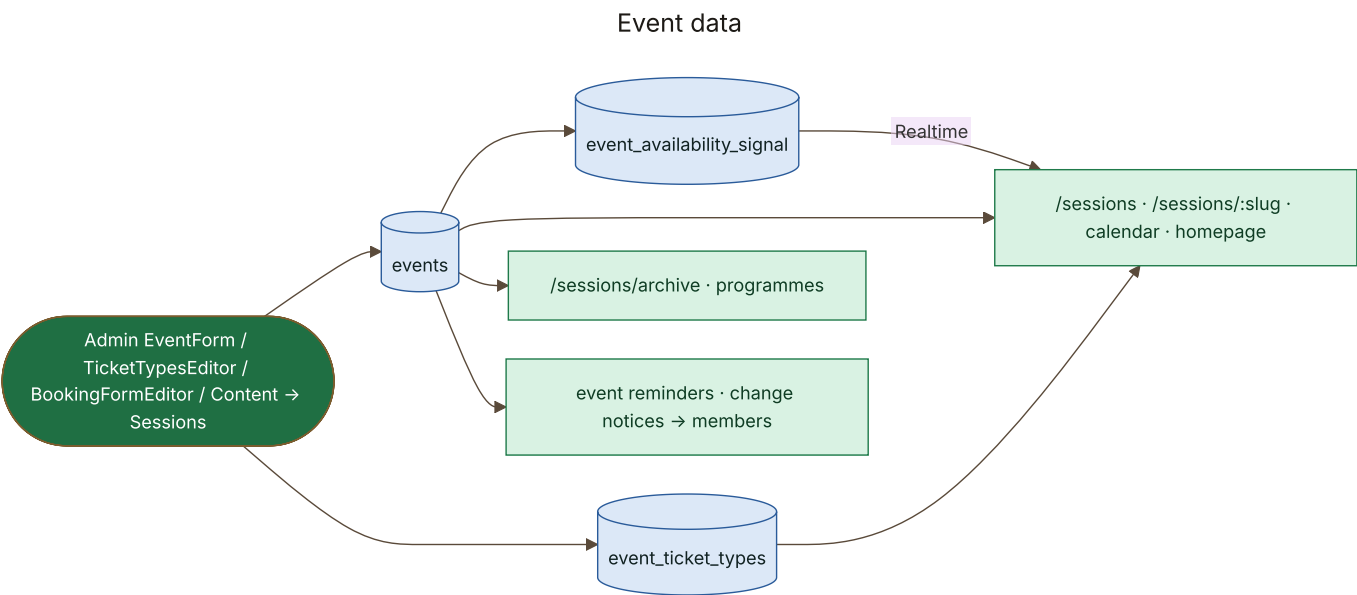


Diagram — Event data

3. Artist data

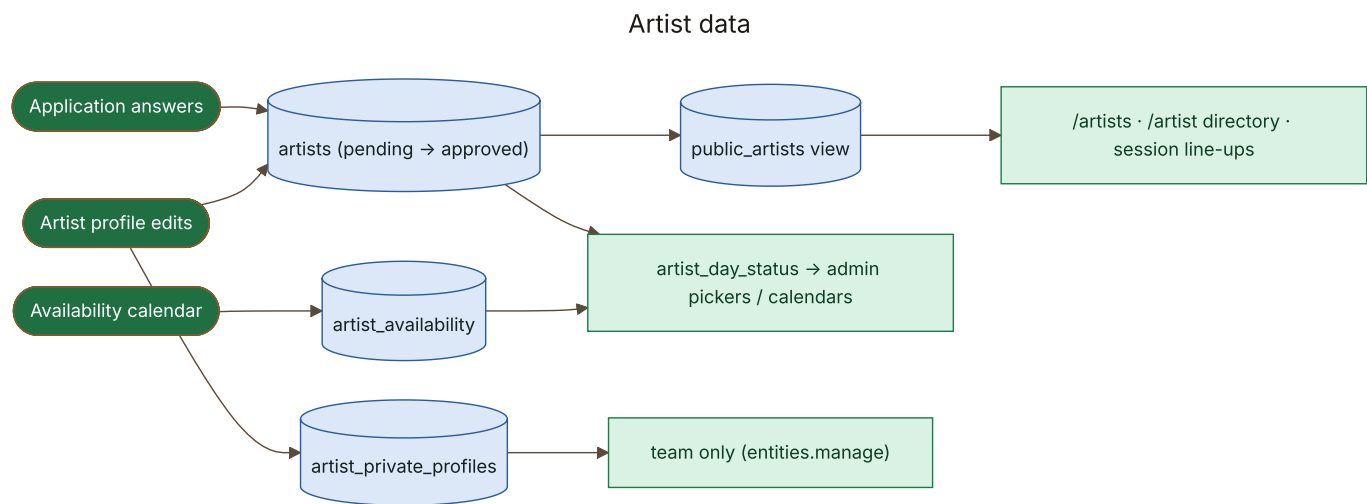


Diagram — Artist data

4. Application data

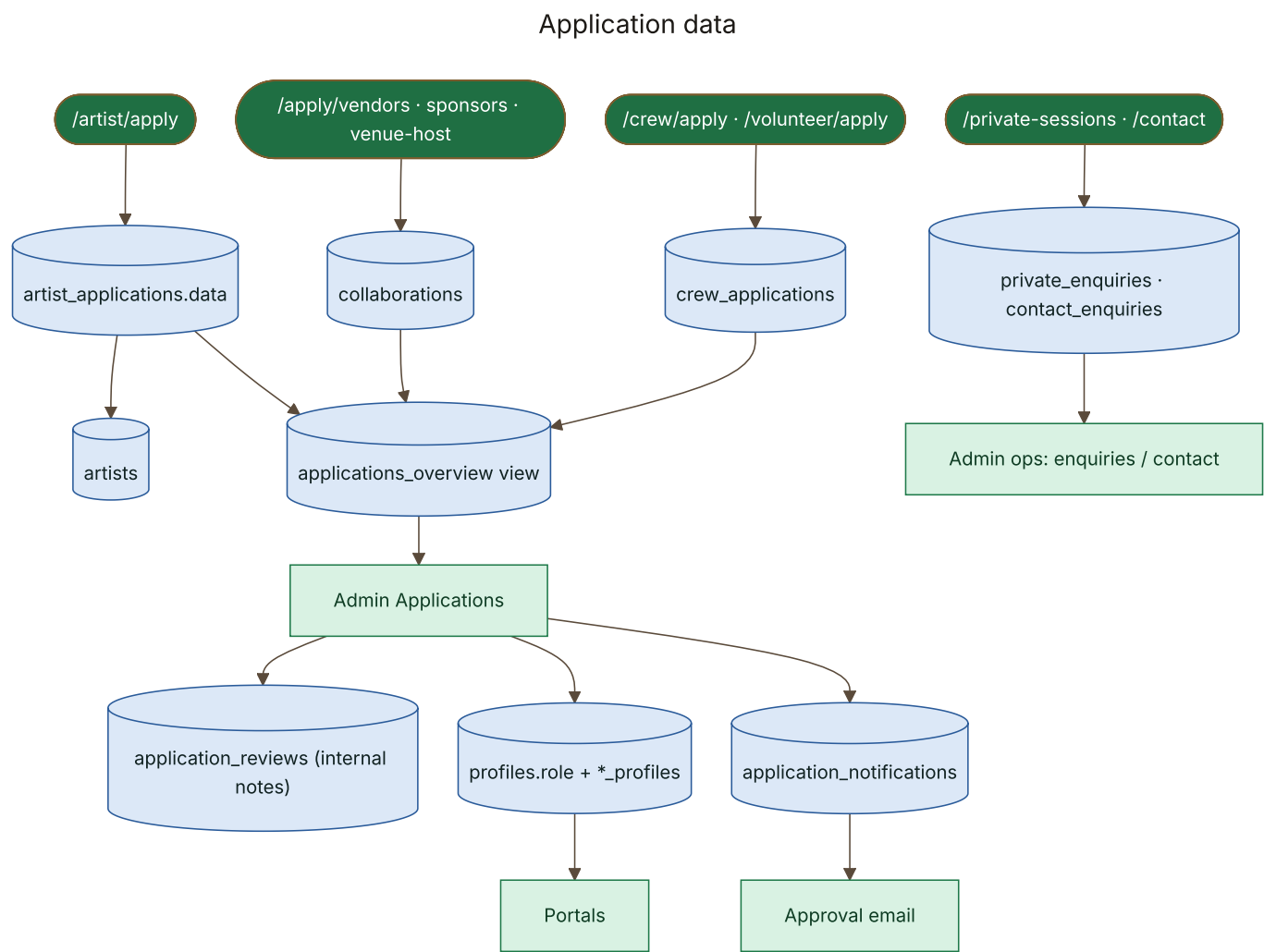


Diagram — Application data

5. Booking data

Booking data

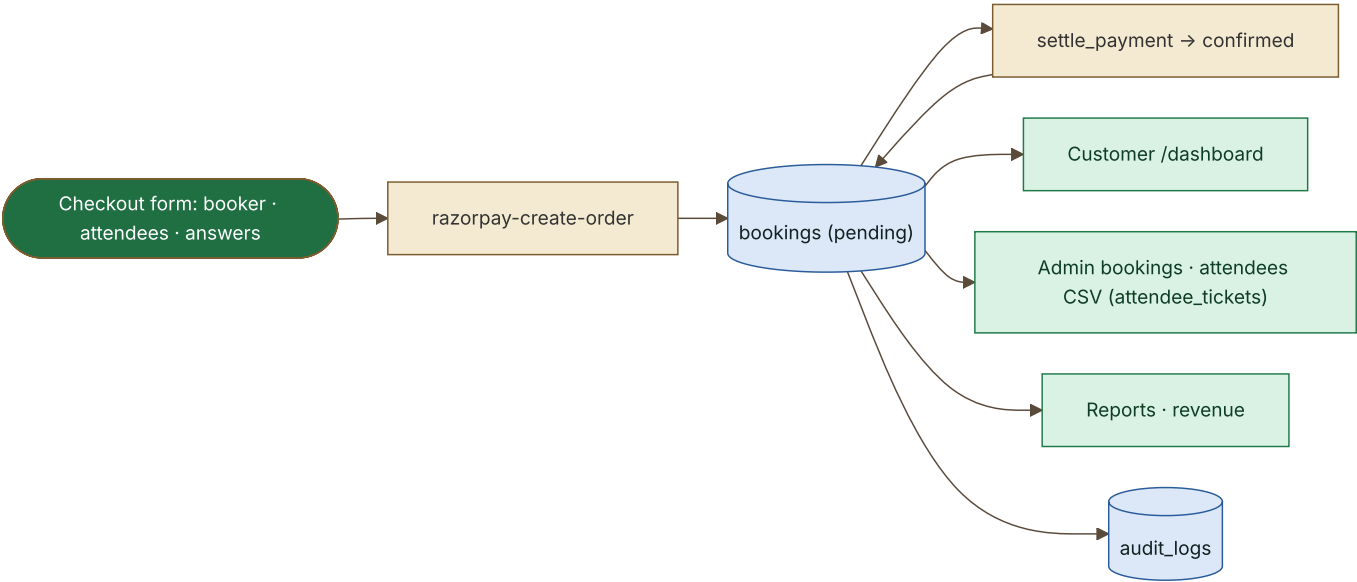


Diagram — Booking data

6. Payment data

Payment data

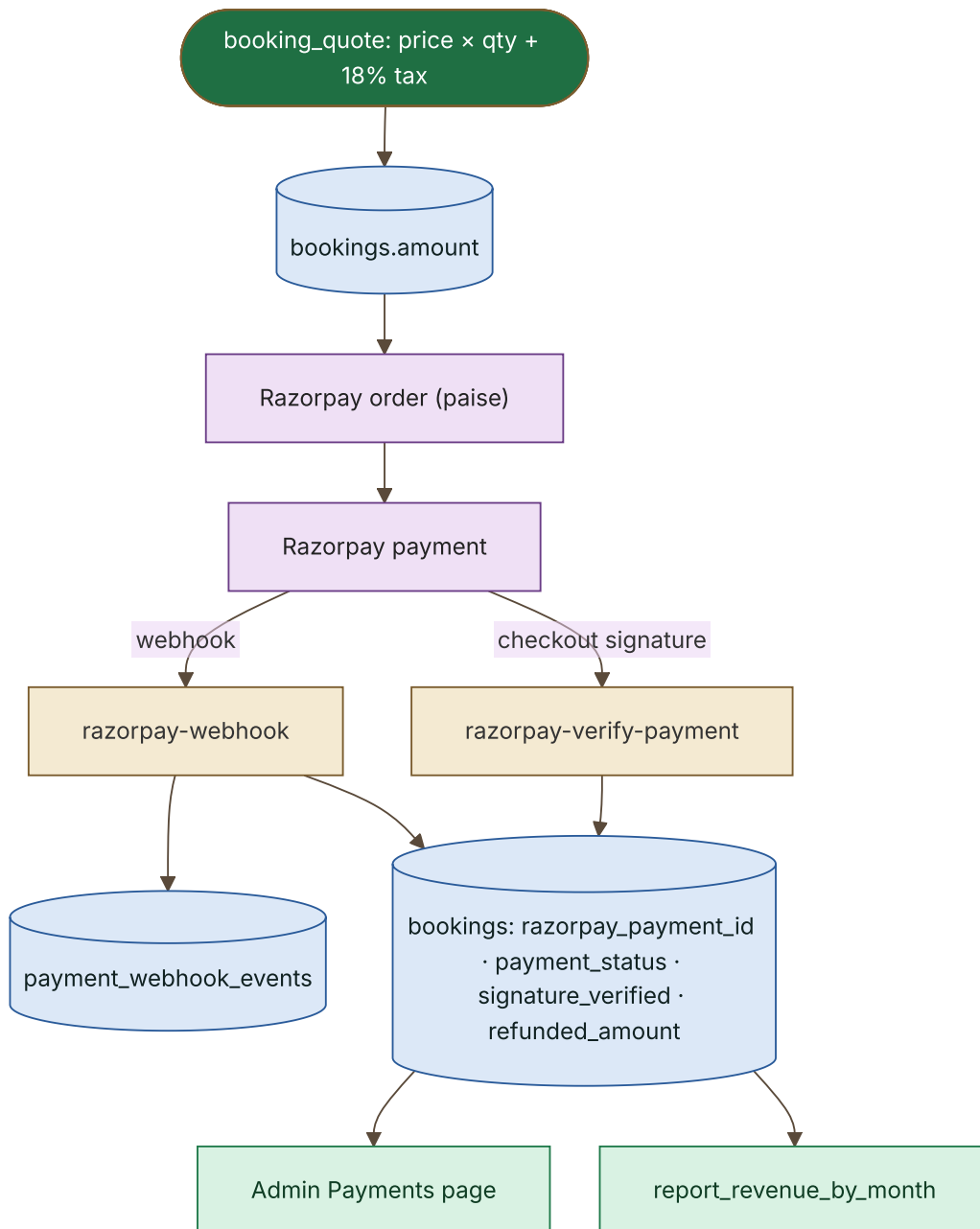


Diagram — Payment data

7. Ticket data

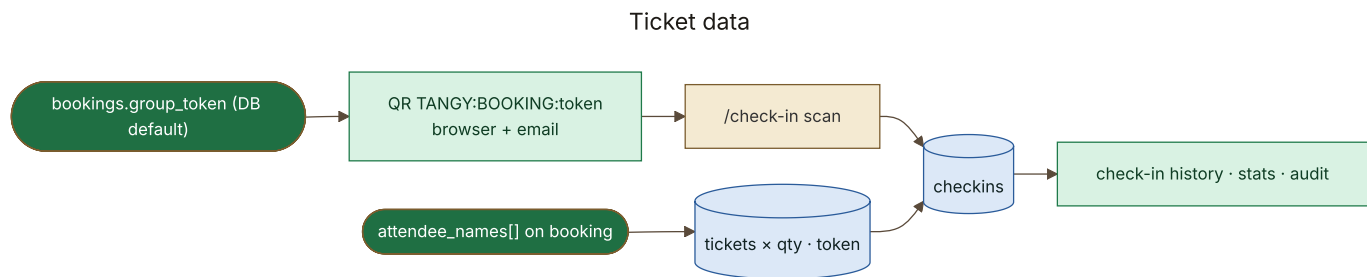


Diagram — Ticket data

8. Waitlist data

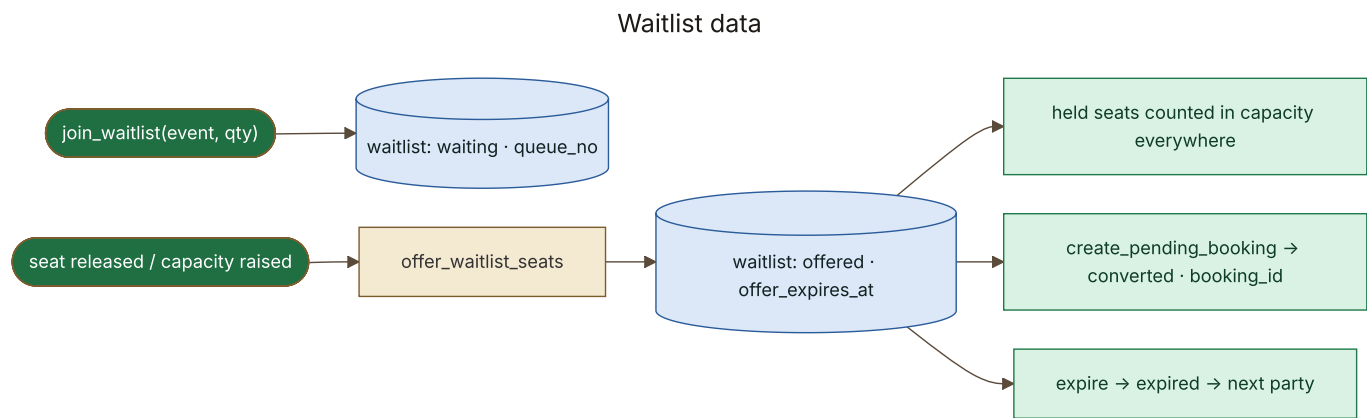


Diagram — Waitlist data

9. Notification data

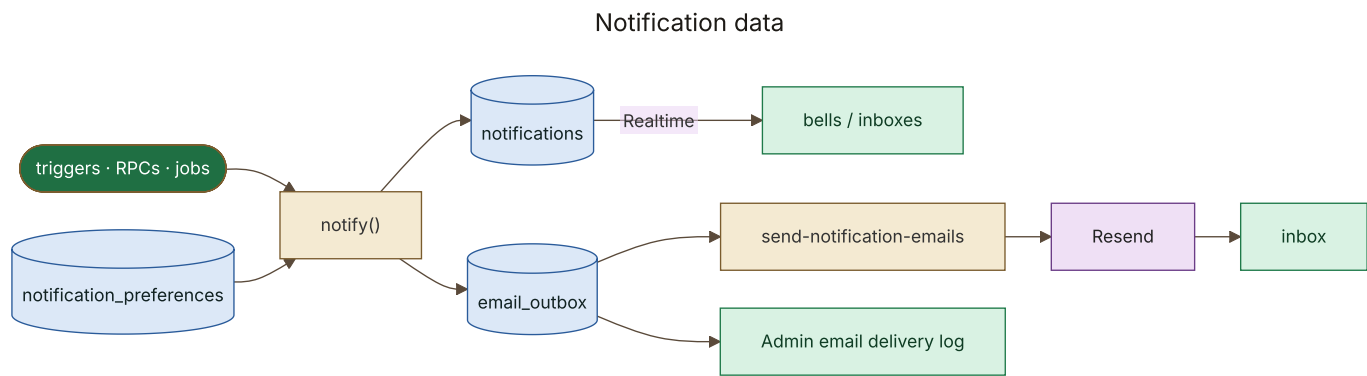


Diagram — Notification data

10. Messaging data

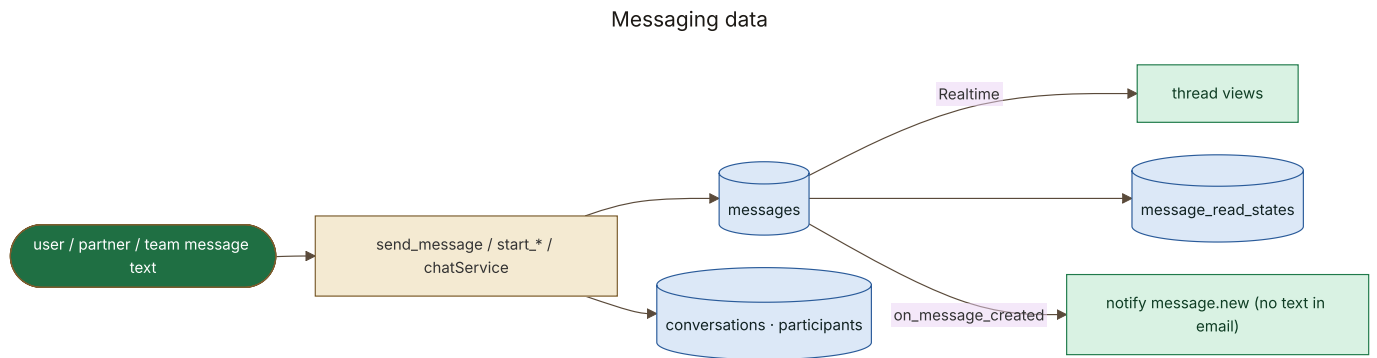


Diagram — Messaging data

11. Content data

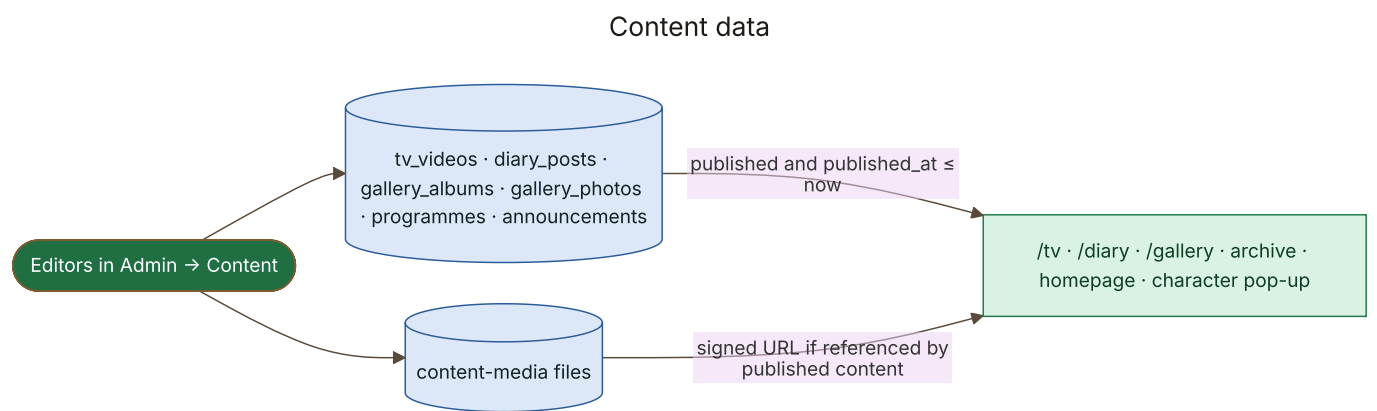


Diagram — Content data

12. Storage data

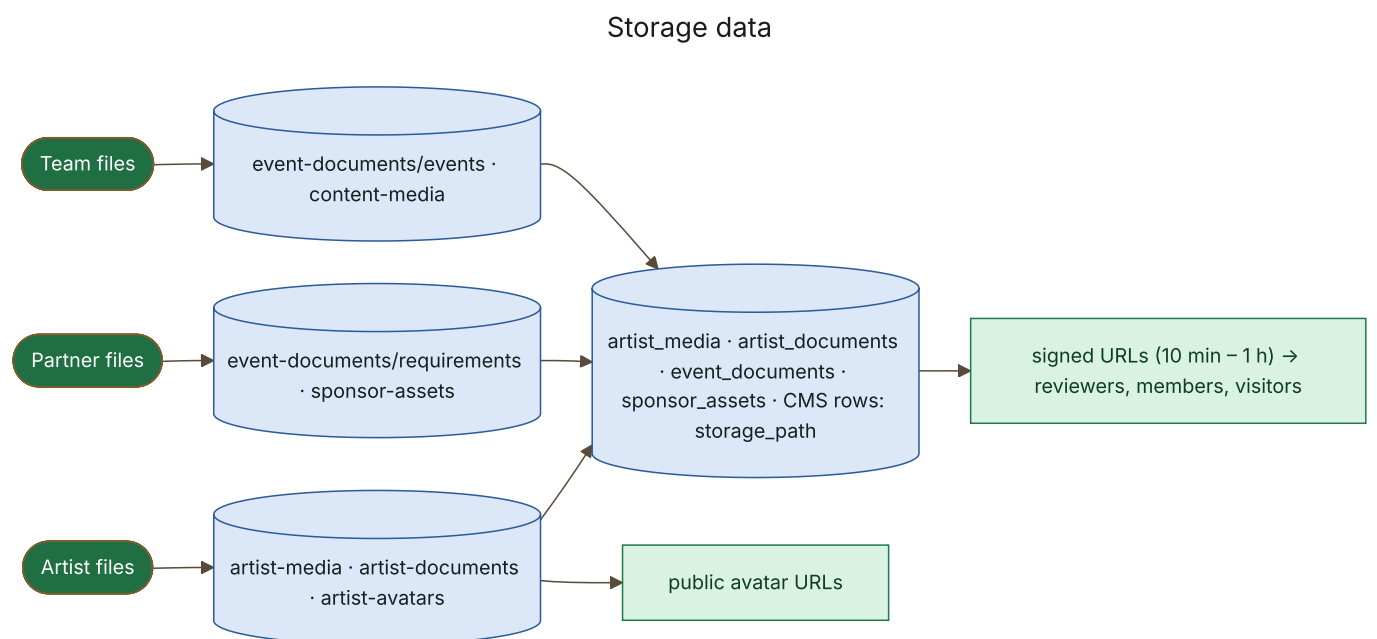


Diagram — Storage data

Part VII — Who Calls What

31 — "Who Calls What" Map

31 — "Who Calls What" Map

Generated from actual imports and call sites:

- `supabase.from()`, `.rpc()`, `functions.invoke()`, `storage.from()`, `.channel()`
- the admin helpers `list/one/insert/update/remove/useAdminList`
- `uploadWithProgress` / `signedUrl`

Part A is the curated end-to-end chain for the major journeys. Part B (appendix) is the raw per-file inventory.

Part A — end-to-end chains

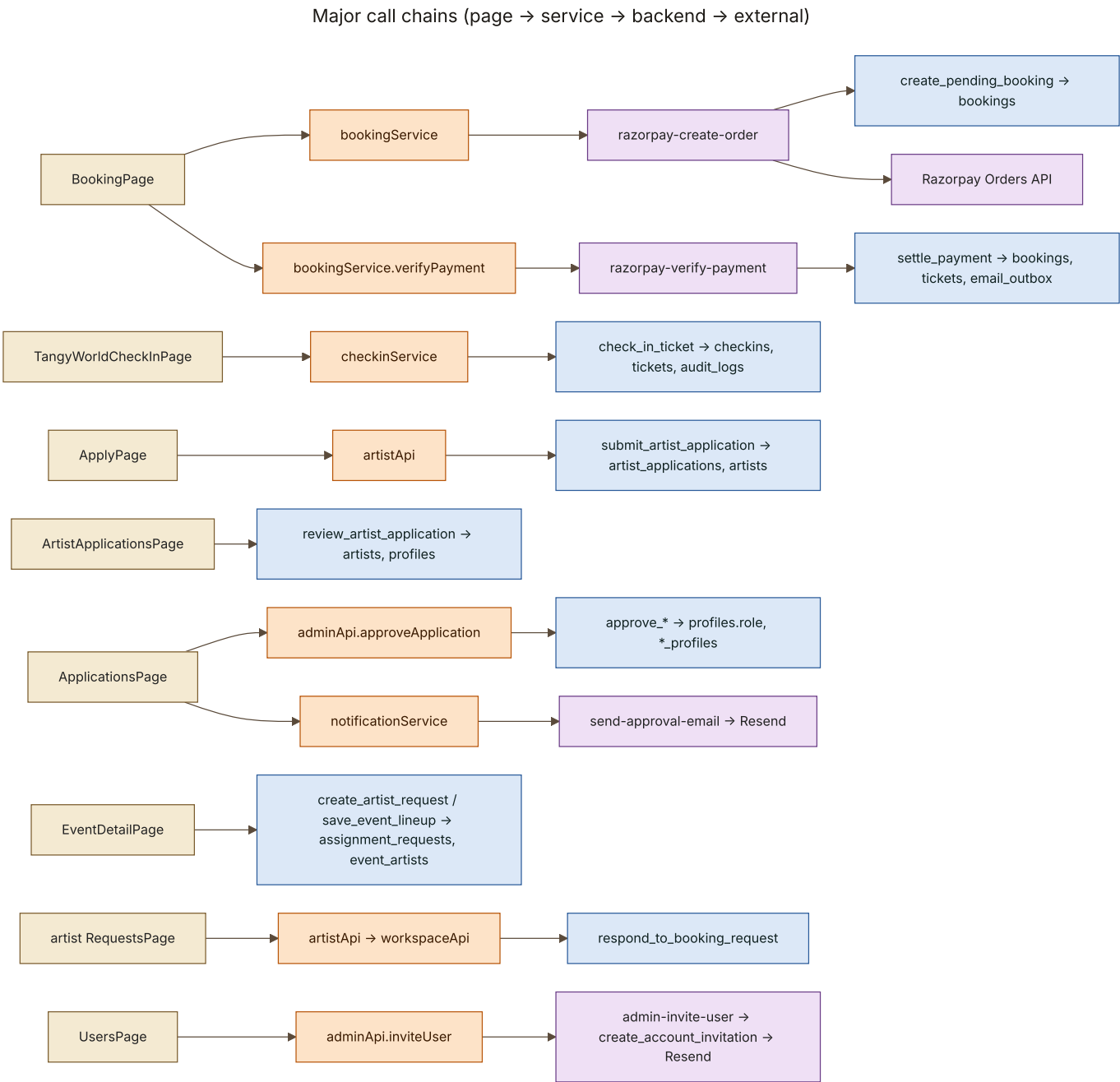


Diagram — Major call chains (page → service → backend → external)

Page	Component	Service	Supabase query / RPC	Edge Function	Tables	External	Result
/sessions/:id BookingPage	useSessionDetail, WaitlistPanel	—	events, event_artists, public_artists, booking_quote, event_availability, my_waitlist, channel availability-<id>	—	events, event_ticket_types, bookings, waitlist	—	Live price and seats
same	pay button	bookingService.createPaymentOrder	(create_pending_booking inside function)	razorpay-create-order	bookings	Razorpay Orders API	Pending booking + order
same	Razorpay handler	bookingService.verifyPayment	(settle_payment, enqueueTicketEmail inside)	razorpay-verify-payment	bookings, tickets, email_outbox, audit_logs	—	Confirmed booking + tickets
same	confirmation	bookingService.sendTicketEmail	—	send-ticket-email	bookings, email_outbox	Resend (fallback only)	No-op if queued
(Razorpay servers)	—	—	payment_webhook_events, settle_payment, record_webhook_failure	razorpay-webhook	bookings, tickets, payment_webhook_events, email_outbox	Razorpay	Source-of-truth settlement
/sessions/:id, /sessions/waitlist	WaitlistPanel, WaitlistDirectory	—	join_waitlist, leave_waitlist, event_availability, my_waitlist	—	waitlist	—	Queue position
/dashboard	PatronDashboard	bookingService.getMyBookings	bookings(*, events, tickets), my_waitlist, profiles, waitlist	—	bookings, tickets, events, waitlist	—	Bookings, QR, passport
/check-in	TangyWorldCheckIn Page	checkinService	check_in_ticket, my_checkin_events, event_checkin_stats, get_checkin_history, attendee_tickets	—	tickets, bookings, checkins, audit_logs	camera (html5-qrcode)	Admit by name
/artist/apply	ApplyPage	artistApi	artist_applications insert/update, submit_artist_application, withdraw_artist_application	—	artist_applications, artists	—	Application submitted
same	MediaStep	uploadWithProgress	storage artist-media/applications/<uid>/	—	storage.objects	—	Video uploaded
/artist/requests	RequestsPage	artistApi → workspaceApi	my_booking_requests, mark_artist_request_viewed, respond_to_booking_request	—	assignment_requests, event_artists, event_artist_details	—	Accept / decline
/artist/availability	AvailabilityPage	artistApi	set_artist_availability, artist_availability_summary, artist_availability	—	artist_availability	—	Calendar saved
/artist/media	MediaPage	uploadWithProgress, signedUrl	artist_media, storage artist-media	—	artist_media	—	Media for review

Page	Component	Service	Supabase query / RPC	Edge Function	Tables	External	Result
/artist/documents	DocumentsPage	artistApi, uploadWithProgress	artist_documents, storage artist-documents	—	artist_documents	—	Private docs
/artist/profile	ProfilePage	artist authService / workspaceApi	artists, artist_private_profiles, artist_profile_completion, storage artist-avatars	—	artists	—	Profile
/apply/vendors · sponsors · venue-host	*ApplyPage	— (direct client)	insert collaborations	—	collaborations	—	Pending application
/crew/apply, /volunteer/apply	CrewApplicationForm, VolunteerApplyPage	—	insert crew_applications; events	—	crew_applications	—	Pending application
/private-sessions, /contact	PrivateEnquiryForm, ContactEnquiryForm	—	insert private_enquiries / contact_enquiries	—	enquiries	—	Receipt notification
Partner portals	PartnerPortal, PortalSections, PartnerExtras, MessagesPanel	portalApi	my_portal_events, submit_requirement, start_partner_conversation, send_message, my_notifications, request_checkin_access, event_requirements, event_documents, sponsor_assets, partner_invoices	—	many	—	Partner workspace
/admin-portal/applications	ApplicationsPage	adminApi, notificationService	applications_overview, approve_*/reject_*, application_notifications	send-approval-email	applications, profiles	Resend	Decision + email
/admin-portal/artists/application/:id	ArtistApplicationsPage	direct supabase.rpc	review_artist_application, artist_applications, application_reviews	—	artist_applications, artists, profiles	—	Review step
/admin-portal/events/:id	EventDetailPage, EventEditor, EventOps, TicketTypesEditor	adminApi, direct	save_event_lineup, create_artist_request, manage_artist_request, update_event_artist, remove_event_artist, artist_schedule_check, event_health, event_command_center, review_requirement, close_requirement, booking_quote	—	events, event_artists, assignment_requests, event_requirements, event_documents, event_ticket_types	—	Event operations
/admin-portal/bookings	BookingsPage, Bookings.jsx	adminApi, bookingService, checkinService	admin_cancel_booking, admin_record_refund, admin_cancel_ticket, admin_create_comp_booking, booking_checkin_history	send-ticket-email (force)	bookings, tickets, audit_logs	Resend	Booking admin

Page	Component	Service	Supabase query / RPC	Edge Function	Tables	External	Result
/admin-portal/users	UsersPage	adminApi	admin_set_user_role, admin_set_user_active, list_account_invitations, revoke_account_invitation	admin-invite-user	profiles, account_invitations	Resend	Roles / invites
/admin-portal/volunteers	VolunteersPage	direct	volunteers_overview, grant_temporary_access, revoke_temporary_access, decline_access_request, volunteer_access_activity	—	temporary_access, access_requests	—	Access windows
/admin-portal/content/*	ContentPage, ContentCollections, MediaLibrary, Announcements	contentService	CMS tables, update_session_content, storage content-media	—	CMS tables	—	Publish content
/admin-portal/messages	MessagesPanel	portalApi	admin_conversations, conversation_messages, send_message, set_conversation_status, set_conversation_meta	—	conversations, messages	—	Replies
Header	AdminShell	adminApi	admin_search, log_auth_event, applications_overview (badge), conversations (badge)	—	—	—	Search / badges
Assistant	TangyAssistant, AgentRequestForm	messageService (mock), aiSupportService (mock), conversationService	get_or_create_support_conversation, messages	—	conversations, messages	—	Agent handoff

Part B — appendix: per-file Supabase access (generated)

Files under `src/` that call Supabase directly or through the admin helpers. Files that only call other services are omitted. The mock services

(`src/services/{bookingService,collaborationService,enquiryService,eventService,userService,waitlistService,messageService,aiSupportService}.js`) make no Supabase calls.

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
<code>src/admin/AdminShell.jsx</code>	—	<code>applications_overview</code> , <code>conversations</code>	—	—	—
<code>src/admin/api.js</code>	<code>admin_cancel_booking</code> , <code>admin_cancel_ticket</code> , <code>admin_create_comp_booking</code> , <code>admin_dashboard_summary</code> , <code>admin_operations_overview</code> , <code>admin_record_refund</code> , <code>admin_search</code> , <code>admin_set_user_active</code> , <code>admin_set_user_role</code> , <code>check_in_ticket</code> , <code>create_booking_request</code> , <code>event_checkin_stats</code> , <code>event_health</code> , <code>get_checkin_history</code> , <code>get_runtime_settings</code> , <code>list_account_invitations</code> , <code>log_auth_event</code> , <code>my_checkin_events</code> , <code>my_permissions</code> , <code>report_applications</code> , <code>report_event_performance</code> , <code>report_revenue_by_month</code> , <code>report_staff_activity</code> , <code>revoke_account_invitation</code> , <code>staff_dashboard</code> , <code>update_system_setting</code>	—	<code>admin-invite-user</code>	—	—
<code>src/admin/components/Announcements.jsx</code>	—	<code>announcements</code>	—	—	—
<code>src/admin/components/ArtistAvailability.jsx</code>	<code>find_available_artists</code>	—	—	—	—
<code>src/admin/components/Attendees.jsx</code>	—	<code>attendee_tickets</code>	—	—	—
<code>src/admin/components/BookingFormEditor.jsx</code>	—	<code>events</code>	—	—	—
<code>src/admin/components/Bookings.jsx</code>	—	<code>audit_logs</code> , <code>bookings</code> , <code>event_ticket_types</code> , <code>events</code>	—	—	—
<code>src/admin/components/ContentCollections.jsx</code>	—	<code>events</code> , <code>gallery_photos</code> , <code>programme_events</code>	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/admin/components/CustomNotification.jsx	send_custom_notification	—	—	—	—
src/admin/components/EventEditor.jsx	save_event_lineup	event_artists, events	—	—	—
src/admin/components/EventForm.jsx	—	events, venue_profiles, venues	—	—	—
src/admin/components/EventOps.jsx	close_requirement, event_command_center, review_requirement	audit_logs, event_artist_details, event_artists, event_assignments, event_documents, event_requirements, profiles	—	event-documents	—
src/admin/components/MediaLibrary.jsx	—	—	—	content-media	—
src/admin/components/Tasks.jsx	—	event_assignments, event_tasks	—	—	—
src/admin/components/Team.jsx	—	event_assignments, event_tasks, profiles	—	—	—
src/admin/components/TicketTypesEditor.jsx	booking_quote	event_ticket_types	—	—	—
src/admin/pages/ApplicationsPage.jsx	—	application_notifications, applications_overview, bookings, profiles	—	—	—
src/admin/pages/ArtistApplicationsPage.jsx	review_artist_application	application_reviews, artist_applications	—	artist-media	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/admin/pages/ArtistDetailPage.jsx	admin_start_partner_conversation, artist_availability_summary, artist_calendar, start_private_artist_conversation	artist_applications, artist_media, artists, assignment_requests, conversations, event_artists	—	—	—
src/admin/pages/BookingsPage.jsx	—	bookings, payment_webhook_events	—	—	—
src/admin/pages/CalendarPage.jsx	admin_calendar	—	—	—	—
src/admin/pages/ContentPage.jsx	update_session_content	artists, events	—	—	—
src/admin/pages/DashboardPage.jsx	artist_day_summary	access_requests, announcements, audit_logs, bookings, conversations, event_requirements, event_tasks	—	—	—
src/admin/pages/EventDetailPage.jsx	artist_schedule_check, create_artist_request, manage_artist_request, remove_event_artist, save_event_lineup, update_event_artist	artists, assignment_requests, bookings, event_artist_details, event_artists, events, sponsor_deliverables, sponsor_profiles, tickets, venue_profiles, venues	—	—	—
src/admin/pages/InvoicesPage.jsx	—	partner_invoices, profiles	—	—	—
src/admin/pages/MyEventsPage.jsx	—	event_assignments	—	—	—
src/admin/pages/PeoplePage.jsx	—	event_artists, event_assignments, events, sponsor_deliverables	—	—	—
src/admin/pages/ReportsPage.jsx	report_platform_activity	—	—	—	—
src/admin/pages/RolesPage.jsx	set_role_permission	role_permissions	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/admin/pages/SettingsPage.jsx	—	role_permissions, system_settings	—	—	—
src/admin/pages/StaffAnnouncementsPage.jsx	—	announcements	—	—	—
src/admin/pages/StaffEventPage.jsx	—	announcements, event_assignments, events, venues	—	—	—
src/admin/pages/TeamPage.jsx	—	event_assignments, events, profiles	—	—	—
src/admin/pages/UsersPage.jsx	—	audit_logs, profiles	—	—	—
src/admin/pages/VolunteersPage.jsx	decline_access_request, grant_temporary_access, revoke_temporary_access, volunteer_access_activity, volunteers_overview	event_assignments, events, profiles	—	—	—
src/admin/sections/EnquiriesSection.jsx	—	contact_enquiries, private_enquiries	—	—	—
src/admin/sections/NotificationsSection.jsx	—	application_notifications, email_outbox	—	—	—
src/admin/sections/WaitlistSection.jsx	admin_offer_waitlist, admin_remove_waitlist_entry	waitlist	—	—	—
src/artist/components/ArtistEventDrawer.jsx	—	partner_invoices	—	—	—
src/artist/pages/ArtistDetailsPage.jsx	—	event_artists, public_artists	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/artist/pages/ArtistsDirectoryPage.jsx	—	public_artists	—	—	—
src/artist/pages/MediaPage.jsx	—	—	—	artist-media	—
src/artist/pages/ProfilePage.jsx	—	artists	—	artist-avatars	—
src/artist/portals/api.js	artist_availability_summary, mark_artist_request_viewed, set_artist_availability, submit_artist_application, withdraw_artist_application	artist_applications, artist_availability, artist_documents	—	—	—
src/artist/portals/pages/ApplyPage.jsx	—	—	—	artist-media	—
src/artist/portals/pages/InboxPages.jsx	—	—	—	artist-documents	—
src/artist/services/authService.js	—	artists	—	—	—
src/artist/services/workspaceApi.js	artist_profile_completion, my_booking_requests, my_notification_preferences, respond_to_booking_request, set_notification_preference	artist_availability, artist_media, artist_private_profiles, artists	—	—	—
src/components/archive/ArchiveIndex.jsx	—	diary_posts, events, gallery_albums, programmes, public_artists, tv_videos	—	—	—
src/components/booking/WaitlistDirectory.jsx	event_availability, my_waitlist	—	—	—	—
src/components/booking/WaitlistPanel.jsx	join_waitlist, leave_waitlist	—	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/components/calendar/calendarData.js	artist_calendar	—	—	—	—
src/components/contact/ContactEnquiryForm.jsx	—	contact_enquiries	—	—	—
src/components/crew/CrewApplicationForm.jsx	—	crew_applications	—	—	—
src/components/museum/UserLoginModal.jsx	—	checkins	—	—	—
src/components/private/PrivateEnquiryForm.jsx	—	private_enquiries	—	—	—
src/context/UserAuthContext.jsx	—	profiles	—	—	—
src/hooks/useEvents.js	—	events	—	—	—
src/hooks/useSessionDetails.js	booking_quote, event_availability, my_waitlist	event_artists, events, public_artists	—	—	availability-\${eventId}
src/lib/archiveService.js	—	diary_posts, event_artists, events, gallery_albums, programme_events, programmes, public_artists, tv_videos	—	—	—
src/lib/bookingService.js	—	bookings	razorpay-create-order, razorpay-verify-payment, send-ticket-email	—	—
src/lib/checkinService.js	booking_checkin_history, check_in_ticket, event_checkin_stats, get_checkin_history, my_checkin_events	attendee_tickets	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/lib/contentService.js	—	diary_posts, event_artists, gallery_albums, gallery_photos, public_artists, tv_videos	—	content-media	—
src/pages/InvitationPage.jsx	accept_account_invitation, invitation_preview	—	—	—	—
src/pages/ProfilePage.jsx	—	profiles	—	—	—
src/pages/SponsorApplyPage.jsx	—	collaborations	—	—	—
src/pages/VendorApplyPage.jsx	—	collaborations	—	—	—
src/pages/VenueHostApplyPage.jsx	—	collaborations	—	—	—
src/pages/admin/AdminArtistPreview.jsx	—	artist_availability, artist_media, artists, assignment_requests, event_artists	—	—	—
src/pages/admin/AdminEntitySelector.jsx	—	artists, crew_profiles, profiles, sponsor_profiles, vendor_profiles, venue_profiles, volunteer_profiles	—	—	—
src/pages/admin/AdminPortalPreview.jsx	—	profiles	—	—	—
src/pages/dashboards/CrewDashboard.jsx	—	crew_applications, crew_profiles, event_assignments, event_tasks	—	—	—
src/pages/dashboards/PatronDashboard.jsx	my_waitlist	profiles, waitlist	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/pages/dashboards/SponsorDashboard.jsx	—	collaborations, sponsor_deliverables, sponsor_profiles	—	—	—
src/pages/dashboards/VendorDashboard.jsx	—	collaborations, event_assignments, vendor_profiles	—	—	—
src/pages/dashboards/VenueDashboard.jsx	—	collaborations, events, venue_profiles	—	—	—
src/pages/dashboards/VolunteerDashboard.jsx	—	crew_applications, event_assignments, volunteer_profiles	—	—	—
src/pages/subsections/VolunteerApplyPage.jsx	—	crew_applications, events	—	—	—
src/portals/MessagesPanel.jsx	—	profiles	—	—	—
src/portals/NotificationPreferences.jsx	my_notification_preferences, set_notification_preference	—	—	—	—
src/portals/PartnerExtras.jsx	—	event_tasks, partner_invoices, sponsor_assets, sponsor_deliverables	—	sponsor-assets	—
src/portals/PortalSections.jsx	—	event_assignments	—	event-documents	—
src/portals/portalApi.js	admin_conversations, admin_start_partner_conversation, conversation_messages, mark_conversation_read, mark_notifications_read, my_checkin_access, my_conversations, my_notifications, my_portal_events, notification_unread_count, portal_announcements, request_checkin_access, send_message, set_conversation_meta, set_conversation_status, start_partner_conversation, submit_requirement	event_documents, event_requirements	—	—	—
src/services/announcementService.js	—	announcements	—	—	—

Source file	RPCs	Tables / views	Edge Functions	Storage	Realtime
src/services/assignmentRequestService.js	cancel_assignment_request, create_assignment_request, respond_to_assignment_request	artists, assignment_requests, events	—	—	—
src/services/chatService.js	assign_conversation, get_or_create_support_conversation, reopen_conversation	artists, conversations, message_read_states, messages, profiles	—	—	conversations:inbox, messages:\${conversationId}
src/services/notificationService.js	—	application_notifications	send-approval-email	—	—

Part VIII — Complete Business Flow

32 — Complete Business Flow

32 — Complete Business Flow

One lifecycle that connects every subsystem, as implemented. Dashed boxes are **not implemented** or **production configuration**.

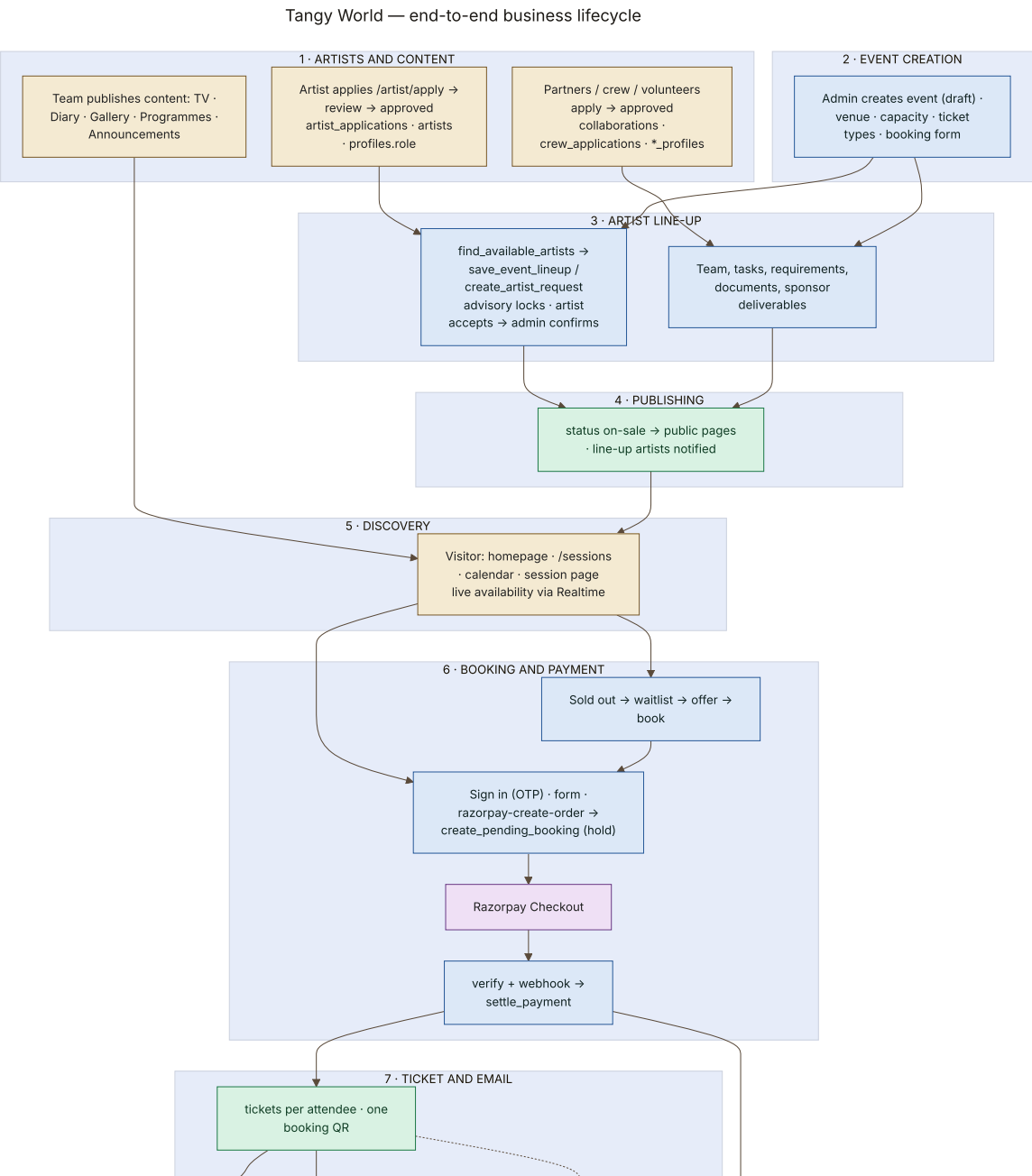


Diagram — Tangy World — end-to-end business lifecycle (part 1 of 2)

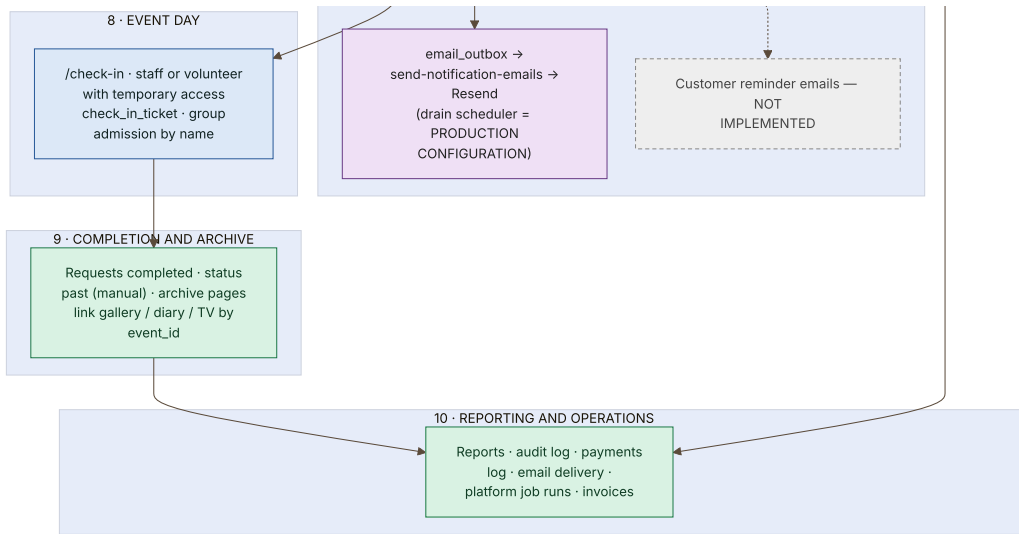


Diagram — Tangy World — end-to-end business lifecycle (part 2 of 2)

Systems touched at each stage

Stage	Frontend	Database	Edge / external	People notified
Artists & content	ApplyPage, ArtistApplicationsPage, ApplicationsPage, Content	artist_applications, artists, collaborations, crew_applications, CMS tables	send-approval-email → Resend	applicant, reviewers
Event creation	EventForm, TicketTypesEditor, BookingFormEditor	events, event_ticket_types	—	—
Line-up	EventDetailPage, artist Requests / Availability	event_artists, event_artist_details, assignment_requests, artist_availability	—	artist, requesting admin
Publishing	event status	events	—	line-up artists
Discovery	public pages	events, public_artists, availability signal	Realtime	—
Booking & payment	BookingPage	bookings, waitlist, payment_webhook_events	razorpay-* → Razorpay	payments.view on problems
Ticket & email	confirmation, dashboard	tickets, email_outbox	send-notification-emails → Resend	customer
Event day	/check-in	checkins, audit_logs, temporary_access	camera	—
Completion & archive	archive pages	assignment_requests, events, CMS	—	—
Reporting & operations	Reports, Audit, Payments, Email delivery	report_* RPCs, audit_logs, platform_job_runs	pg_cron	—

Part IX — Explaining & Presenting

33 — How to Explain Tangy World to Someone · 34 — How I Should Present This System

33 — How to Explain Tangy World to Someone

Plain language, no jargon. Use this when talking to a client, a new team member or anyone non-technical.

The one-sentence version

Tangy World is the website and back-office for Tangy Sessions. Fans discover and book intimate live-music nights. Artists and partners apply and work with the team. The Tangy team runs every event from one control room.

Three layers

Tangy World has three major layers that all share one secure database.

First, the public website. This is what fans see. It looks like a 1970s concert poster and a music museum: a hanging microphone you follow down the page, a diary whose pages turn as you scroll, Tangy TV, a gallery and an archive of past sessions. Each upcoming session has its own page, and that same page is where you book.

Second, the operations system. The Tangy team signs in to the "Admin Portal". There they create sessions, choose artists, set ticket types and prices, see bookings and payments, publish content, answer messages, manage volunteers, read reports and see a full history of who did what. A separate check-in screen runs on a phone at the door.

Third, the artist and partner portals. Artists, sponsors, vendors, venue hosts, crew and volunteers each get their own workspace after the team approves them. Artists mark when they are free, accept or decline performance requests, upload media and documents, and message the team. Partners see the events they are part of, send what the team needs, and see invoices.

Everything is decided by the database

The website only *shows* things. Every rule lives in the database, and it checks every single request no matter how the request was sent. That covers who can see what, how many seats are left, what a ticket costs, and whether a payment really happened. Hiding a button on screen is never the security; the database is.

Walk-throughs

When a customer books a ticket...

1. They open a session and pick a ticket type and how many people.
2. The price, including tax, is calculated by the database, not by the browser.
3. They sign in with a one-time code sent to their email.
4. They enter a name for each person coming.
5. The system holds those seats for 30 minutes and opens the Razorpay payment window.

When payment succeeds...

1. Two independent messages confirm it: one from the customer's browser and one sent directly by Razorpay to our server.
2. Each is checked with a secret cryptographic signature.
3. Whichever arrives first confirms the booking and creates one ticket per named person. The second does nothing extra, so nobody is double-charged or double-ticketed.
4. Even if the customer closes the browser, Razorpay's own message still confirms the booking.
5. The customer sees one QR code for their whole group. The same QR is emailed to them once email is switched on.

When a session sells out...

1. Fans can join a waitlist.
2. If someone cancels, or the team adds seats, the next person in line gets those seats held for two hours and is notified.
3. If they don't book in time, the seats move to the next person.

When someone arrives at the event...

1. Staff (or a volunteer given temporary access for that night) scan the QR on their phone.
2. The system checks the ticket is real, paid and for tonight's event.
3. It shows the list of names on the booking. Staff tick off who has arrived, and latecomers can be checked in later from the same QR.
4. Every check-in is recorded.

When an artist applies...

1. They fill in an 8-step application: about them, their music, experience, links, a performance video, technical needs, availability, then a final review.
2. Their answers save automatically as they go.
3. The team reviews it and can ask for more information. The artist updates and resubmits.
4. Once approved, the artist's account becomes an artist account and their portal opens.

When the team creates an event...

1. They set the date, venue, capacity, ticket types and the booking form.
2. Then they build the line-up. The system shows which artists are free, tentative, unavailable or already booked that day.
3. It refuses to double-book an artist, even if two team members try at the same moment.

When an artist accepts a performance...

1. They open the request in their portal and accept it (or decline with a reason).
2. The system checks again that they are not booked elsewhere at that time, then adds them to the line-up.
3. The team confirms, and the artist sees their call time, soundcheck, set time and fee.

When the team publishes the event...

1. It appears on the public site.
2. The artists already on the line-up are told they are performing.

Honest status

The platform is fully built and tested on a local copy of the system. Going live still needs the production setup:

- loading the database into the production project
- adding the Razorpay and email service keys
- setting up the email sending schedule
- pointing the website at the real domain

None of that is done yet.

Two things fans might expect are not built yet:

- reminder emails to ticket holders before the event
- automatic messages to ticket holders if an event is cancelled

The public "Tangy AI" assistant answers from a fixed list of questions rather than a real AI model, and hands over to a human when it can't help.

34 — How I Should Present This System

A recommended **25-minute** order; trim to 15 by skipping the starred (★) items. Each step names the document and the diagram to put on screen.

Presentation order (25 min)

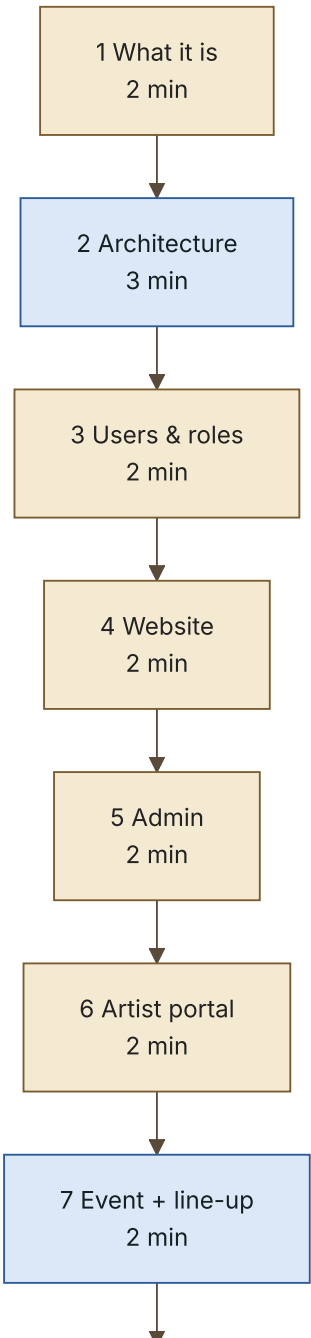


Diagram — Presentation order (25 min) (part 1 of 2)

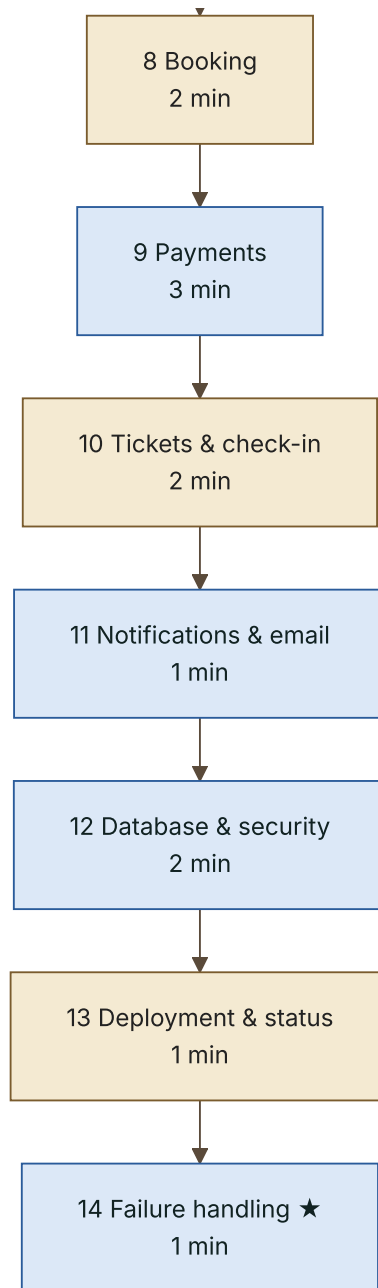


Diagram — Presentation order (25 min) (part 2 of 2)

#	Topic	Show	Key message
1	What Tangy World is	01 three-layers diagram	One app: public experience, operations console, partner portals, on one database
2	High-level architecture	01 system context	React SPA (Vercel) → Supabase (Auth, Postgres + RLS, Storage, Realtime, Edge Functions) → Razorpay, Resend
3	Users	02 "How each role is obtained"	10 roles; nobody grants themselves a role; approvals and invitations provision access; permissions live in a table
4	Website	03 sitemap	The session page is the checkout; the content is CMS-driven; the archive is built from past sessions
5	Admin	06 generic request path	Navigation is generated from permissions; the database re-checks every action; everything is audited
6	Artist portal	07 lifecycle + 08 state machine	8-step autosaved application → review loop → portal
7	Event workflow	09 + 10 availability decision	One function decides availability; advisory locks stop double-booking
8	Booking workflow	11 journey	Seat hold under an event row lock; price from the database

#	Topic	Show	Key message
9	Payment workflow	12 sequence + <code>settle_payment</code> tree	Two paths (browser + webhook), HMAC-verified, converge on one idempotent settlement; webhook = source of truth
10	Ticket / check-in	13 + 14	One opaque QR per booking; admit people by name; volunteers get time-boxed access
11	Notifications / email	16	<code>notify()</code> → in-app + email outbox → drain → Resend; preferences; nothing sensitive in email
12	Database & security	21 domain map + 25 layers	~240 RLS policies, definer functions with pinned <code>search_path</code> , revoked EXECUTE, append-only audit
13	Deployment	27	Built for Vercel + Supabase; backend deployment is an operator checklist not yet done
14 ★	Failure handling	29 payments diagram	Every critical failure has a recovery path

Ten-minute version

1. (1 min) What it is: the poster-style site, the console, the portals.
2. (2 min) Architecture diagram; the rule *"the database decides"*.
3. (2 min) Booking + payment sequence, with the webhook as the source of truth.
4. (1 min) Check-in by group QR.
5. (2 min) Artist application → line-up with availability and locks.
6. (1 min) Security layers.
7. (1 min) Honest status and gaps (below).

Questions to prepare for

Likely question	Answer (where to point)
How do you stop overselling?	Event row lock + capacity check in <code>create_pending_booking</code> ; waitlist holds counted (12)
What if the browser closes after paying?	The Razorpay webhook settles and queues the email (12)
Can a user make themselves admin?	No: the <code>prevent_role_self_escalation</code> trigger; roles change only through audited RPCs (02, 25)
How does staff access stay limited?	Event-scoped functions; <code>is_staff_or_admin()</code> is admin-only since 0017 (25)
Is chat encrypted?	Not end-to-end; protected by HTTPS, auth and RLS; private Super Admin ↔ artist threads (18)
Is it live?	Code complete and tested locally; production configuration pending (27)

Gaps to mention honestly

- Backend not deployed; production secrets, the email drain scheduler and the OTP template are pending.
- No customer reminder emails; no automatic customer notification or refund when an event is cancelled; refunds are manual in Razorpay.
- `sold-out` / `past` event statuses are set manually (capacity itself is enforced automatically).
- Artist approval from the Artist Applications page sends no branded email (in-app only).
- No CSP / security headers in `vercel.json`.
- The Tangy AI assistant is a knowledge-base lookup, not a model.